

10 - Data Scientist:

Analyze Data with

R

Syllabus

1. What Is R?
 - Welcome To Analyze Data With R
 - 1 Informational
 - Learn R: Introduction
 - 1 Lesson, 1 Project, 1 Quiz, 2 Videos
2. Introduction To Programming In R
 - Welcome to Programming in R
 - 1 Informational
 - Intro to Programming in R
 - 1 Lesson, 3 Articles, 1 Quiz, 1 Project
 - Introduction To Off Platform Projects In R
 - 1 Informational
3. Statistics In R
 - Welcome To Statistics In R
 - 1 Informational
 - Learn R: Mean, Median, and Mode
 - 3 Lessons, 1 Project, 1 Quiz
 - Learn R: Variance and Standard Deviation
 - 2 Lessons, 1 Quiz, 1 Project
 - Learn R: Quartiles, Quantiles, and Interquartile Range
 - 3 Lessons, 1 Quiz, 1 Project
 - Learn R: Hypothesis Testing
 - 1 Lesson, 1 Quiz, 1 Project
4. Working With Data In R
 - Welcome To Working With Data In R
 - 1 Informational
 - Learn R: Data Frames
 - 2 Lessons, 1 Quiz, 1 Project
 - Learn R: Data Cleaning
 - 1 Lesson, 1 Quiz, 1 Project
 - Learn R: Aggregates
 - 1 Lesson, 1 Quiz, 1 Project
 - Learn R: Joining Tables
 - 1 Lesson, 1 Quiz, 1 Project
 - Working With Data Off Platform Project
 - 1 Informational
5. Data Visualization In R
 - Welcome To Data Visualization In R
 - 1 Informational
 - Learn R: Fundamentals of Data Visualization with ggplot2
 - 1 Lesson, 1 Quiz, 1 Project
 - Intermediate Data Visualization With ggplot2
 - 1 Lesson, 1 Quiz, 1 Project
 - Data Visualization In R Off Platform Project
 - 1 Informational
6. Introduction To Machine Learning In R
 - Welcome To Machine Learning In R
 - 1 Informational
 - Linear Regression In R
 - 1 Lesson, 1 Article, 1 Quiz, 1 Project
 - Linear Regression In R Off Platform Project
 - 1 Informational
7. Analyze Data With R Capstone Project
 - Analyze Data With R Capstone Project
 - 1 Informational

Analyze Data With R Introduction

Welcome to the Analyze Data With R Skill Path! Let's talk about what you'll learn in this path.

Welcome to the Analyze Data With R Skill Path! This path will take you from writing your first line of R code all the way to creating a professional data science report.

The path is broken into 7 main sections.

What is R? — We'll teach you some R basics and help you set up R on your own computer.

Introduction to Programming in R — Learn fundamental programming topics like loops, functions, and control flow. Also learn about different data structures that we'll use throughout the course.

Statistics in R — Get a crash course in introductory statistics and learn how to use them in R.

Working with Data in R — Start working with larger datasets. Learn how to import and manipulate spreadsheet data in R.

Data Visualization in R — Learn how to make visualizations to help visually explain findings in your data.

Introduction to Machine Learning in R — Dip your toes into the world of machine learning as you begin to build statistical models using real data.

Capstone Project — Put all of your knowledge to the test and build a data report based on a topic of your choosing.

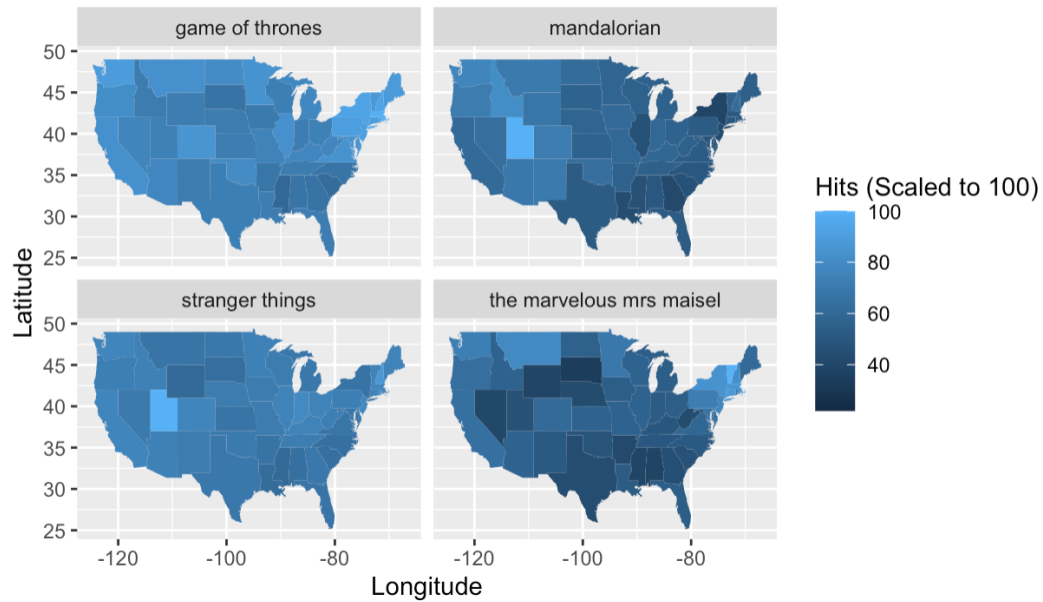
Throughout the path, you'll work on some really fun projects using real data! Take a look below for a preview of the type of project you'll be working on.

Analyze Citi Bike User Patterns

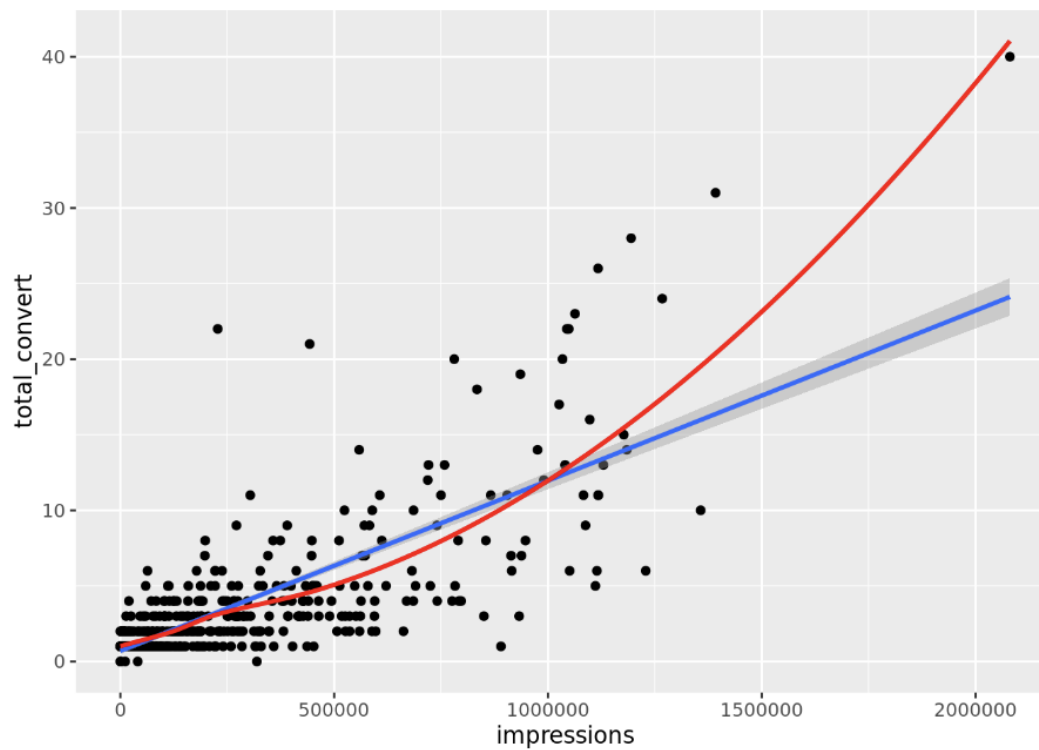


Look At Google Trends Data For TV Show

Google Trend Data For Streaming Service Shows By State



Create A Statistical Model For Businesses



Introduction to R Syntax

Why R?

Ahoy! We R excited for you to start your learning adventure with a language built for data enthusiasts! The R community is made up of people passionate about the intersection of numbers, data, analysis, and code. In this lesson, we will introduce you to some basic R syntax and discuss how R classifies

[data types](#)

Preview: Docs Loading link description

so that it can mathematically process them in analysis.

But before you go any further, let's talk about how R is different from most programming languages. Unlike with other languages, most beginners who want to learn R do so because they want to analyze data. In this way, R is more of a tool to understand data than a programming language used to build software applications. Our approach to teaching you R will be similar to teaching you how to use a new tool.

What is the tool for? R is powerful for conducting statistics and other specialized data analysis. It was invented by scientists for statistical computing and a community of specialized packages has been built around the language.

Some housekeeping: over the next exercises you'll be conducting all of your work inside R-notebook files. You'll see the **.Rmd** file format will allow you to write blocks of code and see their output as a webpage. You will see the blocks wrapped inside the following syntax:

```
This is regular markdown text that will render as regular text.
` `` `{r}
7 + 7 # This is R code
` `` `
```

Copy to Clipboard

This format allows you to group blocks of code with common logic, see their output, and then render all of it together as an HTML file that can be viewed on a browser as a static webpage. The **.Rmd** format is the preferred standard in the data science industry.

Note: This lesson assumes no prior coding knowledge and will introduce programming concepts as you need them in order to continue using R as a tool to conduct analysis.

Introduction to R Syntax

Calculations

Let's start with the basic syntax for mathematical calculations in R. R performs addition, subtraction, multiplication, and division with **+**, **-**, *****, and **/**:

```
# Results in "500"
573 - 74 + 1

# Results in "50"
25 * 2
```

```
# Results in "2"  
10 / 5
```

Copy to Clipboard

Mathematical operations in R follow the standard mathematical [order of operations](#). Let's write your first line of R code and calculate some basic math!

Instructions

Checkpoint 1 Enabled

1.

Calculate the result of this equation: `25 * 4 + 9 / 3` in the R code block in your **notebook.Rmd** R-markdown file. Before you press run, try to figure out what the answer would be based on the order of operations. Your answer should match the output!

Introduction to R Syntax

Comments

Ironically, the second thing we're going to do is show you how to tell R to ignore a part of your program. We promise it's very useful to know how to do this. Text written in a program but not run by the computer is called a *comment*. R interprets anything after a `#` as a comment.

Why would anyone want the computer to ignore a part of their file? Multiple reasons!

Comments

Preview: Docs Code comments are used to document and clarify parts of the code for better readability and maintenance.

can:

Provide context for why something is written the way it is:

```
# This code will be used to count the number of times anyone tweets the word  
persnickety  
persnickety_count <- 0
```

Copy to Clipboard

Help other people reading the code understand it faster:

```
# This code will calculate the likelihood that it will rain tomorrow  
complicated_rain_calculation_for_tomorrow()
```

Copy to Clipboard

Ignore a line of code and see how a program will run without it:

```
# useful_value <- old_sloppy_code()  
useful_value <- new_clean_code()
```

Copy to Clipboard

Annotating or documenting your code can help other people read your program later! It could also help your future-self understand the code when you go back and read an old file trying to remember how it works. Documenting your code will help others reproduce it, and it will help you become a better programmer too. Note: In R notebooks, you're also allowed to add documentation using markdown text outside the code blocks.

Instructions

Checkpoint 1 Enabled

1.

Add a comment inside the code block that explains that the line of code inside **notebook.Rmd** is computing the volume of a cube.

```
---
title: "Introduction to R Syntax"
output: html_notebook
---
```{r}
This is a comment added by Lei Soares
3 * 3 * 3
```
```

Introduction to R Syntax

Data Types

Now that you know how to calculate basic math and add

[comments](#)

Preview: Docs Loading link description

explaining your code, let's dive into how R "thinks about" different types of data. In R and in programming, *data types* are the classifications we give different kinds of information pieces. In this lesson, we will explore the following R data types:

Numeric: Any number with or without a decimal point: `23`, `0.03` and the numeric null value `NA`.

Character: Any grouping of characters on your keyboard (letters, numbers, spaces, symbols, etc.) or text.

Most

[strings](#)

Preview: Docs Loading link description

are surrounded by single quotes: `'... '` or double quotes `"... "`, though we prefer single quotes.

Sometimes you will hear this type referred to as "string."

Logical: This data type only has two possible values— either `TRUE` or `FALSE` (without quotes). It's helpful to think of logical types or booleans as on and off switches or as the answers to a "yes" or "no" question.

Vectors: A list of related data that is all the same type.

NA: This data type represents the absence of a value, and is represented by the keyword `NA` (without quotes) but it has its own significance in the context of the different types. That is there is a numeric NA, a character NA, and a logical NA.

Let's get comfortable with checking the data type of the following:

```
class(2) # numeric
class('hello') # character
class('23') #character
class (FALSE) #logical
class(NA) #logical
```

Copy to Clipboard

In the example above, notice that the third line is labeled a character type. Why? Because the characters `23` are in quotes, so it gets classified as a character.

Instructions

Checkpoint 1 Enabled

1.

In order to print a value, you must put the value inside the following syntax: `print()`. Print your name as a character string.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print your age as a numeric type.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

On a new line of code, print your age as a character type.

```
---
title: "Introduction to R Syntax"
output: html_notebook
---
```{r}
print('String - name: Lei soares')
print(123)
print('25')
```
```

Introduction to R Syntax

Variables

Now that you know how R classifies some of the basic information types, let's figure out how to store them. In programming, variables allow us to store information and associate that information with a name. In R, we *assign* variables by using the assignment operator, an arrow sign (`<-`) made with a carat and a dash.

```
full_name <- "Natalia Rodriguez Nuñez"
```

Copy to Clipboard

In the example above, we store the string value "Natalia Rodríguez Nuñez" in a variable called `full_name`. Variables can't have spaces or symbols in their names other than an underscore (`_`). They can't begin with numbers but they can have numbers after the first letter (e.g., `cool_variable_5` is OK).

It's no coincidence we call these creatures "variables". If we need to update a variable but perform the same logical process on it, we can change its value! For example, take the variable `message_string`:

```
# Greeting
message_string <- "Hello there"
print(message_string)

# Farewell
message_string <- "Hasta la vista"
print(message_string)
```

Copy to Clipboard

Above, we create the variable `message_string`, assign a welcome message, and print the greeting. After we greet the user, we want to wish them goodbye. We then update `message_string` to a departure message and print that out.

Note: You can also use `=` instead of `<-` to assign a value but R-tists(R programmers) prefer to do it with an arrow.

Instructions

Checkpoint 1 Enabled

1.

Create a variable `name` with your first name as a string.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.
Create a variable `age` with your age as a number.

Introduction to R Syntax

Vectors

We mentioned

[Vectors](#)

Preview: Docs Loading link description
when we introduced

[data types](#)

Preview: Docs Loading link description
earlier. In R, vectors are a list-like structure that contain items of the same data type.
Take a look here:

```
spring_months <- c("March", "April", "May", "June")
```

Copy to Clipboard

In the example above, we created a new variable with the value of a new vector. We created this vector by separating four character

[strings](#)

Preview: Docs Loading link description
with a comma and wrapping them inside `c()`.

A few things you should know how to do with vectors:

You can check the type of elements in a vector by using `typeof(vector_name)`

You can check the length of a vector by using `length(vector_name)`

You can access individual elements in the vector by using `[]` and placing the element position inside the brackets. For example, if we wanted to access the second element we would write: `vector_name[2]`. Note: In R, you start counting elements at position one, not zero.

Instructions

Checkpoint 1 Enabled

1.

Create a numeric vector named `phone` that contains your phone number as three numbers: your area code, the next three digits, and then the last four.

```
---  
title: "Introduction to R Syntax"  
output: html_notebook  
---  
<<<{r}  
phone <- c(543, 232, 1293)  
<<<
```

Introduction to R Syntax

Conditionals

In R, we will often perform a task based on a condition. For example, if we are analyzing data for the summer, then we will only want to look at data that falls in June, July, and August.

We can perform a task based on a condition using an if statement:

```
if (TRUE) {  
  print('This message will print!')  
}
```

Copy to Clipboard

Notice in the example above, we have an *if* statement. The if statement is composed of:

- The if keyword followed by a set of parentheses `()` which is followed by a code block, or block statement, indicated by a set of curly braces `{}`.

- Inside the parentheses `()`, a condition is provided that evaluates to `TRUE` or `FALSE`.

- If the condition evaluates to true, the code inside the curly braces `{}` runs, or executes.

- If the condition evaluates to false, the code inside the block won't execute.

Knowing how to use if statements will help you introduce logic in your data analysis. There is also a way to add an *else statement*. An else statement must be paired with an if statement, and together they are referred to as an if...else statement.

```
if (TRUE) {  
  print("Go to sleep!")  
} else {  
  print("Wake up!")  
}
```

Copy to Clipboard

In the example above, the else statement:

- Uses the else keyword following the code block of an if statement.

- Has a code block that is wrapped by a set of curly braces `{}`.

- The code inside the else statement code block will execute when the if statement's condition evaluates to false. These if...else statements allow us to automate solutions to yes-or-no questions, also known as binary decisions.

Instructions

Checkpoint 1 Enabled

1.

Create a conditional statement in **notebook.Rmd** such that it will change the value of the variable `message` to `'I execute this when true!'` when the condition is `TRUE`, and the value of `message` to `'I execute this when false!'` when it is `FALSE`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Print the value of `message` after the if...else statement by using `print(message)`.

```
---  
title: "Introduction to R Syntax"  
output: html_notebook  
---  
{r}  
message <- 'I change based on a condition.'  
if (TRUE) {  
  message <- 'I execute this when true!'  
} else {  
  message <- 'I execute this when false!'
```

```
}  
  
print(message)  
```\n
```

# Introduction to R Syntax

## Comparison Operators

When writing conditional statements, sometimes we need to use different types of operators to compare values. These operators are called comparison operators.

Here is a list of some handy comparison operators and their syntax:

```
Less than: <
Greater than: >
Less than or equal to: <=
Greater than or equal to: >=
Is equal to: ==
Is NOT equal to: !=
```

Comparison operators compare the value on the left with the value on the right. For instance:

```
10 < 12 # Evaluates to TRUE
```

Copy to Clipboard

It can be helpful to think of comparison statements as questions. When the answer is “yes”, the statement evaluates to true, and when the answer is “no”, the statement evaluates to false. The code above would be asking: is 10 less than 12? Yes! So 10 < 12 evaluates to true.

### Instructions

Checkpoint 1 Enabled

1.

Use a comparison operator to check if 56 is greater than or equal to 129. What should this expression evaluate to? Check by running your code!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

In a new line, use the right comparison operator to check if 56 is NOT equal to 129. What should this expression evaluate to? Check by running your code!

```

title: "Introduction to R Syntax"
output: html_notebook

```\n{r}  
56 >= 129  
56 != 129  
```\n
```

# Introduction to R Syntax

# Logical Operators

Working with

[conditionals](#)

Preview: Docs Loading link description

means that we will be using logical, true or false values. In R, there are

[operators](#)

Preview: Docs Loading link description

that work with logical values known as logical operators. We can use logical operators to add more sophisticated logic to our conditionals. There are three logical operators:

the AND operator (&)

the OR operator (|)

the NOT operator, otherwise known as the bang operator (!)

When we use the & operator, we are checking that two things are true:

```
if (stopLight == 'green' & pedestrians == 0) {
 print('Go!');
} else {
 print('Stop');
}
```

Copy to Clipboard

When using the & operator, both conditions must evaluate to true for the entire condition to evaluate to true and execute. Otherwise, if either condition is false, the & condition will evaluate to false and the else block will execute.

If we only care about either condition being true, we can use the | operator:

```
if (day == 'Saturday' | day == 'Sunday') {
 print('Enjoy the weekend!')
} else {
 print('Do some work.')
}
```

Copy to Clipboard

When using the | operator, only one of the conditions must evaluate to true for the overall statement to evaluate to true. In the code example above, if either `day == 'Saturday'` or `day == 'Sunday'` evaluates to true the if's condition will evaluate to true and its code block will execute. If the first condition in an | statement evaluates to true, the second condition won't even be checked.

The ! NOT operator reverses, or negates, the value of a TRUE value:

```
excited <- TRUE
print(!excited) # Prints FALSE
```

Copy to Clipboard

Essentially, the ! operator will either take a true value and pass back false, or it will take a false value and pass back true.

Logical operators are often used in conditional statements to add another layer of logic to our code.

## Instructions

Checkpoint 1 Enabled

1.

There are two variables in your code, `weather` and `high_chance_of_rain`. Write a conditional statement that:



Checks to see if `weather` is equal `cloudy` and there is a `high_chance_of_rain`.  
If it is both, the code block should assign the value of the variable `message` to be "Pack umbrella!"  
Otherwise, the code block should assign the value of the variable `message` to "No need for umbrella!"  
Print the `message` variable after the conditional statement. Based on the condition, what should its value be?

```

title: "Introduction to R Syntax"
output: html_notebook

{r}

message <- 'Should I pack an umbrella?'
weather <- 'cloudy'
high_chance_of_rain <- TRUE

if (weather == 'cloudy' & high_chance_of_rain) {
 message <- "Pack umbrella!"
} else {
 message <- "No need for umbrella!"
}

print(message)

```

# Introduction to R Syntax

## Calling a Function

### Functions

Preview: Docs Functions are blocks of code that perform specific tasks and can be repeatedly called within other parts of the code.

are actions we can perform. R provides a number of functions, and you've actually been using a few of them even though you maybe didn't realize!

We *call*, or use, these functions by stating the name of the function and following it with an opening and closing parentheses: ie. `functionName()`. In addition, between the parenthesis, we usually pass in an *argument*, or a value that the function uses to conduct an action, i.e. `functionName(value)`.

Does that syntax look a little familiar? When we have used

`print()`

Preview: Docs Loading link description

we're calling the `print` function. When we made a vector, we actually used the combine `c()` function! Let's see

`print()` and some real functions in action!

```
sort(c(2,4,10,5,1)); # Outputs c(1,2,4,5,10)
length(c(2,4,10,5,1)); # Outputs 5
sum(5,15,10) #Outputs 30
```

Copy to Clipboard

Let's look at each of the lines above:

On the first line, the `sort()` function is called with a parameter of the vector `c(2,4,10,5,1)`. The result is a sorted vector `c(1,2,4,5,10)` with the values in ascending order.

On the second line, we called a function we've seen before: `length()` and it returned the value 5 because there were five items in the vector.

On the third line, we called a function `sum()` which added up all of the arguments we passed to it.

Understanding how to call a function and what arguments it needs is a fundamental part of leveraging R as a tool to conduct analysis. Let's practice calling some useful functions!

#### Instructions

Checkpoint 1 Enabled

1.

The `unique()` function takes a vector argument and returns a vector with only the unique elements in that vector (removing all duplicates).

Call this function and pass in the argument `data`.

Save that result inside a variable named `unique_vals`

Print `unique_vals` variable so you can see what is inside the vector with only unique values.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Get the `sqrt()` square root of the number 49 by calling the function with the specified argument. Save the result inside a variable named `solution`.

Print the `solution` variable so you can see confirm `sqrt()` computed the square root correctly.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

The `floor()` function rounds a decimal down to the next integer, and the `ceiling()` function will round up to the next integer. Call both functions on the number 3.14, and save each result inside two new variables you create: `round_down` and `round_up` respectively.

Print both variables so you can see what's in them!

```

title: "Introduction to R Syntax"
output: html_notebook

```{r}
data <- c(120,22,22,31,15,120)

unique_vals <- unique(data)
print(unique_vals)
solution <- sqrt(49)
print(solution)

round_down <- floor(3.14)
round_up <- ceiling(3.14)
print(round_up)
print(round_down)
```
```

# Introduction to R Syntax

## Importing Packages

R's popularity is also largely due to the many fantastic packages available in the language! A package is a bundle of code that makes coding certain tasks easier. There are all sorts of packages for all sorts of purposes, ranging from visualizing and cleaning data, to ordering pizza or posting a tweet. In fact, most R-grammers (R programmers) use packages when they code. This is why you might hear them differentiate packages from "Base R." Base R refers the R language by itself and all that it can do without importing any packages.

Base R is very powerful, but most of the time, you'll want to import a package to make your life easier. You only need to run this command the first time you install a package, after that there is no need to run it:

```
install.packages('package-name')
```

Copy to Clipboard

To import a package you simply:

```
library(package-name)
```

Copy to Clipboard

You can look up documentation for different packages available in R at the [CRAN](https://cran.r-project.org/) (Comprehensive R Archive Network).

In this lesson, we coded in Base R but let's practice importing one of the most popular R packages: dplyr. Dplyr is a package used to clean, process, and organize data which you will use as you learn about R.

### Instructions

Checkpoint 1 Enabled

1.

Currently, the code inside **notebook.Rmd** throws an error because it uses the `dplyr` and `readr` packages without importing them. Import the `dplyr` and `readr` packages by using the `library()` function and then press run so that you can observe the packages do their magic!

```

title: "Introduction to R Syntax"
output: html_notebook

```{r}
# load libraries

```
```{r}
# load data frame
artists <- read_csv('artists.csv')
```
```{r}
# inspect data frame
artists
head(artists)
summary(artists)
```
```{r}
artists %>%
  select(-country, -year_founded, -albums) %>%
  filter(spotify_monthly_listeners > 20000000, genre != 'Hip Hop') %>%
  arrange(desc(youtube_subscribers))
```
```

# Review

Congrats on finishing your first R lesson!

Here's a summary of some of the concepts you've learned:

R is a powerful statistical programming language with a large community of data enthusiasts.  
You can calculate arithmetic with R and it will follow the normal order of operations  
Data types allow us to classify different pieces of information.  
You can store that information inside of variables  
You can use conditional statements and [operators](#)  
Preview: Docs Loading link description  
to introduce logic to your code  
You can call a function in R by using the `()` and passing in the correct arguments  
R programmers have published lots of useful packages that specialize in different tasks, which are all available for you to use in your programs after you install them.

# Introduction to the RStudio IDE

- <https://www.codecademy.com/paths/analyze-data-with-r/tracks/what-is-r/modules/learn-r-introduction/videos/rstudio-ide-introduction>
- <https://www.youtube.com/watch?v=T44GWKIFFRI>

## How is R Special?

- <https://www.codecademy.com/paths/analyze-data-with-r/tracks/what-is-r/modules/learn-r-introduction/videos/how-is-r-special>
- [https://www.youtube.com/watch?v=EjV9XUQ3w5g&ab\\_channel=Codecademy](https://www.youtube.com/watch?v=EjV9XUQ3w5g&ab_channel=Codecademy)
- Easy exploratory data analysis
- Easy statistical models
- Vectorized calculations
- Data visualizations with **ggplot2**
- Interpreted language
- Open source

- Wide variety of packages
  - CRAN
  - Tidyverse and caret
  - Dplyr
- Different from other programming languages
- 

# Introduction to Programming in R

## Basic data structures, control flow, loops, and functions in R.

Now that we've introduced what R is at a surface-level, we can develop a stronger grasp of the basic data structures and essential programming concepts in R. This understanding will help us write more complex, versatile programs in R.

How to create and manipulate **basic data structures** will be covered in one lesson. Those data structures include: vectors, lists, and matrices. Then, **control flow**, **loops**, and **functions** will each be discussed in individual interactive articles. Along the way, code exercises and projects will solidify basic programming skills in R.

## Introduction

In this lesson, we'll be covering the basic data structures in R:

## [vectors](#)

Preview: Docs Loading link description

,

## [matrices](#)

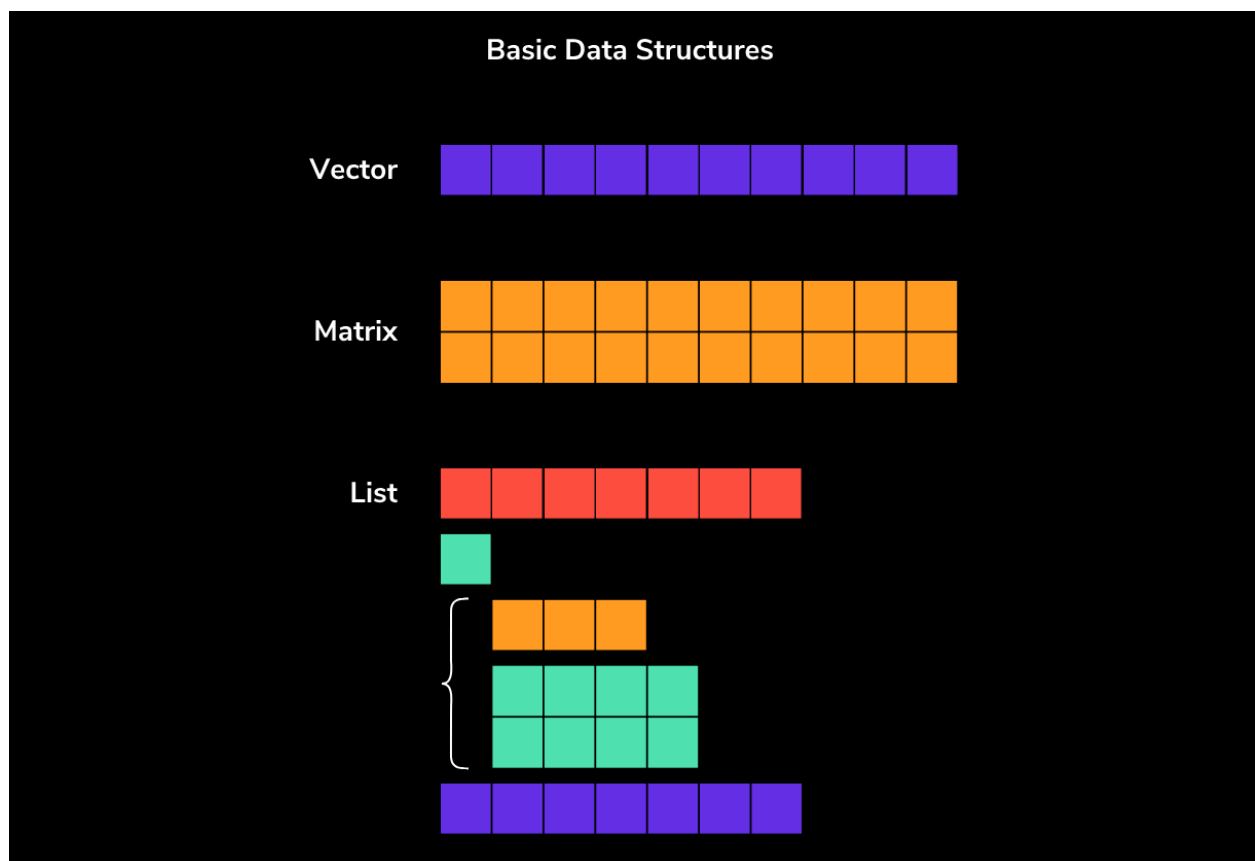
Preview: Docs Matrices in R are a fundamental data type used for storing two-dimensional tabular data with rows and columns.

, and **lists**. In all programming languages, data structures are how data gets stored and organized as the code executes. Vectors, matrices, and lists in R each serve their own function.

We'll go through each data structure separately, then review the upsides and downsides of each at the end. We'll also give you a preview of R's frequently used **data frame** – a data structure that borrows the best features from vectors, matrices, and lists.

By the end, we'll understand how to use each data structure in our own base R programming.

Without further ado, let's dive in!



# Vectors

## [Vectors](#)

Preview: Docs Vectors are used to store multiple values of the same data type.

are the most basic data structure in R. A vector contains elements of the same type, whether they're numericals, doubles, characters, logical (`TRUE` or `FALSE`), or complex numbers. Vectors are one-dimensional, meaning they have just one length.

We create vectors using the function `c()`, separating values inside by commas. Below, we're creating a vector variable called `low_temps` with seven numerical values and a vector called `cities` with 4 character values.

```
a vector of low temperatures (Fahrenheit) of week
low_temps <- c(35, 36, 29, 30, 31, 33, 30)

a vector of city names
cities <- c("Paris", "Lagos", "Shanghai", "Toronto")
```

Copy to Clipboard

There are two basic

### [built-in functions](#)

Preview: Docs Loading link description  
that can be used with vectors.

`length(vec)` returns the number of elements in a vector called `vec`  
`typeof(vec)` gets the data type of elements in `vec`

```
length(low_temps)
output: [1] 7
typeof(cities)
output: [1] "character"
```

Copy to Clipboard

What happens if we try to create a vector with different data types? R will force all items to be one data type. Let's take a look at this example where R converted everything to character types:

```
trying_to_be_different <- c(45, "apple", 0.2, TRUE)
print(trying_to_be_different)
output: [1] "45" "apple" "0.2" "TRUE"

typeof(trying_to_be_different)
#output: [1] "character"
```

Copy to Clipboard

Here are some shortcuts for making numerical vectors.

We can make a vector of consecutive integers (whole numbers) like so:

```
one_thru_ten <- c(1:10)
print(one_thru_ten)
#output: [1] 1 2 3 4 5 6 7 8 9 10
```

Copy to Clipboard

We can also define a range and an interval size using `seq()`. `seq(start_number, end_number, by = interval_size)` Here, we want a vector with 0.5 size intervals between 3 and 5.

```
intervals <- seq(3, 5, by = 0.5)
print(intervals)
output: [1] 3.0 3.5 4.0 4.5 5.0
```

Copy to Clipboard

## Instructions

Checkpoint 1 Enabled

1.

Create a vector named `years` that contains all year numbers from 2017 to the current year by listing the individual years.

On the next line, use `print()` to output `years`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create another vector named `years2` that contains all year numbers from 2017 to the current year by using one of the shortcuts above for consecutive numbers.

On the next line, use `print()` to output `years2`.

```

title: "Data Structures in R"
output: html_notebook

```{r}
# create vector called years:
years <- c(2017:2025)
print(years)
```

```{r}
# create vector called years2:
years2 <- seq(2017,2025)
print(years2)
```
```

## Vectors II

Vectors are sequential, meaning they keep data in order. The indexing, or the way you refer to the position of the data, starts from 1. We can look up the `i`th element of a vector `v` by calling `v[i]`. Below, we're defining an example vector and accessing elements from it. We'll use this same example vector throughout.

```
example_vector <- c("I", "have", "several", "items", "inside", "me")
example_vector[1] # gets the first element "I"
example_vector[3] # gets the third element "several"
```

Copy to Clipboard

We can also select multiple elements by specifying indices in a vector:

```
select third and fifth element
```



```
example_vector[c(3, 5)]
output: [1] "several" "inside"
```

Copy to Clipboard

Let's break this down. Inside of the `[ ]`, the vector declared by `c(3, 5)` means the individual indexes `3` and `5`. Note that they do not have to be in order.

How would we get the first four elements? We would need a vector that contains the indices `1` through `4`. We know how to make a vector of consecutive integers `1` through `4` using `c(1:4)`. So we'll put that index vector in the bracket like so:

```
selects the first four elements of the vector
example_vector[c(1:4)]
output: [1] "I" "have" "several" "items"
```

Copy to Clipboard

Suppose we try to access an index that doesn't exist. If we check `length(example_vector)`, we'll get `6`. If we try to access an index greater than that length, the result will be `NA`, which is R's way of saying a non-existent value.

```
attempt to select eighth element
example_vector[8]
output: [1] NA
```

Copy to Clipboard

## Instructions

Checkpoint 1 Enabled

1.

The vector `top_movies_2019` contains the top ten grossing movies from 2019, in order of their rank so that the top movie is at the beginning of the vector. Create a variable `top_six_movies` that stores the 6th highest-grossing movie from the vector. On the next line, print `top_six_movies`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable `top_three_movies` that stores the top 3 movies from `top_movies_2019`, accessing the elements with an index vector.

On the next line, print `top_three_movies`.

```

title: "Data Structures in R"
output: html_notebook

```{r}  
top_movies_2019 = c(  
  "Avengers: Endgame",  
  "The Lion King",  
  "Toy Story 4",  
  "Frozen 2",  
  "Captain Marvel",  
  "Star Wars: Episode IX - The Rise of Skywalker",  
  "Spider-Man: Far from Home",  
  "Aladdin",  
  "Joker",  
  "It Chapter Two"  
)  
```:~  
```{r}  
# create variable top_six_movies:  
top_six_movies <- top_movies_2019[6]  
  
print(top_six_movies)
```

```
# create vector called top_three_movies:
top_three_movies <- top_movies_2019[c(1:3)]
print(top_three_movies)
``
```

Vectors III

We can modify an item in a vector or the vector as a whole in various ways. To modify an item, we can call its index and setting it to a new value. Below, we're setting the second element of the vector to 7.

```
change_me <- c(7, 11, -28, 32, 5, 19)
# sets second element to 7
change_me[2] <- 7
print(change_me)
# output [1] 7 7 -28 32 5 19
```

Copy to Clipboard

We can also use comparison

[operators](#)

Preview: Docs Loading link description

– which will generate a `TRUE` or `FALSE` – to select elements that fit a certain condition. Here, we're also using the `&` (AND) logical operator. In the code below, once we've selected those values, we set them to 0.

```
change_me <- c(7, 11, -28, 32, 5)
# set elements in vector greater than 0 AND less than 10 to 0
change_me[change_me > 0 & change_me < 10] <- 0
print(change_me)
# output: [1] 0 11 -28 32 0
```

Copy to Clipboard

Note that we use the vector name inside of the brackets too. We're using comparison operators combined with logical operators to check whether each element the vector is between 0 and 10.

We can also cut a vector short like so:

```
change_me <- c(7, 11, -28, 32, 5)
# Cut the vector to the portion containing second to the fourth element
change_me <- change_me[2:4]
print(change_me)
# output: [1] 11 -28 32
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Correct the typo in the existing vector `vegetables` so that the third element says "spinach" instead. Then, print `vegetables` on the next line.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

See the vector `scores`. You're a generous teacher and want to round some really low scores up. Modify the vector so that every score below 60 is set to 60.

Print `scores` after.

```
# Legal variable names:
myvar <- "John"
my_var <- "John"
```

```

myVar <- "John"
MYVAR <- "John"
myvar2 <- "John"
.myvar <- "John"

# Illegal variable names:
2myvar <- "John"
my-var <- "John"
my var <- "John"
_my_var <- "John"
my_v@ar <- "John"
TRUE <- "John"

```

```

---
title: "Data Structures in R"
output: html_notebook
---

```{r}
vegetables <- c("celery", "squash", "spinach", "kale", "radish", "carrot", "onion")
```

```{r}
correct to spinach here:
vegetables[3] <- "spinach"
print(vegetables)
```

```{r}
scores <- c(44, 65, 77, 49, 56, 75, 59, 90, 72)
```

```{r}
#modify scores below:
scores[scores < 60] <- 60
print(scores)
```

```

Matrix I

A matrix (plural: matrices) is a 2-D data structure that holds elements of the same type. Matrices are just like vectors, but in 2-D. They're table-like, and we refer to the two dimensions as rows and columns. In math, engineering, and economics, matrices have tons of applications, which R has built-in functions for. They're also just a common way of storing data.

Below is what we call a 4 by 5 matrix, since it has 4 rows and 5 columns (we generally refer to a matrix's dimensions by row first).

5 columns

4 rows

| | | | | |
|----|----|----|----|----|
| 3 | 8 | 9 | 13 | 23 |
| 8 | 23 | 11 | 5 | 39 |
| 9 | 4 | 12 | 4 | 8 |
| 10 | 8 | 9 | 12 | 10 |

To create a matrix, use the `matrix()` function to specify what the matrix should contain, and how many rows and columns it should have. The `matrix()` function takes three arguments:

The first argument should be data you'd like to keep in the matrix. If we specify just one value, the entire matrix will be filled with it. If we specify a vector, R will determine whether it should repeat throughout the matrix or if it fills the whole matrix, depending on the vector's length. The vector's data is inserted column-wise by default.

`nrow` = number of rows
`ncol` = number of columns

See the [R matrix documentation](#) for more optional arguments you can use to specify a desired matrix.

The code below will create a 2 (rows) by 3 (columns) matrix of all 0s.

```
two_by_three_matrix <- matrix(0, nrow = 2, ncol = 3)
print(two_by_three_matrix)
# output:
      [,1] [,2] [,3]
[1,]    0    0    0
[2,]    0    0    0
```

Copy to Clipboard

This will create a 3 by 4 matrix with each column being "a" "b" "c"

```
abc_repeat <- matrix(c("a", "b", "c"), nrow = 3, ncol = 4)
print(abc_repeat)
# output:
      [,1] [,2] [,3] [,4]
[1,] "a"  "a"  "a"  "a"
[2,] "b"  "b"  "b"  "b"
[3,] "c"  "c"  "c"  "c"
```

Copy to Clipboard

What about vectors that won't fit evenly down each column? The matrix below is created from a vector of numbers 1-8, which will repeat in a 3 by 4 matrix and cut off whenever there is no more space.

```
eight_matrix <- matrix(c(1:8), nrow = 3, ncol = 4)
print(eight_matrix)
# output:
      [,1] [,2] [,3] [,4]
[1,]    1    4    7    2
[2,]    2    5    8    3
[3,]    3    6    1    4
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Create a matrix called `one_hundred_greetings` that has 10 rows and 10 columns full of "hi"s

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a number matrix called `different_neighbors` in R based on what's shown below:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

You can print `different_neighbors` to make sure it looks right.

```
---
title: "Data Structures in R"
output: html_notebook
---

```{r}
create one_hundred_greetings matrix below:
one_hundred_greetings <- matrix("hi", nrow = 10, ncol = 10)
print(one_hundred_greetings)
```

```{r}
create different_neighbors matrix below:
different_neighbors <- matrix(c(1, 0), nrow = 3, ncol = 5)
print(different_neighbors)
```
```

Matrix II

If we think of a matrix as a table, we can look up individual elements by referring to their row first, then the column in that row. We'll find the element at the `x`th row, `y`th column using `my_matrix[x,y]`.

Let's use this code

```
my_matrix <- matrix(c(1:25), nrow = 5, ncol = 5)
```

Copy to Clipboard

to make this matrix below, which we'll refer to in the rest of the examples:

5 columns

| | | | | | |
|--------|---|----|----|----|----|
| 5 rows | 1 | 6 | 11 | 16 | 21 |
| | 2 | 7 | 12 | 17 | 22 |
| | 3 | 8 | 13 | 18 | 23 |
| | 4 | 9 | 14 | 19 | 24 |
| | 5 | 10 | 15 | 20 | 25 |

```
# select the element in the second row, third column (position in the row)
my_matrix[2, 3]
# result: 12
```

Copy to Clipboard

You can also grab an entire rows or columns by only specifying the row or only specifying the column, leaving the other side of the comma blank. A single row or column will be returned as a vector.

```
# select the entire second row as a vector
my_matrix[2, ]
# result: [1] 2 7 12 17 22

#select the entire third column as a vector
my_matrix[, 3]
# result: [1] 11 12 13 14 15
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

See the matrix defined as `twenty_eight_matrix`. Get the element in the 2nd row, 4th column and store it in a variable called `middle_element`.

Then print `middle_element`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Get the 6th column of the matrix and store it in a variable called `sixth_column`.

Then print `sixth_column`.

```
---
title: "Data Structures in R"
output: html_notebook
---

```{r}
twenty_eight_matrix <- matrix(c(1:28), nrow = 4, ncol = 7)
print(twenty_eight_matrix)
```

```{r}
get the element in second row, 4th column
middle_element <- twenty_eight_matrix[2,4]
```

```
print(middle_element)

sixth_column <- twenty_eight_matrix[,6]
print(sixth_column)
```

```

Lists I

Lists are data structures that can contain multiple

[data types](#)

Preview: Docs Loading link description

, unlike

[vectors](#)

Preview: Docs Loading link description

. They can also store lists, meaning there can be lists of lists... of lists.

Generally, a list is created using the syntax `my_list <- list(a, b, c)` where `a`, `b`, `c` could be different data types.

```
# list storing a string, two numbers, and a list
my_list <- list("Elephant", 900, 80.3, list("pencil", "pens"))
```

Copy to Clipboard

To access items in a list in R by their position, we can call the index we want in double brackets like so:

```
# access item by position in list
third_item <- my_list[[3]]
print(third_item)
# output: [1] 80.3
```

Copy to Clipboard

Beware: if we use single bracket notation, like `my_list[1]`, a list object will be returned instead of the value itself.

Modifying elements is also straightforward.

```
# Modify the third element
my_list[[3]] <- 85.2
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Create a list called `fruit_basket` that contains, in this order:

Your favorite fruit in a string

The number of kilograms of the fruit you want to buy e.g. 3.5

A vector of at least two friends' names to gift the fruits to

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Hmm. Seems like you have just enough cash to get 10 kg. Modify the second element of the list, where you've stored how many kg's you'll buy, to `10`.

Then print `fruit_basket`.

```
---
title: "Data Structures in R"
output: html_notebook
---

```{r}
create list fruit basket below:
fruit_basket <- list("peach", 3, c("David", "Penny"))
print(fruit_basket)

modify the second element of list to 10
fruit_basket[[2]] <- 10
print(fruit_basket)
```

# Lists II

Lists are useful as an organizational tool in R because you can label each piece of data inside the list with a name. Instead of having to call the position of the item, you can call it by a name you gave it. The syntax to create a list with named items inside is `my_list <- list(name1 = a, name2 = b, name3 = c)`. For example:

```
a list that has three vectors named Penny, Xavier, and Kiara
test_scores <- list(Penny = c(84, 79, 85), Xavier = c(89, 90, 82), Kiara = c(92, 89, 87))
```

Copy to Clipboard

Adding names to the elements stored inside makes it easy to access them by name. To access whatever element has name1, you would ask for `mylist$name1`. In the above example, if you wanted to get Penny's test scores vector:

```
get Penny's test scores (vector)
penny_scores <- test_scores$Penny
print(penny_scores)
output: [1] 84 79 85
```

Copy to Clipboard

You can name sublists as well. Recreating the list above with named list inside

```
a list that has three vectors named Penny, Xavier, and Kiara with the name of each exam inside the vectors
test_scores <- list(
 Penny = list(exam1 = 84, exam2 = 79, exam3 = 85),
 Nick = list(exam1 = 77, exam2 = 81, exam3 = 83),
 Kiara = list(exam1 = 92, exam2 = 89, exam3 = 87)
)
```

Copy to Clipboard

You can ask for Kiara's test score from exam 1 by adding another \$ with the sublist name, resulting in `$Kiara$exam1`.

```
print(test_scores$Kiara)
$exam1
[1] 92
$exam2
[1] 89
$exam3
[1] 87
> test_scores$Kiara$exam1
[1] 92
```

Copy to Clipboard

## Instructions

Checkpoint 1 Enabled

1.

See the `test_scores` list created.

Create a variable `nick_scores` that stores Nick's scores for all the exams. Print `nick_scores`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create another variable `nick_exam2` that stores Nick's score for exam 2. Print `nick_exam2`.



```

title: "Data Structures in R"
output: html_notebook

```{r}
test_scores <- list(
  Penny = list(exam1 = 84, exam2 = 79, exam3 = 85),
  Nick = list(exam1 = 77, exam2 = 81, exam3 = 83),
  Kiara = list(exam1 = 92, exam2 = 89, exam3 = 87)
)
```

```{r}
# create variable nick_scores
nick_scores <- test_scores$Nick
print(nick_scores)

# create variable nick_exam2
nick_exam2 <- test_scores$Nick$exam2
print(nick_exam2)
```

```

## Special Lists: Data Frames

At this point, we've introduced the basics of

[vectors](#)

Preview: Docs Loading link description

,

[matrices](#)

Preview: Docs Loading link description

, and lists, which allow us to do quite a lot when we're programming in base R.

Big data sets, especially ones we want to import into our R workspace, take on a common standard: a table with named columns. We can hold this data in a special form of a list called a **data frame**.

A **data frame** is a table-like list with rows and columns, where each column has a name and each row contains a set of values, one from each column. Data frames are a combination of lists (where you can name things) and matrices (tabular). They're meant to hold spreadsheet-like data.

Let's think about the list we made last exercise, with Penny, Xavier, and Kiara's test scores. We could organize them into a table with named columns like so:

| Student  | Exam 1 | Exam 2 | Exam 3 | Extra Credit? |
|----------|--------|--------|--------|---------------|
| "Penny"  | 84     | 79     | 85     | TRUE          |
| "Xavier" | 77     | 81     | 83     | TRUE          |

|         |    |    |    |       |
|---------|----|----|----|-------|
| "Kiara" | 92 | 89 | 87 | FALSE |
|---------|----|----|----|-------|

and create a data frame from the table.

```
student_scores <- data.frame(
 student = c("Penny", "Xavier", "Kiara"),
 exam1 = c(84, 77, 92),
 exam2 = c(79, 81, 89),
 exam3 = c(85, 83, 87),
 extra_credit = c(TRUE, TRUE, FALSE)
)
```

Copy to Clipboard

Note: each column is a vector.

Most of the time though,

### [data frames](#)

Preview: Docs Loading link description

are not made from scratch. They're created through importing external CSV (spreadsheet) files or datasets from R packages. Importing data is done through a variety of [R read functions](#).

We can select the column names using the `$` sign, just like in a regular list! Below, we're selecting the `extra_credit` column from the data frame we just made:

```
print(student_scores$extra_credit)
output: [1] TRUE TRUE FALSE
```

Copy to Clipboard

Learning how to use data frames is an important step in becoming proficient in R.

### Instructions

Checkpoint 1 Enabled

1.

Notice the dataframe named `plant_data` that we've created through reading in a CSV file. Select the column called `height` by using the `$` sign, and store it in a variable called `plant_heights`. Then, print `plant_heights`.

```

title: "Data Structures in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load data frame
plant_data <- read.csv('plant_traits.csv')
```

```{r}
# select the height column here
plant_heights <- plant_data$height
print(plant_heights)
```
```

# Review

In this lesson, we learned a lot about data structures in R!

We started with `vector`, which is a one-dimensional sequence that can hold only a single data type. We index elements by their position inside a square bracket `[ ]`.

We then learned about

[matrices](#), which have two dimensions—rows and columns—of only a single data type. Matrix elements have two indices to hold row and column number, and we can select entire rows and columns. Vectors and matrices hold homogenous

[data types](#)

Preview: Docs R supports many data types, each with different uses and characteristics.

`vector`, but this strictness also means we can depend on them to be standardized. That means we can call one operation (like `mean` or `sort`) on all the elements and know what data type that operation will use.

However, not all data is homogenous. That's where lists provide more flexibility. **Lists** can fit different data types in the same list — they can contain vectors, matrices, and even other lists. The shape and indexing of a list is determined by the programmer, since we can name elements inside and refer to them by those names using the `$` sign.

Most R datasets are stored in a special table-like form of lists, called **data frames**. Data frames have named columns where each column is a vector of the same length, but each column can hold a different data type. Data kept in a spreadsheet is often imported into R as a data frame. The `$` sign is used to select a column by its name, and we can perform operations on entire columns.

# Control Flow in R

In this article, we will be covering what control flow is conceptually, and how we can establish control flow in R. We will introduce if/else blocks through various examples and challenges.

How do we program decision-making into our code? In this article, we will be covering what control flow is conceptually, and how we can establish control flow in R. Note that this is written with a beginner programming level in mind.

First, take a look at the diagram above. This is a series of decisions we might be making each morning, where a YES will send us one way, and a NO will send us the other. Different choices open up, while other outcomes become impossible.



We want our programs, in R or otherwise, to have a similar way of making decisions based on whether certain conditions are true at different steps. A program will run and start moving through its checklists and ask "is this condition met?" then execute a code block if the answer is yes, or skip it if the answer is no. This is the *Control Flow* of a program.

R is a script language, meaning every code file executes from the top down, until there is nothing left to run. It is the programmer's job to include gateways throughout, known as conditional statements, to tell the computer whether or not it should execute certain blocks of code.

In this article, we will cover how to build conditional statements that check logical (**TRUE** or **FALSE**) values, just like the YES's or NO's from above, and manage the control flow in our code.

## If statements

The most basic kind of conditional statement is an **if** statement. If a certain condition is true, execute this code. The syntax looks like so:

```
if (condition_to_check) {
 # code to execute
}
```

A short example:

```
a <- 10
if (a < 12) {
 print("The variable a is less than 12")
}
```

`if` is an R keyword, followed by a condition inside the parentheses. The code for the condition should evaluate to a `TRUE` or `FALSE`. As a refresher, in order for a code expression to evaluate to a `TRUE` or `FALSE` value, we can use comparison operators like `<`, `>`, `>=`, `<=`, `==`, and `!=`. We can combine `TRUE` and `FALSE` values with logical operators like `&` (AND), `|` (OR), and `!` (NOT).

Coding question

## Questions

Check out the code below. The conditional allows us to switch whether we want to cut a data frame short or not. In the `if` statement, we check the value of the variable `use_short_data`. If it is `TRUE`, we will only use the first 50 rows.

In the last line of code, we print how many rows are in the dataframe currently. Press run and see that the code inside the `if` statement is executed by noting the output number.

Then set `use_full_data` to `FALSE`. What happens? What is the output number now? When you're done, click "Check Answer".

Run your code to check your answer

**If, Else**

If we want a program to execute some code if a condition IS true, or some other code in case the condition is NOT true, you could add an `else` block to the `if`. Below is the syntax. This is the conventional way of connecting them, with the `else` block directly following the closing bracket of the `if` block. Only one of the code options will execute.

```
if (condition_to_check) {
 # execute some code
} else {
 # execute some other code
}
```

Multiple choice

## Questions

Take a moment to read through this code that includes `if` and `else`.

`amount_per_student` will equal the `funding` number divided by `number_of_students`. If that value is at least 500, then we print one statement. If not, then we print another.

Do the division on your own to find the value of `amount_per_student`. Which statement will be printed in the end?

## Code

```
number_of_students <- 88
funding <- 30000
amount_per_student <- funding / number_of_students

if (amount_per_student >= 500) {
 print("We have enough funding for each student!")
} else {
 print("We're lacking funds this year.")
}
```

## Answer Choices

"We have enough funding for each student!"

"We're lacking funds this year."

**Else if statements**

Let's say that we want to give our code more options outside of just yes or no. Following our last example, if the `amount_per_student` was at least 400 but less than 500, maybe we want to print a message saying "We cannot afford stationery and planners for students." If it was between 300 and 400, maybe we want another message.

That's where `else ifs` come in. They allow us to define options that are separate from one another that we check in a strict order. Whichever condition is true first, the accompanying code will run, and other conditions are not considered. If none of the statements in the `if` and `else if` are true, then we will default to the code in the `else` block, if the `else` exists. Note that `else if` always comes after `if`.

Multiple choice

## Questions

Read through these conditionals, which include `else ifs`. Note that the variables are the same values as in the last example, and remember that `&` means AND.

Which statement will be printed in the end?

## Code

```
number_of_students <- 88
funding <- 30000
amount_per_student <- funding / number_of_students

if (amount_per_student >= 500) {
 print("We have enough funding for each student!")
} else if (amount_per_student < 500 & amount_per_student >= 400) {
 print("We cannot afford stationery and planners for students.")
} else if (amount_per_student < 400 & amount_per_student >= 300) {
 print("We cannot afford stationery, planners, and have to cut down on printing.")
} else {
 print("We're lacking funds this year.")
}
```

## Answer Choices

"We cannot afford stationery and planners for students."

"We cannot afford stationery, planners, and have to cut down on printing."

"We're lacking funds this year."

"We have enough funding for each student!"

### Nesting conditionals

So far, the code that we've seen hits just one outcome and ends there. If we want to add a condition that should only be checked *inside* of another condition, we can nest `if/else` blocks. In the code below, we nest `if` and `else` statements in the outermost `if` statement. Make sure when you nest conditionals that the flow of the code is still clear to you, the programmer!

```
if (condition_to_check) {
 if (inner_condition_to_check) {
 # code that will execute if both condition_to_check and inner_condition_to_check
 are true
 } else {
 # code that will execute if condition_to_check is TRUE but
 inner_condition_to_check is FALSE
 }
} else {
 # code that will execute if condition_to_check is FALSE
}
```

Fill in the blank

## Questions

Building on the same example, let's say we again first want to check if `amount_per_student` is at least 500. If it is, we want it to print "We have enough funding for each student!" and calculate a `leftover` value. If `leftover` is at least 2000, we want the code to print the statement "We can update the AV equipment this year!". If not, we'll print "Let's save for AV equipment next year".

Pick the right keywords for each conditional!

## Code

```
number_of_students <- 88
funding <- 30000
amount_per_student <- funding / number_of_students

blank 1
(amount_per_student >= 500) {
 print("We have enough funding for each student!")
 leftover = funding - (number_of_students * 500)

blank 2
(leftover >= 2000) {
 print("We can also update the AV equipment this year.")
}

blank 3
{
 print("Let's save for AV equipment next year.")
}
} else {
 print("We're lacking funds this year.")
}
```

## Answer Choices

else  
check  
if  
for  
check  
if  
else if

Click or drag and drop to fill in the blank

Now we've established basic knowledge on how to incorporate decision-making in our code. This makes programs much more versatile and able to account for all the possibilities of inputs.

Coding question

## Questions

Use a series of `if`, `else`, and `else if` statements to code the following scenario:

You just heard about a top-secret event where you have to confirm a series of checks in order to enter. In the workspace, you're given the `access_event` and `access_music_show` variables that start at `FALSE`, and the `age`, `possible_codeword`, and `real_codeword` variables as well.

First, there should be a check on whether `age` is 21+ or 18+.

If `age` is 21 or above, set `access_event` to `TRUE` and print "I got in for the night!". If 21+, you can also check whether you have the right codeword to access the secret music show at 11. If your `possible_codeword` matches the `real_codeword` using the `==` comparison operator, set `access_music_show` to `TRUE` and print "I need to get a good spot!". Hint: this codeword check should be a nested conditional.

But if `age` is 18+ but not 21+, you have to check whether the `current_hour` is earlier than 11. If it is, you can set `access_event` to `TRUE` and you should print "I can stay until 11!" Hint: you can either nest the first check into the second, or use an `&` (AND) operator to combine them.  
If `age` is not 18+ or 21+, both `access_event` and `access_music_show` should stay `FALSE` and you should print "I guess I'll go somewhere else."

```

title: "Conditionals challenge"
output: html_notebook

```{r}
# defining the variables
access_event <- FALSE
access_music_show <- FALSE
age <- 22
current_hour <- 9
possible_codeword <- "redrobbin"
real_codeword <- "bluejay"
```

```{r}
# conditional code goes here:
```
```



# Loops in R

This article will introduce loops in R and why they are useful as a basic programming structure. We will learn to use loops to iterate through data structures and to execute code a set number of times or based on a condition.

A **loop** is a control structure in programming where the instructions are written once, but a computer can execute them multiple times. Each execution of those instructions is called an **iteration**. Loops are commonly used in programming because, compared to repeated lines of code, they save time, reduce error, and are easy to read.

A loop allows a set of steps to repeat, either with a predefined number of repetitions or with a condition that keeps the loop going.

There are two types of loops in R: **for loops** and **while loops**. **For loops** repeat a piece of code for a known number of iterations, and a **while loop** repeats while a condition holds true.

## For loops

Using **for loops**, you can tell a piece of code to run a defined X number of times or to iterate through the length of a data structure. The for loop requires a variable or counter that keeps track of where the current iteration is.

When we know an exact number of times we want the code to repeat, we can set up our for loop to count from 1 to X, and the code will stop executing after the Xth iteration. Below, we show the basic outline of a for loop that has a counter. We need the `for` keyword, a `loop_variable` to serve as the counter, the `in` keyword, and a range of 1 to `number_of_iterations`. We refer to the code inside of the curly `{ }` brackets as the body of the loop.

```
for (loop_variable in 1:number_of_iterations) {
 # execute this code
}
```

`loop_variable` can be used inside the loop if the number of the current iteration affects the code inside, but the variable only has meaning inside the loop. If you try to use that variable outside the loop, you'll get an error.

Coding question

## Questions

Let's say we want to print "Hello" 10 times without having 10 lines of `print("Hello")`.

Run the code below. Here, the variable `i` is the placeholder for whatever the current count is between 1 and 10. On the first iteration the variable `i` is equal to 1. Then it's equal to 2. At the last iteration, `i` is equal to 10. The print statement is the same at each iteration.

In the code editor, change the number of iterations from 10 to 20. Click Run to see the result this time.

## Code

Another use case of for loops is iterating through a data structure's elements. The syntax for this also requires a loop variable, the keyword `in`, and the data structure's name.

```
for (loop_variable in data_structure) {
 # execute this code
}
```

The code below will print out the words in the vector `my_vector`, in order, one by one. The loop variable `word` inside the `for` statement here is the placeholder for whichever is the current element in `my_vector`. The print statement will

therefore print that element. On the first iteration, `word` equals `"this"`. On the second, `word` equals `"vector"`. And so on. In this instance, the loop variable is used in the body of the loop.

```
my_vector <- c("this", "vector", "is", "being", "read", "by", "a", "for", "loop")
for(word in my_vector){
 print(word)
}
```

Multiple choice

## Questions

How many iterations are in this for loop?

### Code

```
my_vector <- c("this", "vector", "is", "being", "read", "by", "a", "for", "loop")
for (word in my_vector){
 print(word)
}
```

## Answer Choices

9  
18  
3

We can also nest for loops. An important use case is if we want to iterate through two separate indices, like rows and columns of a matrix. Note that in this example, we are using the counters `row` and `col` as loop variables.

```
for (row in 1:nrow(a_matrix)) {
 for (col in 1:ncol(a_matrix)) {
 print(a_matrix[row, col])
 }
}
```

It's important to pay attention to the order of our nesting. In each iteration of the outermost for loop, there are multiple iterations of the inner loop. In the code example above, since the row number is iterated in the outer loop, the code prints out all the matrix elements in all the columns of the first row, then the elements of the second row, and so on.

Coding question

## Questions

Edit the code block to print the matrix elements column by column. First, run the code as is to see that it prints the elements of this matrix row-by-row. Notice how the `row` and `col` variables are increasing. Hint: you should make the outer loop iterate through the column indices, and the inner loop iterate through the row indices.



### Code

## While Loops

**While loops** repeat a code block *while* a certain condition is true. While loops take a little more reasoning because we want to make sure the loop ends when we intend for it to end. We also need to make sure the condition starts as true so the loop runs in the first place. First, the syntax looks like this:

```
while (condition to check) {
```

```
} # execute this code
```

Usually, the code inside the while loop will change the value of a variable in the condition. This allows the condition to eventually become `FALSE`.

Let's say we want a piece of code to simulate flipping a coin until it lands on heads twice. Take a pause to think about how you would code just the loop part (not the coin-flipping).

What would be our `while` condition? We want it to be a condition that will become `FALSE` when we want the loop to end. The condition could be that the flipping code repeats while the number of heads is still less than two. That means we want a variable, which we can call `num_heads` and define as `0` to begin with, to keep track of how many times we get heads. Consider what would happen if our while condition was `1 < 2`, a statement that is always true. Our while loop would never end! But if our condition was something like `num_heads < 2`, and in the loop we incremented `num_heads` by `1` if the coin landed on heads, then the `while` loop would definitely end if `num_heads` ever became `2` or more.

Coding question

## Questions

The coin-flipping example is translated into code below. Read through the code thoroughly. In the `while` loop, we are printing the result at every flip so we can visualize each flip and see how many it takes until we reach our heads goal. Run the code several times to simulate different trials of coin-flipping.

Here, the `heads_goal` is set at `2`. Set the variable to `5`, and run several times again.

## Code

We can also nest while loops, and the relationship between the inner while loop iterations and outer loop iterations is similar to that in a nested for loop.

Again, we can use loops for any time we want a code block to repeat, and the style of the loop depends on whether we want a specific number of iterations, we are iterating through a data structure, or if we want a loop to keep running while a condition is true. Loops add another layer of functionality that we'll see time and time again.

## Try these challenges below!

Coding question

## Questions

Write a **for loop** to multiply all the numbers of a vector `numbers` together and keep the product in a variable called `product`, which we've set to `1` at the beginning. The `product`, which will be printed, should equal `81` at the end.

## Code

Coding question

## Questions

Write a **while loop** that multiplies all the elements in the `numbers` vector, one by one in order, but stops multiplying once the next number to multiply is `5`. To access the vector at a particular index `i`, use `number[i]`. We defined the index `i` as `1` to start, but you'll want to increment that in your loop.

Beware: if your while condition does not eventually become `FALSE`, you'll see the neverending spinning circle after pressing Run — that's an infinite loop! Make sure the `while` condition, and any variables related to it, checks out.

# Functions in R

Learn how to use parameters and return statements, and define functions in R programming.

A function is a collection of several lines of code. By calling the function's name, we execute the code inside of it. We can call the function over and over again, with different arguments (inputs). The point of a function is to make code reusable, modular, and easier to read.

R comes with many built-in functions that make data analysis easier. For example, the function `mean()` takes in a numerical R object and gives us back the average of the values in that numerical object. If we had a vector `measurements` with numbers and wanted to find the average of all those measurements, we could call the function like `mean(measurements)`.

```
measurements <- c(30, 67, 40, 44, 25, 31)
mean(measurements)
output: [1] 39.5
```

Some basic built-in R functions include `seq()`, `c()`, and `matrix()`. There are also functions that come with different libraries like `dplyr`. Each function has its own name and expected arguments. Each of those functions has a definition, which we can find in [R documentation](#).

## Defining a Function

We can define our own functions like so:

```
function_name <- function(parameter_1, parameter_2,) {
 # do something with parameter_1 and parameter_2
 return(some_value)
}
```

A **parameter** is the name of an expected input in the function's definition. Parameter names are used in the function definition as a placeholder for the input value. The **return** statement defines where the code inside the function ends and gives an output. See the example below where we define a function that converts feet to meters.

First, let's choose a descriptive name for the function like... `feet_to_meters`. We expect the function to take in one number representing how many feet. So let's call that parameter `feet`. Inside the function, we should take the value in the `feet` parameter, and multiply it by `0.3048` to get `meters`. The function will return `meters`.

```
feet_to_meters <- function(feet) {
 meters <- feet * 0.3048
 return(meters)
}
```

Now in the same file that we wrote this function in, we could call it multiple times. We call the values that are passed into the function **arguments**. Below, `30` and `40` are arguments.

```
feet_to_meters(30)
output: [1] 9.144
feet_to_meters(40)
output: [1] 12.192
```

Fill in the blank

## Questions

Now let's say we want a function called `meters_to_feet` that takes a number of meters and converts it to the equivalent number of feet. Then we want to call the function to convert `80` meters to feet.

## Code

```
blank_1
<-
blank_2
```

```

(meters) {
 feet <-
blank 3
 * 3.28084

blank 4
 (feet)
}

meters_to_feet(
blank 5
)

```

## Answer Choices

```

{
 print
 function
}
feet_to_meters
meters
meters_to_feet
define
output
return
80

```

Click or drag and drop to fill in the blank

## apply()

In R, functions can be particularly useful when they're used on a set of values stored in a data structure (like a vector). Let's say we had a vector of decimal numbers between 0 and 1 that we want to convert to percentages. If the value was 0.2, we want the result to be 20.

First, we define a function called `turn_to_percent` to convert one value.

```

turn_to_percent <- function(decimal) {
 decimal * 100
}

```

Now let's say that we have a vector of numbers. We want to convert all of those numbers to percents. One way we could do that is by iterating through the vector with a loop. We would call the function on each individual value and tack the new value onto the resulting vector.

```

decimals <- c(0.45, 0.13, 0.25, 0.78, 0.66, 0.92, 0.84, 0.56)
percentages <- c()
for (d in decimals) {
 percentages <- c(percentages, turn_to_percent(d))
}
percentages
output: 45 13 25 78 66 92 84 56

```

But that takes a lot of memory. Wouldn't it be better if you could apply the function on all the values in the vector with just one line of code? It turns out you can by using R's built-in `apply()` functions. `apply()` functions take a data structure and a function's name as parameters. They then apply the function to the objects in the data structure, and returns the resulting objects in the same kind of data structure. When we say "one of the `apply()` functions", we mean that in addition to `apply()`, there are also variations like `sapply()` and `lapply()`, which we'll introduce below too.

Let's break down how to use them, starting with `apply()`.

`apply()` takes a data frame or matrix, a margin value, and a function name. The margin value determines whether the function is applied to the rows (1) or columns (2) of the matrix. The following code returns a vector containing the sums of either each column or each row, using `apply()` and the built-in R function `sum()`.

```

matrix with 2 rows, 3 columns, all elements are 1
my_matrix <- matrix(1, nrow = 2, ncol = 3)
apply the sum() function over each row (margin value 1)
apply(my_matrix, 1, sum)

```

```
output: [1] 3 3
apply the sum() function over each column (margin value 2)
apply(my_matrix, 2, sum)
output: [1] 2 2 2
```

`sapply()` takes any data basic data structure —vector, matrix, or list— and a function name and transforms the elements using the function. `sapply()` returns a vector or matrix of the same length, depending on the number of dimensions required.

```
a_vector_or_matrix <- sapply(any_basic_data_structure, function_name)
```

`lapply()` takes any data basic data structure — vector, matrix, or list — and a function name. `lapply()` transforms the elements and returns a list of the same length. The `l` in `lapply()` stands for list!

```
a_list <- lapply(any_basic_data_structure, function_name)
```

The difference between `sapply()` and `lapply()` is minimal. Choose which function to use based on the data structure you want returned.

Let's see them both in action. The code below accomplishes the same thing as the for loop above.

Use of `sapply()`:

```
turn_to_percent <- function(decimal) {
 decimal * 100
}

decimals <- c(0.45, 0.13, 0.25, 0.78, 0.66, 0.92, 0.84, 0.56)
percentages_sapply <- sapply(decimals, turn_to_percent)
percentages_sapply
output: [1] 45 13 25 78 66 92 84 56
```

Use of `lapply()`:

```
percentages_lapply <- lapply(decimals, turn_to_percent)
output: [[1]]
[1] 45

[[2]]
[1] 13

[[3]]
[1] 25

[[4]]
[1] 78

[[5]]
[1] 66

[[6]]
[1] 92

[[7]]
[1] 84

[[8]]
[1] 56
```

There are a few other functions in the `apply()` family, like `mapply()`, which allows functions with multiple arguments.

In this article, we introduced how functions work in R. Namely, we covered how to define a function, what we mean by a function **parameter** and **argument**, and how to use the `apply()` functions to make our lives even easier. Functions are a basic coding tool that use the same piece of code for

different inputs. We will keep encountering useful functions in various R libraries, and now we have the knowledge to define functions that suit our own R program's needs.

## Try this challenge!

Coding question

## Questions

There is an R function `tolower()` that takes in a string value and converts all the characters to lowercase. For example, `tolower("OMG")` would return `"omg"`. In the workspace, we have a vector `names` that has names all in uppercase. Use an appropriate `apply()` function to create a list `names_lower` where all the names in `names` are now lowercase. Call `names_lower` so you can see it outputted.

Ahoy! We R excited for you to start your learning adventure with a language built for data enthusiasts! The R community is made up of people passionate about the intersection of numbers, data, analysis, and code. In this lesson

# Exploring RStudio Off Platform Project

In this off-platform project, we will learn how to write our first program in RStudio.

Welcome to an off-platform project! The goal of this project is to get you comfortable working in RStudio. Follow these instructions to download the project and complete it using RStudio.

## Getting Started with RStudio

If you've never used RStudio before, we recommend watching this video to familiarize yourself with the tool:  
If you have not already done so, [download R](#), and [download RStudio](#).  
Download the [Explore RStudio project](#) zip file.  
Unzip the file by double-clicking on it in your Downloads folder.  
Navigate to the newly unzipped folder and open `ExploreRStudio_Starting.Rmd` in RStudio to see the project instructions.  
Follow the instructions by writing code in the code blocks found in `ExploreRStudio_Starting.Rmd`.

If you are stuck on the project, we have included our solution in the file `ExploreRStudio_Solution.Rmd`. You can also watch the solution video posted below:

# Welcome To Statistics In R

In this section of Analyze Data With R, we will dive into the world of statistics. Starting with a handful of descriptive statistics like mean, median, mode, variance, and IQR, we will use R to better describe our datasets.

We'll then learn how to make hypotheses and test those hypotheses.

By learning these skills in R, you will grow your ability to describe and analyze your data! Let's get started!

# Mean in R

## Introduction

Finding the center of a dataset is one of the most common ways to summarize statistical findings. Often, people communicate the center of data using words like, *on average*, *usually*, or *often*.

In this lesson, you will learn how to calculate the *mean* of a dataset, a common measure of a dataset's center. We will use the mean to help us answer the question,

*When are adults their most creative and productive?*

You could define “creative” and “productive” in a lot of ways, making this question impossible to fully answer by the end of this lesson. However, you will form an informed opinion on the question using data of the one hundred greatest novels of all time.

We collected the dataset from a survey administered by the French literary magazine, Le Monde. From the dataset, you will calculate the average age of the authors when their books were published.

### Instructions

The histogram to the right displays the ages of 100 authors from the Le Monde survey. Where do you think the data is centered?

```

title: "Mean in R"
output: html_notebook

title: "Mode in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
library(ggplot2)

# load data frame
greatest_books <- read_csv('top-hundred-books.csv')

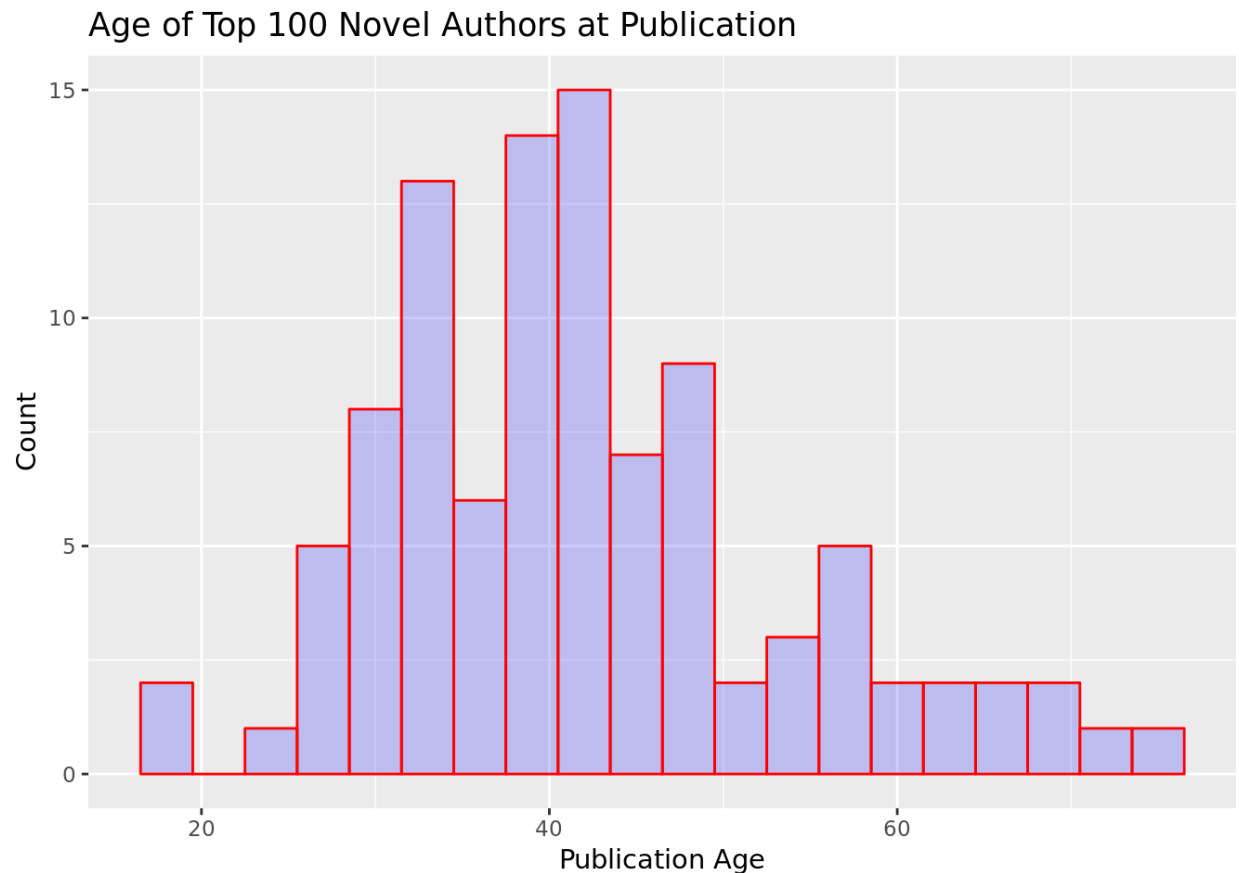
#plot data
hist <- qplot(greatest_books$Ages,
  geom='histogram',
  binwidth = 3,
  main = 'Age of Top 100 Novel Authors at Publication',
  xlab = 'Publication Age',
```



```
ylab = 'Count',
fill=I("blue"),
col=I("red"),
alpha=I(.2))
```

```
hist
```

```



Mean in R

## Calculating Mean

The *mean*, often referred to as the *average*, is a way to measure the center of a dataset.

The average of a set is calculated using a two-step process:

Add all of the observations in your dataset.

Divide the total sum from step one by the number of points in your dataset.

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

The equation above is used to calculate mean.  $x_1, x_2, \dots, x_n$  are observations from a dataset of  $n$  observations.

### Example

Imagine that we wanted to calculate average of a dataset with the following four observations:

```
data <- c(4, 6, 2, 8)
```

Copy to Clipboard

### Step One: Calculate the total

4+6+2+8=20

### Step Two: Divide by the number of observations

The total is equal to 20, and the number of observations is equal to 4.

$$20/4=5$$

The average of this dataset is equal to 5.

### Instructions

Checkpoint 1 Enabled

1.

In this exercise, you will use R to find the average age of the first four authors in Le Monde's top 100 books.

29,49,42,43

29,49,42,43

Add the values together, and set `total` equal to the answer. Print `total`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Divide `total` by the number of values in the dataset, and set `mean_value` to the answer.

Print `mean_value`. Keep that number in your head as you progress through the lesson.

```

title: "Mean in R"
output: html_notebook

```{r}
total <- 29 + 49 + 42 + 43
print(total)
mean_value <- total / 4
print(mean_value)
```
```

# Mean in R

While you've shown that you can calculate the average yourself, it becomes time-consuming as the size of your dataset increases — imagine adding all of the numbers in a dataset with 10,000 observations.

The R `mean()` function can do the work of adding and dividing for you. In the example below, we use `mean()` to calculate the average of a dataset with ten values:

```
example_data <- c(24, 16, 30, 10, 12, 28, 38, 2, 4, 36)

example_average <- mean(example_data)

print(example_average)
```

Copy to Clipboard

The code above calculates the average of `example_data` and saves the value to `example_average`. The resulting average of this array is 20.

### Instructions

Checkpoint 1 Enabled

1.

Use R to calculate the average value of the `author_ages` array. Save the result to `average_age` and print it.  
Does the average age of the authors surprise you? If so, how? Is it older, or younger than you expected?

```

title: "Mean in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# import libraries
library(readr)
library(dplyr)
# Read author data
greatest_books = read_csv("top-hundred-books.csv")

# Set author ages to a vector
author_ages <- greatest_books$Ages

# Use R to calculate mean
average_age <- mean(author_ages)
print(average_age)
```
```

# Median in R

## Introduction

In this lesson, you will learn how to find the *median* of a dataset — a common measure of a dataset's center. Each of the next three exercises will cover the following topics:

- Manually finding the median of a dataset

- Using R's median function to find the median of a dataset

- Interpreting what it means for a dataset to have similar and different median and mean values

In the lesson, we will use a dataset of the 100 greatest novels, determined by a French literary magazine, Le Monde. From the dataset, you will use the median to answer the question:

*When are great authors most likely to publish their best work?*

If you are not familiar with mean, also known as average, we recommend that you learn about it in our lesson on [average](#).

### Instructions

The histogram to the right displays the age of authors, at publication, for the top 100 novels. The red line represents the average value of this dataset.

You can think of the median as being the observation in your dataset that falls right in the middle.

Using this informal definition of the median and the graph to the right, see if you can determine whether the median of this dataset falls to the right or the left of the mean. We will show you the correct answer in the last exercise.

```

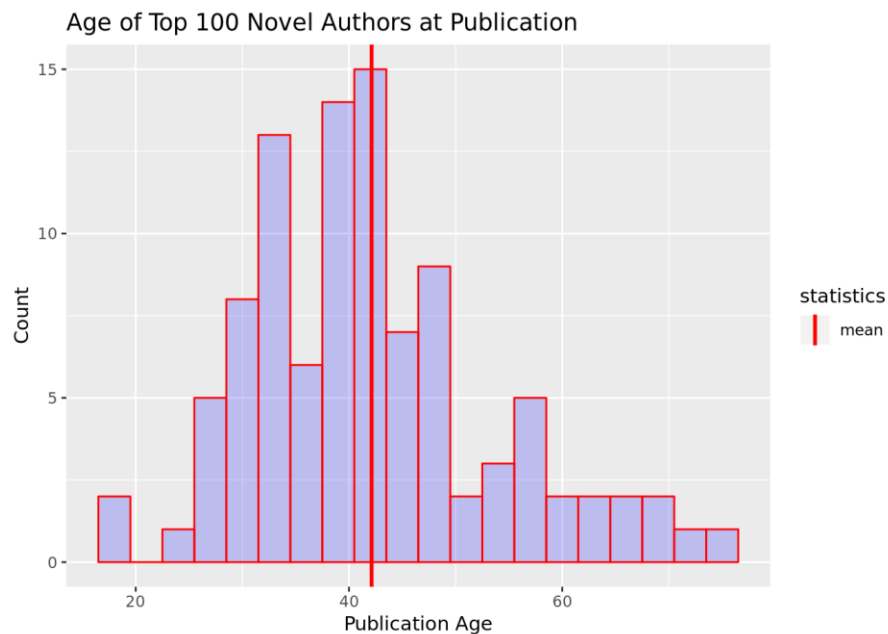
title: "Median in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
library(ggplot2)

# load data frame
greatest_books <- read_csv('top-hundred-books.csv')

#plot data
hist <- qplot(greatest_books$Ages,
  geom='histogram',
  binwidth = 3,
  main = 'Age of Top 100 Novel Authors at Publication',
  xlab = 'Publication Age',
  ylab = 'Count',
  fill=I("blue"),
  col=I("red"),
  alpha=I(.2)) +
  geom_vline(aes(xintercept=mean(greatest_books$Ages),
    color="mean"), linetype="solid",
    size=1) +
  scale_color_manual(name = "statistics", values = c(mean = "red"))

hist
---
```



Median

The formal definition for the median of a dataset is:

The value that, assuming the dataset is ordered from smallest to largest, falls in the middle. If there are an even number of values in a dataset, you either report both of the middle two values or their average.

There are always two steps to finding the median of a dataset:

Order the values in the dataset from smallest to largest
Identify the number(s) that fall(s) in the middle

Example One: Even Number of Values

Say we have a dataset with the following ten numbers:

24, 16, 30, 10, 12, 28, 38, 2, 4, 36

24, 16, 30, 10, 12, 28, 38, 2, 4, 36

The first step is to order these numbers from smallest to largest:

2, 4, 10, 12, [16, 24], 28, 30, 36, 38

2, 4, 10, 12, [16, 24], 28, 30, 36, 38

Because this dataset has an even number of values, there are two medians: 16 and 24 — 16 has four datapoints to the left, and 24 has four datapoints to the right.

Although you can report both values as the median, people often average them. If you averaged 16 and 24, you could report the median as 20.

Example Two: Odd Number of Values

If we added another value (say, 24) to the dataset and sorted it, we would have:

2, 4, 10, 12, 16, [24], 24, 28, 30, 36, 38

2, 4, 10, 12, 16, [24], 24, 28, 30, 36, 38

The new median is equal to 24, because there are 5 values to the left of it, and 5 values to the right of it.

Instructions

Checkpoint 1 Failed, try again

1.

In the next two steps, you will manually sort an array, and then determine which value in the array is the median.

In **notebook.Rmd**, we have a vector with the ages of the first five authors from Le Monde's survey:
29,49,42,43,32

29,49,42,43,32

Under `five_author_ages` there is a variable called `sorted_author_ages`. Change the 0s in `sorted_author_ages` to the values in ascending order from `five_author_ages`.

```
---  
title: "Median in R"  
output: html_notebook  
---
```

```

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)

Array of the first five author ages
five_author_ages <- c(29, 49, 42, 43, 32)

Fill in the empty array with the values sorted
sorted_author_ages <- c(29, 32, 42, 43, 49)

Save the median value to median_value
median_value <- 42

Print the sorted array and median value
cat("The sorted array is:", sorted_author_ages)
cat(paste("The median of the array is: ", median_value))
```

```

Median in R

Finding the median of a dataset becomes increasingly time-consuming as the size of your dataset increases — imagine finding the median of an unsorted dataset with 10,000 observations.

The R `median()` function can do the work of sorting, then finding the median for you. In the example below, we use `median()` to calculate the median of a dataset with ten values:

```

example_data = c(24, 16, 30, 10, 12, 28, 38, 2, 4, 36, 42)

example_median = median(example_data)

print(example_median)

```

Copy to Clipboard

The code above prints the median of the dataset, 24. The mean of this dataset is 22. It's worth noting these two values are close to one another, but not equal.

Instructions

Checkpoint 1 Enabled

1.

Use R to find the median of the `author_ages` array. Save the result to `median_age`.

Does the median age of the authors surprise you? If so, how? Is it older, or younger than you expected?

```

---
title: "Median in R"
output: html_notebook
---
```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)

Load data frame

```

```
greatest_books <- read_csv('top-hundred-books.csv')

Save author ages to author_ages
author_ages <- greatest_books$Ages

Use R to calculate the median age of the top 100 authors
median_age <- median(author_ages)

print(paste("The median age of the 100 greatest authors, according to a survey by Le Monde
is: " , median_age))
```

# Review and Discussion

In this lesson, you learned how to find the median of a dataset in two steps:

- Sort the dataset
- Identify the one or two numbers that fall in the middle of the sorted dataset

You also learned how to calculate the median using R:

```
median(my_data)
```

Copy to Clipboard

## Discussion

Take a look at the histogram to the right. It displays the author age distribution with vertical lines for the mean (red) and median (blue).

Do you feel like the median of our dataset, 40.5, provides us enough information to claim when authors publish their greatest work?

We argue it does not.

Although the median is a good measure of the dataset's center, we cannot make a definitive claim about when authors publish their greatest work — the youngest author published at 18 and the oldest at 76. It would be irresponsible to say anything but, "it seems to be possible at almost any age."

Notice that the mean and the median are nearly equal. This is not a surprising result, as both statistics are a measure of the dataset's center. However, it's worth noting that these results will not always be so close.

In the instructions below, we've written a brief explanation that puts median in the context of our problem.

## Instructions

The median age of authors, when they publish their best work, from Le Monde's 100 greatest books is 41. While this does not tell us much about which year is an author's greatest year, it does indicate that half of the authors from the survey find their greatest success before the age of 41 and half find their greatest success after the age of 41.

```

title: "Median in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
```

```

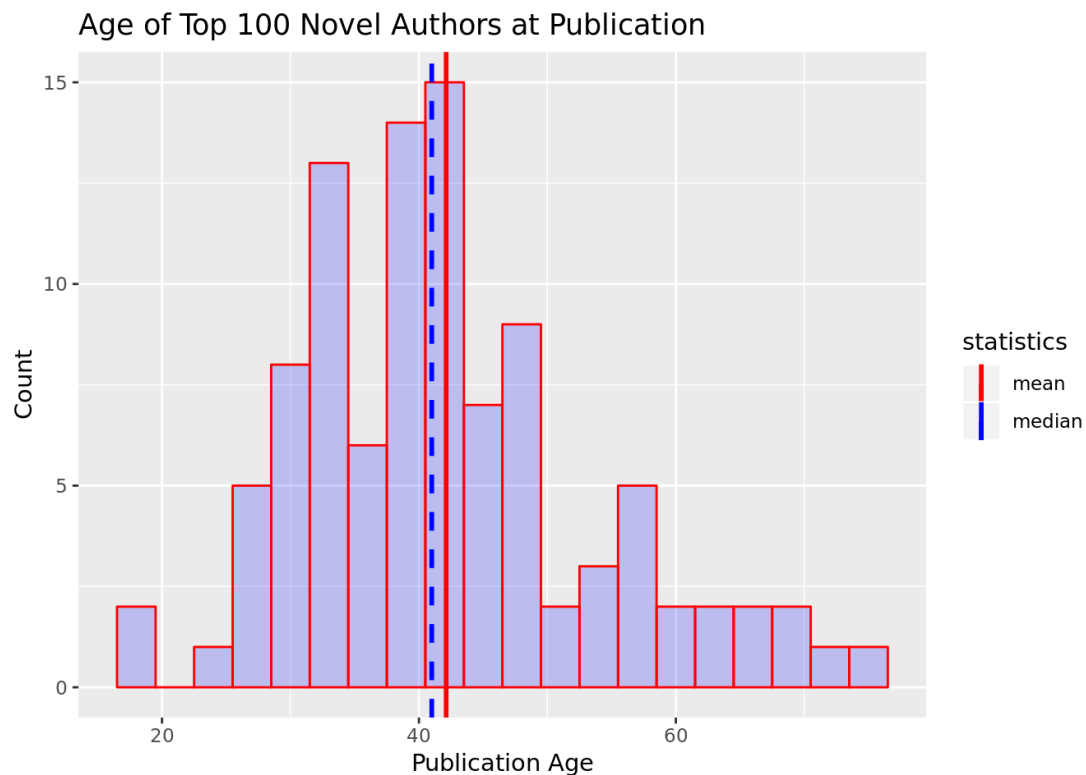
library(readr)
library(dplyr)
library(ggplot2)

# load data frame
greatest_books <- read_csv('top-hundred-books.csv')

#plot data
hist <- qplot(greatest_books$Ages,
  geom='histogram',
  binwidth = 3,
  main = 'Age of Top 100 Novel Authors at Publication',
  xlab = 'Publication Age',
  ylab = 'Count',
  fill=I("blue"),
  col=I("red"),
  alpha=I(.2)) +
  geom_vline(aes(xintercept=median(greatest_books$Ages),
    color="median"), linetype="dashed",
    size=1) +
  geom_vline(aes(xintercept=mean(greatest_books$Ages),
    color="mean"), linetype="solid",
    size=1) +
  scale_color_manual(name = "statistics", values = c(median = "blue", mean =
"red"))

hist
` ``

```



Mode in R

Introduction

In this lesson, you will learn how to find the *mode* of a dataset. Each of the next three exercises will cover the following:

- Manually finding the mode of a dataset

- Using R's

- [functions](#)

- Preview: Docs Functions are blocks of code that perform specific tasks and can be repeatedly called within other parts of the code.

- to find the mode

- Comparing mode to mean and median values

In the lesson, we will use a dataset of the 100 greatest novels, determined by a French literary magazine, Le Monde. From the dataset, you will use the mode to answer the question:

What is the most common age for a great author to publish their best work?

If you are not familiar with mean, also known as average, or median, we recommend that you learn about it in our lessons on [average](#) and [median](#).

Instructions

The histogram to the right displays the age of authors, at publication, for the top 100 novels from Le Monde's survey. The red line is the mean age, and the blue line is the median age.

Use the definition of mode below and the histogram to the right to guess where the mode falls. You will calculate the correct answer in the last exercise.

The mode is the most common observation in a dataset.

You will not be able to find the exact mode, because the histogram displays bins with a range of values. However, you can guess a range of values where you are most likely to see the mode.

Mode

The formal definition for the mode of a dataset is:

The most frequently occurring observation in the dataset. A dataset can have multiple modes if there is more than one value with the same maximum frequency.

While you may be able to find the mode of a small dataset by simply looking through it, if you have trouble, we recommend you follow these two steps:

- Find the frequency of every unique number in the dataset

- Determine which number has the highest frequency

Example

Say we have a dataset with the following ten numbers:

24, 16, 12, 10, 12, 28, 38, 12, 28, 24

Let's find the frequency of each number:

24	16	12	10	28	38
2	1	3	1	2	1

From the table, we can see that our mode is 12, the most frequent number in our dataset.

Instructions

Checkpoint 1 Enabled

1.

Determine the mode of the ages for the first ten authors in the Le Monde survey:
29,49,42,43,32,38,37,41,27,27

```
---  
title: "Mean in R"  
output: html_notebook  
---  
{r}  
mode_age <- 27  
mode_count <- 2  
---
```

Mode with DescTools

Finding the mode of a dataset becomes increasingly time-consuming as the size of your dataset increases – imagine finding the mode of a dataset with 10,000 observations.

The R package `DescTools` includes a handy `Mode()` function which can do the work of finding the mode for us. In the example below, we use `Mode()` to calculate the mode of a dataset with ten values:

Example: One Mode

```
library(DescTools)  
  
example_data <- c(24, 16, 12, 10, 12, 28, 38, 12, 28, 24)  
  
example_mode <- Mode(example_data)
```

Copy to Clipboard

The code above calculates the mode of the values in `example_data` and saves it to `example_mode`.

The result of `Mode()` is a vector with the mode value:

```
>>> example_mode  
[1] 12
```

Copy to Clipboard

Example: Two Modes

If there are multiple modes, the `Mode()` function will return them as a vector.

Let's look at a vector with two modes, 12 and 24:

```
example_data = c(24, 16, 12, 10, 12, 24, 38, 12, 28, 24)
```

```
example_mode = Mode(example_data)
```

Copy to Clipboard

The result is:

```
>>> example_mode  
[1] 12 24
```

Copy to Clipboard

Instructions

Checkpoint 1 Failed, try again

1.

We have already imported the `DescTools` library for you.

Delete the current value set to `mode_age`.

Find the mode of the observations in the `author_ages` array. Save the result to `mode_age`.

```
---  
title: "Mode in R"  
output: html_notebook  
---  
``{r message=FALSE, warning=FALSE}  
# import libraries  
library(readr)  
library(dplyr)  
library(DescTools)  
  
# Read author data  
greatest_books = read_csv("top-hundred-books.csv")  
  
# Set author ages to  
author_ages <- greatest_books$Ages  
  
mode_age <- Mode(author_ages)  
  
print(paste("The mode age of authors from Le Monde's 100 greatest books is: ",  
mode_age[1]))  
  
...
```

Review and Discussion

In this lesson, you learned how to find the mode of a dataset in two steps:

Find the frequency of every unique number in the dataset

Determine which number has the highest frequency

You also learned how to calculate the mode using `DescTools`:

```
Mode(my_array)
```

Copy to Clipboard

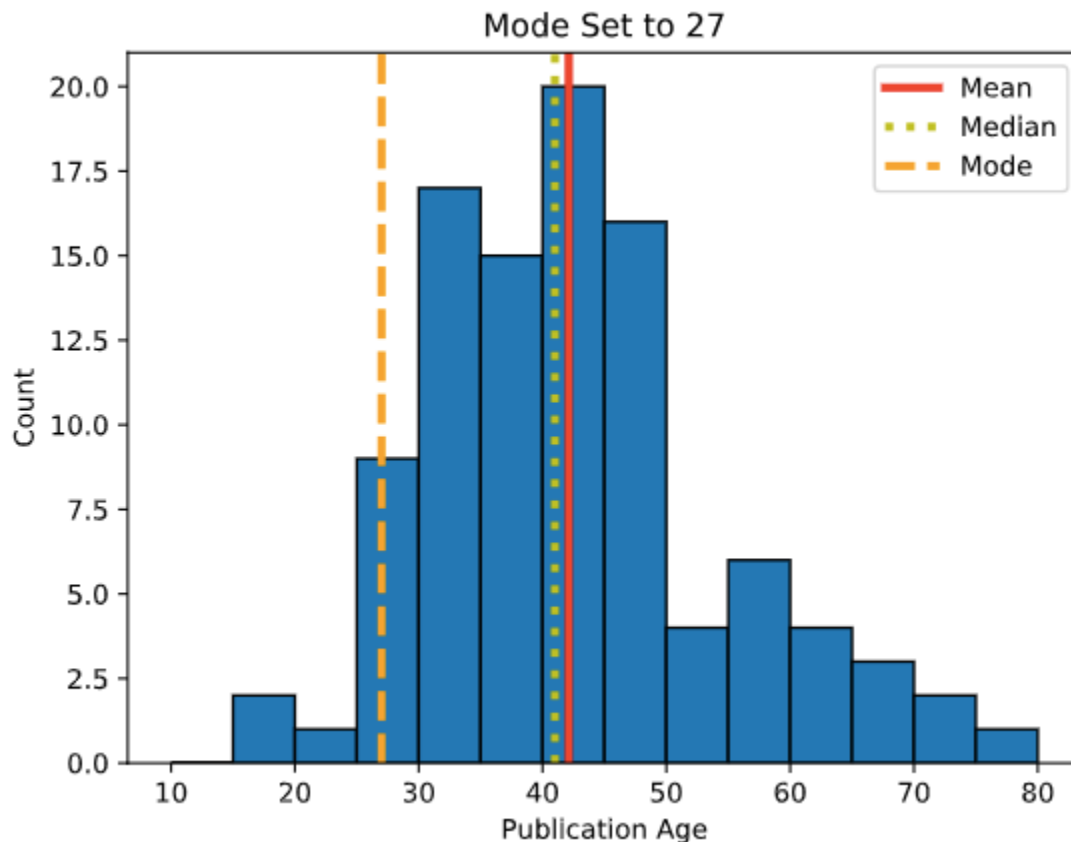
Discussion

In this lesson, you found that 38 was the most common age, at publication, for an author from the Le Monde survey. How does this number compare to your guess from the beginning of the lesson?

The mode is close to the median and mean of the dataset, but it is not in the tallest bucket. This should not be surprising, as the histogram indicates the data is centered between the ages of 30 and 50 — there is a higher chance of a mode in that range than outside of it.

The mode is not always this close to the median and mean, and often will not be in the tallest bucket.

Look at the 25-30 year-old bin. There are nine observations in it. If all the values in that bin happened to be 27, then the dataset's mode would be 27. Although unlikely, it is possible. Below, we show what this would look like:



Based on this graph, it is fair to say the mode may not always be a great measure of where the data is centered. Simply put, mode is a measure of the most frequent observation in the dataset, and is not an indication of the tallest bin in a histogram.

In the instructions below, we've written a brief explanation that puts mode in the context of our problem.

Instructions

The most common age of an author from Le Monde's 100 greatest books, when they publish their best work, is 38.

```
---
title: "Mode in R"
output: html_notebook
---
``{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
library(ggplot2)

# load data frame
```

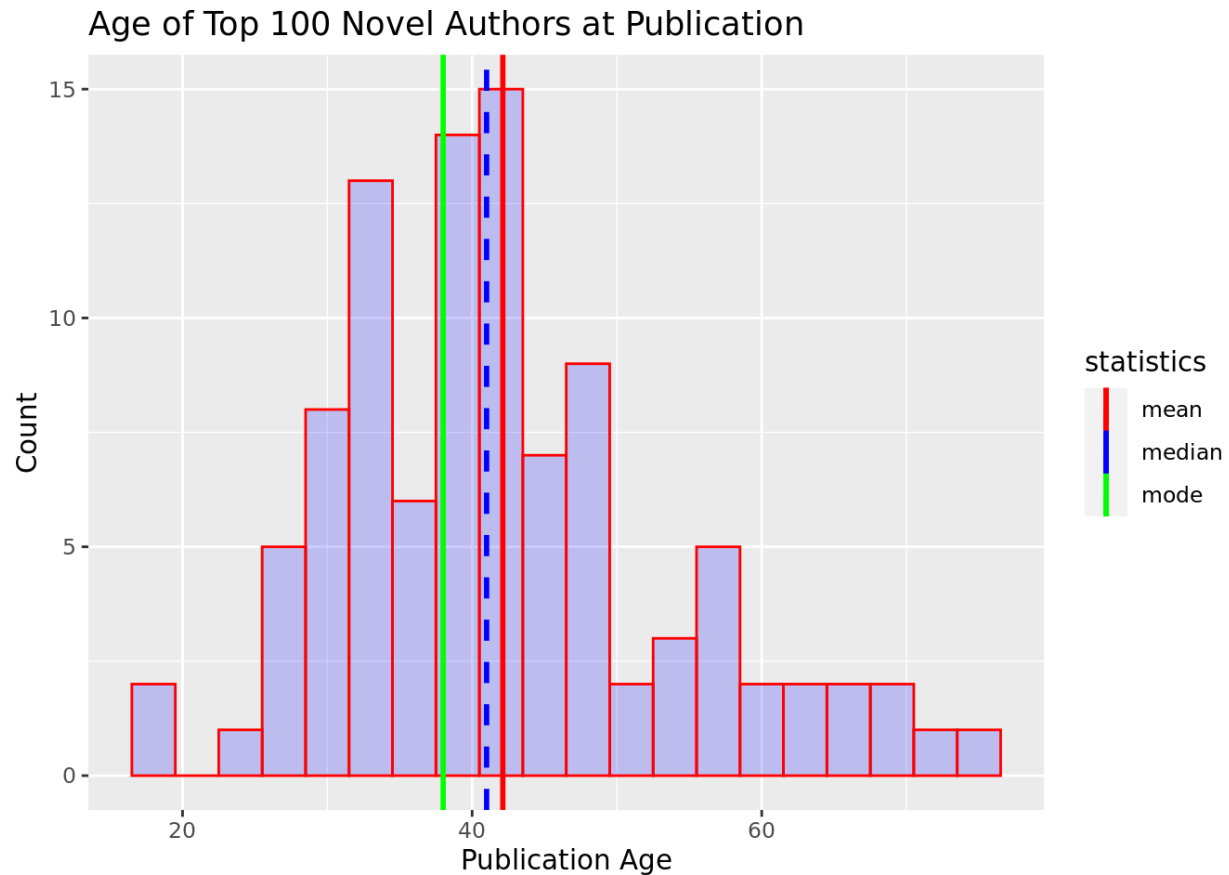
```

greatest_books <- read_csv('top-hundred-books.csv')

#plot data
hist <- qplot(greatest_books$Ages,
  geom='histogram',
  binwidth = 3,
  main = 'Age of Top 100 Novel Authors at Publication',
  xlab = 'Publication Age',
  ylab = 'Count',
  fill=I("blue"),
  col=I("red"),
  alpha=I(.2)) +
  geom_vline(aes(xintercept=median(greatest_books$Ages),
    color="median"), linetype="dashed",
    size=1) +
  geom_vline(aes(xintercept=mean(greatest_books$Ages),
    color="mean"), linetype="solid",
    size=1) +
  geom_vline(aes(xintercept=38,
    color="mode"), linetype="solid",
    size=1) +
  scale_color_manual(name = "statistics", values = c(median = "blue", mean =
"red", mode="green"))

hist
```

```



## Variance

Finding the mean, median, and mode of a dataset is a good way to start getting an understanding of the general shape of your data

However, those three descriptive statistics only tell part of the story. Consider the two datasets below:

```
dataset_one <- c(-4, -2, 0, 2, 4)
dataset_two <- c(-400, -200, 0, 200, 400)
```

[Copy to Clipboard](#)

These two datasets have the same mean and median — both of those values happen to be 0. If we only reported these two statistics, we would not be communicating any meaningful difference between these two datasets.

This is where *variance* comes into play. Variance is a descriptive statistic that describes how spread out the points in a data set are.

### Instructions

1. Run the code and take a look at the two histograms that get created. Also look at the mean of each dataset. These two histograms show the test grades of students from two different teacher's classes. While the datasets have very similar means, their variances are very different. Think about the following questions:
  - Which dataset looks the most spread out?
  - When looking at the spread of a histogram, why are the units on the x-axis so important?
  - What does the spread of the data tell you about these two teachers that you might not understand if you only looked at the mean?

```

title: "Variance in R"
output: html_notebook

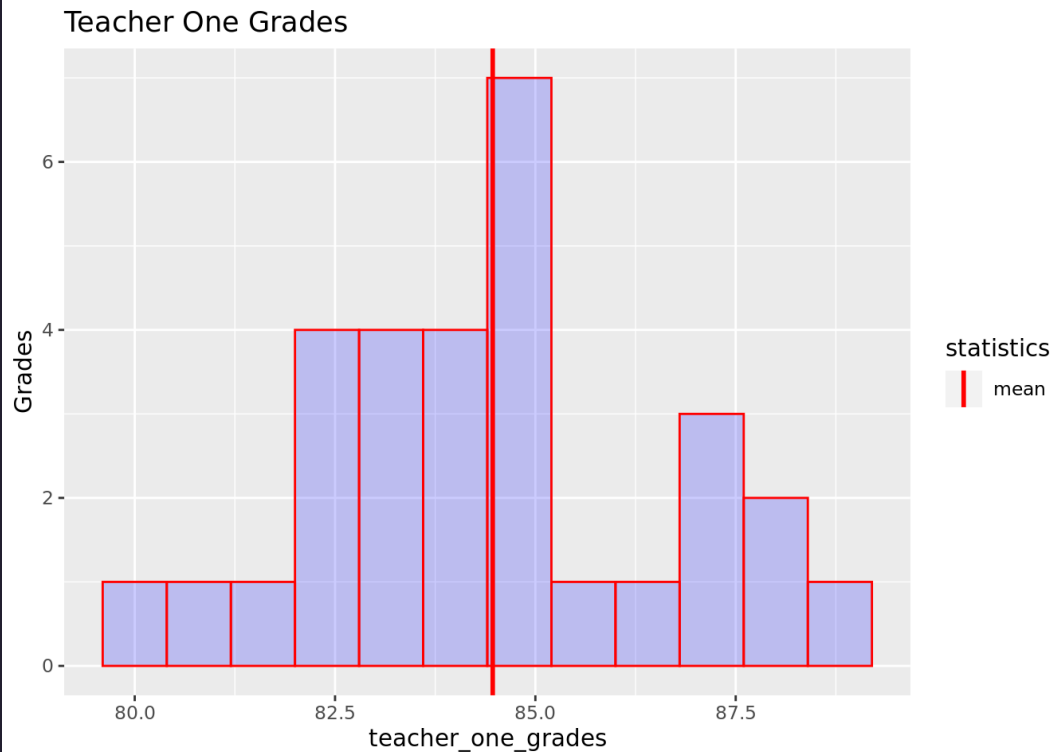
``{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
library(ggplot2)

load data
teacher_one_grades <- c(83.42, 88.04, 82.12, 85.02, 82.52, 87.47, 84.69, 85.18, 86.29,
85.53, 81.29, 82.54, 83.47, 83.91, 86.83, 88.5, 84.95, 83.79, 84.74, 84.03, 87.62, 81.15,
83.45, 80.24, 82.76, 83.98, 84.95, 83.37, 84.89, 87.29)
teacher_two_grades <- c(85.15, 95.64, 84.73, 71.46, 95.99, 81.61, 86.55, 79.81, 77.06,
92.86, 83.67, 73.63, 90.12, 80.64, 78.46, 76.86, 104.4, 88.53, 74.62, 91.27, 76.53, 94.37,
84.74, 81.84, 97.69, 70.77, 84.44, 88.06, 91.62, 65.82)

#plot data
t1_chart <- qplot(teacher_one_grades,
 geom='histogram',
 binwidth = .8,
 main = 'Teacher One Grades',
 ylab = 'Grades',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=mean(teacher_one_grades),
 color="mean"), linetype="solid",
 size=1) +
 scale_color_manual(name = "statistics", values = c(mean = "red"))

t1_chart

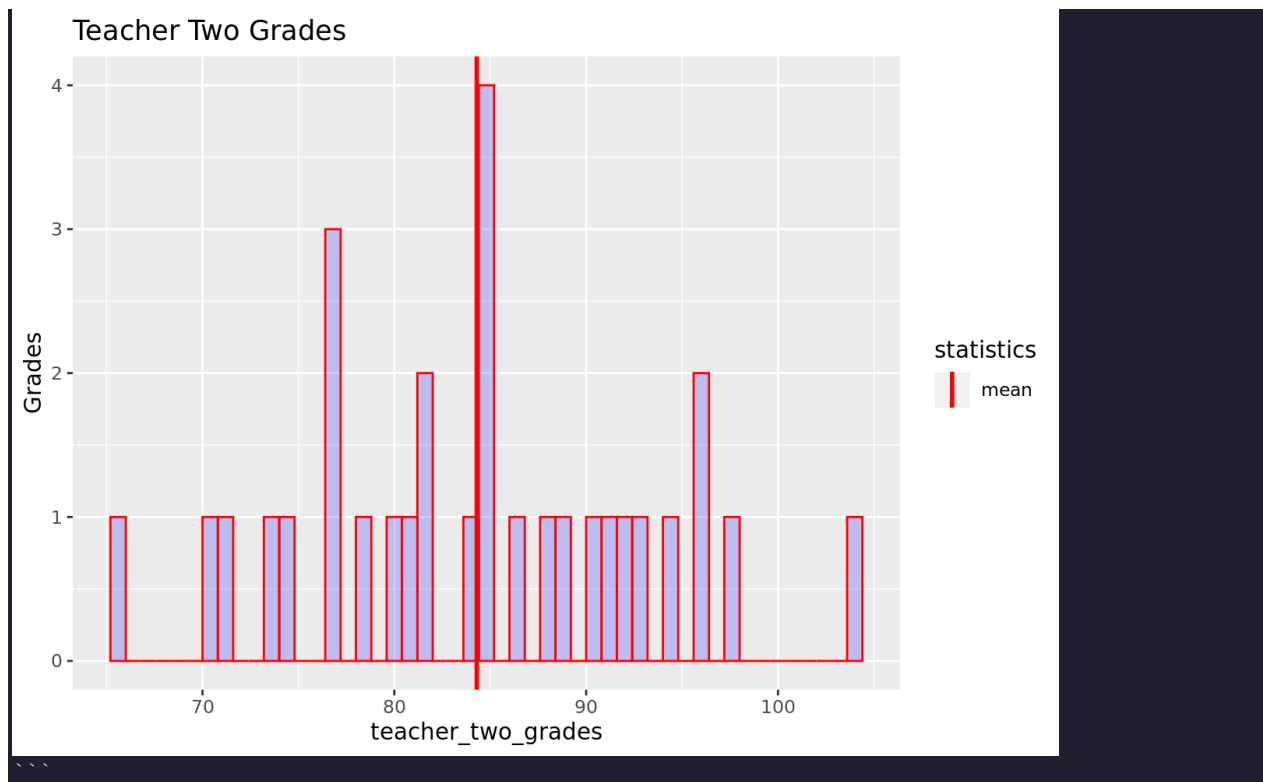
```



```
t2_chart <- qplot(teacher_two_grades,
 geom='histogram',
 binwidth = .8,
 main = 'Teacher Two Grades',
 ylab = 'Grades',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=mean(teacher_two_grades),
 color="mean"), linetype="solid",
 size=1) +
 scale_color_manual(name = "statistics", values = c(mean = "red"))
```

T2\_chart





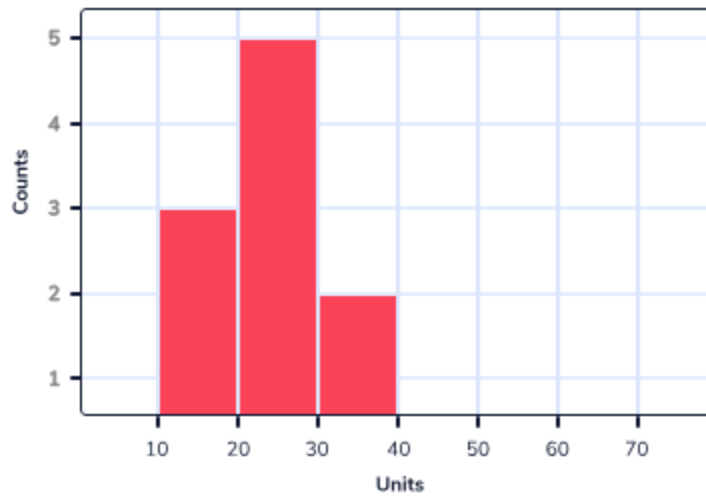
## Distance From Mean

Now that you have learned the importance of describing the spread of a dataset, let's figure out how to mathematically compute this number.

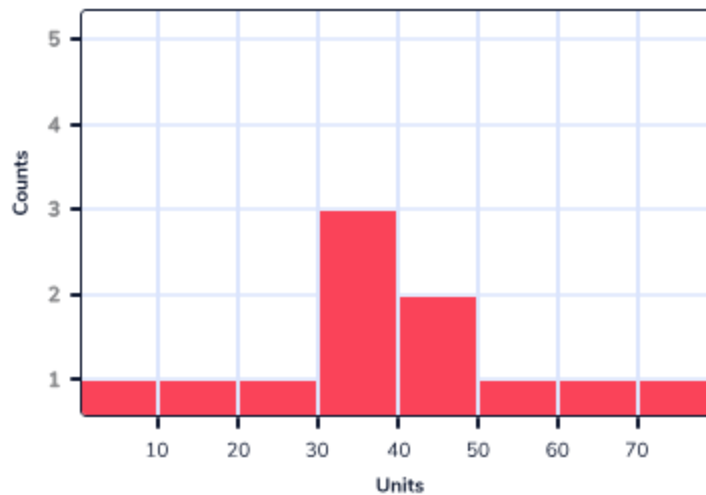
How would you attempt to capture the spread of the data in a single number?

Let's start with our intuition — we want the variance of a dataset to be a large number if the data is spread out, and a small number if the data is close together.

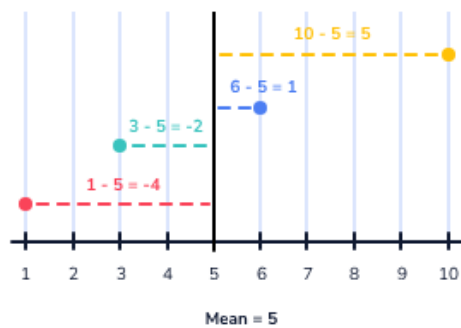
**Smaller Variance Dataset**



**Larger Variance Dataset**



A lot of people may initially consider using the range of the data. But that only considers two points in your entire dataset. Instead, we can include every point in our calculation by finding the difference between every data point and the mean.



If the data is close together, then each data point will tend to be close to the mean, and the difference will be small. If the data is spread out, the difference between every data point and the mean will be larger.

Mathematically, we can write this comparison as

$$\text{Difference} = X - \mu$$

Where  $x$  is a single data point and the Greek letter  $\mu$  is the mean.

### Instructions

Checkpoint 1 Enabled

1.

We've given you a very small dataset of five values named `grades`. These are five randomly chosen grades from the first teacher's class. We've also calculated the mean of this small dataset and stored it in a variable named `mean`.

Let's find the difference between each of these data points and the mean. We've created a variable for each difference. Start with `difference_one`. Change the value of `difference_one` — it should be equal to the first value in the data set minus `mean`.

```

title: "Variance in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load data
grades <- c(88, 82, 85, 84, 90)
mean <- mean(grades)

#Change these five variables
difference_one <- 88 - mean
difference_two <- 82 - mean
difference_three <- 85 - mean
difference_four <- 84 - mean
difference_five <- 90 - mean

print(paste("The mean of the data set is ", mean))

print(paste("The first student is", round(difference_one, digits=2) , "percentage points
away from the mean.))
print(paste("The second student is ", round(difference_two, digits=2) , "percentage points
away from the mean.))
print(paste("The third student is",round(difference_three, digits=2) , "percentage points
away from the mean.))
print(paste("The fourth student is",round(difference_four, digits=2) , "percentage points
away from the mean.))
print(paste("The fifth student is",round(difference_five, digits=2) , "percentage points
away from the mean.))
```
```

## Average Distances

We now have five different values that describe how far away each point is from the mean. That seems to be a good start in describing the spread of the data. But the whole point of calculating variance was to get one number that describes the dataset. We don't want to report five values — we want to combine those into one descriptive statistic.

To do this, we'll take the average of those five numbers. By adding those numbers together and dividing by 5, we'll end up with a single number that describes the average distance between our data points and the mean.

Note that we're not *quite* done yet — our final answer is going to look a bit strange here. There's a small problem that we'll fix in the next exercise.

#### Instructions

Checkpoint 1 Enabled

1.

Sum the five variables `difference_one` through `difference_five` and store the result in the variable `difference_sum`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Divide `difference_sum` by 5 and store the result in the variable named `average_difference`. Think about the answer. Do you think it accurately captures the spread of your data?

## Square the Differences

We're almost there! We have one small problem with our equation. Consider this very small dataset:

```
c(-5, 5)
```

Copy to Clipboard

The mean of this dataset is 0, so when we find the difference between each point and the mean we get  $-5 - 0 = -5$  and  $5 - 0 = 5$ .

When we take the average of  $-5$  and  $5$  to get the variance, we get 0:

$$-5 + 5 = 0$$

Now think about what would happen if the dataset were `c(-200, 200)`. We'd get the same result! That can't possibly be right — the dataset with 200 is much more spread out than the dataset with 5, so the variance should be much larger!

The problem here is with negative numbers. Because one of our data points was 5 units below the mean and the other was 5 units above the mean, they canceled each other out!

When calculating variance, if a data point was above or below the mean — all we care about is how far away it was. To get rid of those pesky negative numbers, we'll square the difference between each data point and the mean.

Our equation for finding the difference between a data point and the mean now looks like this:

$$\text{difference} = (X - \mu)^2$$

#### Instructions

Checkpoint 1 Enabled

1.

Square each of the values stored in the variables `difference_one` through `difference_five`. In R, to square a number, use the `^` operator. The code below gives you an example of squaring the variable `a`.

```
squared_num <- a ^ 2
```

```

title: "Variance in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
grades <- c(88, 82, 85, 84, 90)
mean <- mean(grades)

#Change these five variables
difference_one <- 88 - mean
difference_two <- 82 - mean
difference_three <- 85 - mean
difference_four <- 84 - mean
difference_five <- 90 - mean

#Part 1: Sum the differences
difference_sum <- difference_one + difference_two + difference_three + difference_four
+ difference_five

#Part 2: Average the differences
variance <- difference_sum / 5

#IGNORE CODE BELOW HERE
print(paste("The sum of the differences is ", format(difference_sum, scientific =
FALSE )))
print(paste("The variance difference is " , format(variance, scientific = FALSE)))
```
```

## Variance in R

Well done! You've calculated the variance of a data set. The full equation for the variance is as follows:

$$\sigma^2 = \sum_{i=1}^N \left[ \frac{1}{N} (X_i - \mu)^2 \right] / N$$

Let's dissect this equation a bit.

Variance is usually represented by the symbol sigma squared.

We start by taking every point in the dataset — from point number 1 to point number `N` — and finding the difference between that point and the mean.

Next, we square each difference to make all differences positive.

Finally, we average those squared differences by adding them together and dividing by `N`, the total number of points in the dataset.

All of this work can be done quickly using a function we provided. The `variance()` function takes a list of numbers as a parameter and returns the variance of that dataset.

```
dataset <- c(3, 5, -2, 49, 10)
var <- variance(dataset)
```

Copy to Clipboard

### Instructions

Checkpoint 1 Enabled

1.

We've imported the same two datasets from the beginning of the lesson. Run the code to see a histogram of the two datasets. This time, the histograms are plotted on the same graph to help visualize the difference in spread.

Which dataset do you expect to have a larger variance?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Scroll down in the code to find where we've defined `teacher_one_variance` and `teacher_two_variance`. Set those variables equal to the variance of each dataset using the `variance()` function.

```

title: "Variance in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# Defining the variance of the population mean
variance <- function(x) mean((x-mean(x))^2)

teacher_one_grades <- c(80.24, 81.15, 81.29, 82.12, 82.52, 82.54, 82.76, 83.37, 83.42,
83.45, 83.47, 83.79, 83.91, 83.98, 84.03, 84.69, 84.74, 84.89, 84.95, 84.95, 85.02, 85.18,
85.53, 86.29, 86.83, 87.29, 87.47, 87.62, 88.04, 88.5)
teacher_two_grades <- c(65.82, 70.77, 71.46, 73.63, 74.62, 76.53, 76.86, 77.06, 78.46,
79.81, 80.64, 81.61, 81.84, 83.67, 84.44, 84.73, 84.74, 85.15, 86.55, 88.06, 88.53, 90.12,
91.27, 91.62, 92.86, 94.37, 95.64, 95.99, 97.69, 104.4)

# Set these two variables equal to the variance of each dataset using the var() function in
R
teacher_one_variance <- 0
teacher_two_variance <- 0

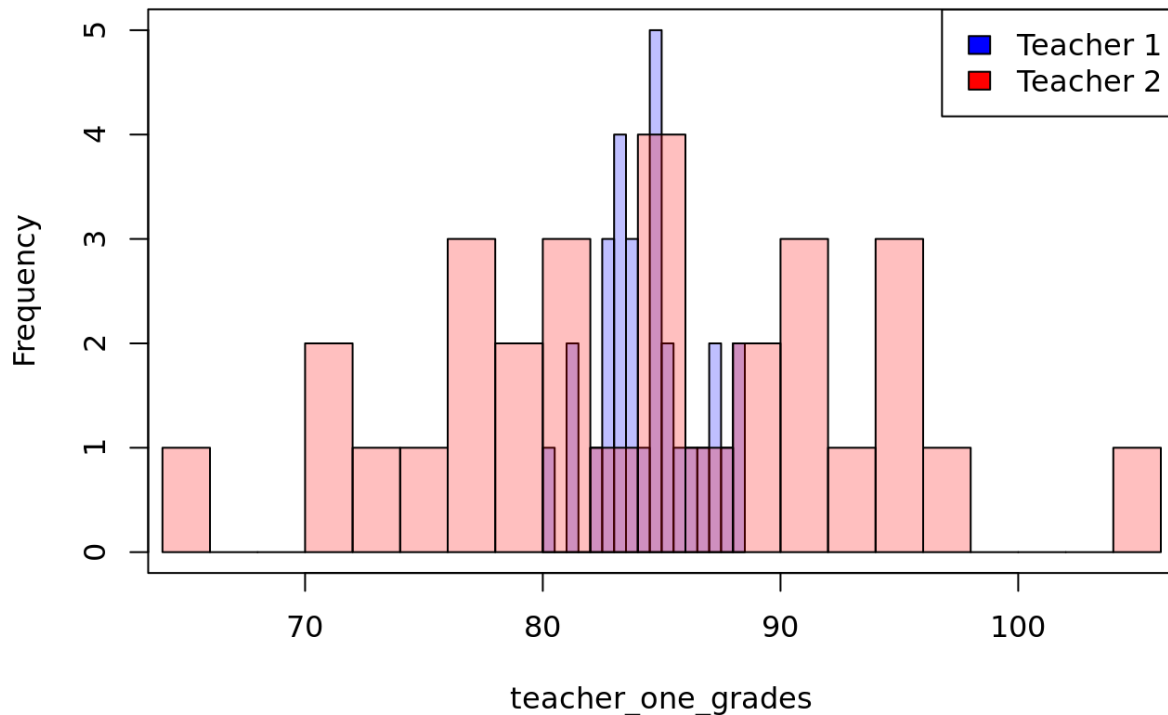
# IGNORE THE CODE BELOW HERE

hist(teacher_one_grades, col=rgb(0,0,1,1/4),xlim=c(65,105), main="Teacher Grades One and
Two", breaks=15)
hist(teacher_two_grades, col=rgb(1,0,0,1/4), add=T, breaks=15)
legend("topright", c("Teacher 1", "Teacher 2"), fill=c("blue", "red"))
box()

print(paste("The mean of the test scores in teacher one's class is ",
mean(teacher_one_grades)))
print(paste("The mean of the test scores in teacher two's class is ",
mean(teacher_two_grades)))

print(paste("The variance of the test scores in teacher one's class is ",
teacher_one_variance))
print(paste("The variance of the test scores in teacher two's class is ",
teacher_two_variance))
```
```

## Teacher Grades One and Two



### Review

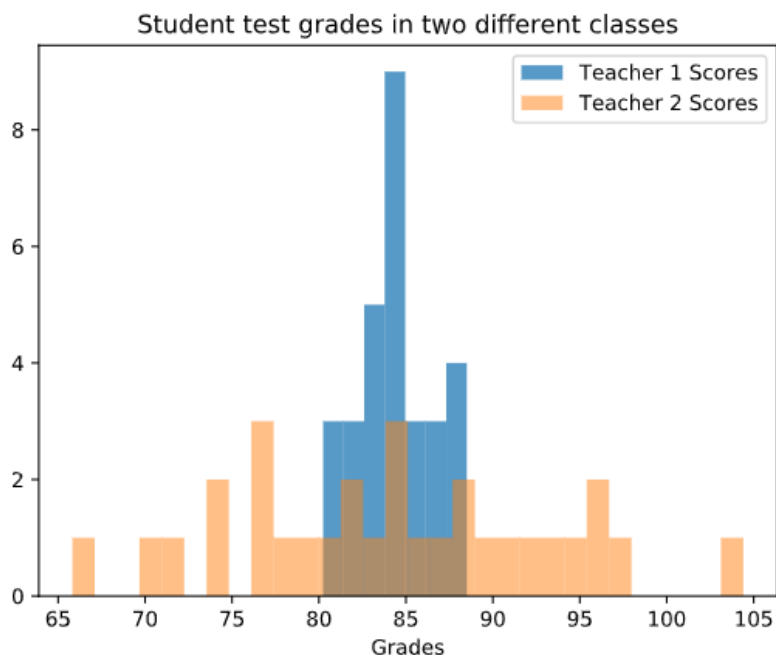
Great work! In this lesson you've learned about variance and how to calculate it.

In the example used in this lesson, the importance of variance was highlighted by showing data from test scores in classes taught by two different teachers. What story does variance tell? What conclusions can we draw from this statistic?

In the class with low variance, it seems like the teacher strives to make sure all students have a firm understanding of the subject, but nobody is exemplary.

In the class with high variance, the teacher might focus more of their attention on certain students. This might enable some students to ace their tests, but other students get left behind.

If we only looked at statistics like mean, median, and mode, these nuances in the data wouldn't be represented.



## Instructions

Checkpoint 1 Enabled

1.

You might be wondering why we need to compute the variance. After all, by comparing histograms, it was fairly easy to tell which dataset had a larger spread.

Variance is useful because it is a *measure* of spread. While we might get a general understanding of the spread by looking at a histogram, computing the variance gives us a numerical value that helps us describe the level of confidence of our comparison.

It is also interesting to consider datasets that don't have the same general curve.

Run the code to see two datasets that have a similar mean, but look very different. You'll also see a printout of their mean and variance. Before looking at the printout, try to guess which dataset has a large

```
hist(dataset_one, col=rgb(0,0,1,1/4), xlim=c(-30,30), breaks=80)
hist(dataset_two, col=rgb(1,0,0,1/4), add=T, breaks=80)
legend("topright", c("Dataset 1", "Dataset 2"), fill=c("blue", "red"))
box()

variance <- function(x) mean((x-mean(x))^2)

print(paste("The mean of dataset_one is", mean(dataset_one)))
print(paste("The mean of dataset_two is", mean(dataset_two)))
print(paste("The variance of dataset_one is", variance(dataset_one)))
print(paste("The variance of dataset_two is", variance(dataset_two)))
```

# Variance Recap



When beginning to work with a dataset, one of the first pieces of information you might want to investigate is the spread — is the data close together or far apart? One of the tools in our statistics toolbelt to do this is the descriptive statistic *variance*:

$$\sigma^2 = \left[ \sum (X_i - \mu)^2 \right] / N$$

By finding the variance of a dataset, we can get a numeric representation of the spread of the data. If you want to take a deeper dive into how to calculate variance, check out our [variance course](#).

But what does that number really mean? How can we use this number to interpret the spread?

It turns out, using variance isn't necessarily the best statistic to use to describe spread. Luckily, there is another statistic — standard deviation — that can be used instead.

In this lesson, we'll be working with two datasets. The first dataset contains the heights (in inches) of a random selection of players from the NBA. The second dataset contains the heights (in inches) of a random selection of users on the dating platform OkCupid.

### Instructions

Checkpoint 1 Passed

1.

Run the code to see a histogram of the datasets. Look at the console to see the mean and variance of each dataset. Try to answer the following questions:

What does it mean for the OkCupid dataset to have a larger variance than the NBA dataset?

What are the units of the mean? Is someone who is 80 inches tall taller than the average of either group? Which group(s)?

In this example, the units of variance are inches squared. Can you interpret what it means for the variance of the NBA dataset to be `13.32` inches squared?

```

title: "Standard Deviation"
output:
 html_notebook:
 code_folding: hide

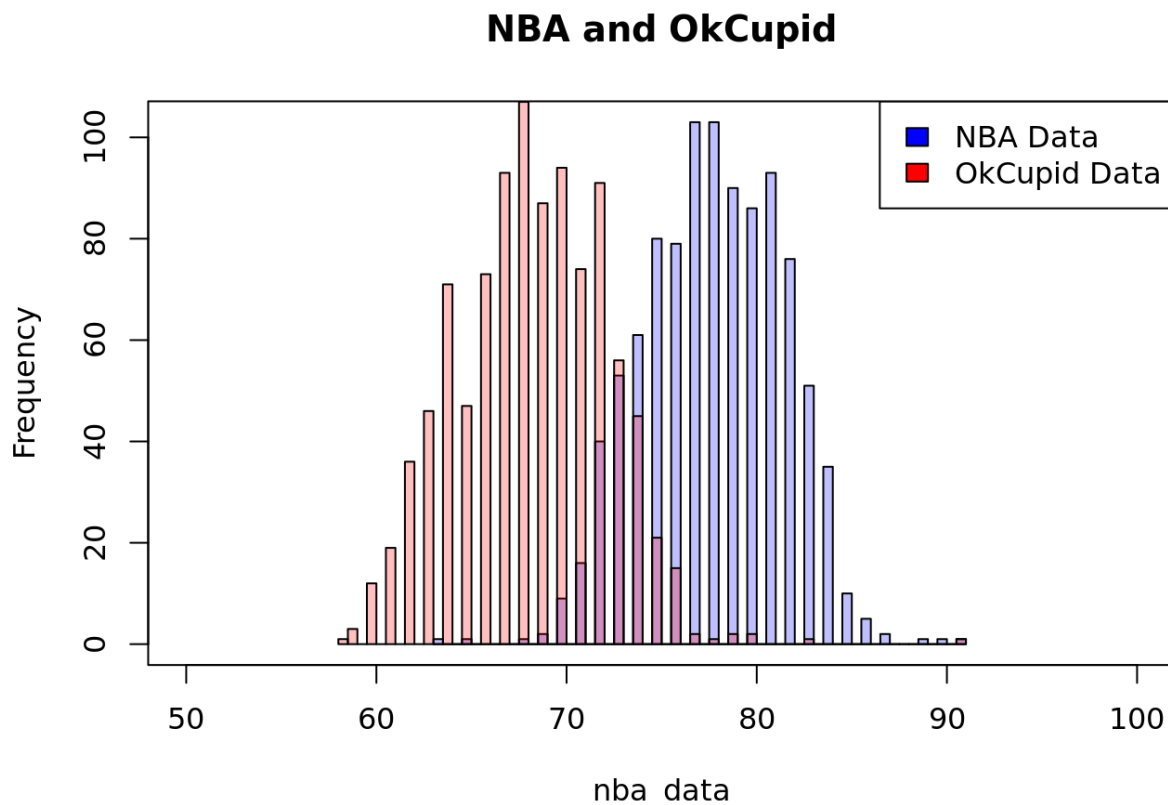
```{r message=FALSE, warning=FALSE}
# load libraries
library(dplyr)
library(tidyr)
```

```{r}
load("lesson_data.Rda")
variance <- function(x) mean((x-mean(x))^2)

hist(nba_data, col=rgb(0,0,1,1/4), xlim=c(50,100), main="NBA and OkCupid", breaks=80)
hist(okcupid_data, col=rgb(1,0,0,1/4), add=T, breaks=80)
legend("topright", c("NBA Data", "OkCupid Data"), fill=c("blue", "red"))
box()

print(paste("The mean of the NBA dataset is ", mean(nba_data)))
print(paste("The mean of the OkCupid dataset is ", mean(okcupid_data)))

print(paste("The variance of the NBA dataset is ", variance(nba_data)))
print(paste("The variance of the OkCupid dataset is ", variance(okcupid_data)))
```
```



## Standard Deviation

Variance is a tricky statistic to use because its units are different from both the mean and the data itself. For example, the mean of our NBA dataset is 77.98 inches. Because of this, we can say someone who is 80 inches tall is about two inches taller than the average NBA player.

However, because the formula for variance includes *squaring* the difference between the data and the mean, the variance is measured in *units squared*. This means that the variance for our NBA dataset is 13.32 inches squared.

This result is hard to interpret in context with the mean or the data because their units are different. This is where the statistic *standard deviation* is useful.

Standard deviation is computed by taking the square root of the variance.  $\sigma$  is the symbol commonly used for standard deviation. Conveniently,  $\sigma^2$  is the symbol commonly used for variance:

$$\sigma^2 = \left[ \sum (X_i - \mu)^2 \right] / N$$

In R, you can take the square root of a number using `^ 0.5` or `sqrt()`, up to you which one you prefer:

```
num <- 25
num_square_root <- num ^ 0.5
```

Copy to Clipboard

### Instructions

Checkpoint 1 Enabled

1.

We've written some code that calculates the variance of the NBA dataset and the OkCupid dataset.

The variances are stored in variables named `nba_variance` and `okcupid_variance`.

Calculate the standard deviation by taking the square root of `nba_variance` and store it in the variable `nba_standard_deviation`. Do the same for the variable `okcupid_standard_deviation`.

```

title: "Standard Deviation"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(dplyr)
library(tidyr)
```

```{r}
# Importing data and calculating variance
load("lesson_data.Rda")
variance <- function(x) mean((x-mean(x))^2)

nba_variance <- variance(nba_data)
okcupid_variance <- variance(okcupid_data)
```

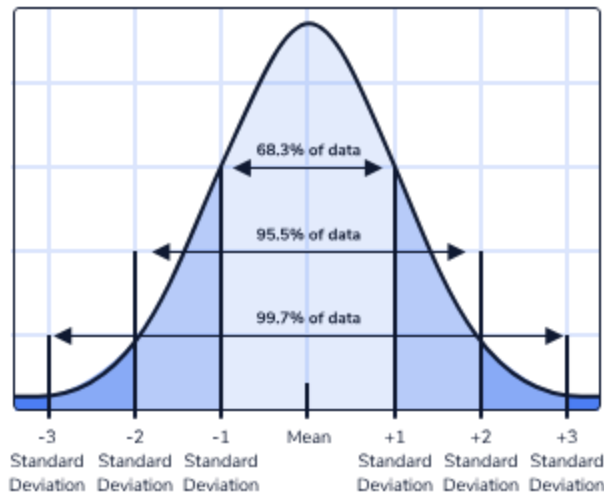
```{r}
# Change these variables to be the standard deviation of each dataset.
nba_standard_deviation <- nba_variance ^ .5
okcupid_standard_deviation <- okcupid_variance ^ .5

#IGNORE CODE BELOW HERE
print(paste("The standard deviation of the NBA dataset is ",nba_standard_deviation))
print(paste("The standard deviation of the OkCupid dataset is ",
okcupid_standard_deviation))
```
```

## Using Standard Deviation

Now that we're able to compute the standard deviation of a dataset, what can we do with it?

Now that our units match, our measure of spread is easier to interpret. By finding the number of standard deviations a data point is away from the mean, we can begin to investigate how unusual that datapoint truly is. In fact, you can usually expect around 68% of your data to fall within one standard deviation of the mean, 95% of your data to fall within two standard deviations of the mean, and 99.7% of your data to fall within three standard deviations of the mean.



If you have a data point that is over three standard deviations away from the mean, that's an incredibly unusual piece of data!

### Instructions

Checkpoint 1 Enabled

1.

Let's find out how many standard deviations away from the mean NBA great LeBron James is. To begin, let's find the difference between LeBron's height (80 inches) and the mean of each dataset.

Set `nba_difference` equal to 80 minus `nba_mean`.

Find the difference between LeBron's height and the OkCupid mean and store it in `okcupid_difference`.

The OkCupid dataset's mean is stored in `okcupid_mean`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

We now want to find out how many times the standard deviation goes into those differences.

Set `num_nba_deviations` equal to `nba_difference` divided by `nba_standard_deviation`.

Do a similar calculation for `num_okcupid_deviations`.

What does that first number tell you about how unusual LeBron James is in the NBA? What does the second number tell you about how unusual LeBron James is in the dating pool?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's check another NBA player. Earl Boykins is one of the smaller NBA players in history at 5'5" (65 inches).

Replace LeBron James' 80 inches with Earl Boykins' 65.

What can you say about how unusual Earl Boykins is with respect to the two different datasets?

We were surprised that Boykins wasn't more standard deviations away from the mean of the OkCupid dataset. Think about why he isn't more of an outlier in this dataset.

```

title: "Standard Deviation"
output: html_notebook

```{r}
# Importing data and calculating variance
load("lesson_data.Rda")
variance <- function(x) mean((x-mean(x))^2)

nba_mean <- mean(nba_data)
okcupid_mean <- mean(okcupid_data)
nba_standard_deviation <- sd(nba_data)
```

```
okcupid_standard_deviation <- sd(okcupid_data)

#Step 1: Calcualte the difference between the player's height and the means
nba_difference <- 0
okcupid_difference <- 0

#Step 2: Use the difference between the point and the mean to find how many standard
deviations the player is away from the mean.
num_nba_deviations <- 0
num_okcupid_deviations <- 0

#IGNORE CODE BELOW HERE
print(paste("Your basketball player is", num_nba_deviations,"standard deviations away from
the mean of NBA player heights,"))
print(paste("Your basketball player is ",num_okcupid_deviations," standard deviations away
from the mean of OkCupid profile heights"))
```

Review

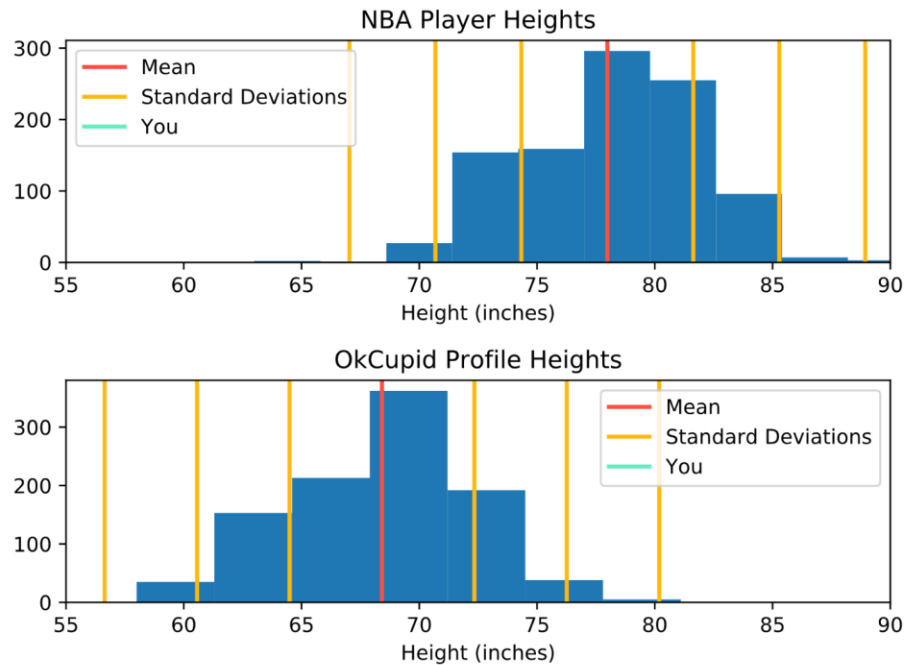
In the last exercise you saw that LeBron James was 0.55 standard deviations above the mean of NBA player heights. He's taller than average, but compared to the other NBA players, he's not absurdly tall.

However, compared to the OkCupid dating pool, he is extremely rare! He's almost three full standard deviations above the mean. You'd expect only about 0.15% of people on OkCupid to be more than 3 standard deviations away from the mean.

This is the power of standard deviation. By taking the square root of the variance, the standard deviation gives you a statistic about spread that can be easily interpreted and compared to the mean.

Instructions

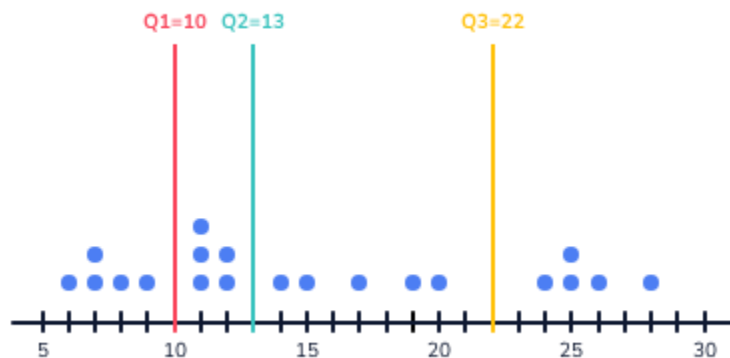
We've created a visualization that shows the mean, first, second, and third standard deviations of each dataset. Compare your height (in inches) to the heights of NBA players and OkCupid daters. You're likely below the first standard deviation for the NBA. Are you below the second? How unusual is your height compared to the OkCupid members?



Quartiles

A common way to communicate a high-level overview of a dataset is to find the values that split the data into four groups of equal size.

By doing this, we can then say whether a new datapoint falls in the first, second, third, or fourth quarter of the data.



The values that split the data into fourths are the *quartiles*.

Those values are called the first quartile (Q1), the second quartile (Q2), and the third quartile (Q3)

In the image above, Q1 is 10, Q2 is 13, and Q3 is 22. Those three values split the data into four groups that each contain five datapoints.

In this lesson, you will learn to calculate the quartiles by hand, and by using base R [functions](#)

Preview: Docs Loading link description

Instructions

In this lesson we'll be looking at a dataset about music. We've plotted a histogram of song lengths (measured in seconds) of 9,975 random songs.

Look up the length of a favorite song of yours. Do you think that song falls in the first, second, third or fourth quarter of the data?

For example, we've picked one of our favorite songs, *Chicago* by Sufjan Stevens. *Chicago* is 364 seconds long — we've plotted it as a blue vertical line. It looks like *Chicago* is in either the third or fourth quarter of the data, but it's hard to say for sure. Let's find the quartiles of the dataset!

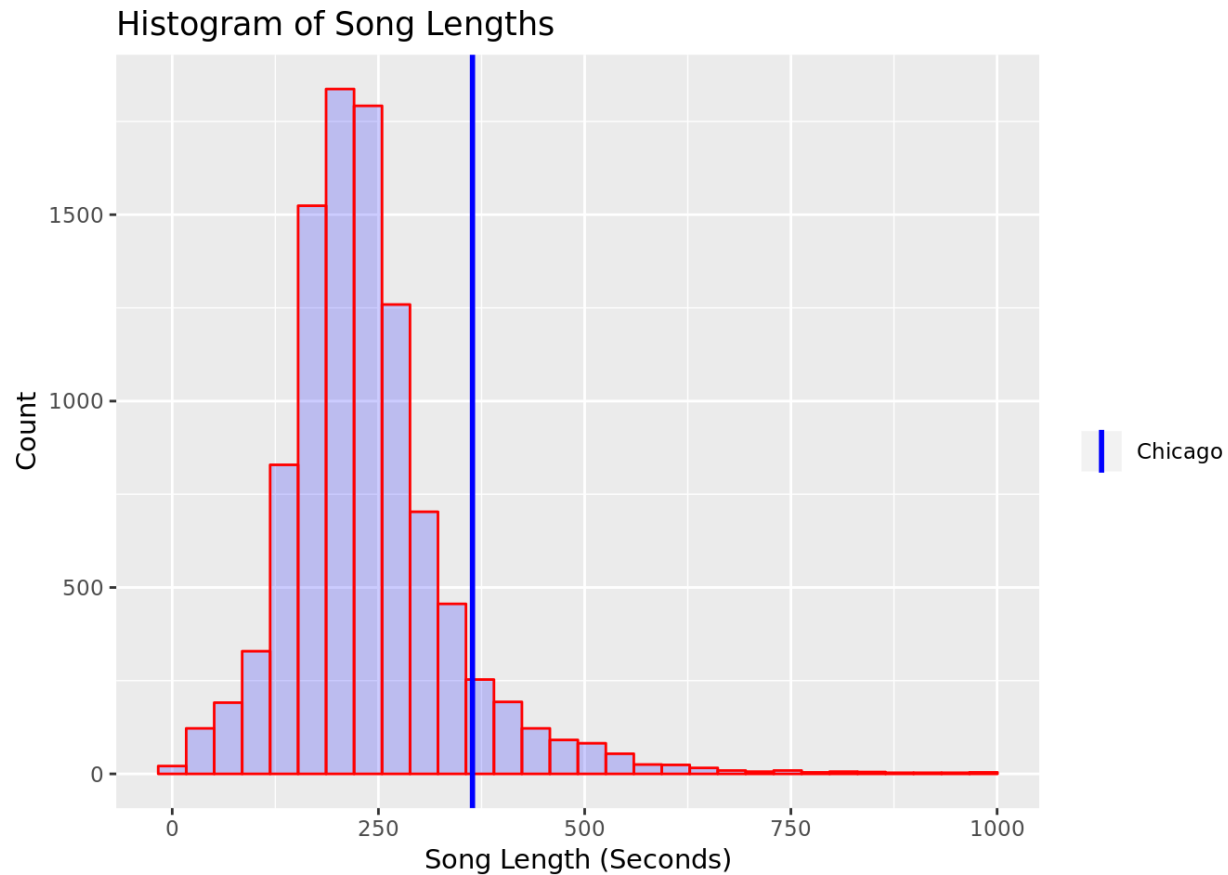
```
---
title: "Quartiles"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(ggplot2)
```

```{r}
load song data
load("songs.Rda")
```

```{r message=FALSE}
plot histogram
hist <- qplot(songs,
 geom="histogram",
 main = 'Histogram of Song Lengths',
 xlab = 'Song Length (Seconds)',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=364,
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 scale_colour_manual(name = "",
 labels =c("Chicago"),
 values=c("blue"))

hist
```
```



The Second Quartile

We'll come back to the music dataset in a bit, but let's first practice on a small dataset.

Let's begin by finding the second quartile (Q2). Q2 happens to be exactly the [median](#). Half of the data falls below Q2 and half of the data falls above Q2.

The first step in finding the quartiles of a dataset is to sort the data from smallest to largest. For example, below is an unsorted dataset:

`c(8,15,4,-108,16,23,42)`

After sorting the dataset, it looks like this:

`c(-108,4,8,15,16,23,42)`

Now that the list is sorted, we can find Q2. In the example dataset above, Q2 (and the median) is 15 — there are three points below 15 and three points above 15.

Even Number of Datapoints

You might be wondering what happens if there is an even number of points in the dataset. For example, if we remove the -108 from our dataset, it will now look like this:

`c(4,8,15,16,23,42)`

Q2 now falls somewhere between 15 and 16. There are a couple of different strategies that you can use to calculate Q2 in this situation. One of the more common ways is to take the average of those two numbers. In this case, that would be 15.5.

Recall that you can find the average of two numbers by adding them together and dividing by two.

Instructions

Checkpoint 1 Enabled

1.

We've included two small unsorted datasets named `dataset_one` and `dataset_two`.

We've also included, as a comment, the sorted version of the first dataset.

By looking at sorted version of `dataset_one`, find the second quartile of the dataset and store it in a variable named `dataset_one_q2`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the second quartile of the `dataset_two` and store it in a variable named `dataset_two_q2`.

Remember to sort the dataset. It might help to write out the sorted dataset as a comment!

Since there are an even number of datapoints in this dataset, the second quartile will fall between two points. The second quartile will be the average of those two points.

```
---
title: "Quartiles"
output: html_notebook
---

```{r}
dataset_one <- c(50, 10, 4, -3, 4, -20, 2)
sorted dataset_one: c(-20, -3, 2, 4, 4, 10, 50)

dataset_two <- c(24, 20, 1, 45, -15, 40)

define the second quartile of both datasets here:

```

```{r}
ignore the code below here:

tryCatch(print(paste("The second quartile of dataset one is", dataset_one_q2)),
error=function(e) {print("You haven't defined dataset_one_q2")})

tryCatch(print(paste("The second quartile of dataset two is", dataset_two_q2)),
error=function(e) {print("You haven't defined dataset_two_q2")})
```
```

Q1 and Q3

Now that we've found Q2, we can use that value to help us find Q1 and Q3. Recall our demo dataset:

`c(-108,4,8,15,16,23,42)`

In this example, Q2 is 15. To find Q1, we take all of the data points smaller than Q2 and find the median of *those* points. In this case, the points smaller than Q2 are:

`c(-108,4,8)`

The median of that smaller dataset is 4. That's Q1!

To find Q3, do the same process using the points that are larger than Q2. We have the following points:

`c(16,23,42)`

The median of *those* points is 23. That's Q3! We now have three points that split the original dataset into groups of four equal sizes.

Instructions

Checkpoint 1 Enabled

1.

Find the first quartile of `dataset_one` and store it in a variable named `dataset_one_q1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the third quartile of `dataset_one` and store it in a variable named `dataset_one_q3`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Find Q1 and Q3 of `dataset_two`. Store the values in variables named `dataset_two_q1` and

`dataset_two_q3`.

```
---
title: "Quartiles"
output: html_notebook
---

```{r}
dataset_one <- c(50, 10, 4, -3, 4, -20, 2)
sorted dataset_one: [-20, -3, 2, 4, 4, 10, 50]

dataset_two <- c(24, 20, 1, 45, -15, 40)

dataset_one_q2 <- 4
dataset_two_q2 <- 22

define the first and third quartile of both datasets here:
dataset_one_q1 <- -3
dataset_one_q3 <- 10

dataset_two_q1 <- 1
dataset_two_q3 <- 40
```

```{r}
ignore the code below here:

tryCatch(print(paste("The first quartile of dataset one is",dataset_one_q1)),
error=function(e) {print("You haven't defined dataset_one_q1")})

tryCatch(print(paste("The third quartile of dataset one is",dataset_one_q3)),
error=function(e) {print("You haven't defined dataset_one_q3")})

tryCatch(print(paste("The first quartile of dataset two is",dataset_two_q1)),
error=function(e) {print("You haven't defined dataset_two_q1")})

tryCatch(print(paste("The third quartile of dataset two is",dataset_two_q3)),
error=function(e) {print("You haven't defined dataset_two_q3")})
```

```{r}
ignore the code below here:
```

```
tryCatch(print(paste("The first quartile of dataset one is",dataset_one_q1)),
error=function(e) {print("You haven't defined dataset_one_q1")})

tryCatch(print(paste("The third quartile of dataset one is",dataset_one_q3)),
error=function(e) {print("You haven't defined dataset_one_q3")})

tryCatch(print(paste("The first quartile of dataset two is",dataset_two_q1)),
error=function(e) {print("You haven't defined dataset_two_q1")})

tryCatch(print(paste("The third quartile of dataset two is",dataset_two_q3)),
error=function(e) {print("You haven't defined dataset_two_q3")})
````
```

Method Two: Including Q2

You just learned a commonly used method to calculate the quartiles of a dataset. However, there is another method that is equally accepted that results in different values!

Note that there is no universally agreed upon method of calculating quartiles, and as a result, two different tools might report different results.

The second method includes Q2 when trying to calculate Q1 and Q3. Let's take a look at an example:

`c(-108,4,8,15,16,23,42)`

Using the first method, we found Q1 to be 4. When looking at all of the points below Q2, we excluded Q2. Using this second method, we *include* Q2 in each half.

For example, when calculating Q1 using this new method, we would now find the median of this dataset:

`c(-108,4,8,15)`

Using this method, Q1 is 6.

Instructions

Checkpoint 1 Enabled

1.

Create a variable named `dataset_one_q1` and set it equal to the first quartile of dataset one. This time, use the second method of finding quartiles.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable named `dataset_one_q3` and set it equal to the third quartile of dataset one. Again, use the second method of finding quartiles.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create two variables named `dataset_two_q1` and `dataset_two_q3` and set them equal to the first and third quartile of dataset two.

Use the second method of calculating quartiles. Since Q2 fell between two data points, this method is no different than the first method!

```
---
title: "Quartiles"
output: html_notebook
---

````{r}
dataset_one <- c(50, 10, 4, -3, 4, -20, 2)
sorted dataset_one: [-20, -3, 2, 4, 4, 10, 50]

dataset_two <- c(24, 20, 1, 45, -15, 40)

dataset_one_q2 <- 4
dataset_two_q2 <- 22
```

```
define the first and third quartile of both datasets here:
dataset_one_q1 <- -0.5
dataset_one_q3 <- 7

dataset_two_q1 <- 1
dataset_two_q3 <- 40
```

```{r}
ignore the code below here:

tryCatch(print(paste("The first quartile of dataset one is",dataset_one_q1)),
error=function(e) {print("You haven't defined dataset_one_q1")})

tryCatch(print(paste("The third quartile of dataset one is",dataset_one_q3)),
error=function(e) {print("You haven't defined dataset_one_q3")})

tryCatch(print(paste("The first quartile of dataset two is",dataset_two_q1)),
error=function(e) {print("You haven't defined dataset_two_q1")})

tryCatch(print(paste("The third quartile of dataset two is",dataset_two_q3)),
error=function(e) {print("You haven't defined dataset_two_q3")})
```
```

Quartiles in R

We were able to find quartiles manually by looking at the dataset and finding the correct division points. But that gets much harder when the dataset starts to get bigger. Luckily, there is a function in base R that will find the quartiles for you.

The base R function that we'll be using is named `quantile()`. You can learn more about quartiles in our [quantiles lesson](#), but for right now all you need to know is that a quartile is a specific kind of quantile.

The code below calculates the third quartile of the given dataset:

```
dataset <- c(50, 10, 4, -3, 4, -20, 2)
third_quartile <- quantile(dataset, 0.75)
```

Copy to Clipboard

The `quantile()` function takes two parameters. The first is the dataset you're interested in. The second is a number between 0 and 1. Since we calculated the third quartile, we used `0.75` — we want the point that splits the first 75% of the data from the rest.

For the second quartile, we'd use `0.5`. This will give you the point that 50% of the data is below and 50% is above.

Notice that the dataset doesn't need to be sorted for R's function to work!

Instructions

Checkpoint 1 Enabled

1.

We've brought back our music dataset. The lengths of 9,975 songs (in seconds) are stored in a variable named `songs`. Use the `quantile()` function to find the first quartile. Store the result in a variable named `songs_q1`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the second and third quartile of the dataset and store the values in two variables named `songs_q2` and `songs_q3`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Look up the length of your favorite song in seconds. Store that value in a variable named `favorite_song`. Does that song fall in the first, second, third, or fourth quarter of the data? Create a variable named `quarter`. Set `quarter` equal to 1 if your favorite song falls in the first quarter of the data. Set it equal to 2 if your song falls in the second fourth. Set it equal to 3 if your song falls in the third fourth. And set it to 4 if your song falls in the final fourth of the data.

```
---
title: "Quartiles"
output: html_notebook
---

```{r}
load song data
load("songs.Rda")
```

```{r}
create the variables songs_q1, songs_q2, and songs_q3 here:
songs_q1 <- quantile(songs,0.25)
songs_q2 <- quantile(songs,0.5)
songs_q3 <- quantile(songs,0.75)
```

```{r}
create the variables favorite_song and quarter here:
favorite_song <- 300
quarter <- 4
```

```{r}
ignore the code below here:

tryCatch(print(paste("The first quartile of dataset one is",songs_q1)), error=function(e)
{print("You haven't defined songs_q1")})

tryCatch(print(paste("The second quartile of dataset two is",songs_q2)), error=function(e)
{print("You haven't defined songs_q2")})

tryCatch(print(paste("The third quartile of dataset one is",songs_q3)), error=function(e)
{print("You haven't defined songs_q3")})
```
```

Quartiles Review

Great work! You now know how to calculate the quartiles of any dataset by hand and with R.

Quartiles are some of the most commonly used descriptive statistics. For example, You might see schools or universities think about quartiles when considering which students to accept. Businesses might compare their revenue to other companies by looking at quartiles.

In fact quartiles are so commonly used that the three quartiles, along with the minimum and the maximum values of a dataset, are called the five-number summary of the dataset. These five numbers help you quickly get a sense of the range, centrality, and spread of the dataset.

Instructions

We've plotted the first, second, and third quartiles on the histogram for our music dataset. Are they where you expected to see them?

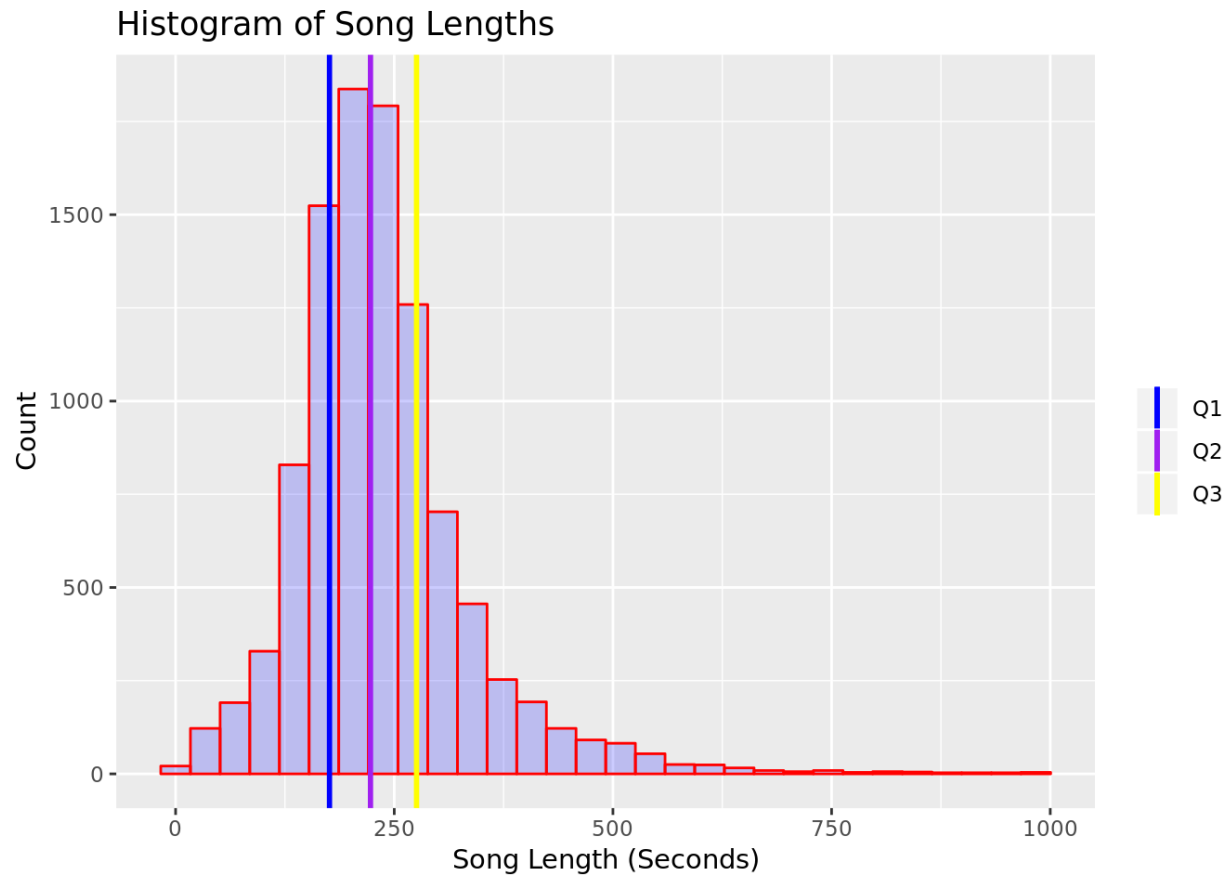
```
---
title: "Quartiles"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(ggplot2)
```

```{r}
load song data
load("songs.Rda")
```

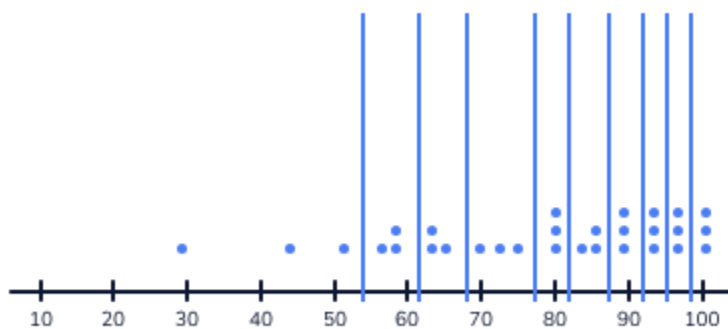
```{r message=FALSE}
plot histogram
hist <- qplot(songs,
 geom="histogram",
 main = 'Histogram of Song Lengths',
 xlab = 'Song Length (Seconds)',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(songs,0.25),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.5),
 color=I("purple")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.75),
 color=I("yellow")),
 linetype="solid",
 size=1,
 show.legend=T) +
 scale_colour_manual(name = "",
 labels =c("Q1","Q2","Q3"),
 values=c("blue","purple","yellow"))

hist
```
```



Quantiles

Quantiles are points that split a dataset into groups of equal size. For example, let's say you just took a test and wanted to know whether you're in the top 10% of the class. One way to determine this would be to split the data into ten groups with an equal number of datapoints in each group and see which group you fall into.



There are nine values that split the dataset into ten groups of equal size — each group has 3 different test scores in it.

Those nine values that split the data are quantiles! Specifically, they are the 10-quantiles, or deciles.

You can find any number of quantiles. For example, if you split the dataset into 100 groups of equal size, the 99 values that split the data are the 100-quantiles, or percentiles.

The [quartiles](#) are some of the most commonly used quantiles. The quartiles split the data into four groups of equal size.

In this lesson, we'll show you how to calculate quantiles using R and discuss some of the most commonly used quantiles.

Instructions

We've imported a dataset of song lengths (measured in seconds). We've drawn a few histograms showing different quantiles.

What do you think a histogram that shows the 100-quantiles would look like?

```
---
title: "Quantiles"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(ggplot2)
```

```{r}
load song data
load("songs.Rda")
```

```{r message=FALSE, echo=FALSE}
4 quantiles
hist <- qplot(songs,
 geom="histogram",
 main = '4-Quantiles',
 xlab = 'Song Length (Seconds)',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(songs,0.25),
 color=I("blue"))),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.5),
 color=I("blue"))),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.75),
 color=I("blue"))),
 linetype="solid",
 size=1,
 show.legend=T)

hist
```

```{r message=FALSE, echo=FALSE}
```



```

5 quantiles
hist <- qplot(songs,
 geom="histogram",
 main = '5-Quantiles',
 xlab = 'Song Length (Seconds)',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(songs,0.2),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.4),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.6),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(songs,0.8),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T)

hist
```



```

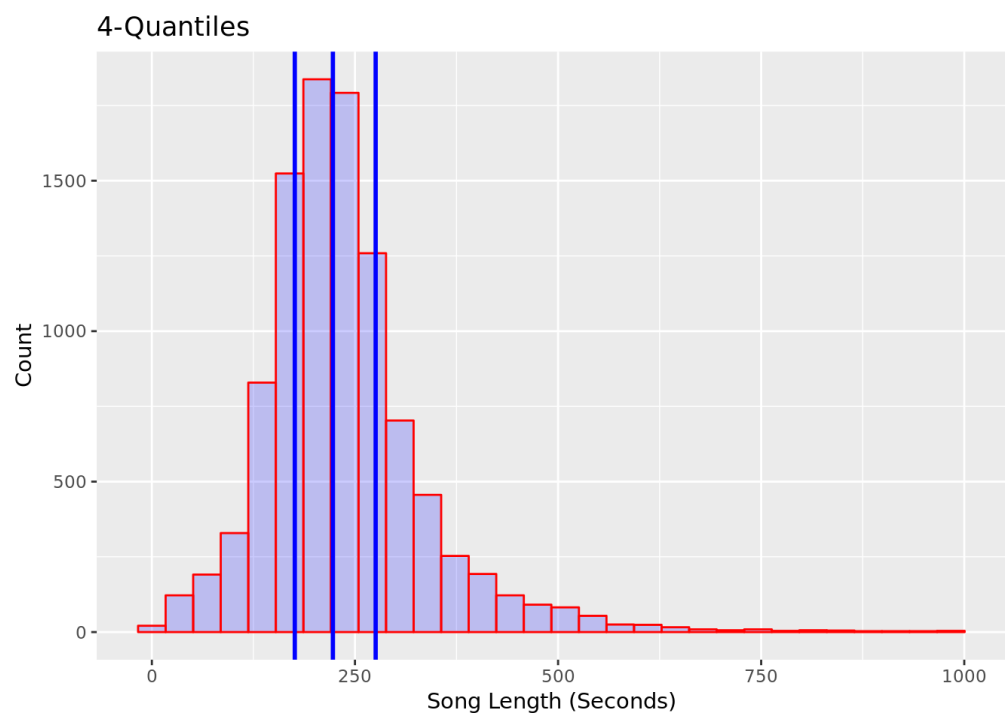
```{r message=FALSE, echo=FALSE}
# 10 quantiles
hist <- qplot(songs,
              geom="histogram",
              main = '10-Quantiles',
              xlab = 'Song Length (Seconds)',
              ylab = 'Count',
              fill=I("blue"),
              col=I("red"),
              alpha=I(.2)) +
  geom_vline(aes(xintercept=quantile(songs,0.1),
                color=I("blue")),
             linetype="solid",
             size=1,
             show.legend=T) +
  geom_vline(aes(xintercept=quantile(songs,0.2),
                color=I("blue")),
             linetype="solid",
             size=1,
             show.legend=T) +
  geom_vline(aes(xintercept=quantile(songs,0.3),
                color=I("blue")),
             linetype="solid",
             size=1,
             show.legend=T) +
  geom_vline(aes(xintercept=quantile(songs,0.4),
                color=I("blue")),
             linetype="solid",
             size=1,
             show.legend=T)

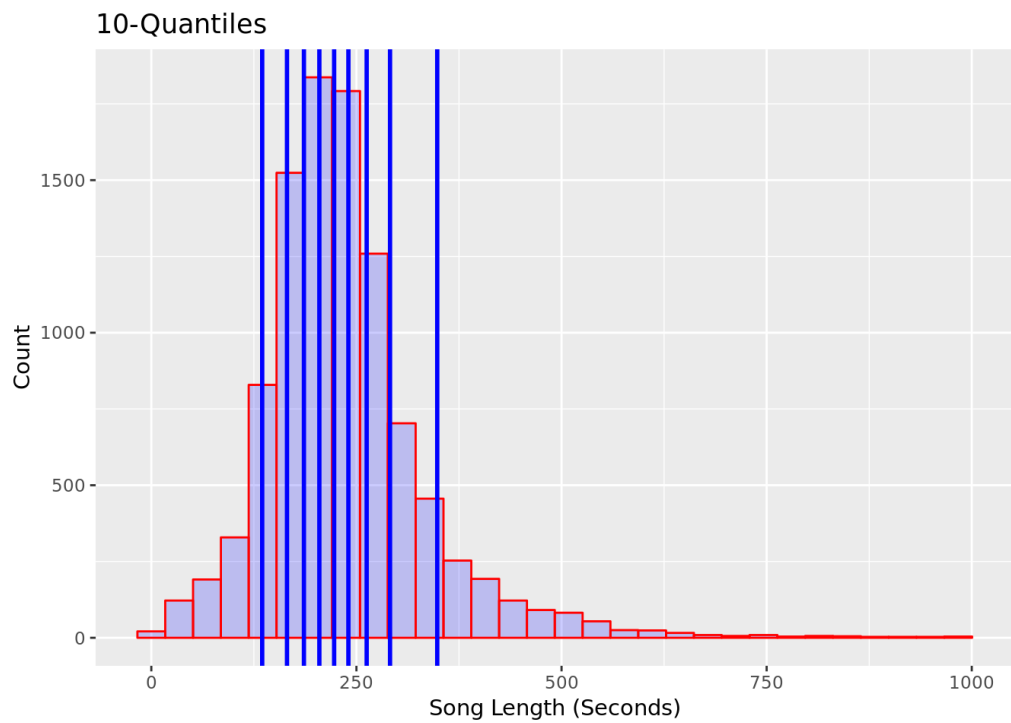
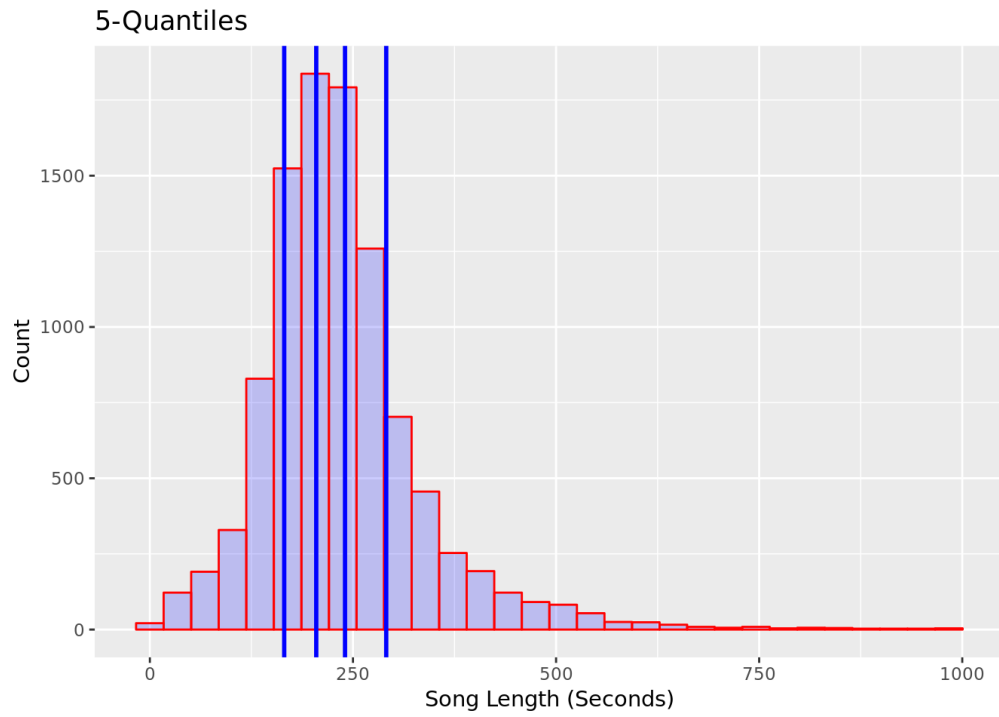
```


```

```
size=1,
show.legend=T) +
geom_vline(aes(xintercept=quantile(songs,0.5),
color=I("blue")),
linetype="solid",
size=1,
show.legend=T) +
geom_vline(aes(xintercept=quantile(songs,0.6),
color=I("blue")),
linetype="solid",
size=1,
show.legend=T) +
geom_vline(aes(xintercept=quantile(songs,0.7),
color=I("blue")),
linetype="solid",
size=1,
show.legend=T) +
geom_vline(aes(xintercept=quantile(songs,0.8),
color=I("blue")),
linetype="solid",
size=1,
show.legend=T) +
geom_vline(aes(xintercept=quantile(songs,0.9),
color=I("blue")),
linetype="solid",
size=1,
show.legend=T)

hist
``,``
```





## Quantiles in R

Base R has a function named `quantile()` that will quickly calculate the quantiles of a dataset for you.

`quantile()` takes two parameters. The first is the dataset that you are using. The second parameter is a single number or a vector of numbers between 0 and 1. These numbers represent the places in the data where you want to split.

For example, if you only wanted the value that split the first 10% of the data apart from the remaining 90%, you could use this code:

```
dataset <- c(5, 10, -20, 42, -9, 10)
ten_percent <- quantile(dataset, 0.10)
```

Copy to Clipboard

`ten_percent` now holds the value `-14.5`. This result *technically* isn't a quantile, because it isn't splitting the dataset into groups of equal sizes — this value splits the data into one group with 10% of the data and another with 90%.

However, it would still be useful if you were curious about whether a data point was in the bottom 10% of the dataset.

### Instructions

Checkpoint 1 Enabled

1.

The dataset containing information about the lengths of songs is stored in a variable named `songs`. Create a variable named `twenty_third_percentile` that contains the value that splits the first 23% of the data from the rest of the data.

```

title: "Quantiles"
output: html_notebook

```{r}
# load song data
load("songs.Rda")
```

```{r}
# define twenty_third_percentile here:
twenty_third_percentile <- quantile(songs,0.23)
```

```{r}
# ignore the code below here:

tryCatch(print(paste("The value that splits 23% of the data
is",twenty_third_percentile)), error=function(e) {print("You haven't defined
twenty_third_percentile.")})
```
```

## Many Quantiles

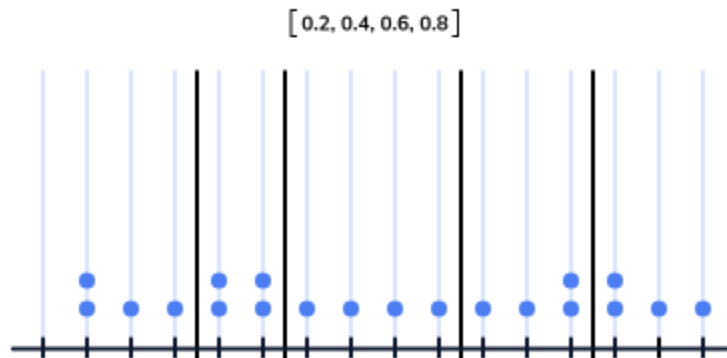
In the last exercise, we found a single “quantile” — we split the first 23% of the data away from the remaining 77%.

However, quantiles are usually a set of values that split the data into groups of equal size. For example, if you wanted to get the 5-quantiles, or the four values that split the data into five groups of equal size, you could use this code:

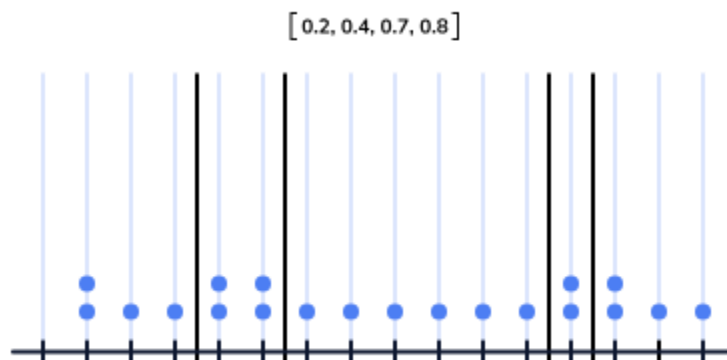
```
dataset <- c(5, 10, -20, 42, -9, 10)
ten_percent <- quantile(dataset, c(0.2, 0.4, 0.6, 0.8))
```

Copy to Clipboard

Note that we had to do a little math in our head to make sure that the values `c(0.2, 0.4, 0.6, 0.8)` split the data into groups of equal size. Each group has 20% of the data.



If we used the values `c(0.2, 0.4, 0.7, 0.8)`, the function would return the four values at those split points. However, those values wouldn't split the data into five equally sized groups. One group would only have 10% of the data, and another group would have 30% of the data!



## Instructions

Checkpoint 1 Enabled

1.

Create a variable named `quantiles` that contains the quantiles of the `songs` dataset.

The quantiles of a dataset split the data into four groups of equal size. Each group should have 25% of the data, so you'll want to use `c(0.25, 0.5, 0.75)` as the second parameter to the `quantile()` function.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable named `deciles`. `deciles` should store the values that split the dataset into ten groups of equal size. Each group should have 10% of the data.

The first value should be at 10% of the data. The next value should be at 20% of the data. The final value should be at 90% of the data.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Look at the printout of the deciles. If you had a song that was 170 seconds long, what tenth of the dataset would it fall in?

Create a variable named `tenth` and set it equal to 1 if you think the 170 second song would fall in the first tenth of the data. Set it equal to 2 if you think the song would fall in the second tenth of the data. If you think the song would fall in the final tenth of the data, set `tenth` equal to 10.

```

title: "Quantiles"
output: html_notebook

```{r}
# load song data
load("songs.Rda")
```

```{r}
# define quartiles and deciles here:
quartiles <- quantile(songs,c(0.25,0.5,0.75))
deciles <- quantile(songs,c(0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9))
```

```{r}
# define tenth here:
tenth <- 3
```

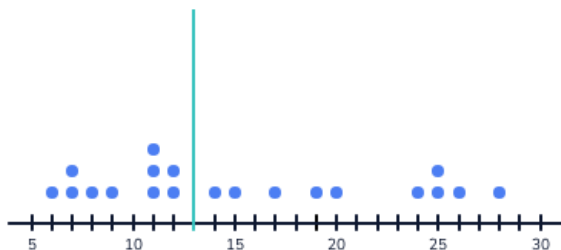
```{r}
# ignore the code below here:

tryCatch(print(paste(c("The quartiles are",quartiles,collapse=" "))),
error=function(e) {print("You haven't defined quartiles.")})

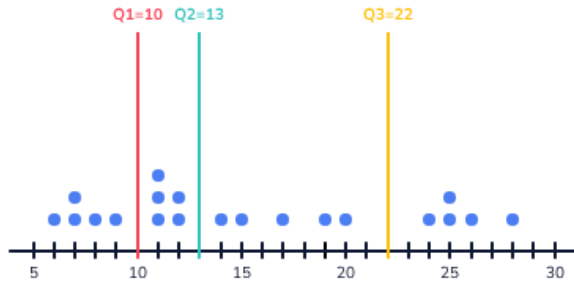
tryCatch(print(paste(c("The deciles are",deciles,collapse=" "))), error=function(e)
{print("You haven't defined deciles.")})
```
```

## Common Quantiles

One of the most common quantiles is the 2-quantile. This value splits the data into two groups of equal size. Half the data will be above this value, and half the data will be below it. This is also known as the [median](#)!



The 4-quantiles, or the [quartiles](#), split the data into four groups of equal size. We found the quartiles in the previous exercise.



Finally, the percentiles, or the values that split the data into 100 groups, are commonly used to compare new data points to the dataset. You might hear statements like “You are above the 80th percentile in height”. This means that your height is above whatever value splits the first 80% of the data from the remaining 20%.

### Instructions

Checkpoint 1 Enabled

1.

We won't make you calculate all 99 percentiles, but let's take a look at one. Find the value that separates the first 32% of the data from the rest.

Store that value in a variable named `percentile`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Look at the printout. If you had a song that was exactly three minutes long, is that song above or below the 32nd percentile?

Create a variable named `answer` and set it equal to either `"above"` or `"below"`. Don't forget to include the quotes!

```

title: "Quantiles"
output: html_notebook

```{r}
# load song data
load("songs.Rda")
```

```{r}
# define percentile and answer here:
percentile <- quantile(songs,0.32)
answer <- "below"
```

```{r}
# ignore the code below here:

tryCatch(print(paste("Your percentile is",percentile)), error=function(e)
{print("You haven't defined percentile")})
```

Quantiles Review

Nice work! Here are some of the major takeaways about quantiles:

Quantiles are values that split a dataset into groups of equal size.

If you have n quantiles, the dataset will be split into $n+1$ groups of equal size.

The median is a quantile. It is the only 2-quantile. Half the data falls below the median and half falls above the median.

Quartiles and percentiles are other common quantiles. Quartiles split the data into 4 groups while percentiles split the data into 100 groups.

Instructions

To the right, we've shown three different histograms along with the deciles. Each histogram shows the SAT scores of the students that a *fake* university has accepted.

If you had an SAT score of 1350, which tenth of the data would you be in for each school? Which schools should you apply to? Would any of the schools be unrealistic options?

```
---
title: "Quantiles"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(ggplot2)
```

```{r}
load school data
load("school_one.Rda")
load("school_two.Rda")
load("school_three.Rda")
```

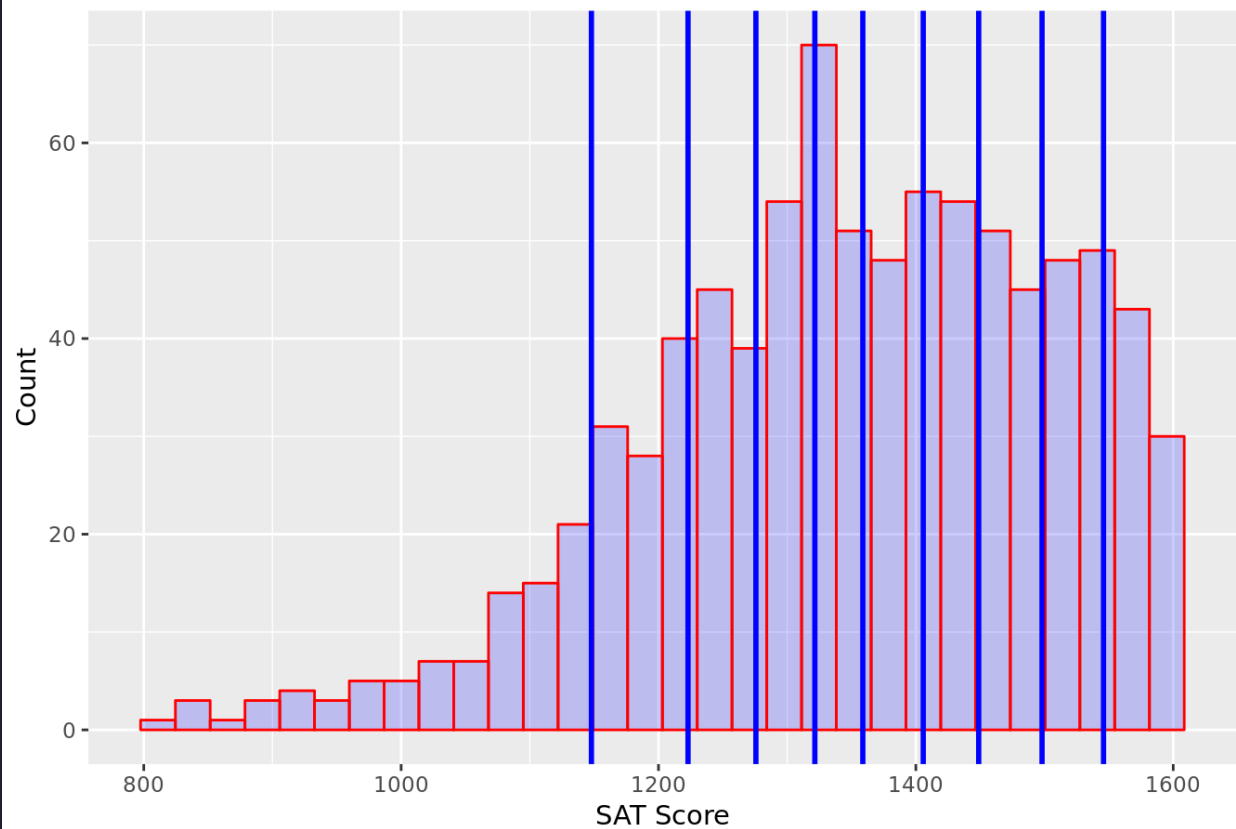
```{r message=FALSE, echo=FALSE}
plot histogram and deciles for school one
hist <- qplot(school_one,
 geom="histogram",
 main = 'School One',
 xlab = 'SAT Score',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(school_one,0.1),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
```



```
geom_vline(aes(xintercept=quantile(school_one,0.2),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.3),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.4),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.5),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.6),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.7),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.8),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_one,0.9),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T)
```

```
hist
` ``
```

## School One



```
``{r message=FALSE, echo=FALSE}
plot histogram and deciles for school two
hist <- qplot(school_two,
 geom="histogram",
 main = 'School Two',
 xlab = 'SAT Score',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(school_two,0.1),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_two,0.2),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_two,0.3),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_two,0.4),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
```

```

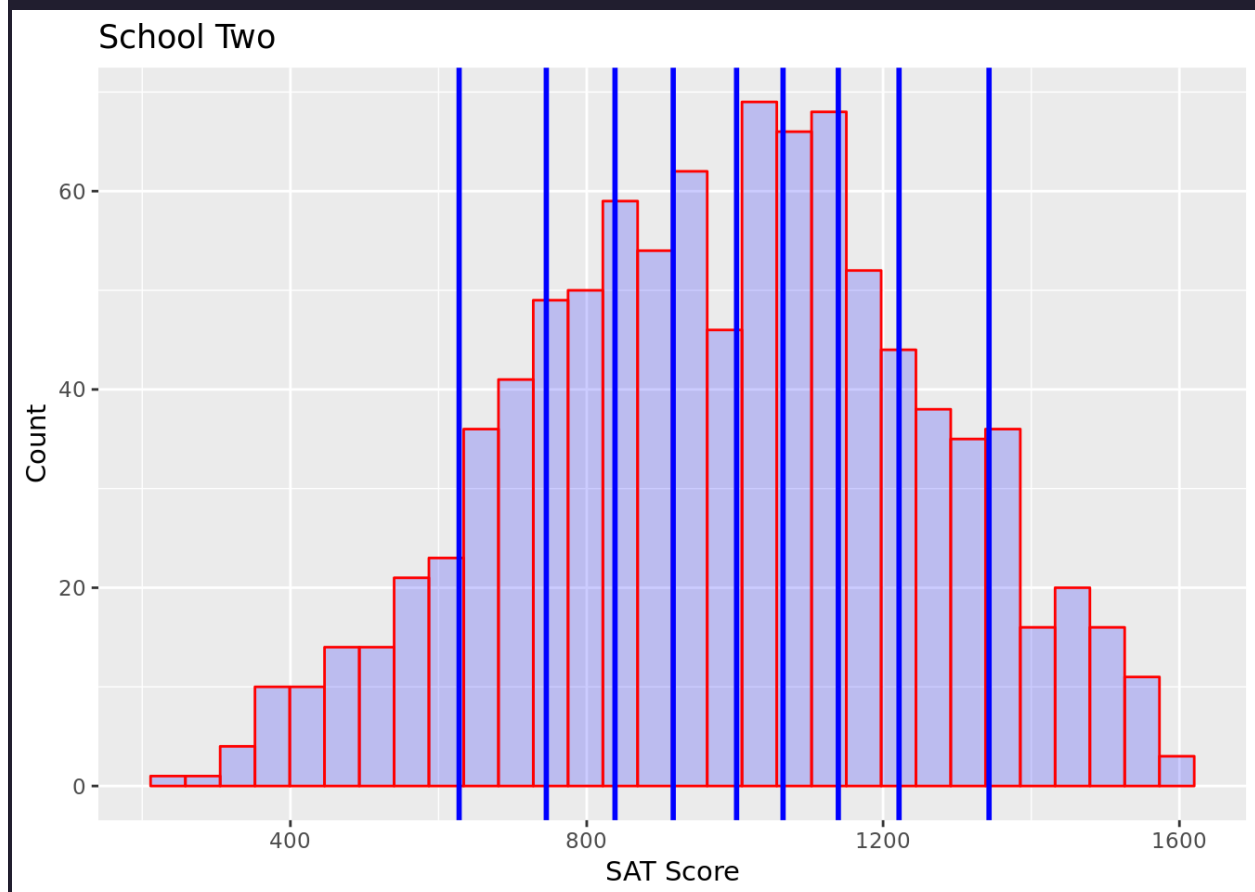
geom_vline(aes(xintercept=quantile(school_two,0.5),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_two,0.6),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_two,0.7),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_two,0.8),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
geom_vline(aes(xintercept=quantile(school_two,0.9),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T)

```

```

hist
```

```



```

```{r message=FALSE, echo=FALSE}
plot histogram and deciles for school three

```

```

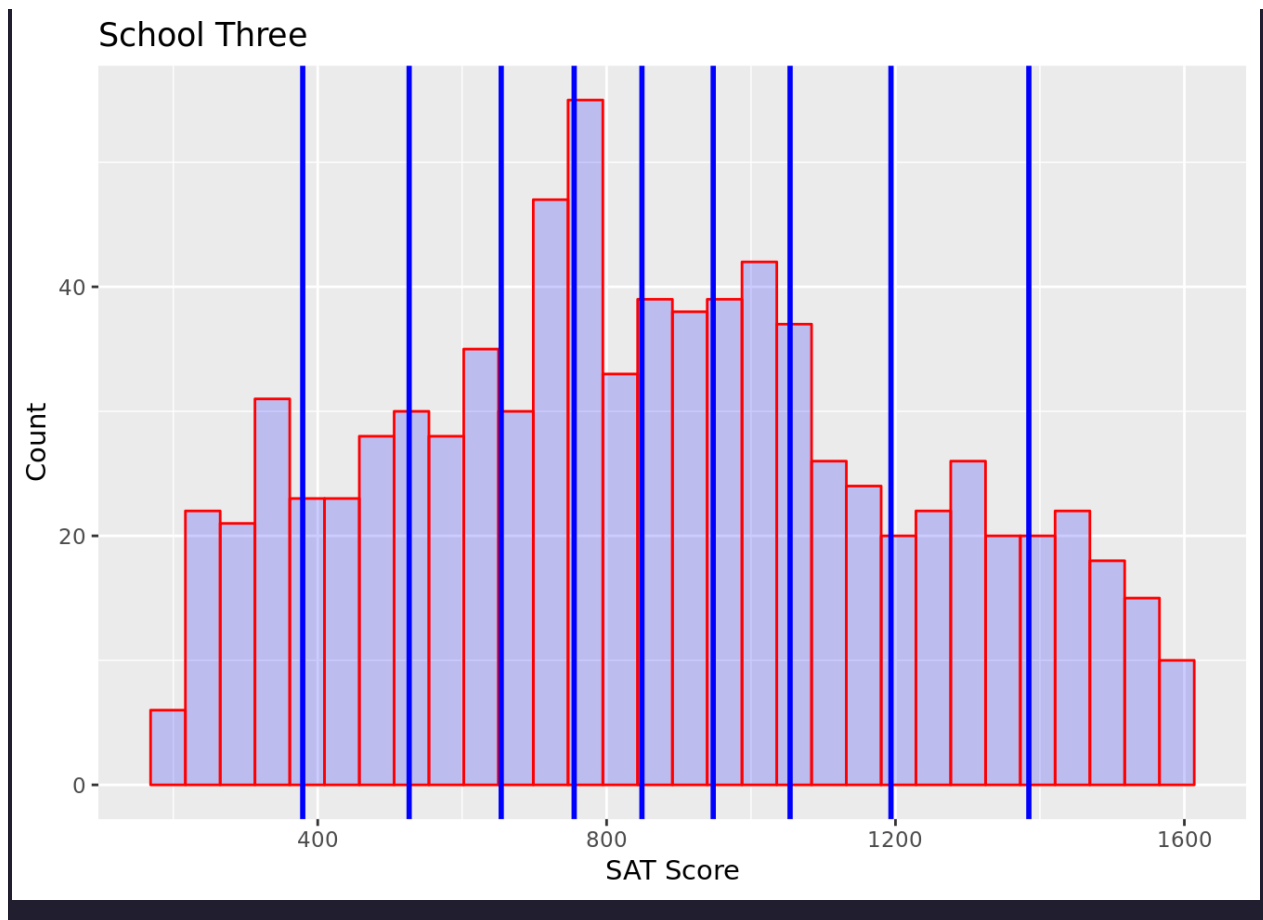
hist <- qplot(school_three,
 geom="histogram",
 main = 'School Three',
 xlab = 'SAT Score',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2)) +
 geom_vline(aes(xintercept=quantile(school_three,0.1),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.2),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.3),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.4),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.5),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.6),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.7),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.8),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T) +
 geom_vline(aes(xintercept=quantile(school_three,0.9),
 color=I("blue")),
 linetype="solid",
 size=1,
 show.legend=T)

```

```

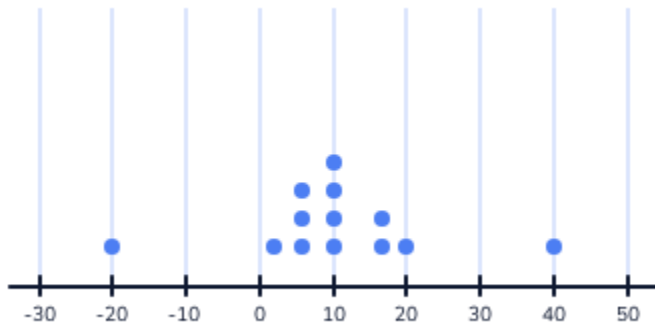
hist
```

```



Range Review

One of the most common statistics to describe a dataset is the *range*. The range of a dataset is the difference between the maximum and minimum values. While this descriptive statistic is a good start, it is important to consider the impact outliers have on the results:



In this image, most of the data is between 0 and 15. However, there is one large negative outlier (-20) and one large positive outlier (40). This makes the range of the dataset 60 (The difference between 40 and -20). That's not very representative of the spread of the majority of the data!

The *interquartile range* (IQR) is a descriptive statistic that tries to solve this problem. The IQR ignores the tails of the dataset, so you know the range around-which your data is centered.

In this lesson, we'll teach you how to calculate the interquartile range and interpret it.

Instructions

Checkpoint 1 Enabled

1.

We've imported a dataset of song lengths (measured in seconds) and plotted a histogram.

It looks like there are some outliers — this might be a good opportunity to use the IQR.

Before we do that, let's calculate the range. We've found the maximum and minimum values of the dataset and stored them in variables named `maximum` and `minimum`.

Create a variable named `song_range` and set it equal to the difference between the maximum and the minimum.

```
---
title: "Interquartile Range"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(ggplot2)
```

```{r}
load song data
load("songs.Rda")
```

```{r}
find maximum and minimum song lengths
maximum <- max(songs)
minimum <- min(songs)
```

```{r}
create variable song_range here:
song_range <- maximum - minimum
```
```

```

```{r message=FALSE, echo=FALSE}
plot histogram
hist <- qplot(songs,
 geom="histogram",
 main = 'Histogram of Song Lengths',
 xlab = 'Song Length (Seconds)',
 ylab = 'Count',
 fill=I("blue"),
 col=I("red"),
 alpha=I(.2))

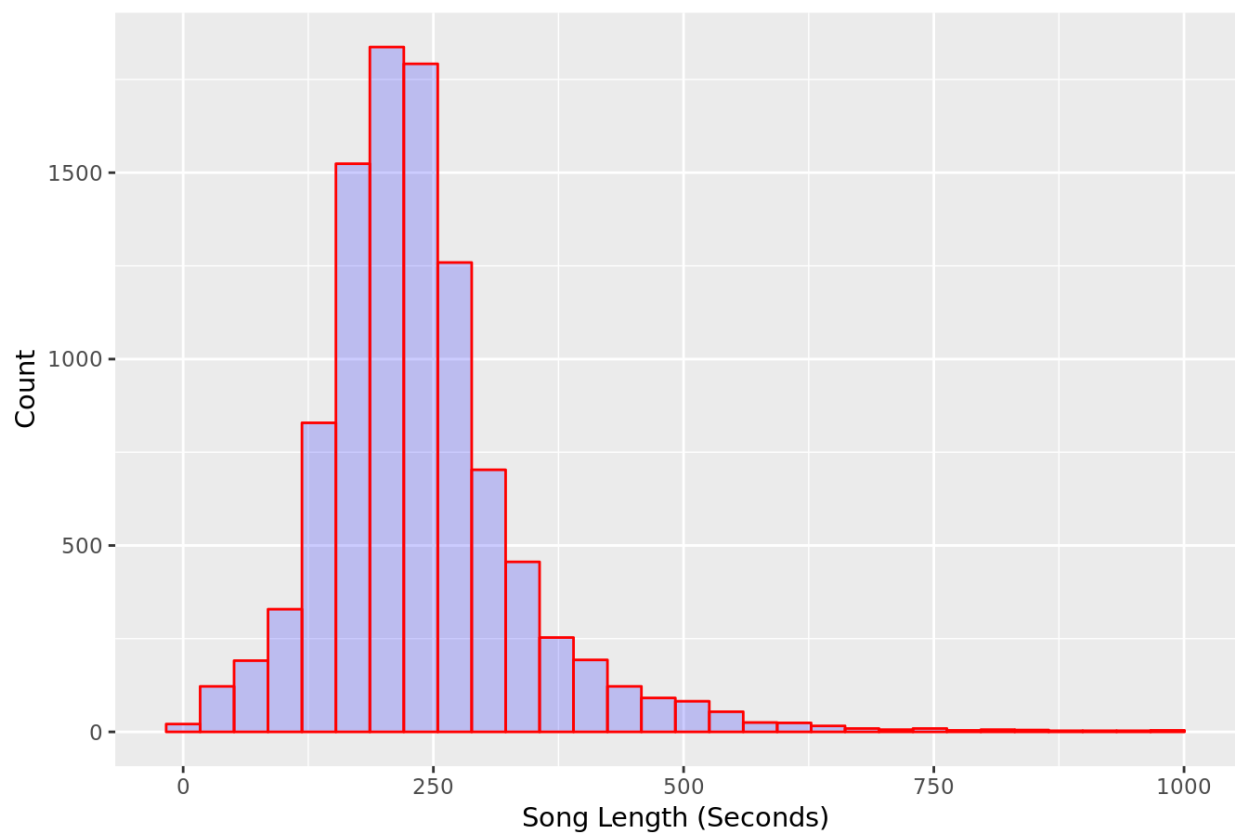
hist
```

```{r}
ignore the code below here:

tryCatch(print(paste("The range of the dataset is",song_range)), error=function(e)
{print("You haven't defined the variable song_range yet")})
```

```

Histogram of Song Lengths

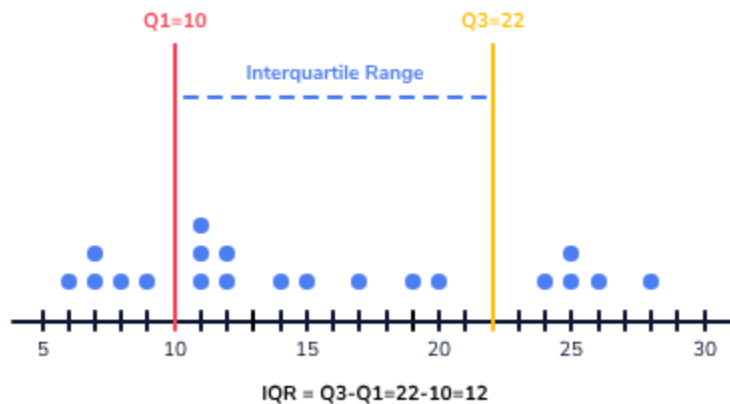


Quartiles

The interquartile range is the difference between the third quartile (Q3) and the first quartile (Q1). If you need a refresher on quartiles, you can take a look at our [lesson](#).

For now, all you need to know is that the first quartile is the value that separates the first 25% of the data from the remaining 75%.

The third quartile is the opposite — it separates the first 75% of the data from the remaining 25%.



The interquartile range is the difference between these two values.

Instructions

Checkpoint 1 Enabled

1.

We've calculated the first quartile of `songs` and stored it in the variable `q1`.

Calculate the third quartile and store it in a variable named `q3`.

To calculate the third quartile, call the same function, but change the second argument to `0.75`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now that we have both the first quartile and the third quartile, let's calculate the IQR.

Create a variable named `interquartile_range` and set it equal to the difference between `q3` and `q1`.

```
---
title: "Interquartile Range"
output: html_notebook
---

```{r}
load song data
load("songs.Rda")
```

```{r}
find the first quartile
q1 <- quantile(songs, 0.25)
```
```



```

```{r}
calculate the third quartile here:
q3 <- quantile(songs,0.75)
```

```{r}
calculate the interquartile range here:
interquartile_range <- q3-q1
```

```{r}
ignore the code below here:

tryCatch(print(paste("The first quartile of the dataset is",q1)), error=function(e)
{print("You haven't defined q1 yet")})

tryCatch(print(paste("The third quartile of the dataset is",q3)), error=function(e)
{print("You haven't defined q3 yet")})

tryCatch(print(paste("The IQR of the dataset is",interquartile_range)),
error=function(e) {print("You haven't defined interquartile_range yet")})
```

```

IQR in R

In the last exercise, we calculated the IQR by finding the quartiles using R and finding the difference ourselves. The stats library has a function that can calculate the IQR all in one step.

The `IQR()` function takes a dataset as a parameter and returns the Interquartile Range.

```

dataset = c(4, 10, 38, 85, 193)
interquartile_range = IQR(dataset)

```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Let's calculate the IQR again, but this time, use the stats function.

Create a variable named `interquartile_range` and set it equal to the result of calling `IQR()` using `songs` as an argument.

```

---
title: "Interquartile Range"
output: html_notebook
---

```{r}
load song data
load("songs.Rda")
```

```{r}
create the variable interquartile_range here
interquartile_range <- IQR(songs)
```

```

```
```{r}
ignore the code below here:

tryCatch(print(paste("The IQR of the dataset is", interquartile_range)),
error=function(e) {print("You haven't defined interquartile_range yet")})
```
```

Review

Nice work! You can now calculate the Interquartile Range of a dataset using R. The main takeaway of the IQR is that it is a statistic, like the range, that helps describe the spread of the center of the data.

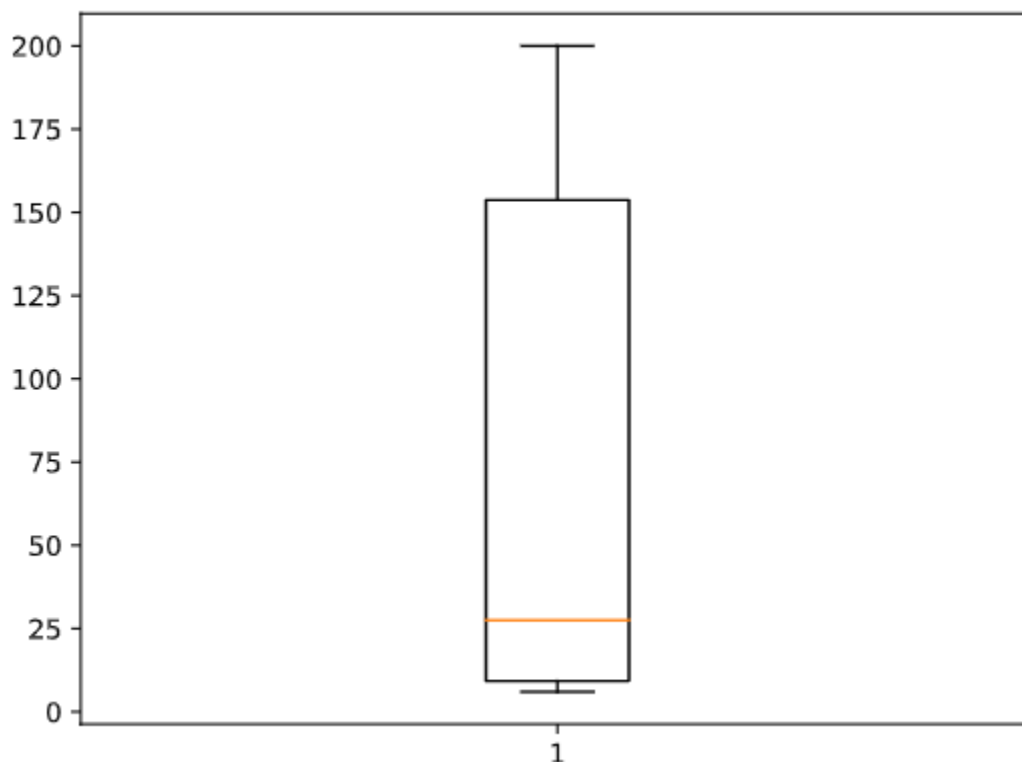
However, unlike the range, the IQR is robust. A statistic is robust when outliers have little impact on it. For example, the IQRs of the two datasets below are identical, even though one has a massive outlier.

```
dataset_one = c(6, 9, 10, 45, 190, 200) # IQR is 144.5
dataset_two = c(6, 9, 10, 45, 190, 20000000) # IQR is 144.5
```

Copy to Clipboard

By looking at the IQR instead of the range, you can get a better sense of the spread of the *middle* of the data.

The interquartile range is displayed in a commonly-used graph — the box plot.



In a box plot, the ends of the box are Q1 and Q3. So the length of the box is the IQR.

Instructions

We've set up a small `dataset` and are printing its range and IQR.

Try changing the maximum number in the dataset to different values.

What happens to the range when you make the maximum value `100000`? What happens to the IQR?

Try changing the minimum value to be more of an outlier as well.

```
---  
title: "Interquartile Range"  
output: html_notebook  
---  
  
```{r}  
small dataset
dataset = c(-50, -24, -13, -2, 0, 12, 15, 18, 73, 90, 100)

calculate range and IQR
dataset_range = max(dataset) - min(dataset)
dataset_iqr = IQR(dataset)

print range and IQR
print(paste("The range of the dataset is ",dataset_range))
print(paste("The IQR of the dataset is ",dataset_iqr))
```
```

Hypothesis Testing with R

Introduction

Say you work for a major social media website. Your boss comes to you with two questions:

- does the demographic of users on your site match the company's expectation?

- did the new interface update affect user engagement?

With terabytes of user data at your hands, you decide the best way to answer these questions is with statistical hypothesis tests!

Statistical hypothesis testing is a process that allows you to evaluate if a change or difference seen in a dataset is "real", or if it's just a result of random fluctuation in the data.

Hypothesis testing can be an integral component of any decision making process. It provides a framework for evaluating how confident one can be in making conclusions based on data. Some instances where this might come up include:

- a professor expects an exam average to be roughly 75%, and wants to know if the actual scores line up with this expectation. Was the test actually too easy or too hard?

- a product manager for a website wants to compare the time spent on different versions of a homepage. Does one version make users stay on the page significantly longer?

In this lesson, you will cover the fundamental concepts that will help you run and evaluate hypothesis tests:

- Sample and Population Mean

- P-Values

- Significance Level

- Type I and Type II Errors

You will then learn about three different hypothesis tests you can perform to answer the kinds of questions discussed above:

- One Sample T-Test

- Two Sample T-Test

- ANOVA (Analysis of Variance)

Let's get started!

Instructions

The code in `notebook.Rmd` performs a hypothesis test on data for a company BuyPie.com. The test evaluates whether the time spent per visitor on the website changes significantly between two weeks.

Read the output at the bottom of the rendered notebook. Do you think there is a difference in time spent per visitor between Week 1 and Week 2?

By the end of the lesson, you will be able to perform and interpret such hypothesis tests yourself!

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r message = FALSE}
load data
load("week_1.Rda")
load("week_2.Rda")
```

```

```
```{r}
calculate week_1_mean and week_2_mean:
week_1_mean <- mean(week_1)
week_1_mean
week_2_mean <- mean(week_2)
week_2_mean
```

```{r}
calculate week_1_sd and week_2_sd:
week_1_sd <- sd(week_1)
week_1_sd
week_2_sd <- sd(week_2)
week_2_sd
```

```{r}
run two sample t-test:
results <- t.test(week_1, week_2)
results
```

```

Hypothesis Testing with R

Sample Mean and Population Mean - I

Suppose you want to know the average height of an oak tree in your local park. On Monday, you measure 10 trees and get an average height of 32 ft. On Tuesday, you measure 12 different trees and reach an average height of 35 ft. On Wednesday, you measure the remaining 11 trees in the park, whose average height is 31 ft. The average height for all 33 trees in your local park is 32.8 ft.

The collection of individual height measurements on Monday, Tuesday, and Wednesday are each called samples. A *sample* is a subset of the entire population (all the oak trees in the park). The mean of each sample is a *sample mean* and it is an estimate of the *population mean*.

Note: the sample means (32 ft., 35 ft., and 31 ft.) were all close to the population mean (32.8 ft.), but were all slightly different from the population mean and from each other.

For a population, the mean is a constant value no matter how many times it's recalculated. But with a set of samples, the mean will depend on exactly which samples are selected. From a sample mean, we can then extrapolate the mean of the population as a whole. There are three main reasons we might use sampling:

- data on the entire population is not available
- data on the entire population is available, but it is so large that it is unfeasible to analyze
- meaningful answers to questions can be found faster with sampling

Instructions

Checkpoint 1 Enabled

1.

In the workspace, we've generated a random population of size 300 that follows a normal distribution with a mean of 65. Update the value of `population_mean` to store the `mean()` of `population`. Does it closely match your expectation?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's look at how the means of different samples can vary within the same population.

The code in the notebook generates 5 random samples from `population`. `sample_1` is displayed and `sample_1_mean` has been calculated.

Replace the "Not calculated" strings with calculations of the means for `sample_2`, `sample_3`, `sample_4`, and `sample_5`.

Look at the population mean and the sample means. Are they all the same? All different? Why?

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r}
generate random population
population <- rnorm(300, mean=65, sd=3.5)

calculate population mean here:
population_mean <- mean(population)
population_mean
```

```{r}
generate sample 1
sample_1 <- sample(population, size=30)
sample_1

calculate sample 1 mean
sample_1_mean <- mean(sample_1)
sample_1_mean
```

```{r}
generate samples 2,3,4 and 5
sample_2 <- sample(population, size=30)
sample_3 <- sample(population, size=30)
sample_4 <- sample(population, size=30)
sample_5 <- sample(population, size=30)
```

```{r}
calculate sample means here:
sample_2_mean <- mean(sample_2)
sample_2_mean
sample_3_mean <- mean(sample_3)
sample_3_mean
sample_4_mean <- mean(sample_4)
sample_4_mean
sample_5_mean <- mean(sample_5)
sample_5_mean
```
```

Hypothesis Testing with R

Sample Mean and Population Mean - II

In the previous exercise, the sample means you calculated closely approximated the population mean. This won't always be the case!

Consider a tailor of school uniforms at a school for students aged 11 to 13. The tailor needs to know the average height of all the students in order to know which sizes to make the uniforms.

The tailor measures the heights of a random sample of 20 students out of the 300 in the school. The average height of the sample is 57.5 inches. Using this sample mean, the tailor makes uniforms that fit students of this height, some smaller, and some larger.

After delivering the uniforms, the tailor starts to receive some feedback — many of the uniforms are too small! They go back to take measurements on the rest of the students, collecting the following data:

11 year olds average height: 56.7 inches

12 year olds average height: 59 inches

13 year olds average height: 62.8 inches

All students average height (population mean): 59.5 inches

The original sample mean was off from the population mean by 2 inches! How did this happen?

The random sample of 20 students was skewed to one direction of the total population. More 11 year olds were chosen in the sample than is representative of the whole school, bringing down the average height of the sample.

This is called a *sampling error*, and occurs when a sample is not representative of the population it comes from. How do you get an average sample height that looks more like the average population height, and reduce the chance of a sampling error?

Selecting only 20 students for the sample allowed for the chance that only younger, shorter students were included.

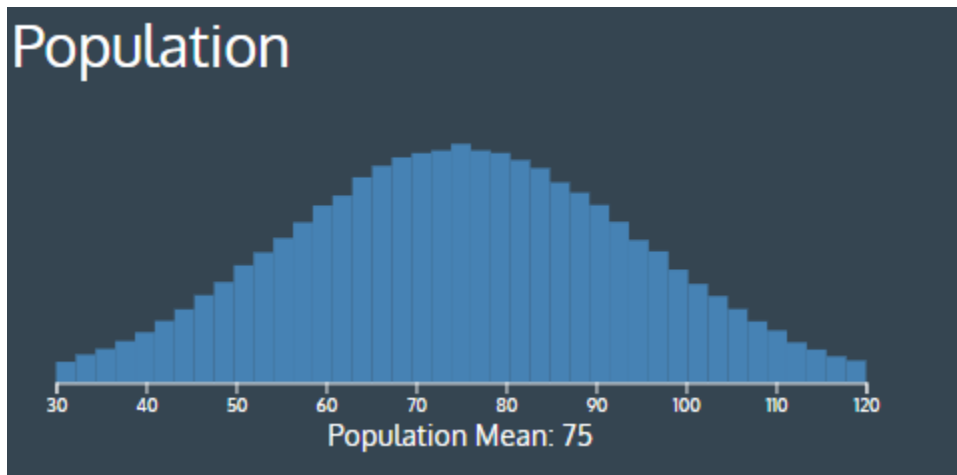
This is a natural consequence of the fact that a sample has less data than the population to which it belongs. If the sample selection is poor, then you will have a sample mean seriously skewed from the population mean.

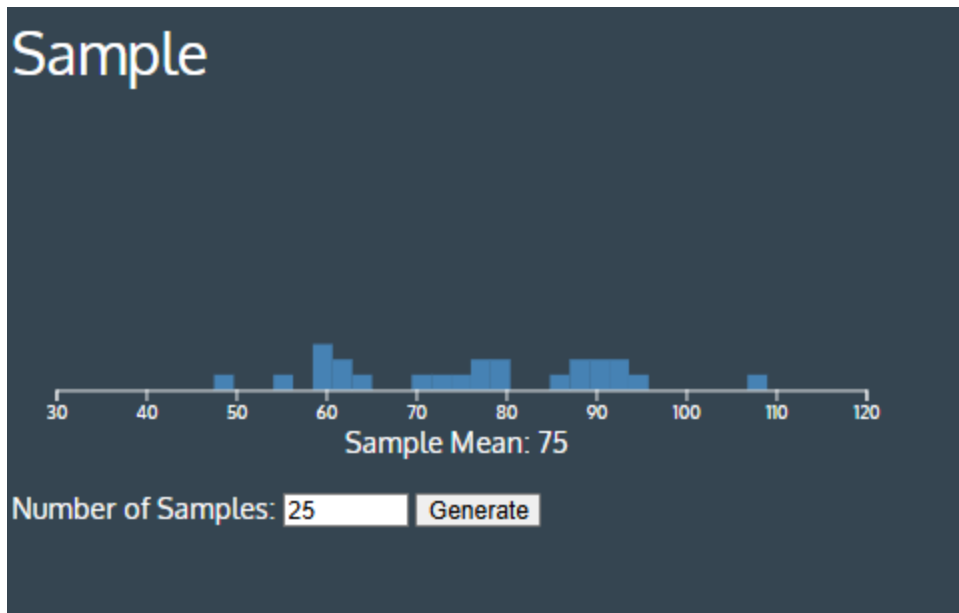
There is one surefire way to mitigate the risk of having a skewed sample mean — take a larger set of samples! The sample mean of a larger sample set will more closely approximate the population mean, and reduce the chance of a sampling error.

Instructions

In the workspace, we have a population that is normally distributed. Generate samples of different sizes and see how the sample mean could differ from the population mean.

What happens to the difference between the sample mean and the population mean as you increase the sample size?





Hypothesis Testing with R

Hypothesis Formulation

You begin the statistical hypothesis testing process by defining a *hypothesis*, or an assumption about your population that you want to test. A hypothesis can be written in words, but can also be explained in terms of the sample and population means you just learned about.

Say you are developing a website and want to compare the time spent on different versions of a homepage. You could run a hypothesis test to see if version A or B makes users stay on the page significantly longer. Your hypothesis might be:

"The average time spent on homepage A is greater than the average time spent on homepage B."

While this is a fine hypothesis to make, data analysts are often very hesitant people. They don't like to make bold claims without having data to back them up! Thus when constructing hypotheses for a hypothesis test, you want to formulate a null hypothesis. A *null hypothesis* states that there is no difference between the populations you are comparing, and it implies that any difference seen in the sample data is due to sampling error. A null hypothesis for the same scenario is as follows:

"The average time spent on homepage A is the same as the average time spent on homepage B."

You could also restate this in terms of population mean:

"The population mean of time spent on homepage A is the same as the population mean of time spent on homepage B."

After collecting some sample data on how users interact with each homepage, you can then run a hypothesis test using the data collected to determine whether your null hypothesis is true or false, or can be rejected (i.e. there is a difference in time spent on homepage A or B).

Instructions

Checkpoint 1 Passed

1.

A researcher at a pharmaceutical company is working on the development of a new medication to lower blood pressure, DeePressurize. They run an experiment with a control group of 100 patients that receive a placebo (a sugar pill), and an experimental group of 100 patients that receive DeePressurize. Blood pressure

measurements are taken after a 3 month period on both groups of patients.

The researcher wants to run a hypothesis test to compare the resulting datasets. Two hypotheses, `hypo_a` and `hypo_b`, are given in `notebook.Rmd`. Which could be a null hypothesis for comparing the two sets of data? Update the value of `null_hypo_1` to the string `"hypo_a"` or `"hypo_b"` based on your answer.

Checkpoint 2 Passed

2.

A product manager at a dating app company is developing a new user profile page with a different picture layout. They want to see if the new layout results in more matches between users than the current layout.

50% of profiles are updated to the new layout, and over a 1 month period the number of matches for users with the new layout and the original layout are recorded.

The product manager wants to run a hypothesis test to compare the resulting datasets. Two hypotheses, `hypo_c` and `hypo_d`, are given in `notebook.Rmd`. Which could be a null hypothesis for comparing the two sets of data? Update the value of `null_hypo_2` to the string `"hypo_c"` or `"hypo_d"` based on your answer.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r}
experiment 1
hypo_a <- "DeePressurize lowers blood pressure in patients."
hypo_b <- "DeePressurize has no effect on blood pressure in patients."
null_hypo_1 <- "hypo_b"
```

```{r}
experiment 2
hypo_c <- "The new profile layout has no effect on number of matches with other
users."
hypo_d <- "The new profile layout results in more matches with other users than the
original layout."
null_hypo_2 <- "hypo_c"
```
```

Hypothesis Testing with R

Designing an Experiment

Suppose you want to know if students who study history are more interested in volleyball than students who study chemistry. Before doing anything else to answer your original question, you come up with a null hypothesis:

"History and chemistry students are interested in volleyball at the same rates."

To test this hypothesis, you need to design an experiment and collect data. You invite 100 history majors and 100 chemistry majors from your university to join an extracurricular volleyball team. After one week, 34 history majors sign up (34%), and 39 chemistry majors sign up (39%). More chemistry majors than history majors signed up, but is this a "real", or significant difference? Can you conclude that students who study chemistry are more interested in volleyball than students who study history?

In your experiment, the 100 history and 100 chemistry majors at your university are samples of their respective populations (all history and chemistry majors). The sample means are the percentages of history majors (34%) and chemistry majors (39%) that signed up for the team, and the difference in sample means is $39\% - 34\% = 5\%$. The population means are the percentage of history and chemistry majors worldwide that would sign up for an extracurricular volleyball team if given the chance.

You want to know if the difference you observed in these sample means (5%) reflects a difference in the population means, or if the difference was caused by sampling error, and the samples of students you chose do not represent the greater populations of history and chemistry students.

Restating the null hypothesis in terms of the population means yields the following:

"The percentage of all history majors who would sign up for volleyball is the same as the percentage of all chemistry majors who would sign up for volleyball, and the observed difference in sample means is due to sampling error."

This is the same as saying, "If you gave the same volleyball invitation to every history and chemistry major in the world, they would sign up at the same rate, and the sample of 200 students you selected are not representative of their populations."

Instructions

Checkpoint 1 Passed

1.

Your friend is a dog walker that specializes in working with Golden Retrievers and Goldendoodles. They are interested in knowing if there is a significant difference in the lengths of the two breeds. After a few weeks of data collection, they give you a spreadsheet of 10 Golden Retrievers' lengths and 10 Goldendoodles' lengths.

The lengths of the dogs are given in `retriever_lengths` and `doodle_lengths`. Calculate the mean of each breed and save the results to `mean_retriever_l` and `mean_doodle_l`. View `mean_retriever_l` and `mean_doodle_l`.

Checkpoint 2 Passed

2.

Calculate the difference between `mean_retriever_l` and `mean_doodle_l` and save the result to `mean_difference`. View `mean_difference`.

Checkpoint 3 Passed

3.

You want to run a hypothesis test to see if there is a significant difference in the lengths of Golden Retrievers and Goldendoodles. Which of the two statements could be a formulation of the null hypothesis? Update the value of `null_hypo` with "st_1" or "st_2" depending on your answer.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r}
load data
load("retriever_lengths.Rda")
load("doodle_lengths.Rda")
```

```{r}
calculate mean_retriever_l and mean_doodle_l here:
mean_retriever_l <- mean(retriever_lengths)
mean_retriever_l
mean_doodle_l <- mean(doodle_lengths)
mean_doodle_l
```

```{r}
calculate mean_difference here:
mean_difference <- mean_retriever_l - mean_doodle_l
mean_difference
```

```{r}
statements:
```

```
st_1 <- "The average length of Golden Retrievers is 2.5 inches longer than the average
length of Goldendoodles."
st_2 <- "The average length of Golden Retrievers is the same as the average length of
Goldendoodles."

update null_hypo here:
null_hypo <- "st_2"
````
```

Hypothesis Testing with R

Type I and Type II Errors

When using automated processes to make decisions, you need to be aware of how this automation can lead to mistakes. Computer programs can be as fallible as the humans who design them. Because of this, there is a responsibility to understand what can go wrong and what can be done to contain these foreseeable problems.

In statistical hypothesis testing, there are two types of error. A *Type I error* occurs when a hypothesis test finds a correlation between things that are not related. This error is sometimes called a “false positive” and occurs when the null hypothesis is rejected even though it is true.

For example, consider the history and chemistry major experiment from the previous exercise. Say you run a hypothesis test on the sample data you collected and conclude that there is a significant difference in interest in volleyball between history and chemistry majors. You have rejected the null hypothesis that there is no difference between the two populations of students. If, in reality, your results were due to the groups you happened to pick (sampling error), and there actually is no significant difference in interest in volleyball between history and chemistry majors in the greater population, you have become the victim of a false positive, or a Type I error.

The second kind of error, a *Type II error*, is failing to find a correlation between things that are actually related. This error is referred to as a “false negative” and occurs when the null hypothesis is not rejected even though it is false.

For example, with the history and chemistry student experiment, say that after you perform the hypothesis test, you conclude that there is no significant difference in interest in volleyball between history and chemistry majors. You did *not* reject the null hypothesis. If there actually is a difference in the populations as a whole, and there is a significant difference in interest in volleyball between history and chemistry majors, your test has resulted in a false negative, or a Type II error.

Instructions

Checkpoint 1 Enabled

1.

In `notebook.Rmd` you will find four vectors: `actual_positive`, `actual_negative`, `experimental_positive`, and `experimental_negative`. These vectors represent outcomes from a statistical experiment.

The base R `intersect()` function can take two vectors as arguments and returns a vector containing the common elements.

Use the `intersect()` function and the vectors provided to define `type_i_errors`. This vector should contain the false positives of the experiment. View `type_i_errors`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, define `type_ii_errors`, the list representing the false negatives of the experiment.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r}
the true positives and negatives:
```

```

actual_positive <- c(2, 5, 6, 7, 8, 10, 18, 21, 24, 25, 29, 30, 32, 33, 38, 39, 42,
44, 45, 47)
actual_negative <- c(1, 3, 4, 9, 11, 12, 13, 14, 15, 16, 17, 19, 20, 22, 23, 26, 27,
28, 31, 34, 35, 36, 37, 40, 41, 43, 46, 48, 49)

the positives and negatives we determine by running the experiment:
experimental_positive <- c(2, 4, 5, 7, 8, 9, 10, 11, 13, 15, 16, 17, 18, 19, 20, 21,
22, 24, 26, 27, 28, 32, 35, 36, 38, 39, 40, 45, 46, 49)
experimental_negative <- c(1, 3, 6, 12, 14, 23, 25, 29, 30, 31, 33, 34, 37, 41, 42,
43, 44, 47, 48)
```

```{r}
define type_i_errors and type_ii_errors here:
type_i_errors <- intersect(experimental_positive, actual_negative)
type_i_errors
type_ii_errors <- intersect(experimental_negative, actual_positive)
type_ii_errors
```

```

Type I and Type II Error

| Null hypothesis is ... | True | False |
|------------------------|--|---|
| Rejected | Type I error
False positive
Probability = α | Correct decision
True positive
Probability = $1 - \beta$ |
| Not rejected | Correct decision
True negative
Probability = $1 - \alpha$ | Type II error
False negative
Probability = β |

| | | Reality | |
|------------------------|-------|---------------------------|--------------------------|
| | | True | False |
| Measured/
Perceived | True | Correct
😊 | Type I
False Positive |
| | False | Type II
False Negative | Correct
😊 |

Truth
(for population studied)

| Decision
(based on sample) | The Null Hypothesis is | True | False |
|-------------------------------|------------------------|---|--|
| | Rejected | Type I Error
False Positive
Probability = α | Correct decision
True Positive
Probability = $1 - \beta$ |
| | Not Rejected | Correct decision
True Negative
Probability = $1 - \alpha$ | Type II Error
False Negative
Probability = β |

Hypothesis Testing with R

P-Values

You know that a hypothesis test is used to determine the validity of a null hypothesis. Once again, the null hypothesis states that there is no actual difference between the two populations of data. But what result does a hypothesis test actually return, and how can you interpret it?

A hypothesis test returns a few numeric measures, most of which are out of the scope of this introductory lesson. Here we will focus on one: p-values. P-values help determine how confident you can be in validating the null hypothesis. In this context, a *p-value* is the probability that, assuming the null hypothesis is true, you would see at least such a difference in the sample means of your data.

Consider the experiment on history and chemistry majors and their interest in volleyball from a previous exercise:

Null Hypothesis: "History and chemistry students are interested in volleyball at the same rates"

Experiment Sample Means: 34% of history majors and 39% of chemistry majors sign up for the volleyball class

Assuming the null hypothesis is true, there is no actual difference in preference for volleyball between all history and chemistry majors, and any difference present in the experiment data is the result of sampling error. Imagine you run a

hypothesis test on this experiment data and it returns a p-value of 0.04. A p-value of 0.04 indicates that you could expect to see a difference of at least 5% (calculated as 39% - 34% = 5%) in the sample means only 4% of the time.

Essentially, if you ran this same experiment 100 times, you would expect to see as large a difference in the sample means only 4 times given the assumption that there is no actual difference between the populations (i.e. they have the same mean).

Seems like a really small probability, right? Are you thinking about rejecting the null hypothesis you originally stated?

Instructions

Checkpoint 1 Enabled

1.

You are big fan of apples, so you gather 10 green and 10 red apples to compare their weights. The green apples average 150 grams in weight, and the red apples average 160 grams in weight.

You run a hypothesis test to see if there is a significant difference in the weight of green and red apples. The test returns a p-value of 0.2. Which statement (st_1, st_2, st_3, or st_4) indicates how this p-value can be interpreted?

Update the value of interpretation with the string "st_1", "st_2", "st_3", or "st_4" depending on your answer.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r}
possible interpretations
st_1 <- "There is a 20% chance that the difference in average weight of green and red
apples is due to random sampling."
st_2 <- "There is a 20% chance that green and red apples have the same average
weight."
st_3 <- "There is a 20% chance red apples weigh more than green apples."
st_4 <- "There is a 20% chance green apples weigh more than green apples."

update the value of interpretation here:
interpretation <- "st_1"
```
```

Significance Level

While a hypothesis test will return a p-value indicating a level of confidence in the null hypothesis, it does not definitively claim whether you should reject the null hypothesis. To make this decision, you need to determine a threshold p-value for which all p-values below it will result in rejecting the null hypothesis. This threshold is known as the *significance level*.

A higher significance level is more likely to give a false positive, as it makes it “easier” to state that there is a difference in the populations of your data when such a difference might not actually exist. If you want to be very sure that the result is not due to sampling error, you should select a very small significance level.

It is important to choose the significance level before you perform a statistical hypothesis test. If you wait until after you receive a p-value from a test, you might pick a significance level such that you get the result you want to see. For instance, if someone is trying to publish the results of their scientific study in a journal, they might set a higher significance level that makes their results appear statistically significant. Choosing a significance level in advance helps keep everyone honest.

It is an industry-standard to set a significance level of 0.05 or less, meaning that there is a 5% or less chance that your result is due to sampling error.

Instructions

Checkpoint 1 Enabled

1.

Before you run a hypothesis test on a set of data, you set your significance level to 0.05. The hypothesis test then returns a p-value of 0.1. Can you reject the null hypothesis? Update the value of `reject_hypothesis` to `TRUE` or `FALSE` depending on your answer.

One Sample T-Test

Consider the fictional business BuyPie, which sends ingredients for pies to your household so that you can make them from scratch. Suppose that a product manager hypothesizes the average age of visitors to BuyPie.com is 30. In the past hour, the website had 100 visitors and the average age was 31. Are the visitors older than expected? Or is this just the result of chance (sampling error) and a small sample size?

You can test this using a One Sample T-Test. A *One Sample T-Test* compares a sample mean to a hypothetical population mean. It answers the question “What is the probability that the sample came from a distribution with the desired mean?”

The first step is formulating a null hypothesis, which again is the hypothesis that there is no difference between the populations you are comparing. The second population in a One Sample T-Test is the hypothetical population you choose. The null hypothesis that this test examines can be phrased as follows: “The set of samples belongs to a population with the target mean”.

One result of a One Sample T-Test will be a *p-value*, which tells you whether or not you can reject this null hypothesis. If the p-value you receive is less than your significance level, normally 0.05, you can reject the null hypothesis and state that there is a significant difference.

R has a function called `t.test()` in the `stats` package which can perform a One Sample T-Test for you.

`t.test()` requires two arguments, a distribution of values and an expected mean:

```
results <- t.test(sample_distribution, mu = expected_mean)
```

Copy to Clipboard

`sample_distribution` is the sample of values that were collected
`mu` is an argument indicating the desired mean of the hypothetical population
`expected_mean` is the value of the desired mean

`t.test()` will return, among other information we will not cover here, a p-value — this tells you how confident you can be that the sample of values came from a distribution with the specified mean.

P-values give you an idea of how confident you can be in a result. Just because you don’t have enough data to detect a difference doesn’t mean that there isn’t one. Generally, the more samples you have, the smaller a difference you can detect.

Instructions

Checkpoint 1 Enabled

1.

We have provided a small dataset called `ages`, representing the ages of customers to BuyPie.com in the past hour, in `notebook.Rmd`.

Even with a small dataset like this, it is hard to make judgments from just looking at the numbers. To understand the data better, let's look at the mean. Calculate the mean of `ages`, and store the result in a variable called `ages_mean`. View `ages_mean`.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r message = FALSE}
load and view data
load("ages.Rda")
ages
```

```{r}
calculate ages_mean here:
ages_mean <- mean(ages)
ages_mean
```

```{r}
perform t-test here:
results <- t.test(ages, mu=30)
results
```
```

Two Sample T-Test

Suppose that last week, the average amount of time spent per visitor to a website was 25 minutes. This week, the average amount of time spent per visitor to a website was 29 minutes. Did the average time spent per visitor change (i.e. was there a statistically significant bump in user time on the site)? Or is this just part of natural fluctuations? One way of testing whether this difference is significant is by using a Two Sample T-Test. A *Two Sample T-Test* compares two sets of data, which are both approximately normally distributed. The null hypothesis, in this case, is that the two distributions have the same mean. You can use R's `t.test()` function to perform a Two Sample T-Test, as shown below:

```
results <- t.test(distribution_1, distribution_2)
```

Copy to Clipboard

When performing a Two Sample T-Test, `t.test()` takes two distributions as arguments and returns, among other information, a p-value. Remember, the p-value let's you know the probability that the difference in the means happened by chance (sampling error).

Instructions

Checkpoint 1 Enabled

1.

We've created two distributions representing the time spent per visitor to BuyPie.com last week, `week_1`, and the time spent per visitor to BuyPie.com this week, `week_2`.

Find the means of these two distributions. Store them in `week_1_mean` and `week_2_mean`. View both means.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Find the standard deviations of these two distributions. Store them in `week_1_sd` and `week_2_sd`. View both standard deviations.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Run a Two Sample T-Test using the `t.test()` function.

Save the results to a variable called `results` and view it. Does the p-value make sense, knowing what you know about these datasets?

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r message = FALSE}
load data
load("week_1.Rda")
week_1
load("week_2.Rda")
week_2
```

```{r}
calculate week_1_mean and week_2_mean here:
week_1_mean <- mean(week_1)
week_1_mean
week_2_mean <- mean(week_2)
week_2_mean
```

```{r}
calculate week_1_sd and week_2_sd here:
week_1_sd <- sd(week_1)
week_1_sd
week_2_sd <- sd(week_2)
week_2_sd
```

```{r}
run two sample t-test here:
results <- t.test(week_1, week_2)
results
```
```

Dangers of Multiple T-Tests

Suppose that you own a chain of stores that sell ants, called VeryAnts. There are three different locations: A, B, and C. You want to know if the average ant sales over the past year are significantly different between the three locations.

At first, it seems that you could perform T-tests between each pair of stores.

You know that the p-value is the probability that you incorrectly reject the null hypothesis on each t-test. The more t-tests you perform, the more likely that you are to get a false positive, a Type I error.

For a p-value of 0.05, if the null hypothesis is true, then the probability of obtaining a significant result is $1 - 0.05 = 0.95$. When you run another t-test, the probability of still getting a correct result is $0.95 * 0.95$, or 0.9025. That means your probability of making an error is now close to 10%! This error probability only gets bigger with the more t-tests you do.

Instructions

Checkpoint 1 Failed, try again

1.

We have created samples `store_a`, `store_b`, and `store_c`, representing the sales at VeryAnts at locations A, B, and C, respectively. We want to see if there's a significant difference in sales between the three locations.

Explore datasets `store_a`, `store_b`, and `store_c` by finding and viewing the means and standard deviations of each one. Store the means in variables called `store_a_mean`, `store_b_mean`, and `store_c_mean`. Store the standard deviations in variables called `store_a_sd`, `store_b_sd`, and `store_c_sd`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Perform a Two Sample T-test between each pair of location data.

Store the results of the tests in variables called `a_b_results`, `a_c_results`, and `b_c_results`. View the results for each test.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Store the probability of error for running three T-Tests in a variable called `error_prob`. View `error_prob`.

```
---
title: "Hypothesis Testing in R"
output: html_notebook
---

```{r message = FALSE}
load data
load("store_a.Rda")
load("store_b.Rda")
load("store_c.Rda")
```

```{r}
calculate means here:
store_a_mean <- mean(store_a)
store_a_mean
store_b_mean <- mean(store_b)
store_b_mean
store_c_mean <- mean(store_c)
store_c_mean
```

```{r}
calculate standard deviations here:
store_a_sd <- sd(store_a)
store_a_sd
store_b_sd <- sd(store_b)
store_b_sd
store_c_sd <- sd(store_c)
store_c_sd
```

```{r}
perform two sample t-test here:
a_b_results <- t.test(store_a, store_b)
a_b_results
a_c_results <- t.test(store_a, store_c)
a_c_results
b_c_results <- t.test(store_b, store_c)
```

```
b_c_results
'''
'''{r}
calculate error_prob here:
error_prob <- (1-(0.95**3))
error_prob
'''
```

## ANOVA

In the last exercise, you saw that the probability of making a Type I error got dangerously high as you performed more t-tests.

When comparing more than two numerical datasets, the best way to preserve a Type I error probability of 0.05 is to use ANOVA. *ANOVA (Analysis of Variance)* tests the null hypothesis that all of the datasets you are considering have the same mean. If you reject the null hypothesis with ANOVA, you're saying that at least one of the sets has a different mean; however, it does not tell you which datasets are different.

You can use the `stats` package function `aov()` to perform ANOVA on multiple datasets. `aov()` takes the different datasets combined into a data frame as an argument. For example, if you were comparing scores on a video game between math majors, writing majors, and psychology majors, you could format the data in a data frame `df_scores` as follows:

group	score
math major	88
math major	81
writing major	92
writing major	80
psychology major	94
psychology major	83

You can then run an ANOVA test with this line:

```
results <- aov(score ~ group, data = df_scores)
```

Copy to Clipboard

**Note:** `score ~ group` indicates the relationship you want to analyze (i.e. how each `group`, or major, relates to `score` on the video game)

To retrieve the p-value from the results of calling `aov()`, use the `summary()` function:

```
summary(results)
```

Copy to Clipboard

The null hypothesis, in this case, is that all three populations have the same mean score on this video game. If you reject this null hypothesis (if the p-value is less than 0.05), you can say you are reasonably confident that a pair of datasets is significantly different. After using only ANOVA, however, you can't make any conclusions on which two populations have a significant difference.

Let's look at an example of ANOVA in action.

### Instructions

Checkpoint 1 Enabled

1.

We've reformatted the store data from the last exercise into a data frame `stores`. View `stores` to see what columns it contains.

Open the hint for an explanation of the columns.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Perform an ANOVA on the `stores` data and save the test results to a variable `results`. Use the `summary()` function to view the p-value of the test. Does this p-value lead you to reject the null hypothesis?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's say the sales at location B have suddenly *soared* (maybe there's an ant convention happening nearby).

The new sales for location B have been updated in the `stores_new` data frame.

Re-run the ANOVA test on `stores_new` and save the test results to a variable `results_new`. Use the `summary()` function to see what the p-value is now. Does this new value make sense?

```

title: "Hypothesis Testing in R"
output: html_notebook

```{r message = FALSE}
# load libraries
library(tidyr)
```

```{r message = FALSE}
# load data
load("stores.Rda")
load("stores_new.Rda")
```

```{r}
# inspect stores here:
stores
```

```{r}
# perform anova on stores here:
results <- aov(sales ~ store, data = stores)
results
```

```
summary(results)
```

```{r}
# perform anova on stores_new here:
results_new <- aov(sales ~ store, data = stores_new)
results_new
summary(results_new)
```
```

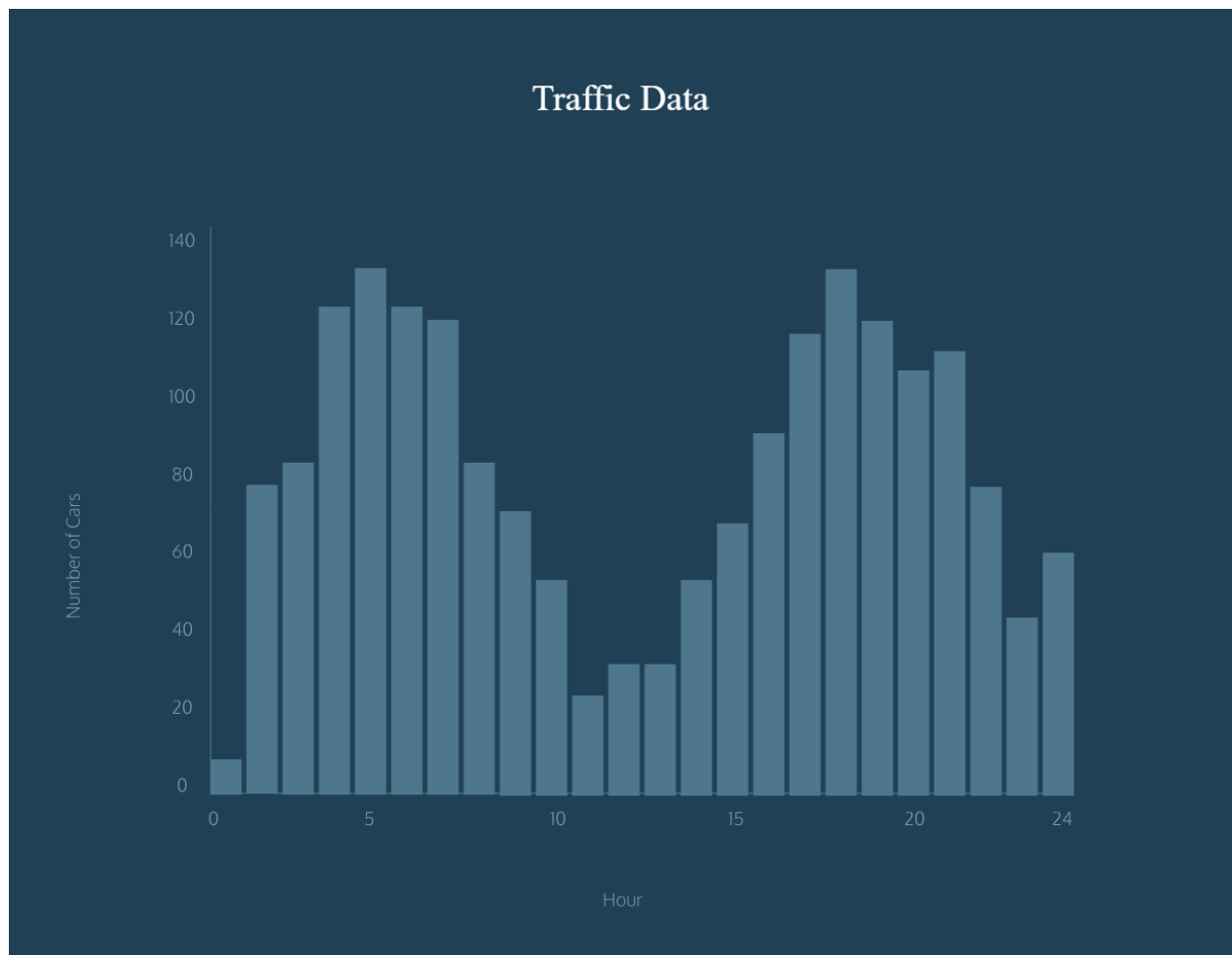
## Assumptions of Numerical Hypothesis Tests

Before you use numerical hypothesis tests, you need to be sure that the following things are true:

### 1. The samples should each be normally distributed...ish

Data analysts in the real world often still perform hypothesis tests on datasets that aren't exactly normally distributed. What is more important is to recognize if there is some reason to believe that a normal distribution is especially unlikely. If your dataset is definitively not normal, the numerical hypothesis tests won't work as intended.

For example, imagine you have three datasets, each representing a day of traffic data in three different cities. Each dataset is independent, as traffic in one city should not impact traffic in another city. However, it is unlikely that each dataset is normally distributed. In fact, each dataset probably has two distinct peaks, one at the morning rush hour and one during the evening rush hour. The histogram of a day of traffic data might look something like this:



In this scenario, using a numerical hypothesis test would be inappropriate.

## 2. The population standard deviations of the groups should be equal

For ANOVA and Two Sample T-Tests, using datasets with standard deviations that are significantly different from each other will often obscure the differences in group means.

To check for similarity between the standard deviations, it is normally sufficient to divide the two standard deviations and see if the ratio is “close enough” to 1. “Close enough” may differ in different contexts, but generally staying within 10% should suffice.

## 3. The samples must be independent

When comparing two or more datasets, the values in one distribution should not affect the values in another distribution. In other words, knowing more about one distribution should not give you any information about any other distribution.

Here are some examples where it would seem the samples are not independent:

- the number of goals scored per soccer player before, during, and after undergoing a rigorous training regimen
- a group of patients' blood pressure levels before, during, and after the administration of a drug

It is important to understand your datasets before you begin conducting hypothesis tests on them so that you know you are choosing the right test.

### Instructions

Checkpoint 1 Enabled

1.

Use the base R `hist()` function to display the histograms for `dist_one`, `dist_two`, `dist_three`, and `dist_four`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Do the distributions look normal?

One of these distributions would probably *not* be a good choice to use in an ANOVA comparison. Create a variable called `not_normal` and set it equal to the distribution number (1, 2, 3, or 4) that would be least suited for use in an ANOVA test.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Calculate the ratio of standard deviations between `dist_two` and `dist_three`, and store the value in a variable called `ratio`. View `ratio`. Is this “close enough” to perform a numerical hypothesis test between the two datasets?

```

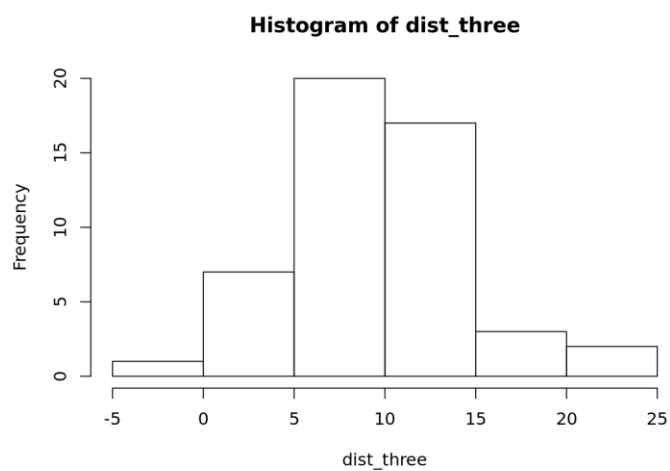
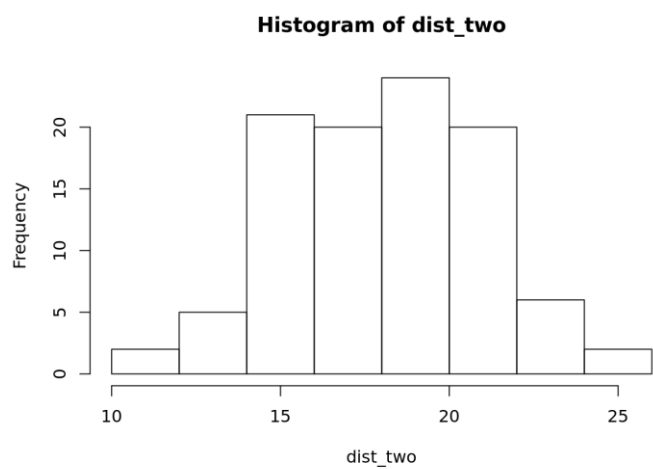
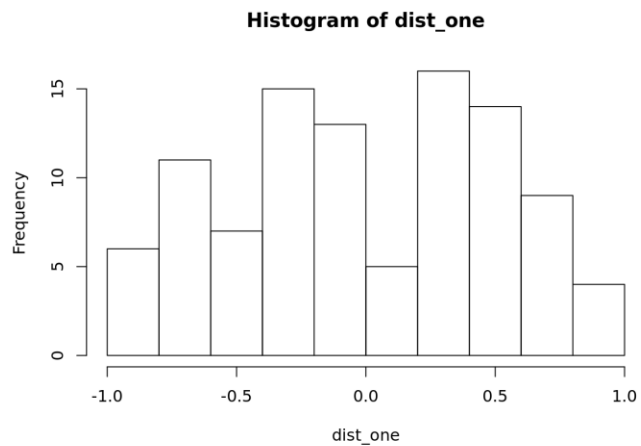
title: "Hypothesis Testing in R"
output: html_notebook

```{r message = FALSE}
# load data
load("dist_one.Rda")
load("dist_two.Rda")
load("dist_three.Rda")
load("dist_four.Rda")
```

```{r}
# plot histograms and define not_normal here:
hist(dist_one)
hist(dist_two)
hist(dist_three)
hist(dist_four)

not_normal <- 4
```

```{r}
# define ratio here:
ratio <- sd(dist_two) / sd(dist_three)
ratio
```
```



## Hypothesis Testing with R



# Review

Phew! Nobody said hypothesis testing is easy, but you made it to the end of the lesson. Congratulations! The world of hypothesis testing is vast. There is much more you can learn, and so many applications where you can use them.

Let's review what you've learned in this lesson:

*Samples* are subsets of an entire *population*, and the *sample mean* can be used to approximate the *population mean*

The *null hypothesis* is an assumption that there is no difference between the populations you are comparing in a hypothesis test

*Type I Errors* occur when a hypothesis test finds a correlation between things that are not related, and *Type II Errors* occur when a hypothesis test fails to find a correlation between things that are actually related

*P-Values* indicate the probability that, assuming the null hypothesis is true, such differences in the samples you are comparing would exist

The *Significance Level* is a threshold p-value for which all p-values below it will result in rejecting the null hypothesis

*One Sample T-Tests* indicate whether a dataset belongs to a distribution with a given mean

*Two Sample T-Tests* indicate whether there is a significant difference between two datasets

*ANOVA (Analysis of Variance)* allows you to detect if there is a significant difference between one of multiple datasets

# Welcome To Working With Data

Let's look ahead to see what's coming up in the Working With Data section of the course.

Welcome to the section of the course focused on working with data in R. In this section, you will learn how to import large spreadsheet-like datasets into R and manipulate their contents.

Rather than working in a spreadsheet, you can import those spreadsheets to R as data frames and apply all of the R skills you have learned so far!

In this section, you will learn how to create and clean data frames. You will also learn how to perform aggregate functions on those data frames. Finally, we will show you how to work with multiple data frames in unison.

Let's get started!

## Introduction

Data lies at the heart of nearly every problem in the business world and society. Having the right tools to manipulate data and organize it in a meaningful way is integral to performing data analyses and discovering unique insights!

The dplyr package in R is designed to make data manipulation tasks simpler and more intuitive than working with base R

[functions](#)

Preview: Docs Loading link description

only. Called a "grammar of data manipulation," dplyr provides functions that solve many challenges that arise when organizing tabular data (i.e., data in a table with rows and columns). Tabular data has a lot of the same functionality as tables from SQL or Excel, but dplyr adds the power of R.

In addition to learning how to load data into R with the readr package, this lesson will introduce how to perform the following data manipulation tasks with dplyr:

- select columns of a table
- filter rows of a table
- arrange rows of a table in order

dplyr and readr are a part of the tidyverse, a collection of R packages designed for data science. In this and future lessons, you will use different packages of the tidyverse to more easily analyze and visualize data!

The tidyverse is a package itself, and it can be imported at the top of your file if you need to use any of the packages it contains.

In our lessons, however, we will explicitly import the packages within the tidyverse that we are using. To get started with readr and dplyr, you can import them at the top of your `.Rmd` R-markdown file or `.R` script:

```
library(readr)
library(dplyr)
```

Copy to Clipboard

### Instructions

Checkpoint 1 Enabled

1.

The `notebook.Rmd` file in the code editor contains the code to load, inspect and manipulate a data frame containing data from the top 7 most popular music groups of 2018.

Load the readr and dplyr libraries in the code block at the top of the notebook so the code can properly run.

By the end of this lesson, you will be able to perform this same analysis!

```

title: "Introduction to Data Frames"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r}
# load data frame
artists <- read_csv('artists.csv')
```

```{r}
# inspect data frame
artists
head(artists)
summary(artists)
```

```{r}
# select columns, filter and arrange rows of artists
artists_manipulated <- artists %>%
  select(-country, -year_founded, -albums) %>%
  filter(spotify_monthly_listeners > 20000000, genre != 'Hip Hop') %>%
  arrange(desc(youtube_subscribers))
artists_manipulated
```
```

[Hide](#)

```
summary(artists)
```

```

 group country genre
Length:7 Length:7 Length:7
Class :character Class :character Class :character
Mode :character Mode :character Mode :character

spotify_monthly_listeners youtube_subscribers year_founded albums
Min. : 8409314 Min. : 1423636 Min. :1962 Min. : 3.000
1st Qu.:12187214 1st Qu.: 4777775 1st Qu.:1995 1st Qu.: 5.000
Median :20929342 Median :13891749 Median :1997 Median : 6.000
Mean :22081324 Mean :11611277 Mean :1997 Mean : 9.286
3rd Qu.:31513052 3rd Qu.:16168444 3rd Qu.:2008 3rd Qu.:10.500
Max. :37830079 Max. :24071114 Max. :2013 Max. :25.000

```

[Hide](#)

```

select columns, filter and arrange rows of artists
artists_manipulated <- artists %>%
 select(-country,-year_founded,-albums) %>%
 filter(spotify_monthly_listeners > 20000000, genre != 'Hip Hop') %>%
 arrange(desc(youtube_subscribers))
artists_manipulated

```

| group<br><chr>  | ge...<br><chr> | spotify_monthly_listeners<br><dbl> | youtube_subscribers<br><dbl> |
|-----------------|----------------|------------------------------------|------------------------------|
| Maroon 5        | Rock           | 35215180                           | 24071114                     |
| Imagine Dragons | Rock           | 37830079                           | 16710940                     |
| Coldplay        | Rock           | 27810924                           | 13891749                     |

3 rows

## What is a Data Frame?

A data frame is an R object that stores tabular data in a table structure made up of rows and columns. You can think of a data frame as a spreadsheet or as a SQL table. While data frames can be created in R, they are usually imported with data from a CSV, an Excel spreadsheet, or a SQL query.

Data frames have rows and columns. Each column has a name and stores the values of one variable. Each row contains a set of values, one from each column. The data stored in a data frame can be of many different types: numeric, character, logical, or NA.

A data frame containing the address, age and name of students in a class could look like this:

| address        | age | name       |
|----------------|-----|------------|
| 123 Main St.   | 34  | John Smith |
| 456 Maple Ave. | 28  | Jane Doe   |
| 789 Broadway   | 51  | Joe Schmo  |

As seen in the first row, the column names of this data frame are `address`, `age`, and `name`.

Note: when working with `dplyr`, you might see functions that take a data frame as an argument and output something called a tibble. Tibbles are modern versions of data frames in R, and they operate in essentially the same way. The terms tibble and data frame are often used interchangeably. Here on Codecademy we will use the term data frame!

### Instructions

Checkpoint 1 Enabled

1.

The code in `notebook.Rmd` loads a data frame named `songs` that contains data about 7 songs from popular music groups (you'll learn how to load a data frame yourself shortly).

Type `songs` in the empty code block and run the code to view the data frame. Make sure to click the arrow to explore each column!

```

title: "Introduction to Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r}
# load data frame
songs <- read_csv('songs.csv')
```

```{r}
head(songs)
```
```

# CSVs

When working with data frames, most of the time you will load in data from an existing data set. One of the most common formats for big datasets is the CSV.

CSV (*comma separated values*) is a text-only spreadsheet format. You can find CSVs in lots of places such as:

- online datasets from governments and companies (here's an example from [data.gov](https://data.gov))
- exported from Excel or Google Sheets
- exported from SQL

The first row of a CSV contains column headings. All subsequent rows contain values. Each column heading and each variable is separated by a comma:

```
column1,column2,column3
value1,value2,value3
value4,value5,value6
```

Copy to Clipboard

That example CSV represents the following table:

| column1 | column2 | column3 |
|---------|---------|---------|
| value1  | value2  | value3  |
| value4  | value5  | value6  |

## Instructions

Checkpoint 1 Failed, try again

1.

You run a cupcake store and want to create a record of all of the cupcakes that you offer.

Write the following data as a CSV in `cupcakes.csv`:

| name            | cake_flavor | frosting_flavor | topping           |
|-----------------|-------------|-----------------|-------------------|
| Red Velvet Cake | chocolate   | cream cheese    | strawberries      |
| Birthday Cake   | vanilla     | vanilla         | rainbow sprinkles |
| Carrot Cake     | carrot      | cream cheese    | almonds           |

Scroll all the way to the right in the table to see each column. And make sure your spelling is correct!

# Loading and Saving CSVs

When you have data in a CSV, you can load it into a data frame in R using `readr`'s `read_csv()` function:

```
df <- read_csv('my_csv_file.csv')
```

Copy to Clipboard

In the example above, the `read_csv()` function is called  
The CSV file `my_csv_file.csv` is passed in as an argument  
A data frame containing the data from `my_csv_file.csv` is returned

You can also save data from a data frame to a CSV using `readr`'s `write_csv()` function:

```
write_csv(df, 'new_csv_file.csv')
```

Copy to Clipboard

In the example above, `write_csv()` takes two arguments:

`df`, which represents a data frame object  
`new_csv_file.csv`, the name of the CSV file that will hold the data from the data frame

By default, this method will save the CSV file to your current directory.

## Instructions

Checkpoint 1 Enabled

1.

`artists.csv` contains data from the top 7 most popular music groups of 2018. Within the empty code block, load the CSV into a data frame `artists`. On the line below, type `artists` and run the code to view the rendered data frame.

Use the arrows to view the different columns of the data frame!

```

title: "Introduction to Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r}
# load data frame
artists <- read_csv('artists.csv')
artists
```
```

# Inspecting Data Frames

When you load a new data frame from a CSV, you want to get an understanding of what the data looks like.

If the data frame is small, you can display it by typing its name `df`. If the data frame is larger, it can be helpful to inspect a few rows of the data frame without having to look at the rest of it.

The `head()` function returns the first 6 rows of a data frame. If you want to see more rows, you can pass an additional argument `n` to `head()`. For example, `head(df, 8)` will show the first 8 rows.

The function `summary()` will return summary statistics such as mean, median, minimum and maximum for each numeric column while providing class and length information for non-numeric columns.

### Instructions

Checkpoint 1 Enabled

1.

View the entire data frame by typing `artists` in the empty code block and running the code.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Within the same code block, inspect `artists` using `head()`. How many rows are returned?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Still within the same code block, call `summary()` on `artists` and run the code to view the summary statistics of the data frame.

```

title: "Introduction to Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
# load data frame
artists <- read_csv('artists.csv')
```

```{r}
# inspect data frame
artists
head(artists)
summary(artists)
```
```

### Introduction to Data Frames in R

# Piping

One of the most appealing aspects of dplyr is the ability to easily manipulate

[data frames](#)

Preview: Docs Data frames are objects that store data into two dimensions of columns and rows.

. Each of the dplyr

[functions](#)

Preview: Docs Loading link description



you will explore takes a data frame as its first argument.

The *pipe operator*, or `%>%`, helps increase the readability of data frame code by piping the value on its left into the first argument of the function that follows it. For example:

```
df %>%
 head()
```

Copy to Clipboard

pipes the data frame `df` into the first argument of `head()`, becoming

```
head(df)
```

Copy to Clipboard

The true power of pipes comes from the ability to link multiple function calls together. Once you learn some of dplyr's functions, we'll revisit pipes and see how they are so useful!

Note: the pipe operator is *not* a part of base R. It comes from the `magrittr` package, but do not worry about loading `magrittr` in your code. Any time you load a package from the tidyverse, like dplyr, `%>%` will automatically be loaded!

### Instructions

Checkpoint 1 Enabled

1.

Use a pipe `%>%` to inspect `artists` with `head()`.

```

title: "Introduction to Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}  
# load libraries  
library(readr)  
library(dplyr)  
```\n\n```{r message=FALSE}  
load data frame
artists <- read_csv('artists.csv')
```\n\n```{r}  
# inspect data frame with pipe  
artists %>%  
  head()  
```\n
```

## Introduction to Data Frames in R

### Selecting Columns

Suppose you have a data frame called `customers`, which contains the ages of your business's customers:

name	age	gender
------	-----	--------

Rebecca Erikson	35	F
Thomas Roberson	28	M
Diane Ochoa	42	NA

For your analysis, you only care about the age and gender of your customers, not their names. The data frame you want looks like this:

age	gender
35	F
28	M
42	NA

You can select the appropriate columns for your analysis using `dplyr`'s `select()` function:

```
select(customers, age, gender)
```

Copy to Clipboard

`select()` takes a data frame as its first argument  
all additional arguments are the desired columns to select  
`select()` returns a new data frame containing only the desired columns

But what about the pipe `%>%`, you ask? Great question. You can simplify the readability of your code by using the pipe:

```
customers %>%
 select(age, gender)
```

Copy to Clipboard

When using the pipe, you can read the code as: from the `customers` table, `select()` the `age` and `gender` columns. From now on we will use the pipe symbol where appropriate to simplify our code.

### Instructions

Checkpoint 1 Enabled

1.

Select the `group` column of `artists` using `select()` and save the result to `artist_groups`. View `artist_groups`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Select the `group`, `spotify_monthly_listeners`, and `year_founded` columns of `artists` using `select()` and save the result to `group_info`. View `group_info`.

```

title: "Introduction to Data Frames in R"
```

```

output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
# load data frame
artists <- read_csv('artists.csv')
```

```{r}
# select one column
artist_groups <- artists %>%
  select(group)
artist_groups
```

```{r}
# select multiple columns
group_info <- artists %>%
  select(group,spotify_monthly_listeners,year_founded)
group_info
```

```

## Introduction to Data Frames in R

# Excluding Columns

Sometimes rather than specify what columns you want to select from a data frame, it's easier to state what columns you do not want to select. `dplyr`'s `select()` function also enables you to do just that! Consider a `customers` data frame that contains biographical information for the customers of your business:

| name         | address        | phone        | age |
|--------------|----------------|--------------|-----|
| Martha Jones | 123 Main St.   | 234-567-8910 | 28  |
| Rose Tyler   | 456 Maple Ave. | 212-867-5309 | 22  |
| Donna Noble  | 789 Broadway   | 949-123-4567 | 35  |

|              |                  |              |    |
|--------------|------------------|--------------|----|
| Amy Pond     | 98 West End Ave. | 646-555-1234 | 29 |
| Clara Oswald | 54 Columbus Ave. | 714-225-1957 | 31 |

You are interested in analyzing where your customers live and how old they are. For your analysis, you do not care about the `name` and `phone` associated with a customer, only their `address` and `age`. To exclude the columns you do not need:

```
customers %>%
 select(-name,-phone)
```

Copy to Clipboard

the data frame `customers` is piped into `select()`  
 the columns to remove, prepended with a `-`, are given as arguments  
 a new data frame without the `name` and `phone` columns is returned

## Instructions

Checkpoint 1 Enabled

1.

Select all columns of `artists` except `albums` using `select()` and save the result to `no_albums`. View `no_albums`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Select all columns of `artists` except `genre`, `spotify_monthly_listeners`, and `year_founded` using `select()` and save the result to `df_cols_removed`. View `df_cols_removed`.

```

title: "Introduction to Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
# load data frame
artists <- read_csv('artists.csv')
```

```{r}
# select all columns except one
no_albums <- artists %>%
  select(-albums)
no_albums
```

```{r}
# select all columns except a set
df_cols_removed <- artists %>%
```

```
select(-genre,-spotify_monthly_listeners,-year_founded)
df_cols_removed
``,`
```

Introduction to Data Frames in R

Filtering Rows with Logic I

In addition to subsetting a data frame by columns, you can also subset a data frame by rows using dplyr's `filter()` function and comparison operators! Consider an `orders` data frame that contains data related to the orders for an e-commerce shoe company:

id	first_name	last_name	email	shoe_type	shoe_material	shoe_color	price
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black	22
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy	32
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black	12
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red	22

Let's say you want to find all orders made by customers with the first name `'Joyce'`.

```
orders %>%
  filter(first_name == 'Joyce')
```

Copy to Clipboard

the `orders` data frame is piped into `filter()`
the condition `first_name == 'Joyce'` is given as an argument
a new data frame containing only the rows where `first_name == 'Joyce'` is returned

What if you have multiple conditions you want to be met? Not a problem! To find all orders made of faux-leather **AND** costing more than 25:

```
orders %>%
  filter(shoe_material == 'faux-leather', price > 25)
```

Copy to Clipboard

the `orders` data frame is again piped into `filter()`
the conditions `shoe_material == 'faux-leather'` **and** `price > 25` are given as arguments

a new data frame containing only the rows where both conditions were met is returned

You can provide any number of conditions that you please, as long as you separate each condition by a comma as its own argument. Note: each condition that you list must be met for a row to be returned!

Instructions

Checkpoint 1 Enabled

1.

Filter the rows of `artists` where the genre is 'Rock' and save the result to `rock_groups`. View `rock_groups`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Filter the rows of `artists` where the genre is 'Rock' **and** `spotify_monthly_listeners` is greater than 20000000. Save the result to `popular_rock_groups`, and view it.

```
---
title: "Introduction to Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
artists <- read_csv('artists.csv')
```

```{r}
filter rows one condition
rock_groups <- artists %>%
 filter(genre == 'Rock')
rock_groups
```

```{r}
filter rows multiple conditions
popular_rock_groups <- artists %>%
 filter(genre == 'Rock', spotify_monthly_listeners > 20000000)
popular_rock_groups
```
```

Introduction to Data Frames in R

Filtering Rows with Logic II

The `filter()` function also allows for more complex filtering with the help of logical operators! Take a look at the same `orders` data frame from the last exercise:

| id | first_name | last_name | email | shoe_type | shoe_material | shoe_color | price |
|----|------------|-----------|-------|-----------|---------------|------------|-------|
|----|------------|-----------|-------|-----------|---------------|------------|-------|

| | | | | | | | |
|-------|---------|----------|------------------------------|--------------|--------------|-------|----|
| 54791 | Rebecca | Lindsay | RebeccaLindsay57@hotmail.com | clogs | faux-leather | black | 22 |
| 53450 | Emily | Joyce | EmilyJoyce25@gmail.com | ballet flats | faux-leather | navy | 32 |
| 91987 | Joyce | Waller | Joyce.Waller@gmail.com | sandals | fabric | black | 12 |
| 14437 | Justin | Erickson | Justin.Erickson@outlook.com | clogs | faux-leather | red | 22 |

You are interested in seeing all orders that were for 'clogs' **OR** that cost less than 20. Using the or operator (`|`):

```
orders %>%
  filter(shoe_type == 'clogs' | price < 20)
```

Copy to Clipboard

the `orders` data frame is piped into `filter()`
the compound conditional statement `shoe_type == 'clogs' OR price < 20` is given as an argument
a new data frame is returned containing only rows where `shoe_type` is 'clogs' or `price` is less than 20

What if you want to find all orders where shoes in any color **but** red were purchased. Using the not or bang operator (`!`):

```
orders %>%
  filter(!(shoe_color == 'red'))
```

Copy to Clipboard

`orders` is again piped into `filter()`
the condition that should *not* be met is wrapped in parentheses, preceded by `!`, and given as an argument to `filter()`
a new data frame is returned containing only rows where `shoe_color` is not 'red'

Instructions

Checkpoint 1 Enabled

1. Filter the rows of `artists` where the country is 'South Korea' **or** the `year_founded` is before 2000. Save the result to `korea_or_before_2000`, and view it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2. Filter the rows of `artists` where the genre is **not** 'Rock'. Save the result to `not_rock_groups`, and view it.

```
---
title: "Introduction to Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
```

```

library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
artists <- read_csv('artists.csv')
```

```{r}
filter rows with or
korea_or_before_2000 <- artists %>%
 filter(country == 'South Korea' | year_founded < 2000)
korea_or_before_2000
```

```{r}
filter rows with not !
not_rock_groups <- artists %>%
 filter(!(genre == 'Rock'))
not_rock_groups
```

```

Arranging Rows

Sometimes all the data you want is in your data frame, but it's all unorganized! Step in the handy dandy dplyr function `arrange()`! `arrange()` will sort the rows of a data frame in ascending order by the column provided as an argument.

For numeric columns, ascending order means from lower to higher numbers. For character columns, ascending order means alphabetical order from A to Z.

Let's look back at the `customers` data frame for your company:

| name | age | address | phone |
|--------------|-----|----------------|--------------|
| Martha Jones | 28 | 123 Main St. | 234-567-8910 |
| Rose Tyler | 22 | 456 Maple Ave. | 212-867-5309 |

| | | | |
|--------------|----|------------------|--------------|
| Donna Noble | 35 | 789 Broadway | 949-123-4567 |
| Amy Pond | 29 | 98 West End Ave. | 646-555-1234 |
| Clara Oswald | 31 | 54 Columbus Ave. | 714-225-1957 |

To arrange the customers in ascending order by name:

```
customers %>%
  arrange(name)
```

Copy to Clipboard

the `customers` data frame is piped into `arrange()`
 the column to order by, `name`, is given as an argument
 a new data frame is returned with rows in ascending order by `name`

`arrange()` can also order rows by descending order! To arrange the customers in descending order by age:

```
customers %>%
  arrange(desc(age))
```

Copy to Clipboard

the `customers` data frame is again piped into `arrange()`
 the column to order by, `age`, is given as an argument to `desc()`, which is then given as an argument to `arrange()`
 a new data frame is returned with rows in descending order by `age`

If multiple arguments are provided to `arrange()`, it will order the rows by the column given as the first argument and use the additional columns to break ties in the values of preceding columns.

Instructions

Checkpoint 1 Enabled

1.

Arrange the rows of `artists` in *ascending* order by `group`. Save the result to `group_asc`, and view it.
 Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Arrange the rows of `artists` in *descending* order by `youtube_subscribers`. Save the result to `youtube_desc`, and view it.
 Make sure to click the arrows in the rendered notebook to see the entire table!

```
---
title: "Introduction to Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```
```

```

```{r message=FALSE}
load data frame
artists <- read_csv('artists.csv')
```

```{r}
arrange rows in ascending order
group_asc <- artists %>%
 arrange(group)
group_asc
```

```{r}
arrange rows in descending order
youtube_desc <- artists %>%
 arrange(desc(youtube_subscribers))
youtube_desc
```

```

Introduction to Data Frames in R

Review

There you have it! With the power of readr and dplyr in your hands, you can now:

- load data from a CSV into a data frame
- inspect the data frame with `head()` and `summary()`
- `select()` the columns you want to analyze
- `filter()` the rows with comparison and logical operators
- `arrange()` rows in ascending or descending order

You've also been exposed to the pipe `%>%`, a powerful tool for chaining function calls, as well as the general principles of data manipulation.

Now that you are well on your way to being a dplyr master, let's combine what you have learned together to perform an analysis and see the true power of the pipe!

Instructions

Checkpoint 1 Enabled

1.

The code in `notebook.Rmd` completes a sequence of steps:

- columns are selected from `artists` and saved to `chosen_cols`
- `chosen_cols` is filtered and saved to `popular_not_hip_hop`
- `popular_not_hip_hop` is arranged and saved to `youtube_desc`

Notice that to arrive at this result, two intermediate variables `chosen_cols` and `popular_not_hip_hop` were created.

With the power of the pipe, we can clean up this code!

In the last code block, `select()` all columns except `country`, `year_founded`, and `albums` from `artists` using the pipe `%>%`. Save the result to `artists` and view the `head()`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Place a pipe `%>%` after the call to `select()`. This will pipe your selection to the next line, where you should `filter()` all rows where `spotify_monthly_listeners` is greater than 20000000 and `genre` is not equal to 'Hip Hop'. Keep this data frame saved to `artists`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Place a pipe `%>%` after the call to `filter()`. This will pipe your filtered data frame to the next line, where you should `arrange()` the rows in descending order by `youtube_subscribers`. Keep this data frame saved to `artists`.

Did you get the same result as the previous code block?

Make sure you've called `head(artists)` to see the resulting data frame!

```
---
title: "Introduction to Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
artists <- read_csv('artists.csv')
```

```{r}
select columns
chosen_cols <- artists %>%
 select(-country, -year_founded, -albums)
head(chosen_cols)
```

```{r}
filter rows
popular_not_hip_hop <- chosen_cols %>%
 filter(spotify_monthly_listeners > 20000000, genre != 'Hip Hop')
head(popular_not_hip_hop)
```

```{r}
arrange rows
youtube_desc <- popular_not_hip_hop %>%
 arrange(desc(youtube_subscribers))
head(youtube_desc)
```

```{r}
select columns, filter and arrange rows
artists <- artists %>%
 select(-country, -year_founded, -albums) %>%
 filter(spotify_monthly_listeners > 20000000, genre != 'Hip Hop') %>%
 arrange(desc(youtube_subscribers))
head(artists)
```
```

Modifying Data Frames in R

Introduction

When working with

[data frames](#)

Preview: Docs Loading link description

, you often need to modify the columns for your analysis at hand. With the help of the dplyr package, data frame modifications are easily performed.

In this lesson, you'll learn how to modify an existing data frame using dplyr. Some of the skills you'll learn include:

- adding columns to an existing data frame

- adding new columns and dropping existing columns from a data frame

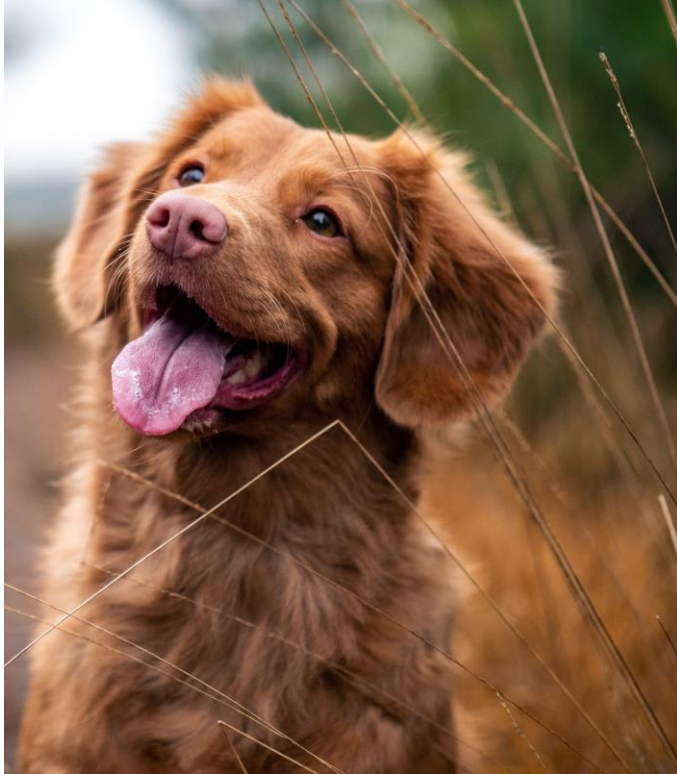
- renaming columns

Instructions

1. In `notebook.Rmd` data from the [American Kennel Club \(AKC\)](#) is loaded into a data frame. The data covers two categories of dog data:
 - [dog breed size](#), including lower and upper bounds for height (inches) and weight (pounds)
 - [dog breed popularity](#), including popularity ranks from 2013, 2014, 2015 and 2016

Review the code and take a look at the rendered notebook. What do you think the fourth and fifth code blocks accomplish?

Proceed to the next exercise when you're ready to start working with this data and modifying data frames!



```
---
title: "Modifying Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r}
load data frame
dogs <- read_csv('dogs.csv')
```

```{r}
inspect data frame
head(dogs)
```

```{r}
rename columns
dogs <- dogs %>%
 rename(popularity_2016 = rank_2016,
 popularity_2015 = rank_2015,
 popularity_2014 = rank_2014,
 popularity_2013 = rank_2013)
colnames(dogs)
```

```

```{r}
# calculate average height, average weight, and change in popularity from 2013 to 2016
dogs <- dogs %>%
  transmute(breed = breed,
            avg_height = (height_low_inches + height_high_inches)/2,
            avg_weight = (weight_low_lbs + weight_high_lbs)/2,
            pop_change_13_to_16 = popularity_2016 - popularity_2013)
head(dogs)
```

```

## Modifying Data Frames in R

# Adding a Column

Sometimes you might want to add a new column to a data frame. This new column could be a calculation based on the data that you already have.

Suppose you own a hardware store called The Handy Woman and have a data frame containing inventory information:

| product_id | product_description | cost_to_manufacture | price |
|------------|---------------------|---------------------|-------|
| 1          | 3 inch screw        | 0.50                | 0.75  |
| 2          | 2 inch nail         | 0.10                | 0.25  |
| 3          | hammer              | 3.00                | 5.50  |
| 4          | screwdriver         | 2.50                | 3.00  |

You can add a new column to the data frame using the `mutate()` function. `mutate()` takes a name-value pair as an argument. The name will be the name of the new column you are adding, and the value is an expression defining the values of the new column in terms of the existing columns. `mutate()` returns a new data frame with the added column.

Maybe you want to add a column to your inventory table with the amount of sales tax that is charged for each item. The following code multiplies each `price` by 0.075, the sales tax in your state:

```

df %>%
 mutate(sales_tax = price * 0.075)

```

Copy to Clipboard

Now the inventory table has a column called `sales_tax`, where the value is `0.075 * price`:

| product_id | product_description | cost_to_manufacture | price | sales_tax |
|------------|---------------------|---------------------|-------|-----------|
| 1          | 3 inch screw        | 0.50                | 0.75  | 0.06      |
| 2          | 2 inch nail         | 0.10                | 0.25  | 0.02      |
| 3          | hammer              | 3.00                | 5.50  | 0.41      |
| 4          | screwdriver         | 2.50                | 3.00  | 0.22      |

## Instructions

Checkpoint 1 Enabled

1.

The data from the American Kennel Club on dog breed size and popularity has been loaded into a data frame `dogs`.

Inspect the data frame using `head()`. Make sure to click the arrows in the rendered notebook to explore each column of the data frame.

You are about to modify this data frame!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add a new column to `dogs` named `avg_height` that is the average of `height_low_inches` and `height_high_inches`. Save this new data frame to `dogs`.

Use `head()` to inspect the new data frame, and use the arrows to navigate to the last column of the data frame, which will now be `avg_height`.

```

title: "Modifying Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
# load data frame
dogs <- read_csv('dogs.csv')
```

```{r}
# inspect data frame
head(dogs)
```
```

```
```{r}
# add average height column
dogs <- dogs %>%
  mutate(avg_height = (height_low_inches + height_high_inches)/2)
head(dogs)
```
```

## Modifying Data Frames in R

# Adding Multiple Columns

Let's refer back to the inventory table for your store, The Handy Woman.

| product_id | product_description | cost_to_manufacture | price | sales_tax |
|------------|---------------------|---------------------|-------|-----------|
| 1          | 3 inch screw        | 0.50                | 0.75  | 0.06      |
| 2          | 2 inch nail         | 0.10                | 0.25  | 0.02      |
| 3          | hammer              | 3.00                | 5.50  | 0.41      |
| 4          | screwdriver         | 2.50                | 3.00  | 0.22      |

You want to add two more new columns to your table. One column will contain the profit made from selling each item ( $\text{price} - \text{cost\_to\_manufacture}$ ), and the other will state whether the item is currently in stock (suppose every item is currently in stock).

`mutate()` can take multiple arguments to add any number of new columns to a data frame:

```
df %>%
 mutate(profit = price - cost_to_manufacture,
 in_stock = TRUE)
```

Copy to Clipboard

`mutate()` takes two arguments, defining new columns `profit` and `in_stock`. `profit` is equal to `price` minus `cost_to_manufacture`. `in_stock`, rather than be derived from values in existing columns, is given the value `TRUE` for all rows.

The inventory table will now look like this:



| product_id | product_description | cost_to_manufacture | price | sales_tax | profit | in_stock |
|------------|---------------------|---------------------|-------|-----------|--------|----------|
| 1          | 3 inch screw        | 0.50                | 0.75  | 0.06      | 0.25   | TRUE     |
| 2          | 2 inch nail         | 0.10                | 0.25  | 0.02      | 0.15   | TRUE     |
| 3          | hammer              | 3.00                | 5.50  | 0.41      | 2.5    | TRUE     |
| 4          | screwdriver         | 2.50                | 3.00  | 0.22      | 0.5    | TRUE     |

## Instructions

Checkpoint 1 Enabled

1.

Update the call to `mutate()` by adding an additional argument that adds a new column `avg_weight` that is the average of `weight_low_lbs` and `weight_high_lbs`.

Use `head()` to inspect the new data frame.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

You want to see how the popularity of dog breeds has changed from 2013 to 2016. The change in rank can be calculated by subtracting the earlier rank from the later rank.

Update the call to `mutate()` by adding one last argument that adds a new column

`rank_change_13_to_16` that is the change in rank from 2013 to 2016 (`rank_2016 - rank_2013`).

```

title: "Modifying Data Frames in R"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
# load data frame
dogs <- read_csv('dogs.csv')
```

```{r}
# inspect data frame
head(dogs)
```

```{r}
# add average height, average weight and rank change columns
```

```
dogs <- dogs %>%
  mutate(avg_height = (height_low_inches + height_high_inches)/2,
         avg_weight = (weight_low_lbs + weight_high_lbs)/2,
         rank_change_13_to_16 = rank_2016 - rank_2013)
head(dogs)
` ``
```

Modifying Data Frames in R

Transmute Columns

When creating new columns from a data frame, sometimes you are interested in only keeping the new columns you add, and removing the ones you do not need. dplyr's `transmute()` function will add new columns while dropping the existing columns that may no longer be useful for your analysis. Let's go back to the original inventory data frame for your store, The Handy Woman.

product_id	product_description	cost_to_manufacture	price
1	3 inch screw	0.50	0.75
2	2 inch nail	0.10	0.25
3	hammer	3.00	5.50
4	screwdriver	2.50	3.00

Like `mutate()`, `transmute()` takes name-value pairs as arguments. The names will be the names of the new columns you are adding, and the values are expressions defining the values of the new columns. The difference, however, is that `transmute()` returns a data frame with *only* the new columns.

To add `sales_tax` and `profit` columns while dropping all other columns from the data frame:

```
df %>%
  transmute(sales_tax = price * 0.075,
           profit = price - cost_to_manufacture)
```

Copy to Clipboard

This inventory table will now look like this:

sales_tax	profit
-----------	--------

0.06

0.25

0.02

0.15

0.41

2.5

0.22

0.5

Instructions

Checkpoint 1 Enabled

1.

Update the code in the last code block to add the columns `avg_height`, `avg_weight`, and `rank_change_13_to_16` to `dogs` while dropping all existing columns.

Inspect the new data frame with `head()`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The new columns have been added, and all the old columns have been dropped. But, wait! What `breed` do the column values refer to?

Add `breed` back into the `dogs` data frame by adding the following line of code as the first argument of the call to `transmute()`:

```
breed = breed
```

Copy to Clipboard

This will keep the `breed` column from being dropped in the transmuted data frame!

```
---
title: "Modifying Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
dogs <- read_csv('dogs.csv')
```

```{r}
inspect data frame
head(dogs)
```

```{r}
update the function call to drop all existing columns
dogs <- dogs %>%
```

```
transmute(breed = breed,
 avg_height = (height_low_inches + height_high_inches)/2,
 avg_weight = (weight_low_lbs + weight_high_lbs)/2,
 rank_change_13_to_16 = rank_2016 - rank_2013)
head(dogs)
````
```

Modifying Data Frames in R

Rename Columns

Since dplyr functions operate on data frames using column names, it is often useful to update the column names of a data frame so they are as clear and meaningful as possible. dplyr's `rename()` function allows you to easily do this.

Say you have a data frame of books, as shown in the table below:

| name | written_by |
|--|--------------------------|
| The Lord of the Rings | J. R. R. Tolkien |
| Le Petit Prince | Antoine de Saint-Exupéry |
| Harry Potter and the Philosopher's Stone | J. K. Rowling |
| 紅樓夢/红楼梦 | Cao Xueqin |

`rename()` can take any number of arguments, where each new column name is assigned to replace an old column name in the format `new_column_name = old_column_name`. `rename()` returns a new data frame with the updated column names.

To update the `name` column to `book_title` and the `written_by` column to `author`:

```
df %>%
  rename(book_title = name,
         author = written_by)
```

Copy to Clipboard

You can confirm the names of the columns have been updated using either of the base R functions `names()` or `colnames()`, which take a data frame as an argument and return a vector containing the column names.

Instructions

Checkpoint 1 Passed

1.

The updated `dogs` data frame from the previous exercise is given to you in `notebook.Rmd`. Save the column names of `dogs` to `original_col_names` and print it.

Checkpoint 2 Passed

2.

Update the name of `avg_height` to `avg_height_inches`, `avg_weight` to `avg_weight_lbs`, and `rank_change_13_to_16` to `popularity_change_13_to_16`. Save the updated data frame to `dogs`.

Checkpoint 3 Passed

3.

Save the new column names of `dogs` to `new_col_names` and print it.

```
---
title: "Modifying Data Frames in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
dogs <- read_csv('dogs.csv')
```

```{r message=FALSE}
add columns and remove existing columns
dogs <- dogs %>%
 transmute(breed = breed,
 avg_height = (height_low_inches + height_high_inches)/2,
 avg_weight = (weight_low_lbs + weight_high_lbs)/2,
 rank_change_13_to_16 = rank_2016 - rank_2013)
```

```{r}
check column names
original_col_names <- colnames(dogs)
original_col_names
```

```{r}
rename data frame columns
dogs <- dogs %>%
 rename(avg_height_inches = avg_height,
 avg_weight_lbs = avg_weight,
 popularity_change_13_to_16 = rank_change_13_to_16)
```

```{r}
check column names
new_col_names <- colnames(dogs)
new_col_names
```
```

Modifying Data Frames in R

Review

With an understanding of how to manipulate

[data frames](#)

Preview: Docs Loading link description

with dplyr, you are well on your way to becoming a data analysis expert! In this lesson you learned how to:

add new columns to a data frame using `mutate()`

add new columns to a data frame and drop existing columns using `transmute()`

change the column names of a data frame using `rename()`



Instructions

Checkpoint 1 Enabled

1.

Add the two columns defined below to `dogs` while dropping all existing columns from the data frame:

```
height_average_feet is the average of height_low_inches and  
height_high_inches divided by 12, since there are 12 inches in 1 foot  
popularity_change_15_to_16 is the difference in rank from rank_2016 to  
rank_2015
```

Save the result to `dogs`, and inspect `dogs` using `head()`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Add `breed` back into the `dogs` data frame by adding the following line of code as the first argument of the call to `transmute()`:

```
breed = breed
```

Copy to Clipboard

This will keep the `breed` column from being dropped in the transmuted data frame!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Arrange the rows of `dogs` by descending popularity change from 2015 to 2016. Save the result to `dogs`.

```
---  
title: "Modifying Data Frames in R"  
output: html_notebook  
---
```

```

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
load data frame
dogs <- read_csv('dogs.csv')
```

```{r}
inspect data frame
head(dogs)
```

```{r}
add new columns, drop existing columns and arrange
dogs <- dogs %>%
 transmute(breed = breed,
 height_average_feet = ((height_low_inches + height_high_inches)/2)/12,
 popularity_change_15_to_16 = rank_2016 - rank_2015) %>%
 arrange(desc(popularity_change_15_to_16))
head(dogs)
```

```

| | |
|------------------------|---|
| Data | |
| %>% | |
| select | Select (var_name1,var_name2,...) |
| filter | filter(var_name1 > 2000, var_name2 == "lisbon") |
| arrange | arrange(desc(prices)) |
| | |
| Mutate (operation) | mutate(avg_height = (height_low_inches + height_high_inches)/2,
avg_weight = (weight_low_lbs + weight_high_lbs)/2,
rank_change_13_to_16 = rank_2016 - rank_2013) |
| Transmute (operations) | transmute(breed = breed,
avg_height = (height_low_inches + height_high_inches)/2,
avg_weight = (weight_low_lbs + weight_high_lbs)/2,
rank_change_13_to_16 = rank_2016 - rank_2013) |

| | |
|--------|--|
| rename | rename(book_title = name, author = written_by) |
| | |

Data Cleaning in R

Diagnose the Data

We often describe data that is easy to analyze and visualize as “tidy data”. What does it mean to have tidy data?

For data to be tidy, it must have:

- Each variable as a separate column
- Each row as a separate observation

For example, we would want to reshape a table like:

Account

Checkings

Savings

| | | |
|------------|-------|-------|
| "12456543" | 8500 | 8900 |
| "12283942" | 6410 | 8020 |
| "12839485" | 78000 | 92000 |

Into a table that looks more like:

| Account | Account Type | Amount |
|------------|--------------|--------|
| "12456543" | "Checking" | 8500 |
| "12456543" | "Savings" | 8900 |
| "12283942" | "Checking" | 6410 |
| "12283942" | "Savings" | 8020 |
| "12839485" | "Checking" | 78000 |
| "12839485" | "Savings" | 92000 |

The first step of diagnosing whether or not a dataset is tidy is using base R and dplyr functions to explore and probe the dataset.

You've seen most of the functions we often use to diagnose a dataset for cleaning. Some of the most useful ones are:

- `head()` — display the first 6 rows of the table
- `summary()` — display the summary statistics of the table
- `colnames()` — display the column names of the table

Instructions

Checkpoint 1 Enabled

1.

Provided in `notebook.Rmd` are two data frames, `grocery_1` and `grocery_2`.

Begin by viewing the `head()` of both `grocery_1` and `grocery_2`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Explore the data frames using the other functions listed.

Which data frame is “clean”, tidy, and ready for analysis? Create a variable named `clean_data_frame` and assign it the value `1` if `grocery_1` is a clean and tidy data frame or `2` if `grocery_2` is a clean and tidy data frame.

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r message=FALSE}
read CSVs
grocery_1 <- read_csv('grocery_1.csv')
grocery_2 <- read_csv('grocery_2.csv')
```

```{r}
inspect data frames
head(grocery_1)
head(grocery_2)
summary(grocery_1)
summary(grocery_2)
print(colnames(grocery_1))
print(colnames(grocery_2))
```

```{r}
clean data frame
clean_data_frame <- 2
```
```

Data Cleaning in R

Dealing with Multiple Files

Often, you have the same data separated out into multiple files.

Let's say that you have a ton of files following the filename structure: `'file_1.csv'`, `'file_2.csv'`, `'file_3.csv'`, and so on. The power of `dplyr` and `tidyr` is mainly in being able to manipulate large amounts of structured data, so you want to be able to get all of the relevant information into one table so that you can analyze the aggregate data.

You can combine the base R

[functions](#)

Preview: Docs Loading link description

`list.files()` and `lapply()` with `readr` and `dplyr` to organize this data better, as shown below:

```
files <- list.files(pattern = "file .*csv")
```

```
df_list <- lapply(files, read_csv)
df <- bind_rows(df_list)
```

Copy to Clipboard

The first line uses `list.files()` and a [regular expression](#), a sequence of characters describing a pattern of text that should be matched, to find any file in the current directory that starts with `'file_'` and has an extension of `csv`, storing the name of each file in a vector `files`

The second line uses `lapply()` to read each file in `files` into a data frame with `read_csv()`, storing the [data frames](#)

Preview: Docs Loading link description

in `df_list`

The third line then concatenates all of those data frames together with dplyr's `bind_rows()` function

Instructions

Checkpoint 1 Enabled

1.

You have 10 different files containing 100 students each. These files follow the naming structure:

```
exams_0.csv
exams_1.csv
... up to exams_9.csv
```

You are going to read each file into an individual data frame and then combine all of the entries into one data frame.

First, create a variable called `student_files` and set it equal to the `list.files()` of all of the CSV files we want to import.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Read each file in `student_files` into a data frame using `lapply()` and save the result to `df_list`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Concatenate all of the data frames in `df_list` into one data frame called `students`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Inspect `students`. Save the number of rows in `students` to `nrow_students`. Did you get all of them?

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(readr)
library(dplyr)
```

```{r}
list files
student_files <- list.files(pattern = 'exams_.*csv')
print(student_files)
```

```{r message=FALSE}
read files
df_list <- lapply(student_files, read_csv)
```

```{r}
concatenate data frames
```

```
students <- bind_rows(df_list)
students
```



```
```{r}
# number of rows in students
nrow_students <- nrow(students)
print(nrow_students)
```
```


```

Data Cleaning in R

Reshaping your Data

Since we want

Each variable as a separate column

Each row as a separate observation

We would want to reshape a table like:

<i>Account</i>	<i>Checking</i>	<i>Savings</i>
"12456543"	8500	8900
"12283942"	6410	8020
"12839485"	78000	92000

Into a table that looks more like:

<i>Account</i>	<i>Account Type</i>	<i>Amount</i>
"12456543"	"Checking"	8500
"12456543"	"Savings"	8900
"12283942"	"Checking"	6410
"12283942"	"Savings"	8020

"12839485"	"Checking"	78000
"12839485"	"Savings"	920000

We can use tidy's `gather()` function to do this transformation. `gather()` takes a data frame and the columns to unpack:

```
df %>%
  gather('Checking', 'Savings', key='Account Type', value='Amount')
```

Copy to Clipboard

The arguments you provide are:

`df`: the data frame you want to gather, which can be piped into `gather()`

`Checking` and `Savings`: the columns of the old data frame that you want to turn into variables

`key`: what to call the column of the new data frame that stores the variables

`value`: what to call the column of the new data frame that stores the values

Instructions

Checkpoint 1 Enabled

1.

The `students` data frame from the previous exercise has been loaded into the notebook for you. Save the columns names to `original_col_names` and print it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

There is a column for the scores on the `fractions` exam, and a column for the scores on the `probability` exam.

We want to make each row an observation, so we want to transform this table to look like:

<i>full_name</i>	<i>exam</i>	<i>score</i>	<i>gender_age</i>	<i>grade</i>
"First Student"	"fractions"	score%	...	...
"First Student"	"probability"	score%	...	...
"Second Student"	"fractions"	score%	...	...
"Second Student"	"probability"	score%	...	...
...	...	...	...	...

Use `gather` to create a new table (still called `students`) that follows this structure. Then view the `head()` of `students`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Save the columns names of the updated `students` data frame to `gathered_col_names` and print it.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

The dplyr function `count()` takes a data frame and a column as arguments and returns a table with counts of the unique values in the named column.

Find the count of each unique value in the `exam` column. Save the result to `exam_counts` and view `exam_counts`.

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(dplyr)
library(tidyr)
```

```{r}
load students data frame
load("students.Rda")
```

```{r}
original column names
original_col_names <- colnames(students)
print(original_col_names)
```

```{r}
gather columns
students <- students %>%
 gather('fractions', 'probability', key='exam', value='score')
head(students)
```

```{r}
updated column names
gathered_col_names <- colnames(students)
print(gathered_col_names)
```

```{r}
unique value counts of exam
exam_counts <- students %>%
 count(exam)
exam_counts
```
```

Data Cleaning in R

Dealing with Duplicates

Often we see duplicated rows of data in the data frames we are working with. This could happen due to errors in data collection or in saving and loading the data.

To check for duplicates, we can use the base R function `duplicated()`, which will return a logical vector telling us which rows are duplicate rows.

Let's say we have a data frame `fruits` that represents this table:

| <i>item</i> | <i>price</i> | <i>calories</i> |
|--------------|--------------|-----------------|
| "banana" | "\$1" | 105 |
| "apple" | "\$0.75" | 95 |
| "apple" | "\$0.75" | 95 |
| "peach" | "\$3" | 55 |
| "peach" | "\$4" | 55 |
| "clementine" | "\$2.5" | 35 |

If we call `fruits %>% duplicated()`, we would get the following vector:

```
>> [1] FALSE FALSE TRUE FALSE FALSE FALSE
```

Copy to Clipboard

We can see that the third row, which represents an "apple" with price "\$0.75" and 95 calories, is a duplicate row. Every value in this row is the same as in another row (the previous row).

We can use the dplyr `distinct()` function to remove all rows of a data frame that are duplicates of another row.

If we call `fruits %>% distinct()`, we would get the table:

| <i>item</i> | <i>price</i> | <i>calories</i> |
|-------------|--------------|-----------------|
| "banana" | "\$1" | 105 |

| | | |
|--------------|----------|----|
| "apple" | "\$0.75" | 95 |
| "peach" | "\$3" | 55 |
| "peach" | "\$4" | 55 |
| "clementine" | "\$2.5" | 35 |

The "apple" row was deleted because it was exactly the same as another row. But the two "peach" rows remain because there is a difference in the *price* column.

If we wanted to remove every row with a duplicate value in the *item* column, we could specify a `subset`:

```
fruits %>%
  distinct(item, .keep_all=TRUE)
```

Copy to Clipboard

By default, this keeps the first occurrence of the duplicate:

| <i>item</i> | <i>price</i> | <i>calories</i> |
|--------------|--------------|-----------------|
| "banana" | "\$1" | 105 |
| "apple" | "\$0.75" | 95 |
| "peach" | "\$3" | 55 |
| "clementine" | "\$2.5" | 35 |

Make sure that the columns you drop duplicates from are specifically the ones where duplicates don't belong. You wouldn't want to drop duplicates with the *price* column as a subset, for example, because it's okay if multiple items cost the same amount!

Instructions

Checkpoint 1 Enabled

1.

The `students` data frame has a column `id` that is neither unique nor required for our analysis. Drop the `id` column from the data frame and save the result to `students`. View the `head()` of `students`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

It seems like in the data collection process, some rows may have been recorded twice. Use the `duplicated()` function on the `students` data frame to make a vector object called `duplicates`.
Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

`table()` is a base R function that takes any R object as an argument and returns a table with the counts of each unique value in the object.

Pipe the result from the previous checkpoint into `table()` to see how many rows are exact duplicates.

Make sure to save the result to `duplicate_counts`, and view `duplicate_counts`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Get rid of the duplicate rows in the `students` data frame and save this new data frame as `unique_students`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Use the `duplicated()` function again to make an object called `updated_duplicates` after dropping the duplicates. Pipe the result into `table()` to see if any duplicates remain, and view `updated_duplicates`.

Are there any `TRUE`s left?

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(dplyr)
library(tidyr)
```

```{r}
load students data frame
load("students.Rda")
```

```{r}
drop id column
students <- students %>%
 select(-id)
head(students)
```

```{r}
find duplicated rows
duplicates <- students %>%
 duplicated()
```

```{r}
count duplicated rows
duplicate_counts <- duplicates %>%
 table()
duplicate_counts
```

```{r}
remove duplicated rows, keep unique rows
unique_students <- students %>%
 distinct()
```

```

```
```{r}
find and count duplicated rows in updated data frame
updated_duplications <- unique_students %>%
 duplicated() %>%
 table()
updated_duplications
```

```

Data Cleaning in R

Splitting By Index

In trying to get clean data, we want to make sure each column represents one type of measurement. Often, multiple measurements are recorded in the same column, and we want to separate these out so that we can do individual analysis on each variable.

Let's say we have a column "birthday" with data formatted in MMDDYYYY format. In other words, "11011993" represents a birthday of November 1, 1993. We want to split this data into day, month, and year so that we can use these columns as separate features.

In this case, we know the exact structure of these strings. The first two characters will always correspond to the month, the second two to the day, and the rest of the string will always correspond to year. We can easily break the data into three separate columns by splitting the strings into substrings using `str_sub()`, a helpful function from the stringr package:

```

# Create the 'month' column
df %>%
  mutate(month = str_sub(birthday,1,2))

# Create the 'day' column
df %>%
  mutate(day = str_sub(birthday,3,4))

# Create the 'year' column
df %>%
  mutate(year = str_sub(birthday,5))

```

Copy to Clipboard

The first command takes the characters starting at index 1 and ending at index 2 of each value in the `birthday` column and puts it into a `month` column.

The second command takes the characters starting at index 3 and ending at index 4 of each value in the `birthday` column and puts it into a `day` column.

The third command takes the characters starting at index 5 and ending at the end of the value in the `birthday` column and puts it into a `year` column.

This would transform a table like:

| <i>id</i> | <i>birthday</i> |
|-----------|-----------------|
| 1011 | "12241989" |

1112 "10311966"

1113 "01052011"

into a table like:

| <i>id</i> | <i>birthday</i> | <i>month</i> | <i>day</i> | <i>year</i> |
|-----------|-----------------|--------------|------------|-------------|
| 1011 | "12241989" | "12" | "24" | "1989" |
| 1112 | "10311966" | "10" | "31" | "1966" |
| 1113 | "01052011" | "01" | "05" | "2011" |

We will practice changing string columns into numerical columns (like converting "10" to 10) in a future exercise.

Instructions

Checkpoint 1 Enabled

1.

Print out the columns of the `students` data frame.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The column `gender_age` sounds like it contains both `gender` and `age`!

View the `head()` of `students` to see what kind of data `gender_age` contains.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

It looks like the first character of the values in `gender_age` contains the gender, while the rest of the string contains the age. Let's separate out the gender data into a new column called `gender`. Save the result to `students`, and view the `head()`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Now, separate out the age data into a new column called `age`. Save the updated data frame to `students`, and view the `head()`.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Now, we don't need that `gender_age` column anymore. Drop `gender_age` from `students`, and save the result to `students`. View the `head()` of `students`.

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(dplyr)
```

```

library(stringr)
```

```{r}
load students data frame
load("students.Rda")
```

```{r}
print columns of students
print(colnames(students))
```

```{r}
view head of students
head(students)
```

```{r}
add gender and age columns
students <- students %>%
 mutate(gender = str_sub(gender_age,1,1),
 age = str_sub(gender_age,2))
head(students)
```

```{r}
drop gender_age column
students <- students %>%
 select(-gender_age)
head(students)
```

```

Data Cleaning in R

Splitting By Character

Let's say we have a column called "type" with data entries in the format "admin_US" or "user_Kenya", as shown in the table below.

| <i>id</i> | <i>type</i> |
|-----------|----------------|
| 1011 | "user_Kenya" |
| 1112 | "admin_US" |
| 1113 | "moderator_UK" |

Just like we saw before, this column actually contains two types of data. One seems to be the user type (with values like "admin" or "user") and one seems to be the country this user is in (with values like "US" or "Kenya").

We can no longer just split along the first 4 characters because `admin` and `user` are of different lengths. Instead, we know that we want to split along the `"_"`. We can thus use the tidy function `separate()` to split this column into two, separate columns:

```
# Create the 'user_type' and 'country' columns
df %>%
  separate(type, c('user_type', 'country'), '_')
```

Copy to Clipboard

`type` is the column to split

`c('user_type', 'country')` is a vector with the names of the two new columns

`'_'` is the character to split on

This would transform the table above into a table like:

| <i>id</i> | <i>user_type</i> | <i>country</i> |
|-----------|------------------|----------------|
| 1011 | "user" | "Kenya" |
| 1112 | "admin" | "US" |
| 1113 | "moderator" | "UK" |

Instructions

Checkpoint 1 Enabled

1.

View the `head()` of `students`. Notice that the students' names are stored in a column called `full_name`. Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Separate the `full_name` column into two new columns, `first_name` and `last_name`, by splitting on the `' '` character.

Provide as an extra argument to the `separate()` function `extra = 'merge'`. This will ensure that middle names or two-word last names will all end up in the `last_name` column.

Save the result to `students`, and view the `head()`.

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(dplyr)
library(tidy)
```

```{r}
load students data frame
load("students.Rda")
```

```{r}
```

```
view the head of students
head(students)
```



```
```{r}
# separate the full_name column
students <- students %>%
  separate(full_name, c('first_name', 'last_name'), ' ', extra = 'merge')
head(students)
```
```


```

Data Cleaning in R

Looking at Data Types

Each column of a data frame can hold items of the same *data type*. The data types that R uses are: character, numeric (real or decimal), integer, logical, or complex. Often, we want to convert between types so that we can do better analysis. If a numerical category like "num_users" is stored as a vector of `characters` instead of `numerics`, for example, it makes it more difficult to do something like make a line graph of users over time.

To see the types of each column of a data frame, we can use:

```
str(df)
```

Copy to Clipboard

`str()` displays the internal structure of an R object. Calling `str()` with a data frame as an argument will return a variety of information, including the data types. For a data frame like this:

<i>item</i>	<i>price</i>	<i>calories</i>
"banana"	"\$1"	105
"apple"	"\$0.75"	95
"peach"	"\$3"	55
"clementine"	"\$2.5"	35

the data types would be:

```
#> $ item:      chr
#> $ price:      chr
#> $ calories:   num
```

Copy to Clipboard

We can see that the `price` column is made up of `characters`, which will probably make our analysis of price more difficult. We'll look at how to convert columns into numeric values in the next few exercises.

Instructions

Checkpoint 1 Enabled

1.

Let's inspect the data types in the `students` table.

Print out the structure of `students`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

If we wanted to make a scatterplot of `age` vs average exam score, would we be able to do it with this type of data?

Paste the following code in the last code block to try and print out the mean of the `score` column of `students`.

```
students %>%  
  summarise(mean_score = mean(score))
```

Copy to Clipboard

What warning do you see?

```
---  
title: "Data Cleaning in R"  
output: html_notebook  
---  
  
```{r message=FALSE, warning=FALSE}  
load libraries
library(dplyr)
```  
  
```{r}  
load students data frame
load("students.Rda")
```  
  
```{r}  
print structure of students
print(str(students))
```  
  
```{r}  
mean of age column
students %>%
 summarise(mean_age = mean(age))
```
```

Data Cleaning in R

String Parsing

Sometimes we need to modify strings in our data frames to help us transform them into more meaningful metrics. For example, in our `fruits` table from before:

| <i>item</i> | <i>price</i> | <i>calories</i> |
|--------------|--------------|-----------------|
| "banana" | "\$1" | 105 |
| "apple" | "\$0.75" | 95 |
| "peach" | "\$3" | 55 |
| "peach" | "\$4" | 55 |
| "clementine" | "\$2.5" | 35 |

We can see that the 'price' column is actually composed of character strings representing dollar amounts. This column could be much better represented as numeric, so that we could take the mean, calculate other aggregate statistics, or compare different fruits to one another in terms of price.

First, we can use a regular expression, a sequence of characters that describe a pattern of text to be matched, to remove all of the dollar signs. The base R function `gsub()` will remove the \$ from the price column, replacing the symbol with an empty string '':

```
fruit %>%
  mutate(price=gsub('\\$', '', price))
```

Copy to Clipboard

Then, we can use the base R function `as.numeric()` to convert character strings containing numerical values to numeric:

```
fruit %>%
  mutate(price = as.numeric(price))
```

Copy to Clipboard

Now, we have a data frame that looks like:

| <i>item</i> | <i>price</i> | <i>calories</i> |
|-------------|--------------|-----------------|
| "banana" | 1 | 105 |
| "apple" | 0.75 | 95 |

| | | |
|--------------|-----|----|
| "peach" | 3 | 55 |
| "peach" | 4 | 55 |
| "clementine" | 2.5 | 35 |

Instructions

Checkpoint 1 Enabled

1.

We saw in the last exercise that finding the mean of the `score` column is hard to do when the data is stored as `characters` and not numbers.

View the `head()` of `students` to take a look at the values in the `score` column.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Remove the `'%'` symbol from the `score` column, and save the resulting data frame to `students`. View `students`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Convert the `score` column to a numerical type using the `as.numeric()` function. Save this new data frame to `students`, and view it.

```
---
title: "Data Cleaning in R"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries
library(dplyr)
```

```{r}
load students data frame
load("students.Rda")
```

```{r}
view head of students
head(students)
```

```{r}
remove % from score column
students <- students %>%
 mutate(score=gsub('%','',score))
head(students)
```

```{r}
change score column to numeric
students <- students %>%
 mutate(score=as.numeric(score))
head(students)
```
```

Data Cleaning in R

Review

Great! We have looked at a number of different methods we may use to get data into the format we want for analysis.

Specifically, we have covered:

- diagnosing the “tidiness” of data
- combining multiple files
- reshaping data
- changing the types of values
- manipulating
 - [strings](#)
- Preview: Docs Loading link description
 - to represent data better

You can use these methods to transform your datasets to be clean and easy to work with!

Instructions

Checkpoint 1 Failed, try again

1.

The `students` data frame is nearly cleaned and ready for analysis! There’s one more change that can be made to the ages of the students to help describe and visualize the data. What could that change be? What is the ideal data type for the ages column?

Make the change to `age` and save the resulting data frame to `students`.

```
---  
title: "Data Cleaning in R"  
output: html_notebook  
---  
  
```{r message=FALSE, warning=FALSE}  
load libraries
library(dplyr)
```  
  
```{r}  
load students data frame
load("students.Rda")
```  
  
```{r}  
view head of students
head(students)
```  
  
```{r}  
update age column
```

```
students <- students %>%
 mutate(age = as.numeric(age))
head(students)
` ` `
```

# Aggregates in R

## Introduction

In this lesson you will learn about aggregates in R using dplyr. An aggregate statistic is a way of creating a single number that describes a group of numbers. Common aggregate statistics include mean, median, and standard deviation.

Additionally, you will learn how you can group data into different subsets based on column values. This can help narrow the focus of a summary statistic to a subset of a dataset. R you ready to get started?

### Instructions

In the workspace to the right we have loaded data from ShoeFly.com, a fictional e-commerce shoe store. The data includes information regarding customer orders as well as the source of page visits to ShoeFly.com's website. Review the code and the output. What calculations are made, and on which subsets of the data do they occur? Throughout the rest of this lesson you will dig into ShoeFly.com's data to learn more about its customer's orders and how users are discovering the website. Proceed to the next exercise to get started!

### Aggregates in R

#### Calculating Column Statistics

In this exercise, you will learn how to *combine* all of the values from a column for a single calculation. This can be done with the help of the dplyr function `summarize()`, which returns a new data frame containing the desired calculation.

Some examples of this type of calculation include:

The data frame `customers` contains the names and ages of all of your customers. You want to find the median age:

```
customers %>%
 select(age)
c(23, 25, 31, 35, 35, 46, 62)
customers %>%
 summarize(median_age = median(age))
35
```

Copy to Clipboard

The data frame `shipments` contains address information for all shipments that you've sent out in the past year. You want to know how many different states you have shipped to.

```
shipments %>%
 select(states)
c('CA', 'CA', 'CA', 'CA', 'NY', 'NY', 'NJ', 'NJ', 'NJ', 'NJ', 'NJ', 'NJ', 'NJ')
shipments %>%
```

```
summarize(n_distinct_states = n_distinct(states))
3
```

Copy to Clipboard

The data frame `inventory` contains a list of types of t-shirts that your company makes. You want to know the standard deviation of the prices of your inventory.

```
inventory %>%
 select(price)
c(31, 23, 30, 27, 30, 22, 27, 22, 39, 27, 36)
inventory %>%
 summarize(sd_price = sd(price))
5.465595
```

Copy to Clipboard

The general syntax for these calculations is:

```
df %>%
 summarize(var_name = command(column_name))
```

Copy to Clipboard

`df` is the data frame you are working with  
`summarize` is a dplyr function that reduces multiple values to a single value  
`var_name` is the name you assign to the column that stores the results of the summary function in the returned data frame  
`command` is the summary function that is applied to the column by `summarize()`  
`column_name` is the name of the column of `df` that is being summarized

The following table includes common summary

### [functions](#)

Preview: Docs Loading link description

that can be given as an argument to `summarize()`:

Command	Description
<code>mean()</code>	Average of all values in column
<code>median()</code>	Median value of column
<code>sd()</code>	Standard deviation of column
<code>var()</code>	Variance of column

<code>min()</code>	Minimum value in column
<code>max()</code>	Maximum value in column
<code>IQR()</code>	Interquartile range of column
<code>n_distinct()</code>	Number of unique values in column
<code>sum()</code>	Sum values of column

## Instructions

Checkpoint 1 Enabled

1.

ShoeFly.com has a new batch of orders stored in the data frame `orders`. Inspect the first 10 rows of the data frame using `head()`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Our finance department wants to know the price of the most expensive pair of shoes purchased. Save your answer to the variable `most_expensive`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Woah, wait a minute! Take a look at the output of the code you just ran. The result for the most expensive pair of shoes is coming back as `NA`. Why is this happening?

If you scroll up in the rendered notebook to where `orders.csv` is loaded, you can see a warning about row 99 of the file. There is a missing column of information! It appears that the `price` for row 99 was not in the file, and this is causing your maximum value calculation to return `NA`.

Add the following as an additional argument to `max()` so that it removes all missing values before computing the maximum value.

```
na.rm = TRUE
```

Copy to Clipboard

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Our fashion department wants to know how many different colors of shoes we are selling. Save your answer to the variable `num_colors`.

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load order data
```

```

orders <- read_csv("orders.csv")

# inspect orders here:
head(orders, 10)
```

```{r}
# define most_expensive here:
most_expensive <- orders %>%
  summarize(max_price = max(price, na.rm = TRUE))
most_expensive
```

```{r}
# define num_colors here:
num_colors <- orders %>%
  summarize(n_distinct_shoe_color = n_distinct(shoe_color))
num_colors
```

```

## Aggregates in R

# Calculating Aggregate Functions I

When we have a bunch of data, we often want to calculate aggregate statistics (mean, standard deviation, median, percentiles, etc.) over certain subsets of the data.

Suppose we have a grade book with columns `student`, `assignment_name`, and `grade`:

| student | assignment_name | grade |
|---------|-----------------|-------|
| Amy     | Assignment 1    | 96    |
| Amy     | Assignment 2    | 87    |
| Bob     | Assignment 1    | 91    |
| Bob     | Assignment 2    | 75    |
| Chris   | Assignment 1    | 83    |

We want to get an average grade for each student across all assignments. We can do this using the helpful `dplyr` function `group_by()`.

For this example, we'd use the following piece of code:

```
grades <- df %>%
 group_by(student) %>%
 summarize(mean_grade = mean(grade))
```

Copy to Clipboard

The output might look something like this:

| student | mean_grade |
|---------|------------|
| Amy     | 91.5       |
| Bob     | 83         |
| Chris   | 85.5       |

In general, we use the following syntax to calculate aggregates:

```
df %>%
 group_by(column_1) %>%
 summarize(aggregate_name = command(column_2))
```

Copy to Clipboard

`column_1` (`student` in our example) is the column that we want to `group_by()`  
`column_2` (`grade` in our example) is the column that we want to apply `command()`, a summary function, to  
using `summarize()`  
`aggregate_name` is the name assigned to the calculated aggregate

In addition to the summary

### [functions](#)

Preview: Docs Loading link description

discussed in the last exercise (`mean()`, `median()`, `sd()`, `var()`, `min()`, `max()`, `IQR()` and `n_distinct()`), another helpful summary function, especially for grouped data, is `n()`. `n()` will return the count of the rows within a group, and does not require a column as an argument. To get the count of the rows in each group of students from our example:

```
grades <- df %>%
 group_by(student) %>%
 summarize(count = n())
```

Copy to Clipboard



## Instructions

Checkpoint 1 Enabled

1.

Let's return to our `orders` data from ShoeFly.com.

In the previous exercise, our finance department wanted to know the most expensive shoe that we sold. Now, they want to know the price of the most expensive shoe for each `shoe_type` (i.e., the price of the most expensive boot, the price of the most expensive ballet flat, etc.). Name the column that shows the most expensive shoe prices `max_price`.

Save your answer to the variable `pricey_shoes`, and view it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

The inventory team wants to know how many of each `shoe_type` has been sold so they can forecast inventory for the future.

Save your answer to the variable `shoes_sold`, and view it.

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")

# inspect orders
head(orders)
```

```{r}
# define pricey_shoes here:
pricey_shoes <- orders %>%
  group_by(shoe_type) %>%
  summarize(max_price = max(price, na.rm = TRUE))
pricey_shoes
```

```{r}
# define shoes_sold here:
shoes_sold <- orders %>%
  group_by(shoe_type) %>%
  summarize(count = n())
shoes_sold
```
```

Aggregates in R

# Calculating Aggregate Functions II

Sometimes, we want to group by more than one column. We can do this by passing multiple column names as arguments to the `group_by` function.

Imagine that we run a chain of stores and have data about the number of sales at different locations on different days:

| location     | date       | day_of_week | total_sales |
|--------------|------------|-------------|-------------|
| West Village | February 1 | W           | 400         |
| West Village | February 2 | Th          | 450         |
| Chelsea      | February 1 | W           | 375         |
| Chelsea      | February 2 | Th          | 390         |
| ...          | ...        | ...         | ...         |

We suspect that sales are different at different locations on different days of the week. In order to test this hypothesis, we could calculate the average sales for each store on each day of the week across multiple months. The code would look like this:

```
df %>%
 group_by(location, day_of_week) %>%
 summarize(mean_total_sales = mean(total_sales))
```

[Copy to Clipboard](#)

And the results might look something like this:

| location | day_of_week | mean_total_sales |
|----------|-------------|------------------|
| Chelsea  | M           | 402.50           |
| Chelsea  | Tu          | 422.75           |
| Chelsea  | W           | 452.00           |

|              |     |     |
|--------------|-----|-----|
| ...          | ... | ... |
| West Village | M   | 390 |
| West Village | Tu  | 400 |
| ...          | ... | ... |

## Instructions

Checkpoint 1 Passed

1.

At ShoeFly.com, our Purchasing team thinks that certain `shoe_type/shoe_color` combinations are particularly popular this year (for example, blue ballet flats are all the rage in Paris).

Find the total number of shoes of each `shoe_type/shoe_color` combination purchased using `group_by`, `summarize()` and `n()`. Name the aggregate count column `count`. Save your result to the variable `shoe_counts`, and view it.

Checkpoint 2 Passed

2.

The Marketing team wants to better understand the different price levels of the kinds of shoes that have been sold on the website, in particular looking at `shoe_type/shoe_material` combinations.

Find the mean price of each `shoe_type/shoe_material` combination purchased using `group_by`, `summarize()` and `mean()`. Assign the name `mean_price` to the calculated aggregate. Save your result to the variable `shoe_prices`, and view it.

Don't forget to include `na.rm = TRUE` as an argument in the summary function that you call!

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")

# inspect orders
head(orders)
```

```{r}
# define shoe_counts here:
shoe_counts <- orders %>%
  group_by(shoe_type, shoe_color) %>%
```

```

    summarize(count = n())
shoe_counts
```

```{r}
# define shoe_prices here:
shoe_prices <- orders %>%
  group_by(shoe_type, shoe_material) %>%
  summarize(mean_price = mean(price, na.rm = TRUE))
shoe_prices
```

```

## Aggregates in R

# Combining Grouping with Filter

While `group_by()` is most often used with `summarize()` to calculate summary statistics, it can also be used with the dplyr function `filter()` to filter rows of a data frame based on per-group metrics.

Suppose you work at an educational technology company that offers online courses and collects user data in an `enrollments` data frame:

| user_id | course       | quiz_score |
|---------|--------------|------------|
| 1234    | learn_r      | 80         |
| 1234    | learn_python | 95         |
| 4567    | learn_r      | 90         |
| 4567    | learn_python | 55         |

You want to identify all the enrollments in difficult courses, which you define as courses with an average `quiz_score` less than 80. To filter the data frame to just these rows:

```

enrollments %>%
 group_by(course) %>%
 filter(mean(quiz_score) < 80)

```

Copy to Clipboard

`group_by()` groups the data frame by `course` into two groups: `learn-r` and `learn-python`. `filter()` will keep all the rows of the data frame whose per-group (per-course) average `quiz_score` is less than 80.

Rather than filtering rows by the individual column values, the rows will be filtered by their group value since a summary function is used! The resulting data frame would look like this:

| user_id | course       | quiz_score |
|---------|--------------|------------|
| 1234    | learn_python | 95         |
| 4567    | learn_python | 55         |

The average `quiz_score` for the `learn-r` course is 85, so all the rows of `enrollments` with a value of `learn-r` in the `course` column are filtered out.

The average `quiz_score` for the `learn-python` course is 75, so all the rows of `enrollments` with a value of `learn-python` in the `course` column remain.

## Instructions

Checkpoint 1 Enabled

1.

Your boss at ShoeFly.com wants to gain a better insight into the orders of the most popular `shoe_types`. Group orders by `shoe_type` and filter to only include orders with a `shoe_type` that has been ordered more than 16 times. Save the result to `most_pop_orders`, and view it.

You can include any of the summary functions as part of an argument to `filter()`, including `n()`!

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")

# inspect orders
head(orders)
```

```{r}
# define most_pop_orders here:
most_pop_orders <- orders %>%
  group_by(shoe_type) %>%
  filter(n() > 16)
most_pop_orders
```
```

## Aggregates in R

## Combining Grouping with Mutate

`group_by()` can also be used with the dplyr function `mutate()` to add columns to a data frame that involve per-group metrics.

Consider the same educational technology company's `enrollments` table from the previous exercise:

| user_id | course       | quiz_score |
|---------|--------------|------------|
| 1234    | learn_r      | 80         |
| 1234    | learn_python | 95         |
| 4567    | learn_r      | 90         |
| 4567    | learn_python | 55         |

You want to add a new column to the data frame that stores the difference between a row's `quiz_score` and the average `quiz_score` for that row's `course`. To add the column:

```
enrollments %>%
 group_by(course) %>%
 mutate(diff_from_course_mean = quiz_score - mean(quiz_score))
```

Copy to Clipboard

`group_by()` groups the data frame by course into two groups: `learn-r` and `learn-python`  
`mutate()` will add a new column `diff_from_course_mean` which is calculated as the difference between a row's individual `quiz_score` and the `mean(quiz_score)` for that row's group (course)

The resulting data frame would look like this:

| user_id | course       | quiz_score | diff_from_course_mean |
|---------|--------------|------------|-----------------------|
| 1234    | learn_r      | 80         | -5                    |
| 1234    | learn_python | 95         | 20                    |
| 4567    | learn_r      | 90         | 5                     |

The average `quiz_score` for the `learn-r` course is 85, so `diff_from_course_mean` is calculated as `quiz_score - 85` for all the rows of enrollments with a value of `learn-r` in the `course` column. The average `quiz_score` for the `learn-python` course is 75, so `diff_from_course_mean` is calculated as `quiz_score - 75` for all the rows of enrollments with a value of `learn-python` in the `course` column.

## Instructions

Checkpoint 1 Enabled

1.

You want to be able to tell how expensive each order is compared to the average `price` of orders with the same `shoe_type`.

Group orders by `shoe_type` and create a new column named `diff_from_shoe_type_mean` that stores the difference in price between an orders `price` and the average `price` of orders with the same `shoe_type`. Save the result to `diff_from_mean`, and view it.

Don't forget to include `na.rm = TRUE` as an argument in the summary function you call!

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")

# inspect orders
head(orders)
```

```{r}
# define diff_from_mean here:
diff_from_mean <- orders %>%
  group_by(shoe_type) %>%
  mutate(diff_from_shoe_type_mean = price - mean(price, na.rm=TRUE))
diff_from_mean
```
```

## Aggregates in R

# Review

This lesson introduced you to aggregates in R using dplyr. You learned:

How to calculate summary statistics with `summarize()`

How to perform *aggregate* statistics over individual rows with the same value or values using `group_by()`

## Instructions

Checkpoint 1 Enabled

1.

Let's examine some more data from ShoeFly.com. This time, in addition to the `orders` data, we'll be looking at data about user visits to the website, stored in the `page_visits` data frame. Inspect the columns of the data frames using the rendered notebook.

Find the average `price` of an order in the `orders` data frame using `summarize()` and the `mean()` summary function. Save the resulting data frame to a variable named `average_price` and view it.

Don't forget to include `na.rm = TRUE` as an argument in the call to `mean()`!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

In the `page_visits` data frame, the column `utm_source` contains information about how users got to ShoeFly's homepage. For instance, if `utm_source = Facebook`, then the user came to ShoeFly by clicking on an ad on Facebook.com.

Use a `group_by` statement to calculate how many visits came from each of the different sources. Save your answer to the variable `click_source`, and view it.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Our Marketing department thinks that the traffic to our site has been changing over the past few months.

Use `group_by` to calculate the number of visits to our site from each `utm_source` for each `month`. Save your answer to the variable `click_source_by_month`, and view it.

```

title: "Aggregates in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load data
orders <- read_csv("orders.csv")
page_visits <- read_csv("page_visits.csv")
```

```{r}
# inspect data frames
head(orders)
head(page_visits)
```

```{r}
# define average_price here:
average_price <- orders %>%
  summarize(mean_price = mean(price, na.rm = TRUE))
average_price
```

```{r}
```



```
# define click_source here:
click_source <- page_visits %>%
  group_by(utm_source) %>%
  summarize(count = n())
click_source
```

```{r}
# define click_source_by_month here:
click_source_by_month <- page_visits %>%
  group_by(utm_source, month) %>%
  summarize(count = n())
click_source_by_month
```
```

# Joining Tables in R

## Introduction

In order to efficiently store data, we often spread related information across multiple tables.

For instance, imagine that we own an e-commerce business and we want to track the products that have been ordered from our website.

We could have one table with all of the following information:

```
order_id
customer_id
customer_name
customer_address
customer_phone_number
product_id
product_description
product_price
quantity
timestamp
```

However, a lot of this information would be repeated. If the same customer makes multiple orders, that customer's name, address, and phone number will be reported multiple times. If the same product is ordered by multiple customers, then the product price and description will be repeated. This will make our orders table big and unmanageable.

So instead, we can split our data into three tables:

```
orders would contain the information necessary to describe an order: order_id, customer_id,
product_id, quantity, and timestamp
products would contain the information to describe each product: product_id, product_description
and product_price
customers would contain the information for each customer: customer_id, customer_name,
customer_address, and customer_phone_number
```

In this lesson, we will learn the dplyr commands that help us work with data stored in multiple tables.

## Instructions

Checkpoint 1 Enabled

1.

In **notebook.Rmd**, we've loaded in three data frames: `orders`, `products`, and `customers`. Begin by inspecting `orders` using `head()`. What columns are in the data frame?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now inspect `products` using `head()`.

Are there any columns in common with `orders`?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Finally inspect `customers` using `head()`.

Are there any columns in common with `orders` or `products`?

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")
customers <- read_csv("customers.csv")
products <- read_csv("products.csv")
```

```{r}
# inspect orders, customers and products here:
head(orders)
head(customers)
head(products)
```
```

## Joining Tables in R

# Joining

Suppose we have the following three tables that describe our eCommerce business:

`orders` — a table with information on each transaction:

| <code>order_id</code> | <code>customer_id</code> | <code>product_id</code> | <code>quantity</code> | <code>timestamp</code> |
|-----------------------|--------------------------|-------------------------|-----------------------|------------------------|
|-----------------------|--------------------------|-------------------------|-----------------------|------------------------|

|   |   |   |   |            |
|---|---|---|---|------------|
| 1 | 2 | 3 | 1 | 2017-01-01 |
| 2 | 2 | 2 | 3 | 2017-01-01 |
| 3 | 3 | 1 | 1 | 2017-01-01 |
| 4 | 3 | 2 | 2 | 2017-02-01 |
| 5 | 3 | 3 | 3 | 2017-02-01 |
| 6 | 1 | 4 | 2 | 2017-03-01 |
| 7 | 1 | 1 | 1 | 2017-02-02 |
| 8 | 1 | 4 | 1 | 2017-02-02 |

`customers` — a table with customer names and contact information:

| <code>customer_id</code> | <code>customer_name</code> | <code>address</code> | <code>phone_number</code> |
|--------------------------|----------------------------|----------------------|---------------------------|
| 1                        | John Smith                 | 123 Main St.         | 212-123-4567              |
| 2                        | Jane Doe                   | 456 Park Ave.        | 949-867-5309              |
| 3                        | Joe Schmo                  | 798 Broadway         | 112-358-1321              |

`products` — a table with product IDs, descriptions, and prices:

| <code>product_id</code> | <code>description</code> | <code>price</code> |
|-------------------------|--------------------------|--------------------|
|-------------------------|--------------------------|--------------------|

|   |                    |    |
|---|--------------------|----|
| 1 | thing-a-ma-jig     | 5  |
| 2 | whatcha-ma-call-it | 10 |
| 3 | doo-hickey         | 7  |
| 4 | gizmo              | 3  |

If we just look at the `orders` table, we can't really tell what's happened in each order. However, if we refer to the other tables, we can get a more complete picture.

Let's examine the order with an `order_id` of 1. It was purchased by Customer 2. To find out the customer's name, we look at the `customers` table and look for the item with a `customer_id` value of 2. We can see that Customer 2's name is Jane Doe and that she lives at 456 Park Ave.

Doing this kind of matching is called **joining** two data frames.

#### Instructions

Checkpoint 1 Enabled

1.

Examine the `orders` and `products` tables.

What is the `description` of the product that was ordered in Order 3?

Give your answer as a string assigned to the variable `order_3_description`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Examine the `orders` and `customers` tables.

What is the `phone number` of the customer in Order 5?

Give your answer as a string assigned to the variable `order_5_phone_number`.

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders data
orders <- read_csv("orders.csv")
customers <- read_csv("customers.csv")
products <- read_csv("products.csv")
```

```{r}
# inspect orders, customers and products
```

```

head(orders)
head(customers)
head(products)
```

```{r}
# define order_3_description here:
order_3_description <- "thing-a-ma-jig"

# define order_5_phone_number here:
order_5_phone_number <- "112-358-1321"
```

```

## Joining Tables in R

# Inner Join I

It is easy to do this kind of matching for one row, but hard to do it for multiple rows.

Luckily, dplyr can efficiently do this for the entire table using the `inner_join()` method.

The `inner_join()` method looks for columns that are common between two

### [data frames](#)

Preview: Docs Data frames are objects that store data into two dimensions of columns and rows.

and then looks for rows where those columns' values are the same. It then combines the matching rows into a single row in a new table.

We can call the `inner_join()` method with two data frames like this:

```

joined_df <- orders %>%
 inner_join(customers)

```

Copy to Clipboard

This will match up all of the customer information to the orders that each customer made.

### Instructions

Checkpoint 1 Enabled

1.

You are an analyst at Cool T-Shirts Inc. You are going to help them analyze some of their sales data.

There are two data frames defined in the file **notebook.Rmd**:

`sales` contains the monthly revenue for Cool T-Shirts Inc. It has two columns: `month` and `revenue`.

`targets` contains the goals for monthly revenue for each month. It has two columns: `month` and `target`.

Create a new data frame `sales_vs_targets` which contains the `inner_join()` of `sales` and `targets`. Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Cool T-Shirts Inc. wants to know the months when they crushed their targets.

Filter `sales_vs_targets` to only include the rows where `revenue` is greater than `target`. Save these rows to the variable `crushing_it`.

```

title: "Joining Tables in R"
output: html notebook

```

```

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load sales and targets data
sales <- read_csv("sales.csv")
targets <- read_csv("targets.csv")
```

```{r}
# inspect orders, customers and products
head(sales)
head(targets)
```

```{r}
# define sales_vs_targets here:
sales_vs_targets <- sales %>%
  inner_join(targets)
sales_vs_targets
```

```{r}
# define crushing_it here:
crushing_it <- sales_vs_targets %>%
  filter(revenue > target)
crushing_it
```

```

## Joining Tables in R

# Inner Join II

In addition to using `inner_join()` to join two

[data frames](#)

Preview: Docs Loading link description

together, we can use the pipe `%>%` to join multiple data frames together at once. The following command would join `orders` with `customers`, and then join the resulting data frame with `products`:

```

big_df <- orders %>%
 inner_join(customers) %>%
 inner_join(products)

```

Copy to Clipboard

### Instructions

Checkpoint 1 Enabled

1.

We have some more data from Cool T-Shirts Inc. The number of men's and women's t-shirts sold per month is in a file called `men_women_sales.csv`. Load this data into a data frame called `men_women`, and inspect it using `head()`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Join all three data frames (`sales`, `targets`, and `men_women`) into one big data frame called `all_data`.

View `all_data`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Cool T-Shirts Inc. thinks that they have more revenue in months where they sell more women's t-shirts.

Filter the rows of `all_data` to only include rows where:

`revenue` is greater than `target`

AND

`women` is greater than `men`

Save your answer to the variable `results`, and view it.

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load and inspect sales and targets data
sales <- read_csv("sales.csv")
targets <- read_csv("targets.csv")
head(sales)
head(targets)
```

```{r}
# load men_women data here:
men_women <- read_csv("men_women_sales.csv")

# inspect men_women here:
head(men_women)
```

```{r}
# define all_data here:
all_data <- sales %>%
  inner_join(targets) %>%
  inner_join(men_women)
all_data
```

```{r}
# define results here:
results <- all_data %>%
  filter(revenue > target) %>%
  filter(women > men)
```



```
results
'''
```

```
# define all_data here:
all_data <- sales %>%
  inner_join(targets) %>%
  inner_join(men_women)
```

```
Joining, by = "month"
Joining, by = "month"
```

all_data

month <chr>	revenue <dbl>	target <dbl>	men <dbl>	women <dbl>
January	300	310	30	35
February	290	270	29	35
March	310	300	31	29
April	325	350	32	28
May	475	475	47	50
June	495	500	49	45

6 rows

```
# define results here:
results <- all_data %>%
  filter(revenue > target) %>%
  filter(women > men)

results
```

month <chr>	revenue <dbl>	target <dbl>	men <dbl>	women <dbl>
February	290	270	29	35

1 row

Joining Tables in R

Join on Specific Columns I

In the previous example, the `inner_join()` function “knew” how to combine tables based on the columns that were the same between two tables. For instance, `orders` and `customers` both had a column called `customer_id`. This won't always be true when we want to perform a join.

Generally, the `orders` data frame would not have the column `order_id` and the `customers` data frame would not have the column `customer_id`. Instead, they would both have a column `id` and it would be implied that the `id` was the `order_id` for the `orders` table and the `customer_id` for the `customers` table. They would look like this:

Orders

<code>id</code>	<code>customer_id</code>	<code>product_id</code>	<code>quantity</code>	<code>timestamp</code>
1	2	3	1	2017-01-01
2	2	2	3	2017-01-01
3	3	1	1	2017-01-01
4	3	2	2	2017-02-01
5	3	3	3	2017-02-01
6	1	4	2	2017-03-01
7	1	1	1	2017-02-02
8	1	4	1	2017-02-02

Customers

<code>id</code>	<code>customer_name</code>	<code>address</code>	<code>phone_number</code>
1	John Smith	123 Main St.	212-123-4567

2	Jane Doe	456 Park Ave.	949-867-5309
3	Joe Schmo	798 Broadway	112-358-1321

How would this affect our joins?

Because the `id` columns would mean something different in each table, our default joins would be wrong.

One way that we could address this problem is to use the dplyr function `rename()` to rename the columns for our joins. In the example below, we will rename the column `id` in the `customers` data frame to `customer_id`, so that `orders` and `customers` now have a common column to join on.

```
customers <- customers %>%
  rename(customer_id = id)
inner_join(orders, customers)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.
The `id` column of `products` stores the same information as the `product_id` column of `orders`. Rename the `id` column of `products` to `product_id`. Save the updated data frame to `products`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.
Join `orders` and `products`. Save the result to the variable `orders_products`, and view it.

```
---
title: "Joining Tables in R"
output: html_notebook
---

```{r message = FALSE}
load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
load orders and products data
orders <- read_csv("orders.csv")
products <- read_csv("products.csv")
```

```{r}
inspect orders and products
head(orders)
head(products)
```

```{r}
rename the id column of products here:
products <- products %>%
 rename(product_id = id)
products
```

```

```
```{r}
define orders_products here:
orders_products <- orders %>%
 inner_join(products)

orders_products
```

```

Joining Tables in R

Join on Specific Columns II

In the previous exercise, we learned how to use `rename()` to join two data frames whose columns don't match.

A better option, however, exists. We can add the `by` argument when calling `inner_join()` to specify which columns we want to join on. In the example below, the "left" table is the one that comes first (`orders`), and the "right" table is the one that comes second (`customers`). This syntax says that we should match the `customer_id` from `orders` to the `id` in `customers`.

```

orders %>%
  inner_join(customers,
             by = c('customer_id' = 'id'))

```

Copy to Clipboard

The resulting data frame will look like this:

| id | customer_id | product_id | quantity | timestamp | customer_name | address | phone_number |
|----|-------------|------------|----------|---------------------|---------------|--------------|--------------|
| 1 | 2 | 3 | 1 | 2017-01-01 00:00:00 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 2 | 2 | 2 | 3 | 2017-01-01 00:00:00 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 3 | 3 | 1 | 1 | 2017-01-01 00:00:00 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 4 | 3 | 2 | 2 | 2016-02-01 00:00:00 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 5 | 3 | 3 | 3 | 2017-02-01 00:00:00 | Joe Schmo | 789 Broadway | 112-358-1321 |

| | | | | | | | |
|---|---|---|---|------------------------|------------|-----------------|------------------|
| 6 | 1 | 4 | 2 | 2017-03-01
00:00:00 | John Smith | 123 Main
St. | 212-123-
4567 |
| 7 | 1 | 1 | 1 | 2017-02-02
00:00:00 | John Smith | 123 Main
St. | 212-123-
4567 |
| 8 | 1 | 4 | 1 | 2017-02-02
00:00:00 | John Smith | 123 Main
St. | 212-123-
4567 |

If we use this syntax, we'll end up with two columns called `id`, one from the first table and one from the second. R won't let you have two columns with the same name, so it will change them to `id_x` and `id_y`.

It will look like this:

| <code>id_x</code> | <code>customer_id</code> | <code>product_id</code> | <code>quantity</code> | <code>timestamp</code> | <code>id_y</code> | <code>customer_name</code> | <code>address</code> | <code>phone_number</code> |
|-------------------|--------------------------|-------------------------|-----------------------|------------------------|-------------------|----------------------------|----------------------|---------------------------|
| 1 | 2 | 3 | 1 | 2017-01-01
00:00:00 | 2 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 2 | 2 | 2 | 3 | 2017-01-01
00:00:00 | 2 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 3 | 3 | 1 | 1 | 2017-01-01
00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 4 | 3 | 2 | 2 | 2016-02-01
00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 5 | 3 | 3 | 3 | 2017-02-01
00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 6 | 1 | 4 | 2 | 2017-03-01
00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |
| 7 | 1 | 1 | 1 | 2017-02-02
00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |

| | | | | | | | | |
|---|---|---|---|---------------------|---|------------|--------------|--------------|
| 8 | 1 | 4 | 1 | 2017-02-02 00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |
|---|---|---|---|---------------------|---|------------|--------------|--------------|

The new column names `id_x` and `id_y` aren't very helpful for us when we read the table. We can help make them more useful by using the keyword `suffix`. We can provide a vector of suffixes to use instead of `"_x"` and `"_y"`.

For example, we could use the following code to make the suffixes reflect the table names:

```
orders %>%
  inner_join(customers,
             by = c('customer_id' = 'id'),
             suffix = c('_order', '_customer'))
```

Copy to Clipboard
The resulting table would look like this:

| id_order | customer_id | product_id | quantity | timestamp | id_customer | customer_name | addresses | phone_number |
|----------|-------------|------------|----------|---------------------|-------------|---------------|--------------|--------------|
| 1 | 2 | 3 | 1 | 2017-01-01 00:00:00 | 2 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 2 | 2 | 2 | 3 | 2017-01-01 00:00:00 | 2 | Jane Doe | 456 Park Ave | 949-867-5309 |
| 3 | 3 | 1 | 1 | 2017-01-01 00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 4 | 3 | 2 | 2 | 2016-02-01 00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 5 | 3 | 3 | 3 | 2017-02-01 00:00:00 | 3 | Joe Schmo | 789 Broadway | 112-358-1321 |
| 6 | 1 | 4 | 2 | 2017-03-01 00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |

| | | | | | | | | |
|---|---|---|---|---------------------|---|------------|--------------|--------------|
| 7 | 1 | 1 | 1 | 2017-02-02 00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |
| 8 | 1 | 4 | 1 | 2017-02-02 00:00:00 | 1 | John Smith | 123 Main St. | 212-123-4567 |

Instructions

Checkpoint 1 Enabled

1.

Join the `orders` and `products` data frames using an `inner_join()`, with `orders` as the first argument and `products` as the second argument. Also include the `by` argument to indicate which columns to join on. Save your results to the variable `orders_products`, and view it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now join the `products` and `orders` data frames using an `inner_join()`, with `products` as the first argument and `orders` as the second argument. Also include the `by` argument to indicate which columns to join on, as well as a `suffix` argument `c('_product', '_order')`. Save your results to the variable `products_orders`, and view it.

```
---
title: "Joining Tables in R"
output: html_notebook
---

```{r message = FALSE}
load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
load orders and products data
orders <- read_csv("orders.csv")
products <- read_csv("products.csv")
```

```{r}
inspect orders and products
head(orders)
head(products)
```

```{r}
define orders_products here:
orders_products <- orders %>%
 inner_join(products,
 by = c('product_id' = 'id'))
orders_products
```

```{r}
define products_orders here:
products_orders <- products %>%
 inner_join(orders,
 by = c('id' = 'product_id'),
```

```
suffix = c('_product', '_order'))
products_orders
``,`
```

## Joining Tables in R

# Mismatched Joins

In our previous examples, there were always matching values when we were performing our joins. What happens when that isn't true?

Let's imagine that our `products` table is out of date and is missing the newest product: Product 5. What happens when someone orders it?

### Instructions

Checkpoint 1 Enabled

1.

We've just released a new product with `product_id` equal to 5. People are ordering this product, but we haven't updated the `products` table.

In **notebook.Rmd**, you'll find two data frames: `orders` and `products`. Inspect these data frames using `head()`.

Notice that the third order in `orders` is for the mysterious new product, but that there is no `product_id` 5 in `products`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Join the `orders` and `products` data frames with `inner join()` and save the resulting data frame to the variable `orders_products`. View `orders_products`.

What happened to `order_id` 3?

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load orders and products data
orders <- read_csv("orders.csv")
products <- read_csv("products.csv")
```

```{r}
# inspect orders and products here:
head(orders)
head(products)
```

```{r}
# define orders_products here:
orders_products <- orders %>%
  inner_join(products)
orders_products
```
```



## Joining Tables in R

# Full Join

In the previous exercise, we saw that when we join two data frames whose rows don't match perfectly, we lose the unmatched rows.

This type of join (where we only include matching rows) is called an *inner join*. There are other types of joins that we can use when we want to keep information from the unmatched rows.

Suppose that two companies, Company A and Company B have just merged. They each have a list of customers, but they keep slightly different data. Company A has each customer's name and email. Company B has each customer's name and phone number. They have some customers in common, but some are different.

---

company\_a

| name          | email                   |
|---------------|-------------------------|
| Sally Sparrow | sally.sparrow@gmail.com |
| Peter Grant   | pgrant@yahoo.com        |
| Leslie May    | leslie_may@gmail.com    |

---

company\_b

| name        | phone        |
|-------------|--------------|
| Peter Grant | 212-345-6789 |
| Leslie May  | 626-987-6543 |
| Aaron Burr  | 303-456-7891 |

---

If we wanted to combine the data from both companies without losing the customers who are missing from one of the tables, we could use a *Full Join*. A *Full Join* would include all rows from both tables, even if they don't match. Any missing values are filled in with `NA`.

```
full_joined_dfs <- company_a %>%
 full_join(company_b)
```

Copy to Clipboard

The resulting table would look like this:

| name          | email                   | phone        |
|---------------|-------------------------|--------------|
| Sally Sparrow | sally.sparrow@gmail.com | NA           |
| Peter Grant   | pgrant@yahoo.com        | 212-345-6789 |
| Leslie May    | leslie_may@gmail.com    | 626-987-6543 |
| Aaron Burr    | NA                      | 303-456-7891 |

## Instructions

Checkpoint 1 Enabled

1.

There are two hardware stores in town: Store A and Store B. Store A's inventory is in data frame `store_a` and Store B's inventory is in data frame `store_b`. They have decided to merge into one big Super Store! Combine the inventories of Store A and Store B using a full join. Save the results to the variable

`store_a_b_full`.

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}  
# load packages  
library(readr)  
library(dplyr)  
```  

```{r message = FALSE}  
# load store_a and store_b data  
store_a <- read_csv("store_a.csv")  
store_b <- read_csv("store_b.csv")  
```  

```{r}  
# inspect store_a and store_b  
head(store_a)
```

```
head(store_b)
```

```{r}
# define store_a_b_full here:
store_a_b_full <- store_a %>%
  full_join(store_b)
store_a_b_full
```
```

## Joining Tables in R

# Left and Right Joins

Let's return to the join of Company A and Company B.

---

### Left Join

Suppose we want to identify which customers are missing phone information. We would want a list of all customers who have `email`, but don't have `phone`.

We could get this by performing a *Left Join*. A *Left Join* includes all rows from the first (left) table, but only rows from the second (right) table that match the first table.

For this command, the order of the arguments matters. If the first data frame is `company_a` and we do a left join, we'll only end up with rows that appear in `company_a`.

By listing `company_a` first, we get all customers from Company A, and only customers from Company B who are *also* customers of Company A.

```
left_joined_df <- company_a %>%
 left_join(company_b)
```

Copy to Clipboard

The result would look like this:

| name          | email                   | phone        |
|---------------|-------------------------|--------------|
| Sally Sparrow | sally.sparrow@gmail.com | NA           |
| Peter Grant   | pgrant@yahoo.com        | 212-345-6789 |
| Leslie May    | leslie_may@gmail.com    | 626-987-6543 |

Now let's say we want a list of all customers who have `phone` but no `email`. We can do this by performing a Right Join.

---

## Right Join

A Right Join is the exact opposite of left join. Here, the joined table will include all rows from the second (right) table, but only rows from the first (left) table that match the second table.

By listing `company_a` first and `company_b` second, we get all customers from Company B, and only customers from Company A who are also customers of Company B.

```
right_joined_df <- company_a %>%
 right_join(company_b)
```

Copy to Clipboard

The result would look like this:

| name        | email                | phone        |
|-------------|----------------------|--------------|
| Peter Grant | pgrant@yahoo.com     | 212-345-6789 |
| Leslie May  | leslie_may@gmail.com | 626-987-6543 |
| Aaron Burr  | NA                   | 303-456-7891 |

## Instructions

Checkpoint 1 Enabled

1.

Let's return to the two hardware stores, Store A and Store B. They're not quite sure if they want to merge into a big Super Store just yet.

Store A wants to find out what products they carry that Store B does not carry. Using a left join, combine `store_a` to `store_b` and save the results to `left_a_b`.

The items with `NA` in `left_a_b` are carried by Store A, but not by Store B.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Now, Store B wants to find out what products they carry that Store A does not carry. Use a left join, to combine the two data frames but in the reverse order (i.e., `store_b` followed by `store_a`) and save the results to the variable `left_b_a`.

Which items are *not* carried by Store A, but *are* carried by Store B?

---

What do you notice about these two data frames?

How are they different?

How are they the same?

```

title: "Joining Tables in R"
output: html_notebook

```

```

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load store_a and store_b data
store_a <- read_csv("store_a.csv")
store_b <- read_csv("store_b.csv")
```

```{r}
# inspect store_a and store_b
head(store_a)
head(store_b)
```

```{r}
# define left_a_b here:
left_a_b <- store_a %>%
  left_join(store_b)
left_a_b
```

```{r}
# define left_b_a here:
left_b_a <- store_b %>%
  left_join(store_a)
left_b_a
```

```

## Joining Tables in R

# Concatenate Data Frames

Sometimes, a dataset is broken into multiple tables. For instance, data is often split into multiple CSV files so that each download is smaller.

When we need to reconstruct a single data frame from multiple smaller data frames, we can use the dplyr `bind_rows()` method. This method only works if all of the columns are the same in all of the data frames.

For instance, suppose that we have two data frames:

**df1**

| name           | email               |
|----------------|---------------------|
| Katja Obinger  | k.obinger@gmail.com |
| Alison Hendrix | alisonH@yahoo.com   |

|                |                        |
|----------------|------------------------|
| Cosima Niehaus | cosi.niehaus@gmail.com |
|----------------|------------------------|

|               |                          |
|---------------|--------------------------|
| Rachel Duncan | rachelduncan@hotmail.com |
|---------------|--------------------------|

**df2**

| name           | email               |
|----------------|---------------------|
| Jean Gray      | jgray@netscape.net  |
| Scott Summers  | ssummers@gmail.com  |
| Kitty Pryde    | kitkat@gmail.com    |
| Charles Xavier | cxavier@hotmail.com |

If we want to combine these two data frames, we can use the following command:

```
concatenated_dfs <- df1 %>%
 bind_rows(df2)
```

Copy to Clipboard

That would result in the following data frame:

| name           | email                    |
|----------------|--------------------------|
| Katja Obinger  | k.obinger@gmail.com      |
| Alison Hendrix | alisonH@yahoo.com        |
| Cosima Niehaus | cosi.niehaus@gmail.com   |
| Rachel Duncan  | rachelduncan@hotmail.com |

Jean Gray                                  jgray@netscape.net

Scott Summers                      ssummers@gmail.com

Kitty Pryde [kitkat@gmail.com](mailto:kitkat@gmail.com)

Charles Xavier cxavier@hotmail.com

## Instructions

Checkpoint 1 Enabled

1.

An ice cream parlor and a bakery have decided to merge.

The bakery's menu is stored in the data frame `bakery`, and the ice cream parlor's menu is stored in the data frame `ice_cream`.

Create their new menu by concatenating the two data frames into a data frame called `menu`.

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load bakery and ice_cream data
bakery <- read_csv('bakery.csv')
ice_cream <- read_csv('ice_cream.csv')
```

```{r}
# inspect bakery and ice_cream
head(bakery)
head(ice_cream)
```

```{r}
# define menu here:
menu <- bakery %>%
  bind_rows(ice_cream)
menu
```

```

## Review

This lesson introduced some methods for combining multiple data frames:

Creating a data frame made by matching the common columns of two

[data frames](#)

Preview: Docs Data frames are objects that store data into two dimensions of columns and rows.  
is called a `join`

We can specify which columns should be matched by using the `by` argument

We can combine data frames whose rows don't all match using `left`, `right`, and `full` joins

We can stack or concatenate data frames with the same columns using `bind_rows()`

## Instructions

Checkpoint 1 Enabled

1.

Cool T-Shirts Inc. just created a website for ordering their products. They want you to analyze two datasets for them:

`visits` contains information on all visits to their landing page

`checkouts` contains all users who began to checkout on their website

Use `head()` to inspect each data frame.

```

title: "Joining Tables in R"
output: html_notebook

```{r message = FALSE}
# load packages
library(readr)
library(dplyr)
```

```{r message = FALSE}
# load visits and checkouts data
visits <- read_csv('visits.csv')
checkouts <- read_csv('checkouts.csv')
```

```{r}
# inspect visits and checkouts here:
head(visits)
head(checkouts)
```

```{r}
# define v_to_c here:
v_to_c <- visits %>%
  inner_join(checkouts) %>%
  mutate(time = checkout_time - visit_time)
v_to_c
```

```{r}
# define avg_time_to_check here:
avg_time_to_check <- v_to_c %>%
  summarize(mean_time = mean(time))
avg_time_to_check
```
```



# R - Settings

## Working with Data in R Cumulative Project

Welcome to the cumulative project for data manipulation in R! Follow these instructions to download the project and complete it using RStudio.

### Getting Started with RStudio

If you've never used RStudio before, we recommend watching this video to familiarize yourself with the tool:

If you have not already done so, [download R](#), and [download RStudio](#).

Download the [Data Manipulation project](#) zip file.

Unzip the file by double-clicking on it in your Downloads folder.

Navigate to the newly unzipped folder and open `Coronavirus_Starting.Rmd` in RStudio to see the project instructions.

Follow the instructions by writing code in the code blocks found in `Coronavirus_Starting.Rmd`.

If you are stuck on the project, we have included our solution in the file `Coronavirus_Solution.Rmd`.

## VIDEO

<https://www.youtube.com/watch?v=T44GWKIFFRI>

<https://www.codecademy.com/paths/analyze-data-with-r/tracks/working-with-data-in-r-skill-path/modules/working-with-data-off-platform-project/informationals/working-with-data-in-r-cumulative-project>

# Welcome To Data Visualization With R

Let's look ahead to the data visualization section of this course.

Welcome to the section of the course that focuses on data visualization! In this section, you will learn how to use one of R's most popular packages — `ggplot`. This package is all about creating beautiful images based on your data. We'll walk you through how to create some of the most common graphs and charts like histograms, scatter plots, and bar charts.

We will also show you how to customize your images. R gives you total control over the colors you want to use, the labels you display, and even the tick marks on your axes. Let's dive in and start making some great images!

## Intro to Visualization with R

## Layers and Geoms

When you learn grammar in school you learn about the basic units to construct a sentence. The basic units in the "grammar of graphics" consist of:

1. **The *data*** or the ***actual information*** you wish to visualize.
2. **The *geometries***, shortened to "***geoms***", describe the shapes that represent our data. Whether it be dots on a scatter plot, bar charts on the graph, or a line to plot the data! The list goes on. Geoms are the shapes that "map" our data.
3. **The *aesthetics***, or the ***visual attributes of the plot***, including the scales on the axes, the color, the fill, and other attributes concerning appearance.

Another key component to understand is that in `ggplot2`, geoms are "added" as *layers* to the original canvas which is just an empty plot with data associated to it.

Once you learn these three basic grammatical units, you can create the equivalent of a basic sentence, or a basic plot. There are more units in the "grammar of graphics," but in this lesson we'll mostly be learning about these three.

### Instructions

Take a look at the code that generates the plot on the graph, you will understand every single line of what it's doing by the end of this lesson! For now, focus on the plus signs! Each plus sign is adding a layer to the plot!

```

title: "Intro To GGPlot2"
output: html_notebook

```

```

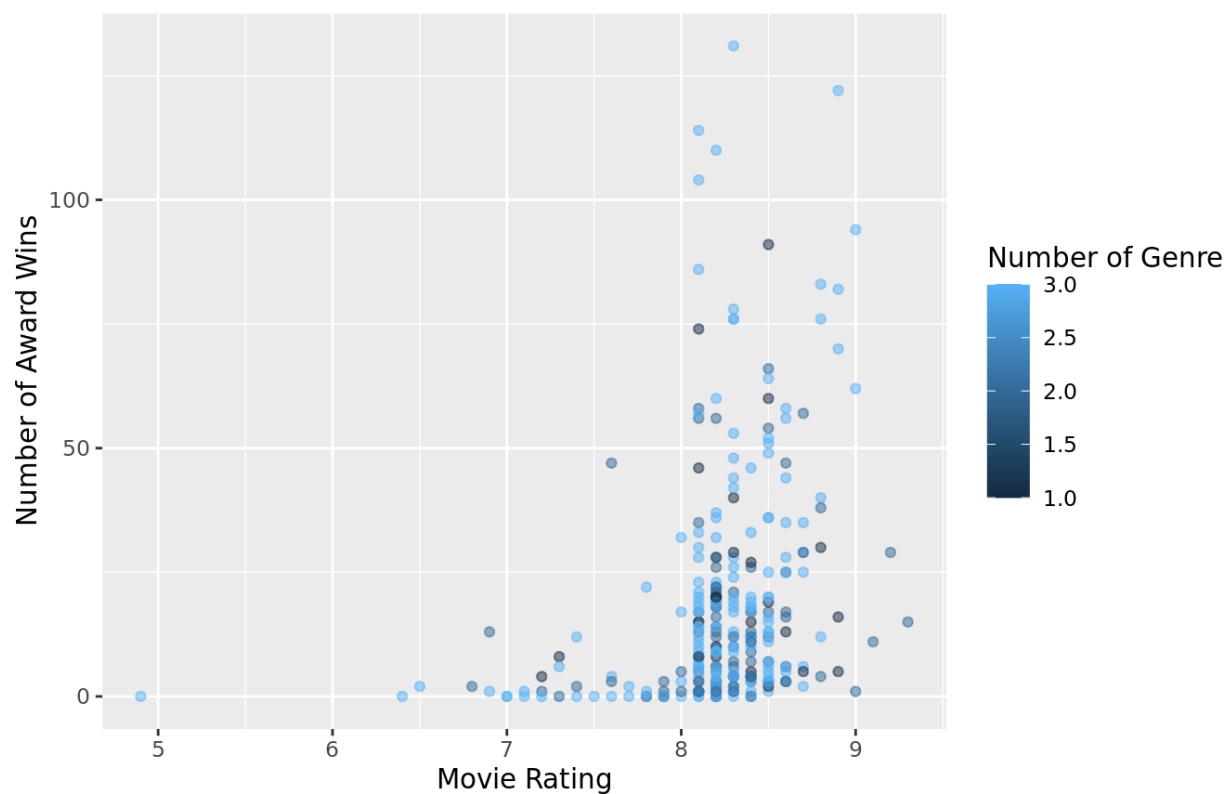
```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Observe layers being added with the + sign
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins)) +
  geom_point(aes(color=nrOfGenre), alpha=0.5) +
  labs(title="Movie Ratings Vs Award Wins", subtitle="From IMDB dataset",
y="Number of Award Wins", x="Movie Rating", color = "Number of Genre")

# Prints the plot
viz
```

```

Movie Ratings Vs Award Wins  
From IMDB dataset



# The ggplot() function

The first thing you'll need to do to create a ggplot object is invoke the `ggplot()` function. Conceptualize this step as initializing the “canvas” of the visualization. In this step, it's also standard to associate the data frame the rest of the visualization will use with the canvas. What do we mean by “the rest” of the visualization? We mean all the layers you'll add as you build out your plot.

As we mentioned, at its heart, a ggplot visualization is a combination of layers that each display information or add style to the final graph. You “add” these layers to a starting canvas, or ggplot object, with a `+` sign. We'll add geometries and aesthetics in the next exercises. For now, let's stop to understand that any arguments inside the `ggplot()` function call are inherited by the rest of the layers on the plot.

Here we invoke `ggplot()` to create a ggplot object and assign the dataframe `df`, saving it inside a variable named `viz`:

```
viz <- ggplot(data=df)

viz
```

Copy to Clipboard

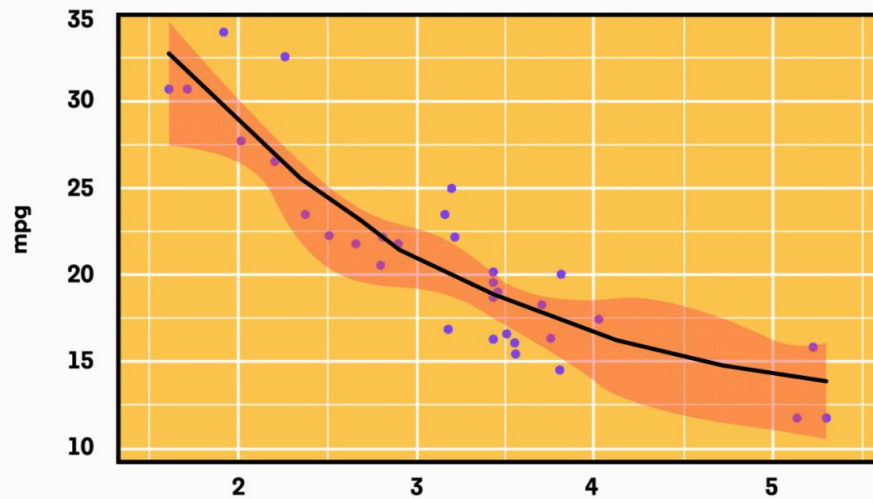
Note: The code above assigns the value of the canvas to `viz` and then states the variable name `viz` after so that the visualization is rendered in the notebook.

Any layers we add to `viz` would have access to the dataframe. We mentioned the idea of aesthetics before. It's important to understand that any aesthetics that you assign as the `ggplot()` arguments will also be inherited by other layers. We'll explore what this means in depth too, but for now, it's sufficient to conceptualize that arguments defined inside `ggplot()` are inherited by other layers.

## Instructions

1. Take a look at the animated diagram to the right.  
Notice how initially, a ggplot object is created as a blank canvas. Notice that the aesthetics are then set inside the `ggplot()` function as an argument.  
You could individually set the scales for each layer, but if all layers will use the same scales, it makes sense to set those arguments at the `ggplot()` “canvas” level.  
When two subsequent layers are added, a scatter plot and a line of best fit, both of those layers are mapped onto the canvas using the same data and scales.

```
geom_point() +
geom_smooth()
```



Intro to Visualization with R

## Associating the Data

Before we go any further, let's stop to understand when the data gets bound to the visualization:

Data is bound to a ggplot2 visualization by passing a data frame as the first argument in the `ggplot()` function call. You can include the named argument like `ggplot(data=df_variable)` or simply pass in the data frame like `ggplot(data frame)`.

Because the data is bound at this step, this means that the rest of our layers, which are function calls we add with a `+` plus sign, all have access to the data frame and can use the column names as [variables](#)

Preview: Docs Loading link description

For example, assume we have a data frame `sales` with the columns `cost` and `profit`. In this example, we assign the data frame `sales` to the `ggplot()` object that is initialized:

```
viz <- ggplot(data=sales) +
 geom_point(aes(x=cost, y=profit))
viz # renders plot
```

Copy to Clipboard

In the example above:

The ggplot object or canvas was initialized with the data frame `sales` assigned to it

The subsequent `geom_point` layer used the `cost` and `profit` columns to define the scales of the axes for that particular geom. Notice that it simply referred to those columns with their column names.

We state the variable name of the visualization ggplot object so we can see the plot.

Note: There are other ways to bind data to layers if you want each layer to have a different dataset, but the most readable and popular way to bind the dataframe happens at the `ggplot()` step and your layers use data from that dataframe.

## Instructions

Checkpoint 1 Enabled

1.

Create a new variable named `viz` and assign it the value of a new ggplot object that you create by invoking the `ggplot()` call and assigning it the dataframe `movies` as the `data` argument. After you've defined `viz` you need to state the variable name on a new line in order to see it.

Click run and watch your code render an empty canvas. Even though no data is displayed, the data is bound to the `viz` ggplot object!

```

title: "Intro To GGPlot2"
output: html_notebook

```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Define variable and print it
viz <- ggplot(movies)
viz
```

```{r message=FALSE}
```
```

## Intro to Visualization with R

# What are aesthetics?

In the context of ggplot, aesthetics are the instructions that determine the visual properties of the plot and its geometries.

Aesthetics can include things like the scales for the x and y axes, the color of the data on the plot based on a property or simply on a color preference, or the size or shape of different geometries.

There are two ways to set aesthetics, by manually specifying individual attributes or by providing *aesthetic mappings*. We'll explore aesthetic mappings first and come back to manual aesthetics later in the lesson. Aesthetic mappings "map"

### [variables](#)

Preview: Docs Loading link description

from the data frame to visual properties in the plot. You can provide aesthetic mappings in two ways using the `aes()` mapping function:

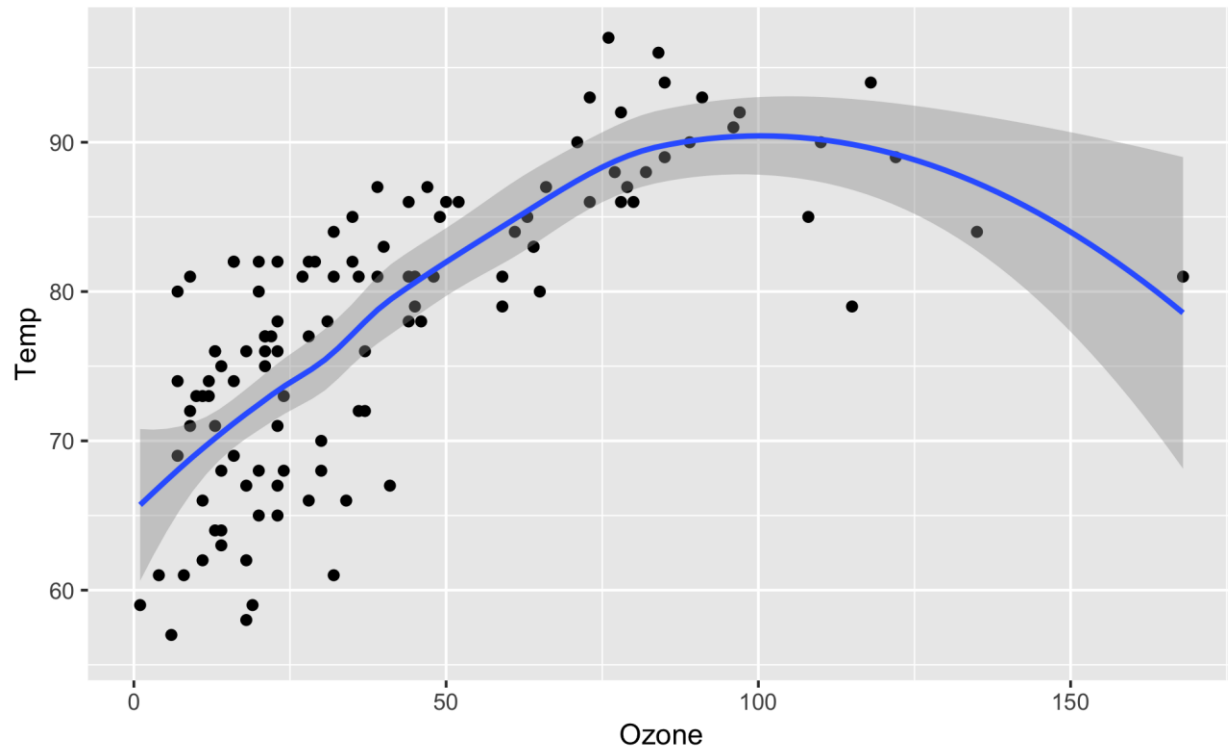
At the canvas level: All subsequent layers on the canvas will inherit the aesthetic mappings you define when you create a ggplot object with `ggplot()`.

At the geom level: Only that layer will use the aesthetic mappings you provide.

Let's discuss *inherited aesthetics* first, or aesthetics you define at the canvas level. Here's an example of code that assigns `aes()` mappings for the `x` and `y` scales at the canvas level:

```
viz <- ggplot(data=airquality, aes(x=Ozone, y=Temp)) +
 geom_point() +
 geom_smooth()
```

Copy to Clipboard



In the example above:

The aesthetic mapping is wrapped in the `aes()` aesthetic mapping function as an additional argument to `ggplot()`.

Both of the subsequent geom layers, `geom_point()` and `geom_smooth()` use the scales defined inside the aesthetic mapping assigned at the canvas level.

You should set aesthetics for subsequent layers at the canvas level if all layers will share those aesthetics.

### Instructions

Checkpoint 1 Enabled

1.

In the visualization we will be creating, we want to plot the Movie Ratings (`imdbRating`) on the `x` axis and the number of awards (`nrOfWins`) on the `y` axis to see if there is a correlation between a movie rating and the number of awards it wins. We will use this scale on the subsequent layers, so create the aesthetic mappings at the canvas level.

```

title: "Intro To GGPlot2"
output: html_notebook

```{r message=FALSE, warning=FALSE}
```

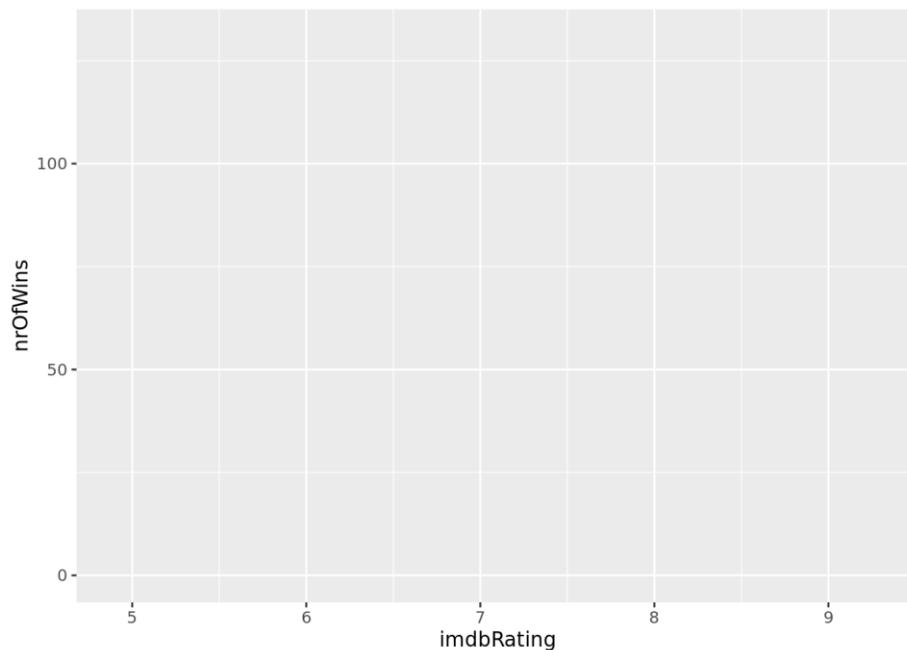
```

# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
#Create aesthetic mappings at the canvas level
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins))
viz
```

```{r message=FALSE}
```

```



## Intro to Visualization with R

### Adding Geoms

Before we teach you how to add aesthetics specific to a geom layer, let's create our first geom! As mentioned before, geometries or geoms are the shapes that represent our data.

In ggplot, there are many types of geoms for representing different relationships in data. You can read all about each one in the layers section of the [ggplot2 documentation](#). Once you learn the basic grammar of graphics, all you'll have to do is read the documentation of a particular geom and you'll be prepared to make a plot with it following the general pattern. For simplicity's sake, let's start with the scatterplot geom, or `geom_point()`, which simply represents each datum as a point on the grid. Scatterplots are great for graphing paired numerical data or to detect a correlation between two

[variables](#)

Preview: Docs Loading link description

.

The following code adds a scatterplot layer to the visualization:



```
viz <- ggplot(data=df, aes(x=col1,y=col2)) +
 geom_point()
```

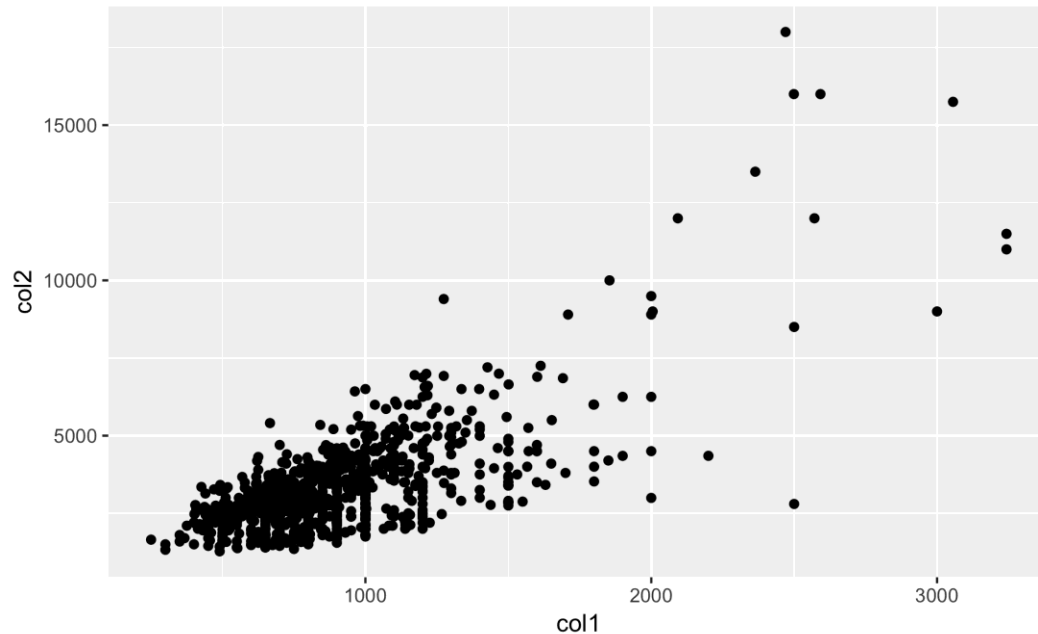
Copy to Clipboard

In the code above:

Notice the layer is being added by using a `+` sign which comes after the `ggplot` object is created, and it comes on the same line.

The `geom_point()` function call is what adds the points layer to the plot. This call can take arguments but we are keeping it simple for now.

The code above would render the following plot:



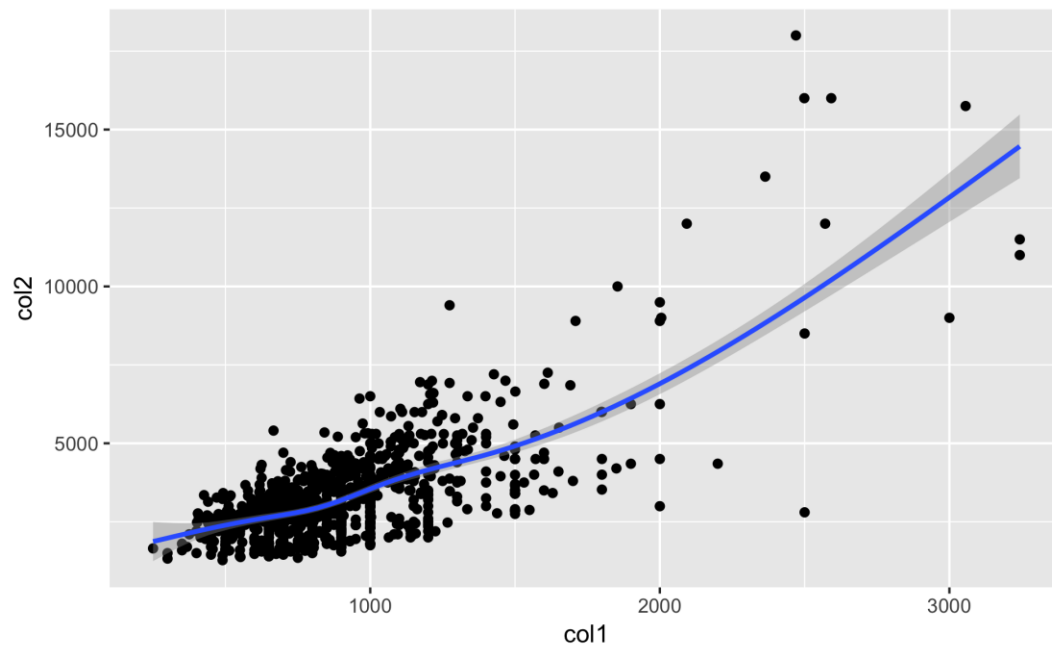
Another popular layer that allows you to eye patterns in the data by completing a line of best fit is the `geom_smooth()` layer. This layer, by nature, comes with a gray error band. You could add a smooth layer to the plot by typing the following:

```
viz <- ggplot(data=df, aes(x=col1,y=col2)) +
 geom_point() +
 geom_smooth()
```

Copy to Clipboard

Notice that you can add layers one on top of the other. We added the smooth line after adding the `geom_point()` layer. We could have just included the point layer, or just the line-of-best-fit layer. But the combination of the two enhances our visual understanding of the data, so they make a great pairing. It is nice to put each layer on its own line although it is not necessary, since it improves readability in the long run if you're collaborating with other people.

The code above would render the following plot:



### Instructions

Checkpoint 1 Enabled

1.

Add a scatter plot of the data to the `viz` ggplot object by using the `geom_point()` layer.

```

title: "Intro To GGPlot2"
output: html_notebook

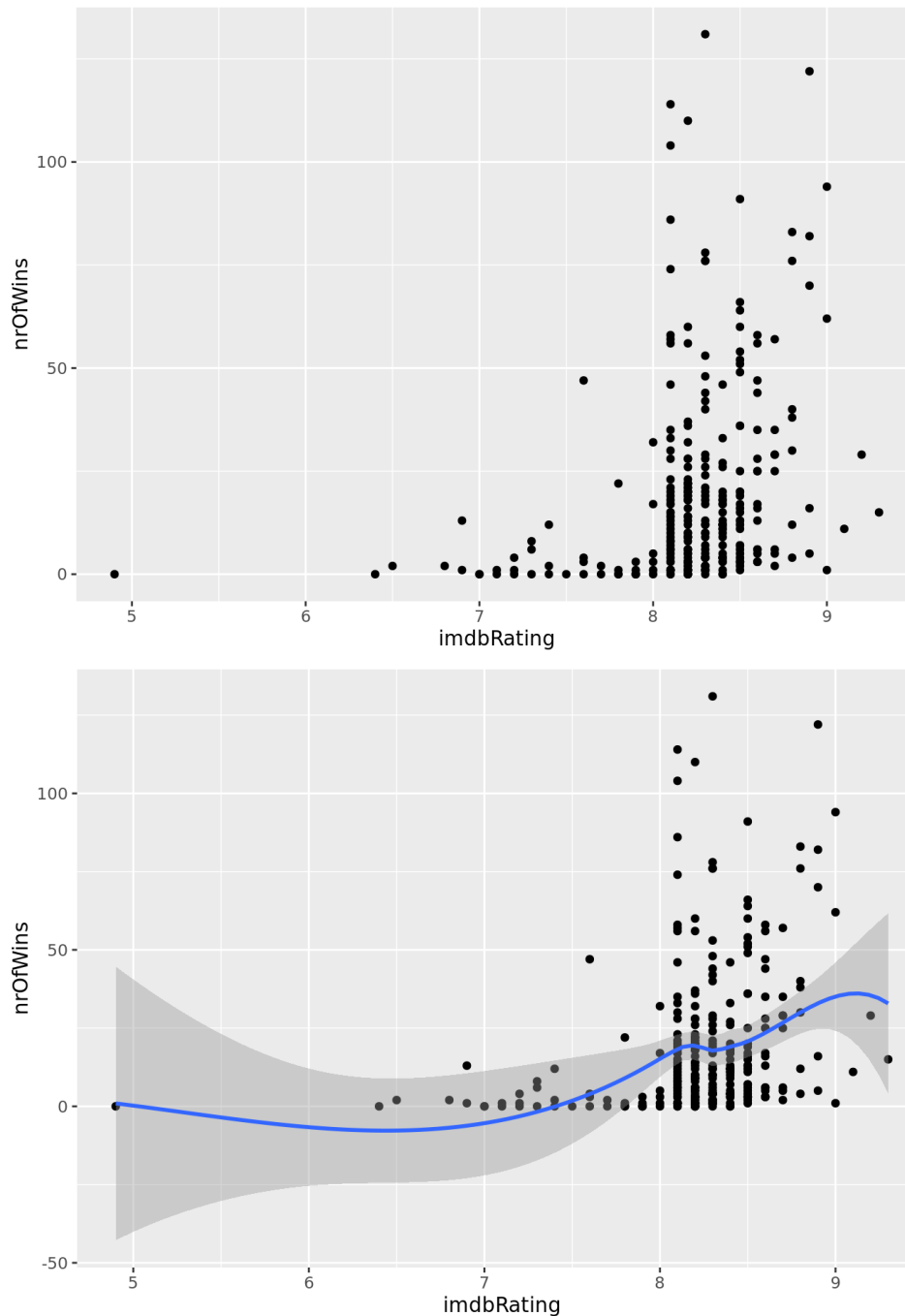
```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Add a geom point layer
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins)) +
  geom_point()

# Prints the plot
viz
```

```{r message=FALSE}
viz + geom_smooth()

viz
```
```



Intro to Visualization with R

## Geom Aesthetics

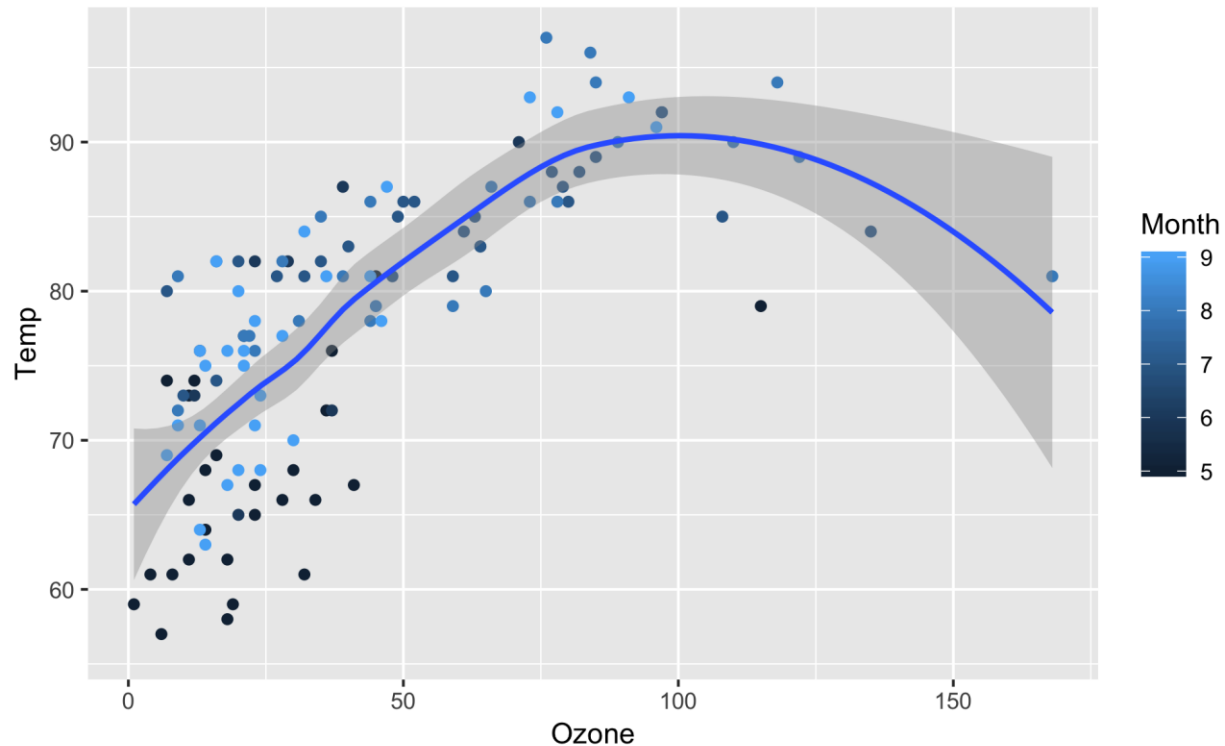
In the previous exercises, we added geoms to the plot and explored the idea of layers inheriting the original aesthetic mappings of the canvas. Sometimes, you'll want individual layers to have their own mappings. For example, what if we wanted the scatterplot layer to classify the points based on a data-driven property? We achieve this by providing an aesthetic mapping for that layer only.

Let's explore the aesthetic mappings for the `geom_point()` layer. What if we wanted to color-code the points on the scatterplot based on a property? It's possible to customize the color by passing in an `aes()` aesthetic mapping with the color based on a data-driven property. Observe this example:

```
viz <- ggplot(data=airquality, aes(x=Ozone, y=Temp)) +
 geom_point(aes(color=Month)) +
 geom_smooth()
```

Copy to Clipboard

The code above would *only* change the color of the point layer, it would not affect the color of the smooth layer since the `aes()` aesthetic mapping is passed at the point layer.



Note: You can read about the individual aesthetics available for each geom when you read its documentation. There are some aesthetics shared across geoms and others that are specific to a particular ones.

### Instructions

Checkpoint 1 Enabled

1.

Inside our `movies` dataset, we have a column named `nrOfGenre` that describes the number of genres a movie is assigned. For example, the movie "Terminator" is classified as both "Action" and "Sci-Fi", so its number of genres is equal to 2. What if we are wondering if the number of genres a movie is assigned, in other words its versatility, is correlated to its movie rating or its number of wins? Are movies better off when they stick to one simple genre or when they explore multiple ones? We want to display this information on our plot.

Add an aesthetic mapping to the `geom_point()` layer that `color` coordinates the data based on `nrOfGenre`.

```

title: "Intro To GGPlot2"
output: html_notebook

```

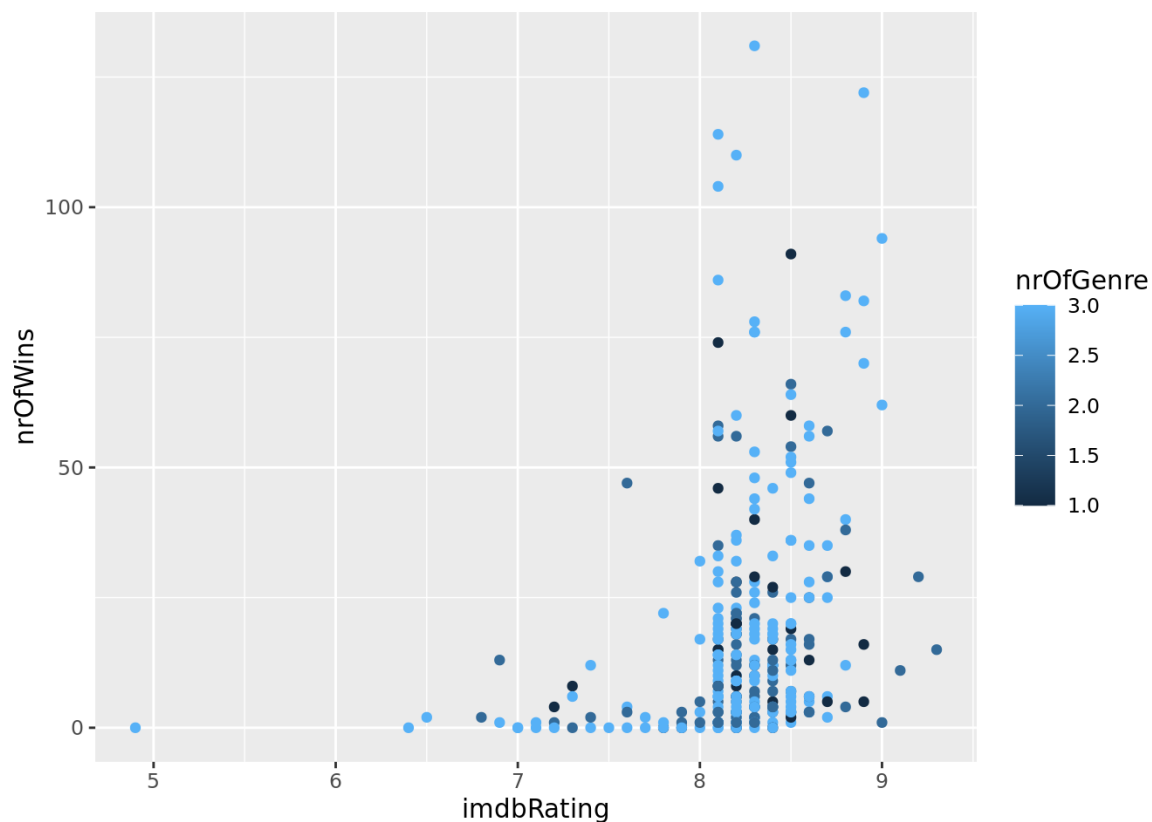
```

```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Add manual alpha aesthetic mapping
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins)) +
  geom_point(aes(color=nrOfGenre))

# Prints the plot
viz
```

```



Intro to Visualization with R

## Manual Aesthetics

We've reviewed how to assign data-driven aesthetic mappings at the canvas level and at the geom level. However, sometimes you'll want to change an aesthetic based on visual preference and not data. You might think of this as "manually" changing an aesthetic.

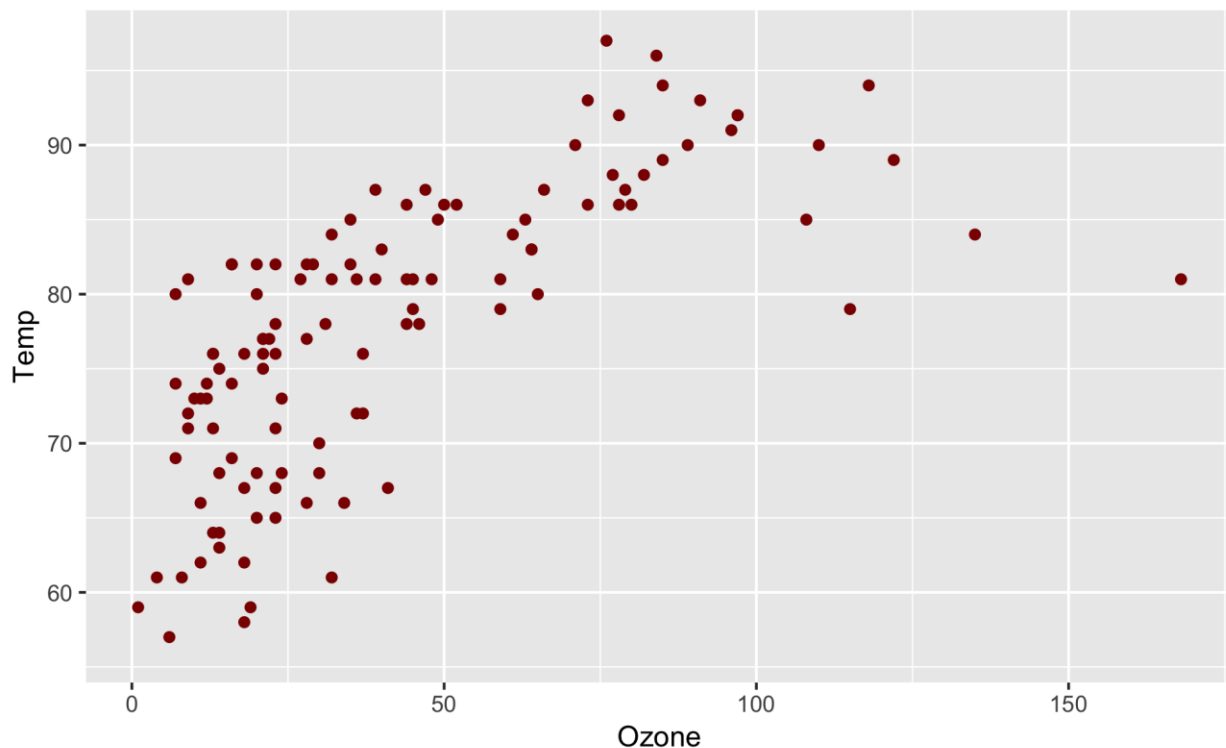
If you have a pre-determined value in mind, you provide a named aesthetic parameter and the value for that property *without* wrapping it in an `aes()`. For example, if you wanted to make all the points on the scatter plot layer dark red because that's in line with the branding of the visualization you are preparing, you could simply pass in a `color` parameter with a manual value `darkred` or any color value like so:

```
viz <- ggplot(data=airquality, aes(x=Ozone, y=Temp)) +
 geom_point(color="darkred")
```

Copy to Clipboard

Note that we did not wrap the `color` argument inside `aes()` because we are manually setting that aesthetic. Here are more aesthetics for the `geom_point()` layer: `x`, `y`, `alpha`, `color`, `fill`, `group`, `shape`, `size`, `stroke`. The `alpha` aesthetic describes opacity of the points, and the `shape` of the dots could be different than a dot. Read more about the values each of these aesthetics take in the [geom\\_point\(\) layer documentation](#).

The code above would render the following plot:



We advise that your aesthetic choices have intention behind them. Too much styling can overcomplicate the appearance of a plot, making it difficult to read.

#### Instructions

Checkpoint 1 Submitting

1.

There seems to be some crowding in our movie scatterplot. Let's change the opacity of our points by making them `.5` translucent. We can accomplish by manually assigning the `alpha` value of the `geom_point()` layer.

```

title: "Intro To GGPlot2"
output: html_notebook

```{r message=FALSE, warning=FALSE}
```

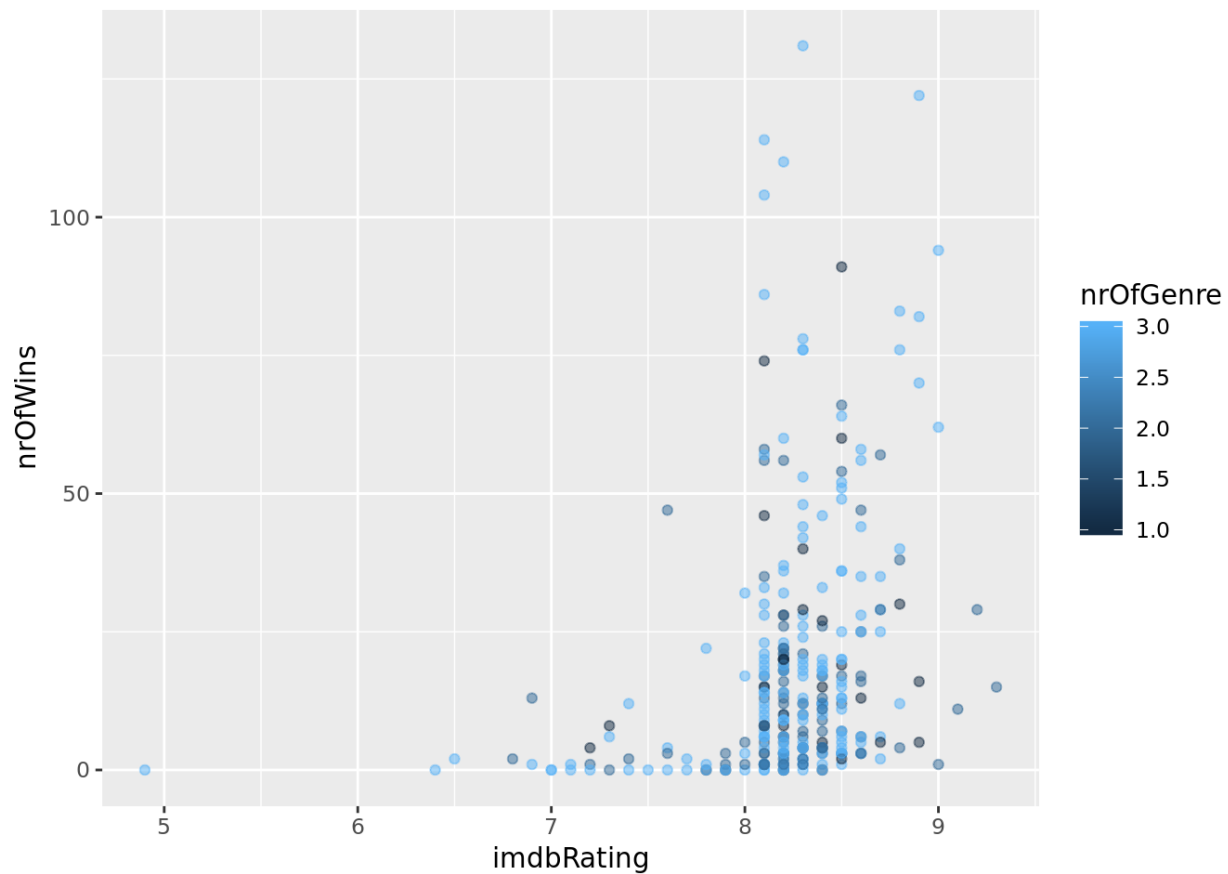
```

# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Add manual alpha aesthetic mapping
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins)) +
  geom_point(aes(color=nrOfGenre), alpha=.5)

# Prints the plot
viz
```

```



Intro to Visualization with R

## Labels

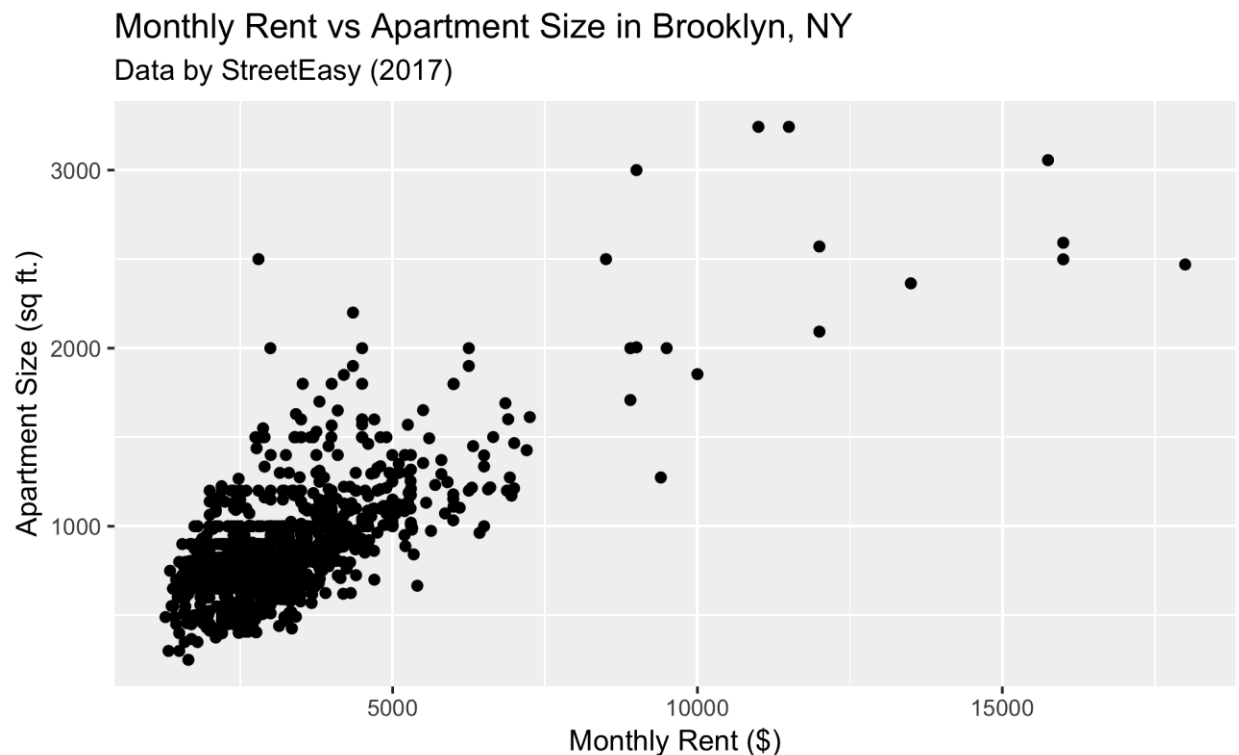
So far, we've reviewed how to add geometries to represent our data. We've also learned how to modify aesthetic values in our plot- whether those aesthetics are data-driven or assigned manually. Another big part of creating a plot is in making sure it has reader-friendly labels. The ggplot2 package automatically assigns the name of the variable corresponding to the different components on the plot as the initial label. Code variable names are unfortunately not always legible to outside readers with no context.

If you wish to customize your labels, you can add a `labs()` function call to your `ggplot` object. Inside the function call to `labs()` you can provide new labels for the `x` and `y` axes as well as a `title`, `subtitle`, or `caption`. You can check out the list of available label arguments in the [labs\(\) documentation](#) here.

The following `labs()` function call and these specified arguments would render the following plot:

```
viz <- ggplot(df, aes(x=rent, y=size_sqft)) +
 geom_point() +
 labs(title="Monthly Rent vs Apartment Size in Brooklyn, NY", subtitle="Data by
StreetEasy (2017)", x="Monthly Rent ($)", y="Apartment Size (sq ft.)")
viz
```

Copy to Clipboard



## Instructions

Checkpoint 1 Enabled

1.

The labels on the plot we've been building could definitely use an update!

Add a `labs()` function call and change the following arguments:

- Change the `title` to "Movie Ratings Vs Award Wins" to contextualize the goal of the plot

- Contextualize details about where the data comes from inside the `subtitle` by adding "From IMDB dataset"

- Change the `x` label to "Movie Rating" and the `y` label to "Number of Award Wins"

- Change the legend label by providing a `color` argument with the string value of "Number of Genre"

```

title: "Intro To GGPlot2"
output: html_notebook

```



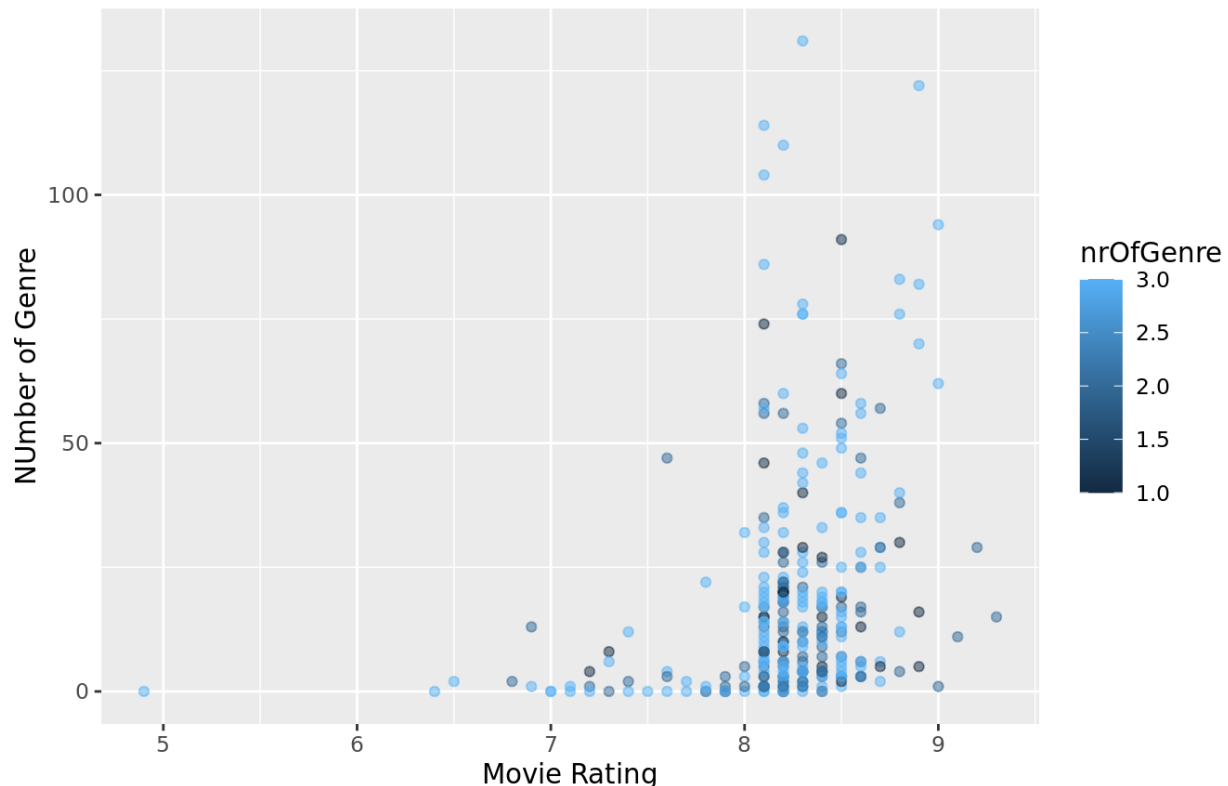
```

```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
movies <- read_csv("imdb.csv")
```

```{r}
# Add labels as specified
viz <- ggplot(data=movies, aes(x=imdbRating, y=nrOfWins)) +
  geom_point(aes(color=nrOfGenre), alpha=0.5) +
  labs(title="Movie Ratings Vs Award Wins", subtitle="From IMDB dataset", x="Movie
Rating", y="Number of Award Wins", color = "Number of Genre")
# Prints the plot
viz
```

```

Movie Ratings Vs Award Wins  
From IMDB dataset



Intro to Visualization with R

## Extending The Grammar

We've gone over each of the basic units in the grammar of graphics: data, geometries, and aesthetics. Let's extend this new knowledge to create a new type of plot: [the bar chart](#). Bar charts are great for showing the distribution of categorical data. Typically, one of the axes on a bar chart will have numerical values and the other will have the names of the different categories you wish to understand.

Let's build a bar chart by using some of the R built-in datasets. These are

## [data frames](#)

Preview: Docs Loading link description

that you can readily access in your code to explore and create visualizations. They are handy because these built-in datasets usually include nicely distributed categorical data.

The `geom_bar()` layer adds a bar chart to the canvas. Typically when creating a bar chart, you assign an `aes()` aesthetic mapping with a single categorical value on the `x` axes and the `aes()` function will compute the count for each category and display the count values on the `y` axis.

Since we're extending the grammar of graphics, let's also learn about how to save our visuals as local image files.

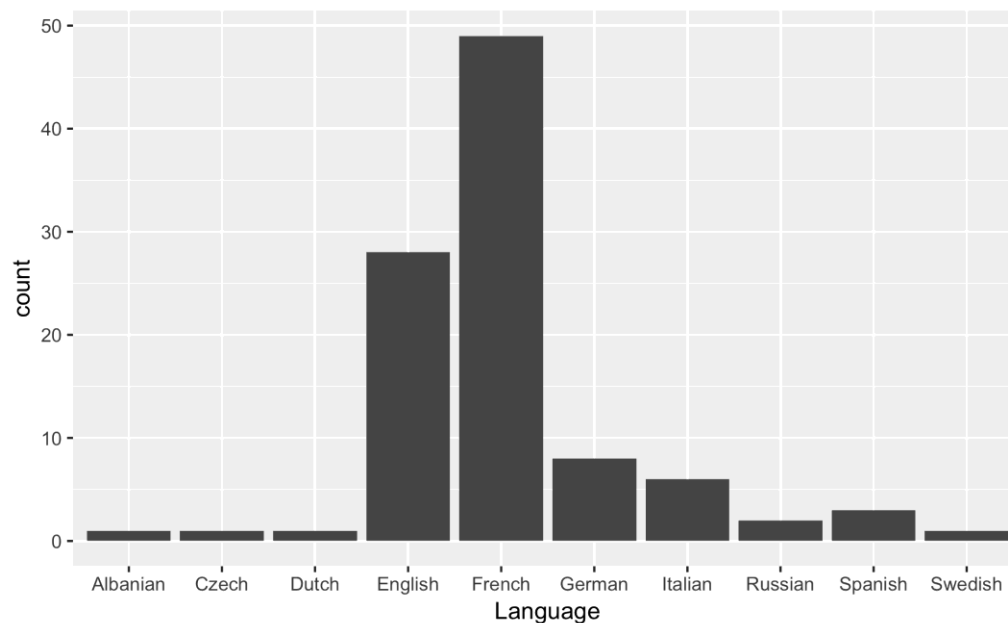
The following code maps the count of each category in the Language column in a dataset of 100 popular books to a bar length and then saves the visualization as a .png file named "bar-example.png":

```
bar <- ggplot(books, aes(x=Language)) + geom_bar()
bar
ggsave("bar-example.png")
```

Copy to Clipboard

Note: The `ggsave()` [function](#) allows you to save visualizations as a local file with the name of your choice. It's a useful function when developing visualizations locally.

The code above outputs the following plot:



## Instructions

Checkpoint 1 Enabled

1.

The `mpg` dataset in R is a built-in dataset describing fuel economy data from 1999 and 2008 for 38 popular models of cars and is included with `ggplot`.

Inspect the built-in dataset `mpg` by printing its `head()`. Take special note of the `class` column which describes vehicle class for the cars with a total of 7 types (compact, SUV, minivan etc.)

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a variable `bar` that is equal to a `ggplot()` object with the `mpg` built-in dataset associated as its `data` argument.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

We want to understand the breakdown of the types of vehicles in the dataset, so provide the canvas, or the `ggplot()` object with an aesthetic mapping `aes()` that makes the `x` axis represent the categorical values of the `class` column in the dataframe. `ggplot2` will count each unique value in the `class` column and automatically designate that value to the `y` axis.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Add a `geom_bar()` layer to `bar`. Be sure to type `bar` after you've declared the variable and added the layer so that the plot can render in your R notebook output.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Let's add some color to the bar chart, by adding an `aes()` aesthetic mapping to the `geom_bar()` layer that fills the color of each bar based on the `class` value.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Our plot could use some context, let's add a title and a sub-title so that users can understand more about what we are displaying with this bar chart and the [mpg dataset](#).

Use the `labs()` function to assign a new `title` that describes this plot is illustrating the Types of Vehicles and a `subtitle` describing the data as From fuel economy data for popular car models (1999-2008)

```

title: "Intro To GGPlot2"
output: html_notebook

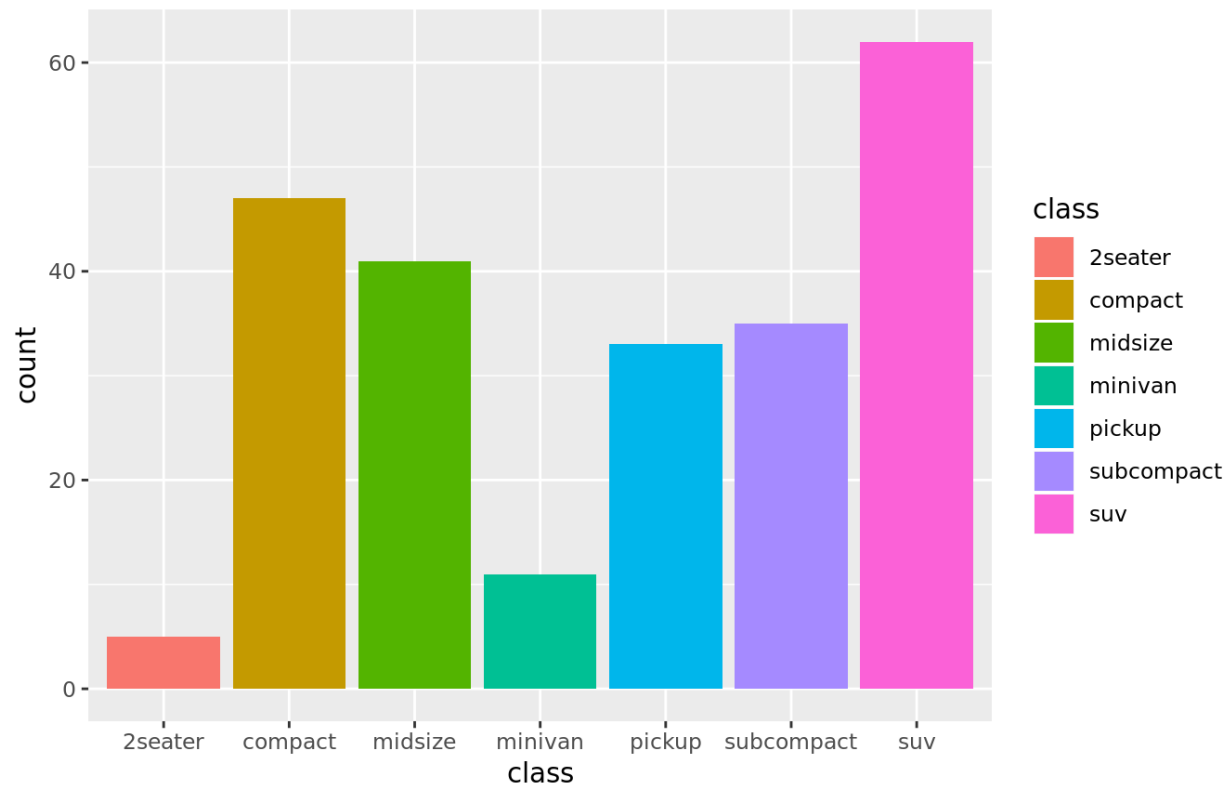
```{r message=FALSE, warning=FALSE}
# load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
```

```{r}
# Inspect the mpg builtin dataset
head(mpg)
```

```{r message=FALSE}
#Create a bar chart
bar <- ggplot(mpg, aes(class)) + geom_bar(aes(fill=class)) +
labs(title="Types of Vehicles", subtitle="From fuel economy data for popular car models
(1999-2008)")
bar
```
```

## Types of Vehicles

From fuel economy data for popular car models (1999-2008)



## Review

You've completed the introduction to ggplot lesson! You're ready to follow the general pattern for creating a visualization:

- Determine what relationship you wish to explore in your data

- Find the right geom(s) in the [ggplot2 documentation](#) to display that relationship and read about the arguments and aesthetics specific to that geom

- Extend the grammar of graphics to follow the pattern learned in this lesson to add layers and create a visualization. Improve graph legibility by polishing labels and styles.

Some of the key concepts you learned in this lesson include:

- The basic units of grammar include data, geoms, and aesthetics.

- The dataframe associated to the plot by using the `ggplot()` function creates a ggplot object that is known as the canvas.

The geometries or geoms are the shapes that display the data. Geometries become layers as you add them to your ggplot object.

The aesthetics are visual instructions you provide the plot. Aesthetics can be inherited or specified at the geom level.

Aesthetic mappings are data-driven visual instructions for the plot.

You can add context to your plot by customizing its labels with the `labs()` function

## Intermediate Data Visualization in R

### Introduction

In this lesson, we'll explore a variety of different visualizations in R's [ggplot2](#) package. We'll also go over different ways we can customize our plots to better communicate data insights.

Let's start with a familiar visualization: the scatter plot, which we can create using a `geom_point()` layer in `ggplot2`. Scatter plots are useful for visualizing the relationship between two

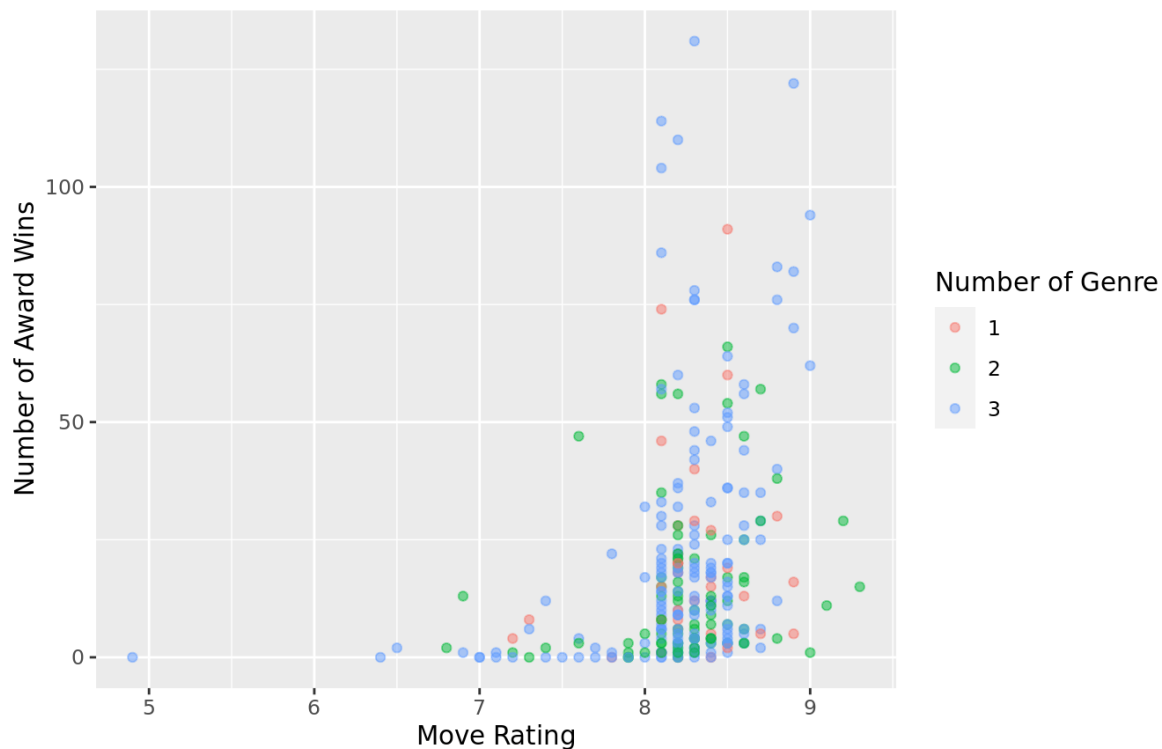
[variables](#)

Preview: Docs Loading link description

. The scatter plot below shows the correlation between a movie's IMDB rating and the number of awards it has won. Unsurprisingly, movies with higher ratings tend to win more awards!

#### Movie Ratings Vs Award Wins

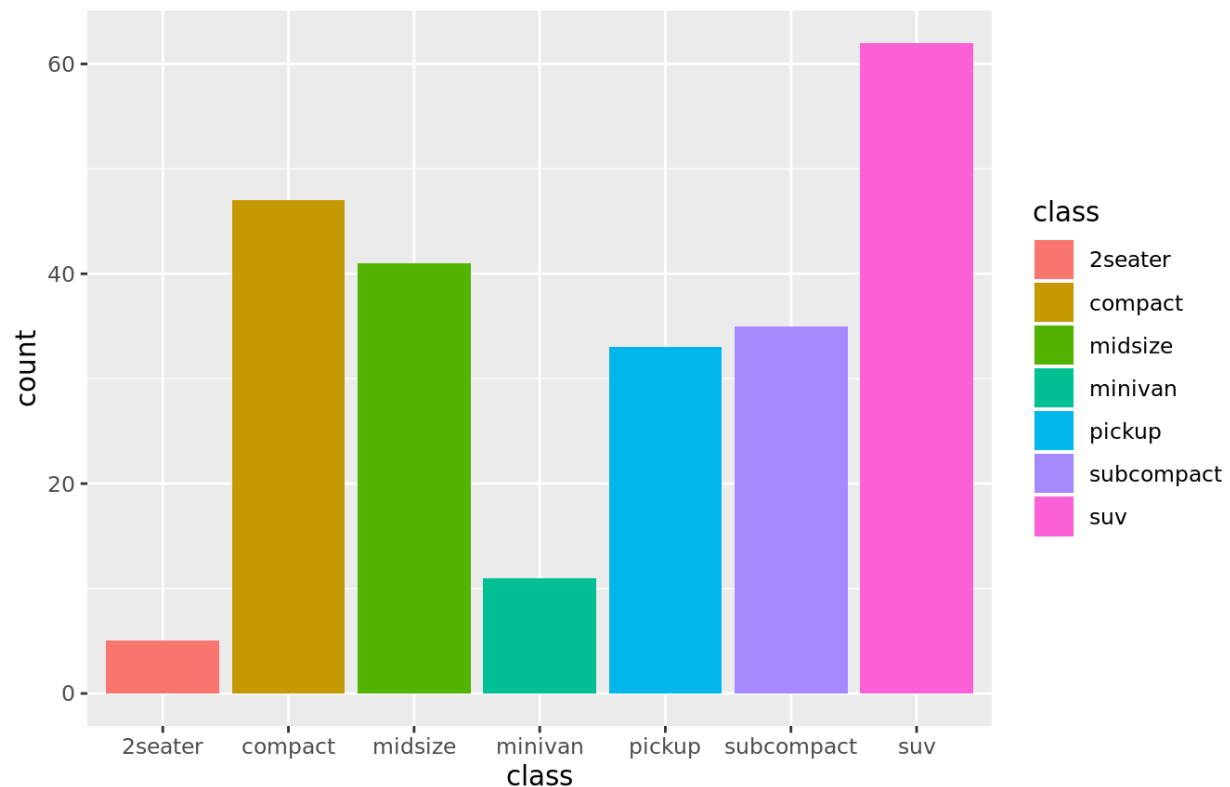
From IMDB dataset



Another common plot is the bar plot. Bar plots are useful for visualizing values for variables that can be counted, such as discrete variables (e.g. integers 1, 2, 3) or categorical variables (e.g. "cat", "dog", "mouse"). We can create bar plots using a `geom_bar()` layer in `ggplot2`. The bar plot below shows how many car models of each class are present in the `mpg` dataset. In this lesson, we'll look at statistical transformations that allow us to display summary values in a bar plot, such as means, and different ways of positioning bars within our bar plots.

## Types of Vehicles

From fuel economy data for popular car models (1999-2008)



While each geom can visualize many kinds of data, some geoms communicate certain insights more clearly than others. For example, if we wanted to see the distribution of height among the U.S. population, a bar plot wouldn't be the best choice because height is a continuous variable with many possible values.

Sometimes we might also need to visualize data across more than two variables. In this lesson, we'll cover the concept of "facets" which let us show additional discrete variables (in addition to  $x$  and  $y$  axes) by dividing a plot into different sections.

Lastly, we'll often want to adjust the axes of our plots, add error bars to show variance, and more. By the end of this lesson, you'll know how to create and customize many kinds of data visualizations using `ggplot2`. Let's get started!

### Instructions

Take a look at the plot to the right showing the sleep patterns of different animals. Notice how our bar plot shows percentages out of 100% — i.e. the percent of each day these animals are awake versus asleep. Notice also how the plot has been divided into facets showing the same  $x$  and  $y$  variables for primates compared to rodents. We'll learn how to create plots like these — and more — in this lesson!



## Intermediate Data Visualization in R

# Histograms

Histograms let us visualize the distribution of a continuous variable, in contrast to bar plots which show counts and other values for discrete and categorical

### [variables](#)

Preview: Docs Loading link description

. Histograms divide values of a variable into bins, which are ranges of values that get counted together. For example, if a variable had values 1 through 100 and we specify that we want 5 bins, each bin would have a range of  $100 / 5 = 20$ . The first bin would count the frequency of values 1 to 20, the second bin would count the frequency of values 21 to 40, and so on.

We can construct a histogram using `geom_histogram()`. The code below creates a histogram using R's built-in `airquality` dataset containing atmospheric measurements from New York City. This histogram shows frequencies of `Ozone` values, measuring the amount of air pollution recorded within a given time period.

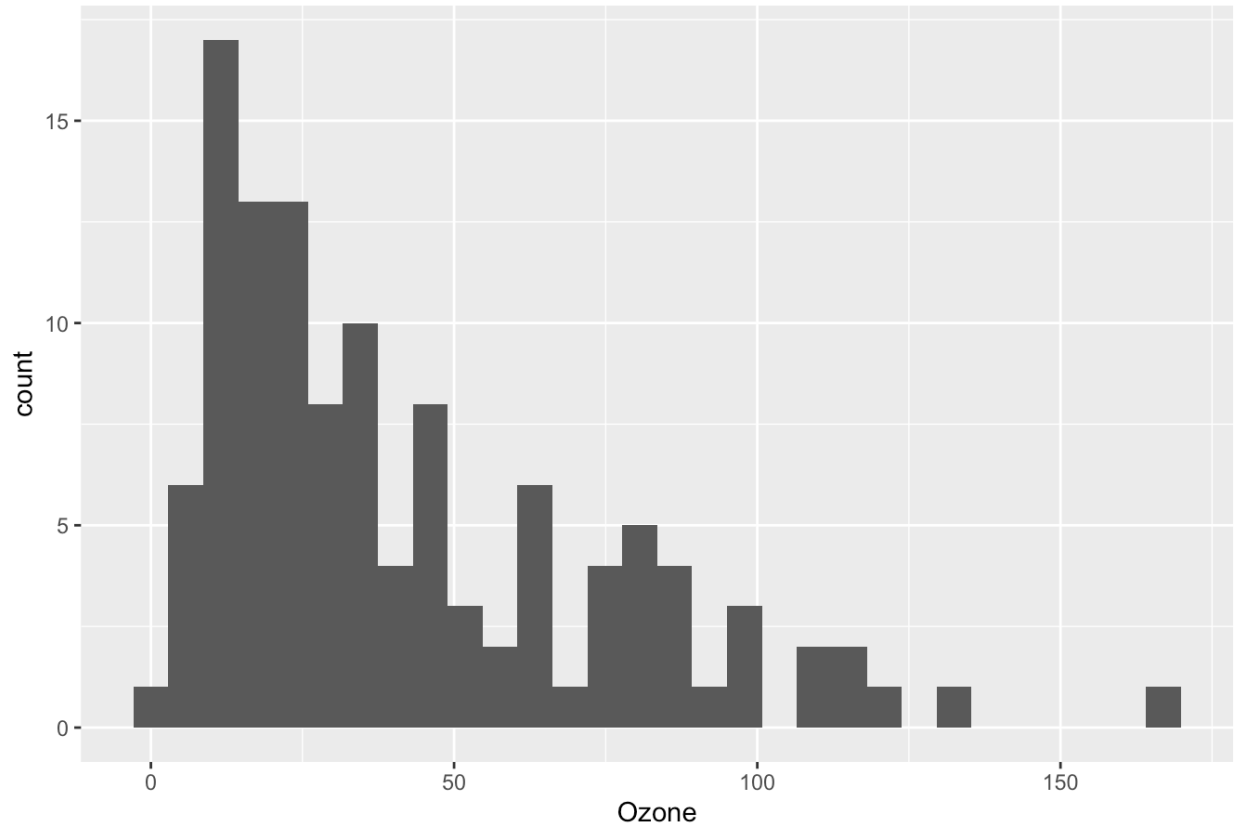
```
airquality_histogram <-
 ggplot(airquality, aes(x = Ozone)) +
 labs(title = "Air Quality: Ozone Distribution") +
 geom_histogram()
```



Copy to Clipboard

This produces the following plot. We see that ozone levels are clustered towards the lower end of the range (a good thing!), though there were days with much higher ozone levels as well.

Air Quality: Ozone Distribution



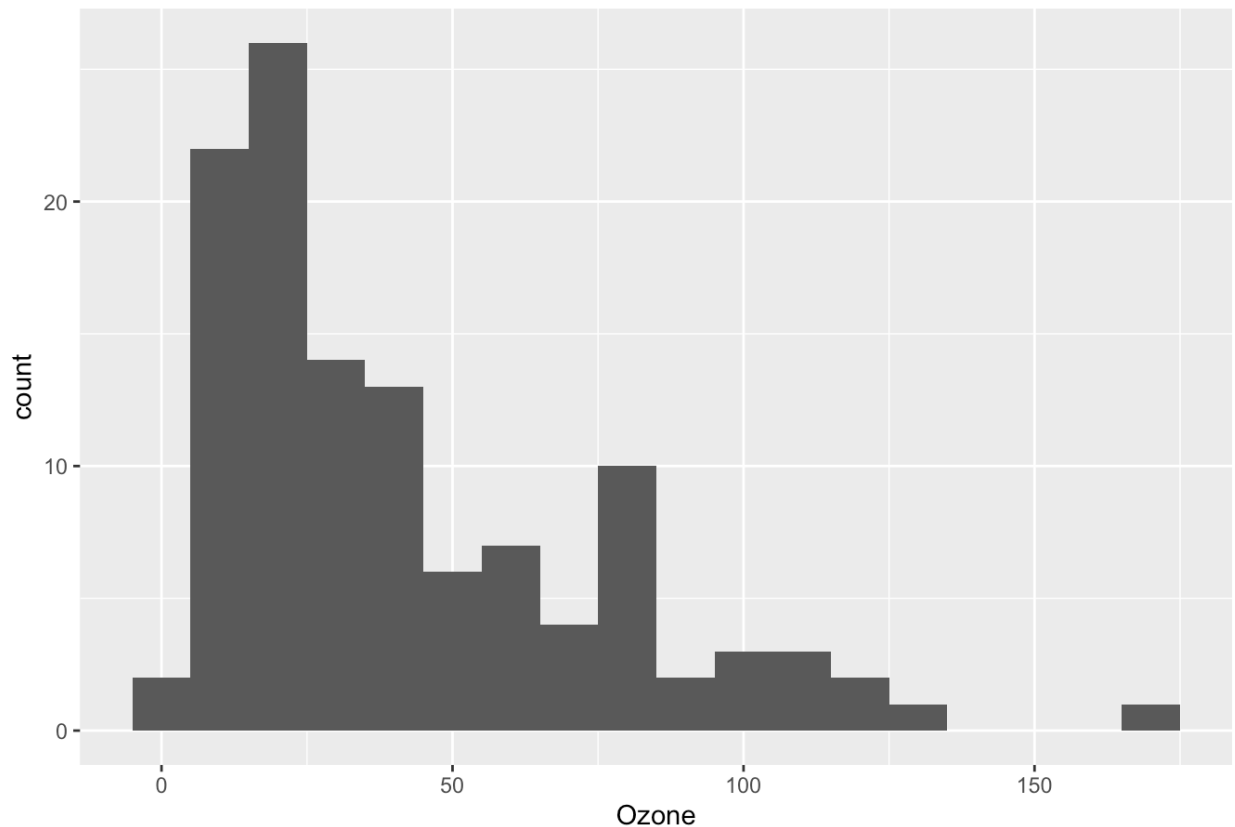
By default, `ggplot2` automatically calculates 30 equally sized bins. Frequently we'll want to specify a range per bin that better fits our data; for example, if we wanted to examine the distribution of weight in pounds for a population of house cats, it would make sense for each bin to represent one pound rather than some arbitrary decimal amount. We can set the width of bins using the `binwidth` argument. The code below creates the same plot as before, now with a `binwidth` of 10.

```
airquality_histogram_binwidth <-
 ggplot(airquality, aes(x = Ozone)) +
 labs(title = "Air Quality: Ozone Distribution") +
 geom_histogram(binwidth = 10)
```

Copy to Clipboard

Take a look at our new plot with `binwidth` set to 10. Notice how the shape of the histogram is now more smooth with fewer local peaks.

## Air Quality: Ozone Distribution



### Instructions

Checkpoint 1 Enabled

1.

Our workspace contains a dataset called `rideshare_df` describing rideshare trips in the city of Chicago. Examine this dataset by calling the `head()` function with `rideshare_df` as an argument. Click through the arrows in the table header to see all of the columns in this data frame.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Lets visualize the distribution of total trip cost across the `rideshare_df` dataset. Construct a histogram called `rideshare_histogram` with the `Trip.Total` variable on the `x` axis. Note that we only supply the `x` variable in our `aes()` mapping because the `y` axis will automatically show frequency counts.

Print your plot after creating it to see what it looks like!

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Create a similar plot called `rideshare_histogram_binwidth`, this time setting the `binwidth` to 5 to count trip totals in intervals of \$5.

Print your plot again to see how it looks with our custom bin width.

```

title: "Histograms"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)

# Read in our rideshare dataset
```

```

rideshare_df <- read.csv("rideshare_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
# Examine rideshare_df using head()
head(rideshare_df)

# Construct a histogram showing the distribution of Trip.Total
rideshare_histogram <-
  ggplot(rideshare_df, aes(x = Trip.Total)) +
  geom_histogram()

# Print the plot to see what it looks like
rideshare_histogram

# Construct our histogram again, this time adding a binwidth of 5
rideshare_histogram_binwidth <-
  ggplot(rideshare_df, aes(x = Trip.Total)) +
  geom_histogram(binwidth = 5)

# Print the plot to see what it looks like
rideshare_histogram_binwidth
```

```

## Intermediate Data Visualization in R

# Heatmaps

Heatmaps let us visualize frequencies along two

### [variables](#)

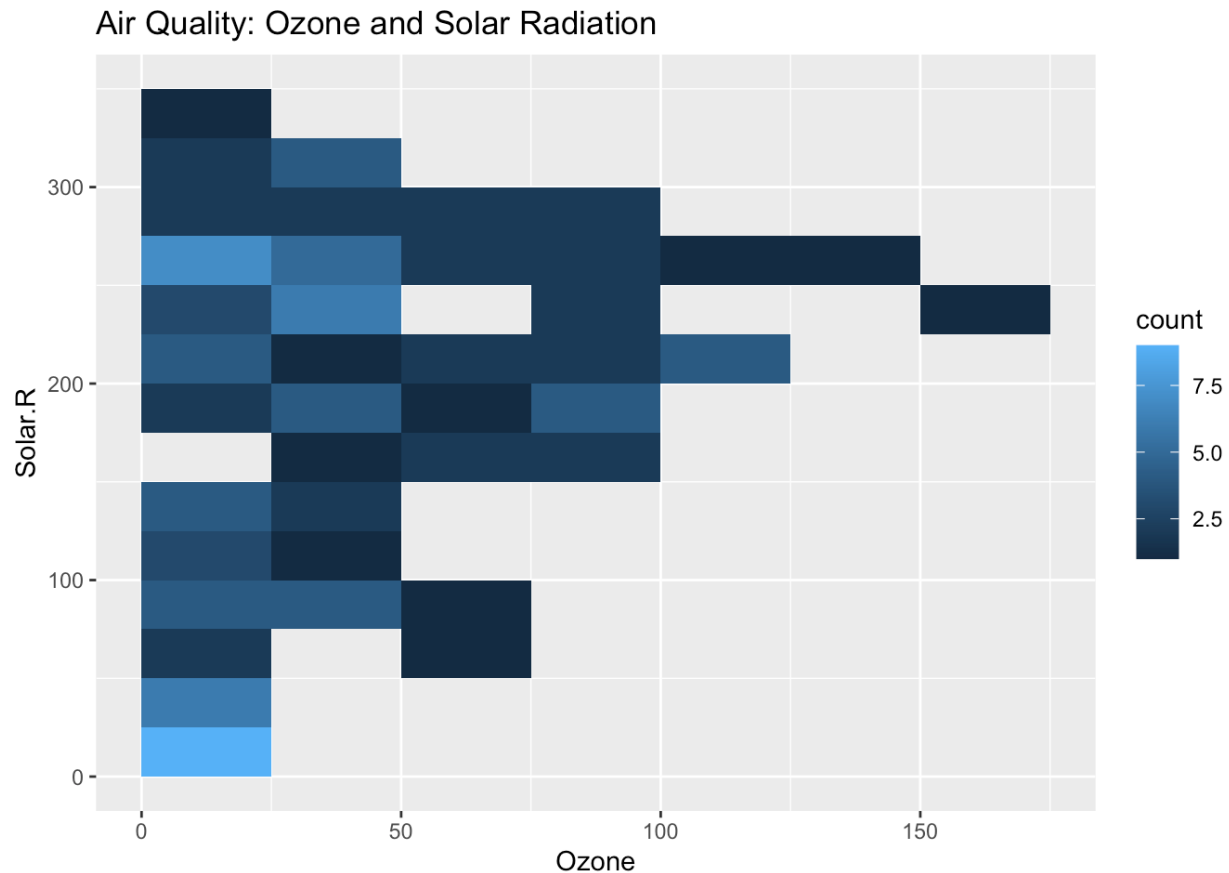
Preview: Docs Loading link description

. A heatmap looks like a scatterplot, but uses color-coded squares rather than individual points to indicate how many cases occurred at the intersection of  $x$  and  $y$  value ranges. Like histograms, we can specify bin widths to control which ranges of values get counted together.

Here's an example heatmap using our `airquality` dataset, mapping `Ozone` values (ozone pollution) on the  $x$  axis and `Solar.R` values (solar radiation) on the  $y$  axis. Notice how each region in the heatmap is color-coded to the number of cases with values in the relevant bin ranges. In this dataset, there are many occurrences where both solar radiation levels and ozone levels are low.

We can create heatmaps using the `geom_bin2d()` layer. This geom takes many of the same arguments as `geom_histogram()`, with slight differences given that a heatmap represents two variables. Like histograms, heatmaps automatically calculate 30 equally sized bins, which may not make sense for many datasets. To specify bin widths for each variable, we can pass a vector of widths instead of a single value, e.g. `geom_bin2d(binwidth = c(1, 5))` to set  $x$  axis bin widths to 1 and  $y$  axis bin widths to 5.

The code below constructs the heatmap shown at the start of this exercise. We set the `binwidth` of both axes using a vector, specifying that each axis should use bins of width 25.



```
airquality_heatmap <-
 ggplot(airquality,
 aes(x = Ozone, y = Solar.R)) +
 labs(title = "Air Quality: Ozone and Solar Radiation") +
 geom_bin2d(binwidth = c(25, 25))
```

Copy to Clipboard

#### Instructions

Checkpoint 1 Enabled

1.

Our `rideshare_df` dataset contains information on the longitude (east-west) and latitude (north-south) where each ride began. We can use a heatmap to visualize the locations where people most frequently began their trips. Create a heatmap named `rideshare_heatmap`, mapping `Pickup.Centroid.Longitude` to the `x` axis and `Pickup.Centroid.Latitude` to the `y` axis.

Print `rideshare_heatmap` to see what it looks like. Notice how the plot looks – how can we improve the way this data is being communicated?

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

To visualize this information more clearly, let's create a similar plot called `rideshare_heatmap_binwidth` with more appropriate bin sizes. Use the same data and `x` and `y` variable mappings. This time, set the `binwidth` to `0.01` for both axes. You should see a denser heatmap with more bins filled in.

Print `rideshare_heatmap_binwidth` to see what it looks like with our specified bin sizes. The region with no values to the right side of the map is Lake Michigan, where it is impossible to pick up a ride!

```

title: "Heatmaps"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)

# Read in our rideshare dataset
rideshare_df <- read.csv("rideshare_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
# Construct a heatmap of trip pickup longitude and latitude
rideshare_heatmap <-
  ggplot(rideshare_df,
    aes(x = Pickup.Centroid.Longitude,
        y = Pickup.Centroid.Latitude)) +
  geom_bin2d()

# Print the plot to see what it looks like
rideshare_heatmap

# Add binwidths of 0.01 to both variables in our heatmap
rideshare_heatmap_binwidth <-
  ggplot(rideshare_df,
    aes(x = Pickup.Centroid.Longitude,
        y = Pickup.Centroid.Latitude)) +
  geom_bin2d(binwidth = c(0.01, 0.01))

# Print the plot to see what it looks like
rideshare_heatmap_binwidth
```

```

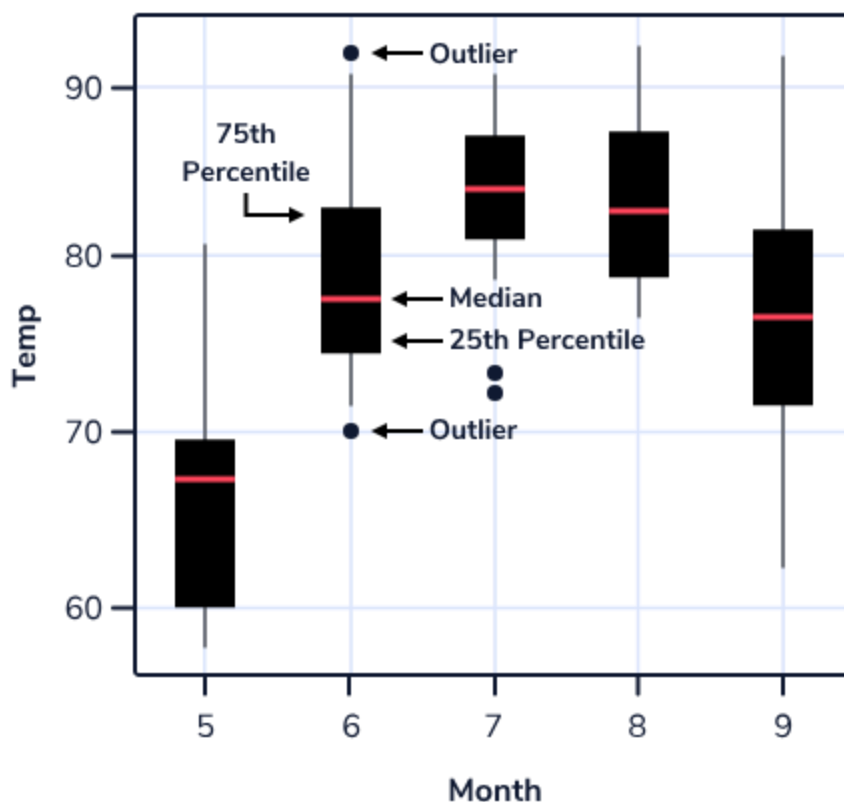
## Intermediate Data Visualization in R

### Box Plots

Box plots, also known as box-and-whisker plots, show the distribution of data by quartiles. Box plots are useful in showing how much a variable varies across values of another variable – are most cases similar in value, or is there a wide range between the highest and lowest values?

In the box plot below, we see the distribution of temperatures for different months within a subset of the `airquality` dataset. As we would expect for New York City, the summer months have the highest temperatures. The center of the box represents the median temperature. The upper and lower bounds of the box show the 75th and 25th percentiles respectively. The whiskers extend up to 1.5 times the distance between the 75th and 25th percentiles. Beyond the whiskers, outliers are shown as points.

## Air Quality: Temperature by Month



We can create a box plot using the `geom_boxplot()` layer. The code below creates the box plot shown above, visualizing temperature by month in the `airquality` data.

```
airquality_boxplot <-
 ggplot(airquality,
 aes(x = Month, y = Temp)) +
 labs(title = "Air Quality: Temperature by Month") +
 geom_boxplot()
```

Copy to Clipboard

Note that box plots show medians, not means. We'll cover how to display mean values using bar plots later in this lesson.

### Instructions

Checkpoint 1 Enabled

1.

Construct a box plot object called `rideshare_boxplot` visualizing the cost of trips in `rideshare` df by month, using the `Trip.Total` and `Month` variables. In your `aes()` mapping, transform `Month` to a factor (`x = factor(Month)`) so that `ggplot` knows to treat each month as a discrete value, rather than a continuous number.

Print the `rideshare_boxplot` object to see what it looks like. Notice what information is depicted in a box plot, compared to what would be included in a bar plot depicting the same data.

```

title: "Box Plots"
output: html_document

```

```

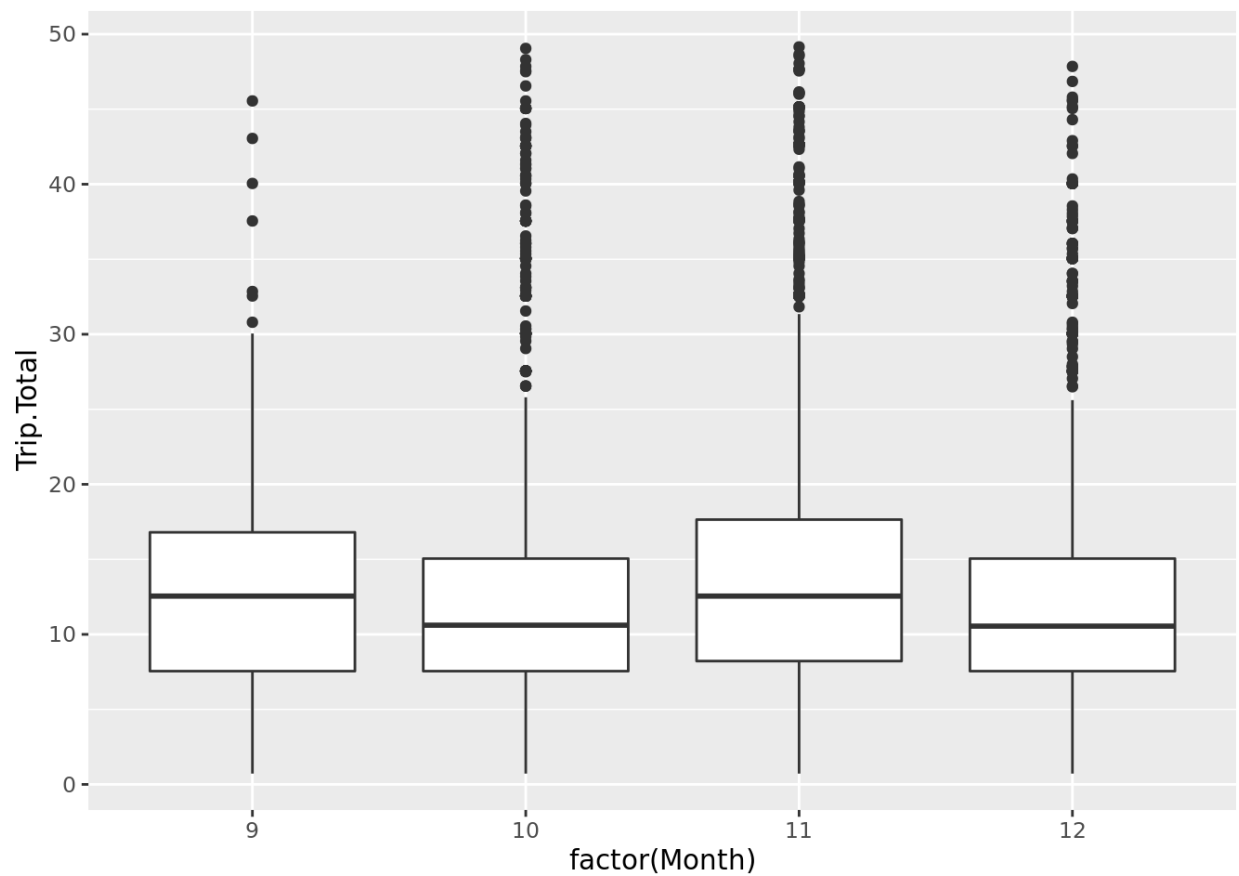
```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)

# Read in our rideshare dataset
rideshare_df <- read.csv("rideshare_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
# Construct a box plot showing Trip.Total by Month
rideshare_boxplot <-
  ggplot(rideshare_df,
    aes(x = factor(Month),
        y = Trip.Total)) +
  geom_boxplot()

# Print the plot to see what it looks like
rideshare_boxplot
```

```



Intermediate Data Visualization in R

## Stacked Bar Plots

Many times we are interested in seeing percentages within our data or how different values add up. We can do this using a stacked bar plot.

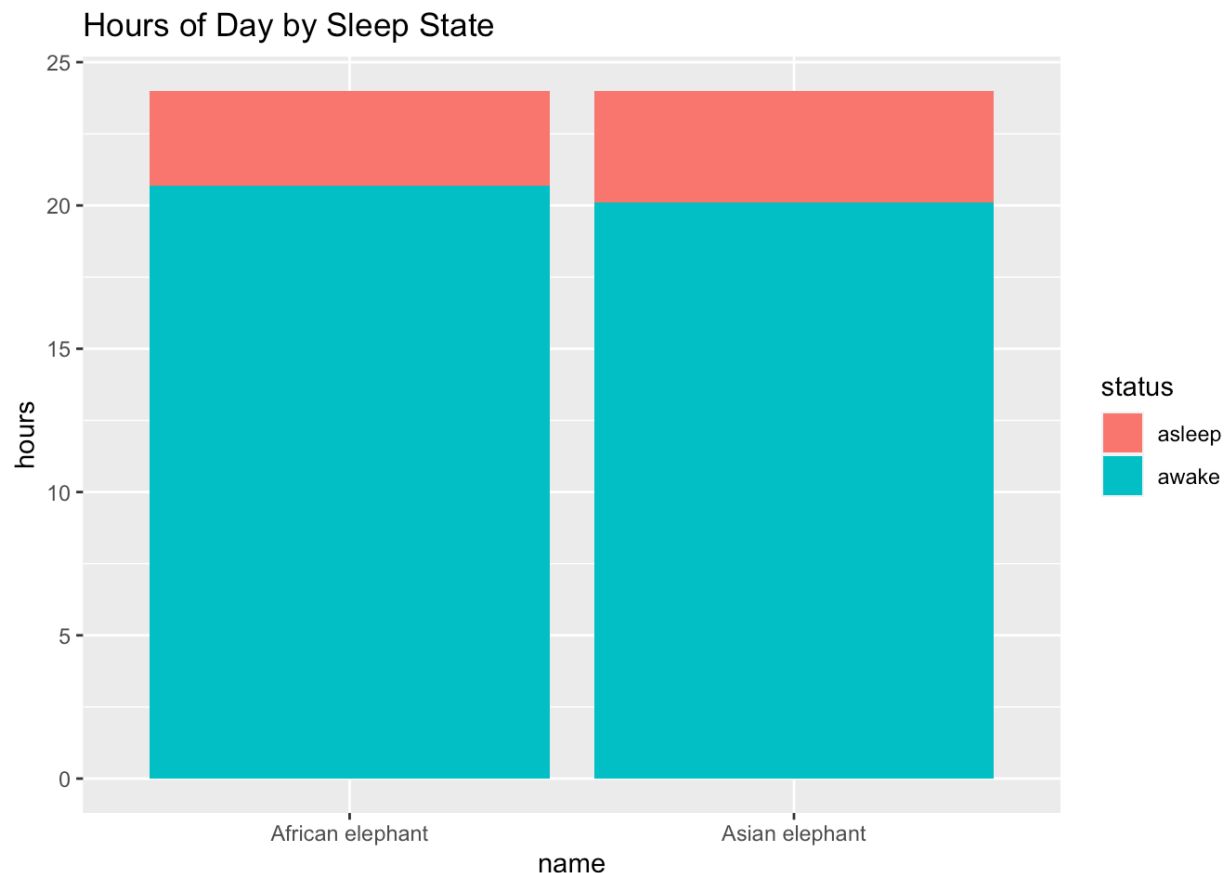
Let's turn to the `msleep` dataset included in `ggplot2` describing the number of hours spent asleep vs awake for various animals. We have pre-processed this data to include just the

[variables](#)

Preview: Docs Loading link description

we care about. Take a look at the output panel to see how this dataset looks.

Now, suppose we want to show the number of hours spent awake versus asleep each day for members of the order Proboscidea, i.e. elephants. We want to show this in a stacked bar plot because the hours awake versus asleep always add up to 24, and we are interested in depicting the proportion of each day spent in each state. The plot below tells us that for both African and Asian elephants, the vast majority of their day is spent awake.



The code below creates the plot we just saw. We specify a `fill` variable in our `aes()` mapping to tell `ggplot2` which variable should be depicted as color-coded segments within our stacked bars. Adding `stat = "identity"` to `geom_bar()` displays the values in our data frame as is, rather than displaying counts.

```
Filter our data to include only elephants
msleep_filtered <- msleep %>%
 filter(order == 'Proboscidea')

Construct a stacked bar plot
msleep_stackedbar <-
 ggplot(msleep_stacked_df,
 aes(x = name,
```



```

 y = hours,
 fill = status)) +
 geom_bar(stat = "identity") +
 labs(title = "Hours of Day by Sleep State")

```

Copy to Clipboard

We can explicitly add `position = "stack"` in our geom telling it to stack different values of the `fill` variable on top of each other; this is also assumed by default if we don't specify any positioning.

```

This creates the same plot!
msleep_stackedbar <-
 ggplot(msleep_stacked_df,
 aes(x = name,
 y = hours,
 fill = status)) +
 geom_bar(position = "stack",
 stat = "identity") +
 labs(title = "Hours of Day by Sleep State")

```

Copy to Clipboard

We can also create this same plot using the `geom_col()` geom, which works just like `geom_bar()` except it assumes `stat = "identity"` by default.

```

This also creates the same plot!
msleep_stackedbar <-
 ggplot(msleep_stacked_df,
 aes(x = name,
 y = hours,
 fill = status)) +
 geom_col() +
 labs(title = "Hours of Day by Sleep State")

```

Copy to Clipboard

## Instructions

Checkpoint 1 Enabled

1.

Let's turn to a new dataset we'll call `graduation_df`. This dataset describes graduation and enrollment rates for different demographic groups at New York City schools over multiple years.

We want to create a stacked bar plot examining the numbers of students graduating, enrolled, or dropped out for students in general education vs special education curricula. We've processed `graduation_df` to include only these two demographic groups and compute summary totals by the `Demographic`, `Cohort`, and `Status` variables. Run the `head()` function on our new `graduation_stacked_df` data frame to see how it looks.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a stacked bar plot named `graduation_stackedbar` with `Demographic` on the `x` axis, `N` on the `y` axis, and `Status` as the fill variable. Construct this bar plot using `geom_col()`.

Print `graduation_stackedbar` to see how it looks.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

The stacked bar plot we just created adds up values within each demographic group. Since there are many more general education students than special education students, it can be hard to compare ratios between the two populations.

We can add a `position = "fill"` argument to our geom, which tells `ggplot2` to represent each bar's total as percentages out of 1 rather than absolute counts. Create the same plot again and call it `graduation_stackedbar_fill`, this time specifying `position = "fill"` in `geom_col()`.

Print our new `graduation_stackedbar_fill` plot. Now it is more apparent how the ratios of students graduating, staying enrolled, and dropping out differ between the two groups.

```

title: "Stacked Bar Plots"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)
library(dplyr)

# Read in our msleep dataset
msleep <- read.csv("msleep_cleaned.csv")

# Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")

# Print the head of msleep
head(msleep)
```

```{r message=FALSE, warning=FALSE}
# Process graduation_df for our stacked bar plot
graduation_stacked_df <- graduation_df %>%
  filter(Demographic %in% c("General Education", "Special Education")) %>%
  group_by(Demographic, Year, Status) %>%
  summarize(N = sum(as.numeric(N)))

# Examine graduation_stacked_df using the head() function
head(graduation_stacked_df)

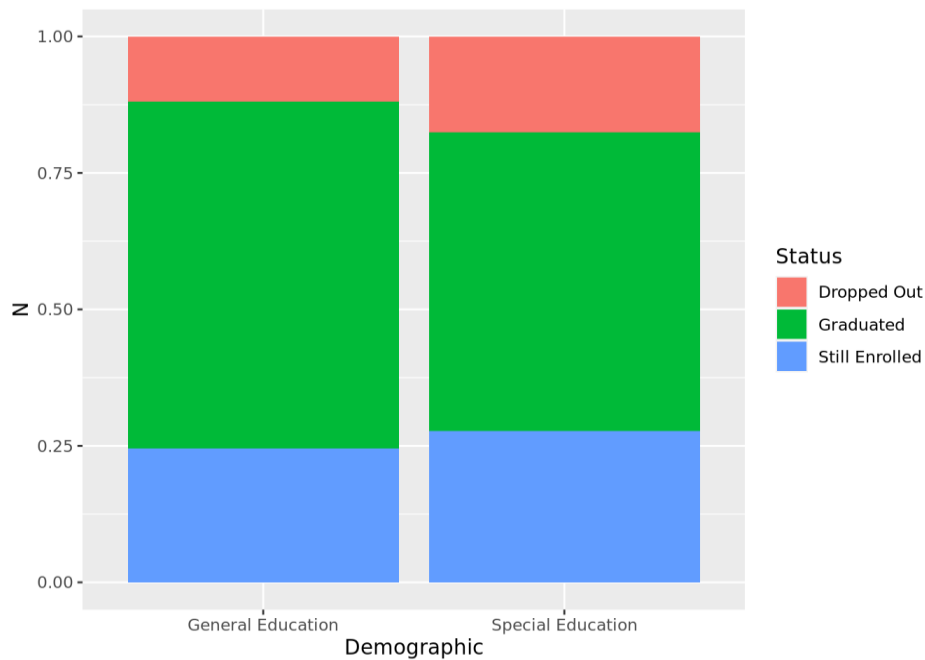
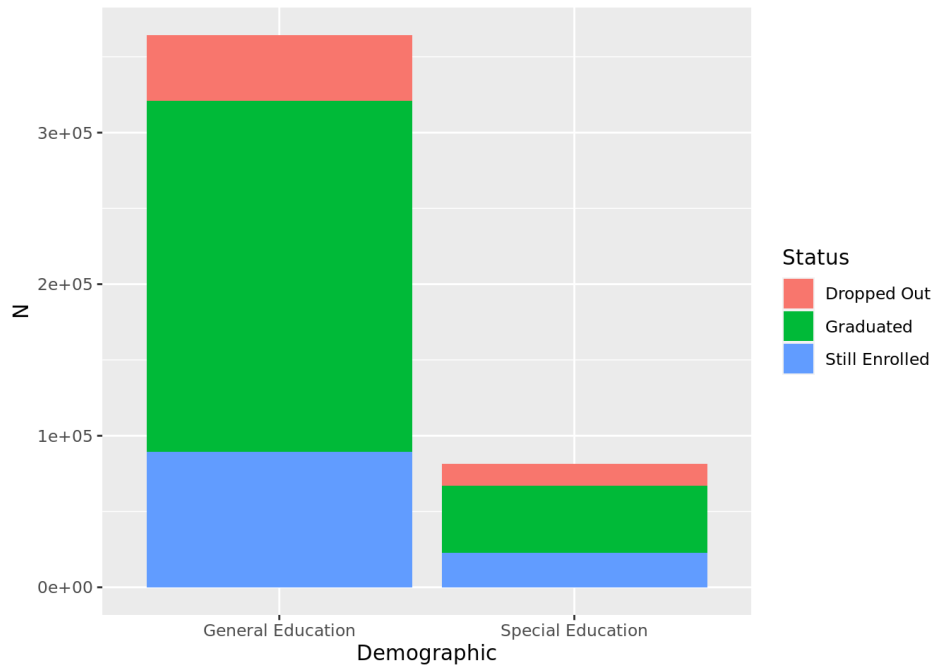
# Create a stacked bar plot named graduation_stackedbar
graduation_stackedbar <-
  ggplot(graduation_stacked_df,
    aes(x = Demographic,
        y = N,
        fill = Status)) +
  geom_col()

# Print the stacked bar plot to see what it looks like
graduation_stackedbar

# Create a stacked bar plot named graduation_stackedbar, this time with position="fill"
graduation_stackedbar_fill <-
  ggplot(graduation_stacked_df,
    aes(x = Demographic,
        y = N,
        fill = Status)) +
  geom_col(position = "fill")

# Print the new stacked bar plot to see what it looks like
graduation_stackedbar_fill
```

```

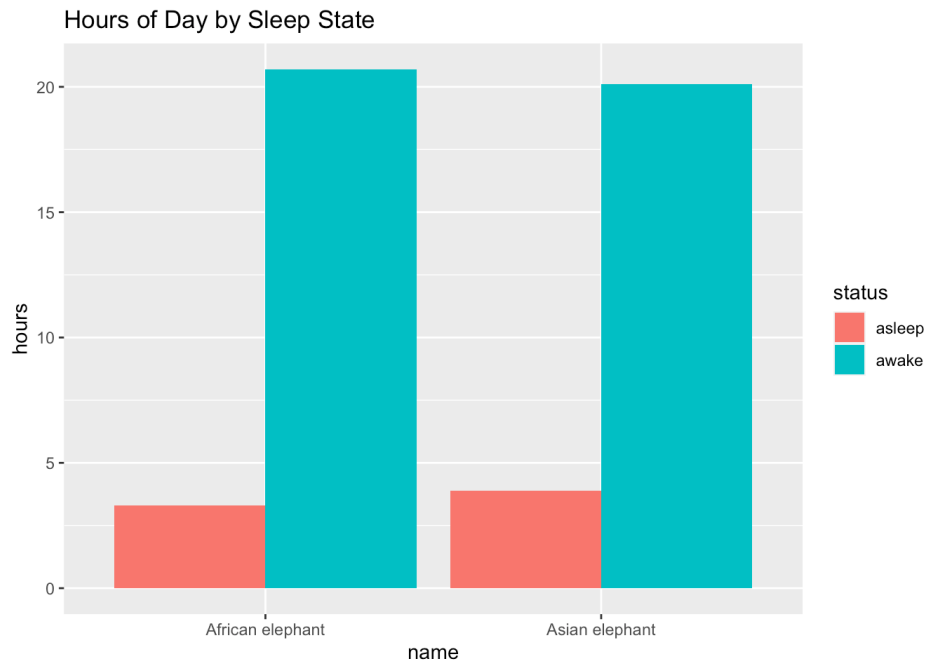


## Intermediate Data Visualization in R

# Clustered Bar Plots

Instead of stacking our `fill` variable in our bar graphs, we can also represent values of the variable side by side. This is known as a clustered bar plot.

The plot below visualizes our `msleep` data for elephants as a clustered bar plot instead of a stacked bar plot. Notice how the values shown are the same as in our stacked bar plot, but the positioning is now different.



We can easily create a clustered bar plot by setting `position = "dodge"` instead of `position = "stack"` in our geom. The `dodge` positioning tells `ggplot2` to display each value of the `fill` variable next to each other (as opposed to on top of each other in `"stack"`). Once again, we can either use `geom_bar(stat = "identity")` or `geom_col()`.

```
Filter our data to include only elephants
msleep_clustered_df <- msleep %>%
 filter(order == "Proboscidea")

Construct a clustered bar plot
msleep_stackedbar <-
 ggplot(msleep_clustered_df,
 aes(x = name,
 y = hours,
 fill = status)) +
 labs(title = "Hours of Day by Sleep State") +
 geom_bar(position = "dodge",
 stat = "identity")

This creates the same plot!
msleep_stackedbar <-
 ggplot(msleep_clustered_df,
 aes(x = name,
 y = hours,
 fill = status)) +
 labs(title = "Hours of Day by Sleep State") +
 geom_col(position = "dodge")
```

Copy to Clipboard

Instructions

## Checkpoint 1 Enabled

1.

Continuing with our `graduation_df` data, we now want to create a clustered bar plot visualizing the percent of students each year who graduated at one school, for students who are English proficient and English language learners.

We've created a new data frame called `graduation_clustered_df` with these records. Examine

`graduation_clustered_df` using the `head()` function.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a clustered bar plot named `graduation_clusteredbar` that maps `Year` to the `x` axis, `Pct` to the `y` axis, and `Demographic` to the `fill` variable. Use `geom_col()`, making sure to specify the correct `position` value.

Print the clustered bar plot we just created to see what it looks like.

```

title: "Clustered Bar Plots"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)
library(dplyr)

# Read in our msleep dataset
msleep <- read.csv("msleep_cleaned.csv")

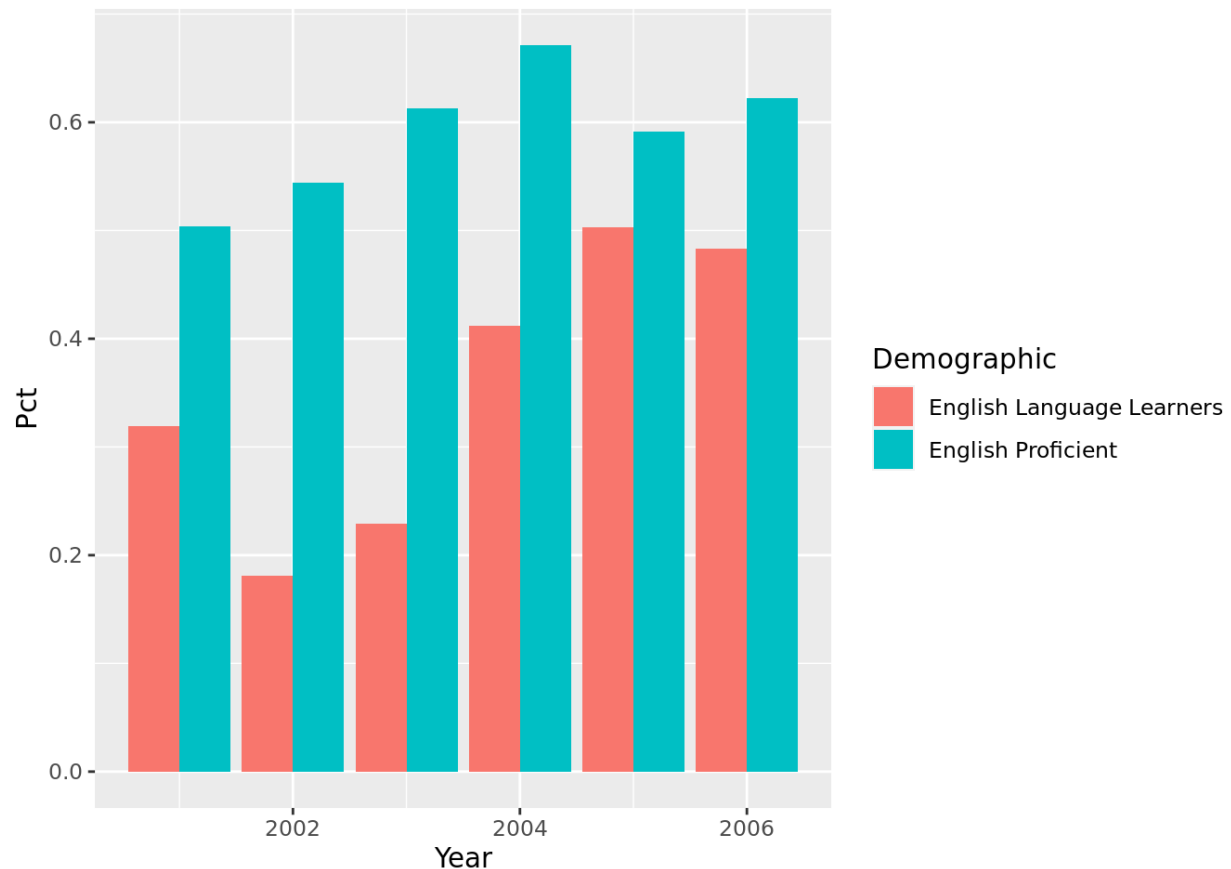
# Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
# Process graduation_df for our clustered bar plot
graduation_clustered_df <- graduation_df %>%
  filter(Demographic %in% c("English Language Learners", "English Proficient")) %>%
  filter(School.ID == "25Q425") %>%
  filter(Status == "Graduated")

# Examine graduation_clustered_df using the head() function
head(graduation_clustered_df)

# Create a clustered bar plot named graduation_clusteredbar
graduation_clusteredbar <-
  ggplot(graduation_clustered_df,
    aes(x = Year,
        y = Pct,
        fill = Demographic)) +
  geom_col(position = "dodge")

# Print the clustered bar plot to see what it looks like
graduation_clusteredbar
```
```



## Intermediate Data Visualization in R

### Statistical Summaries

By default, bar plots using `geom_bar()` show the count of observations for each value. We can also show other types of data, such as calculating and showing the mean instead.

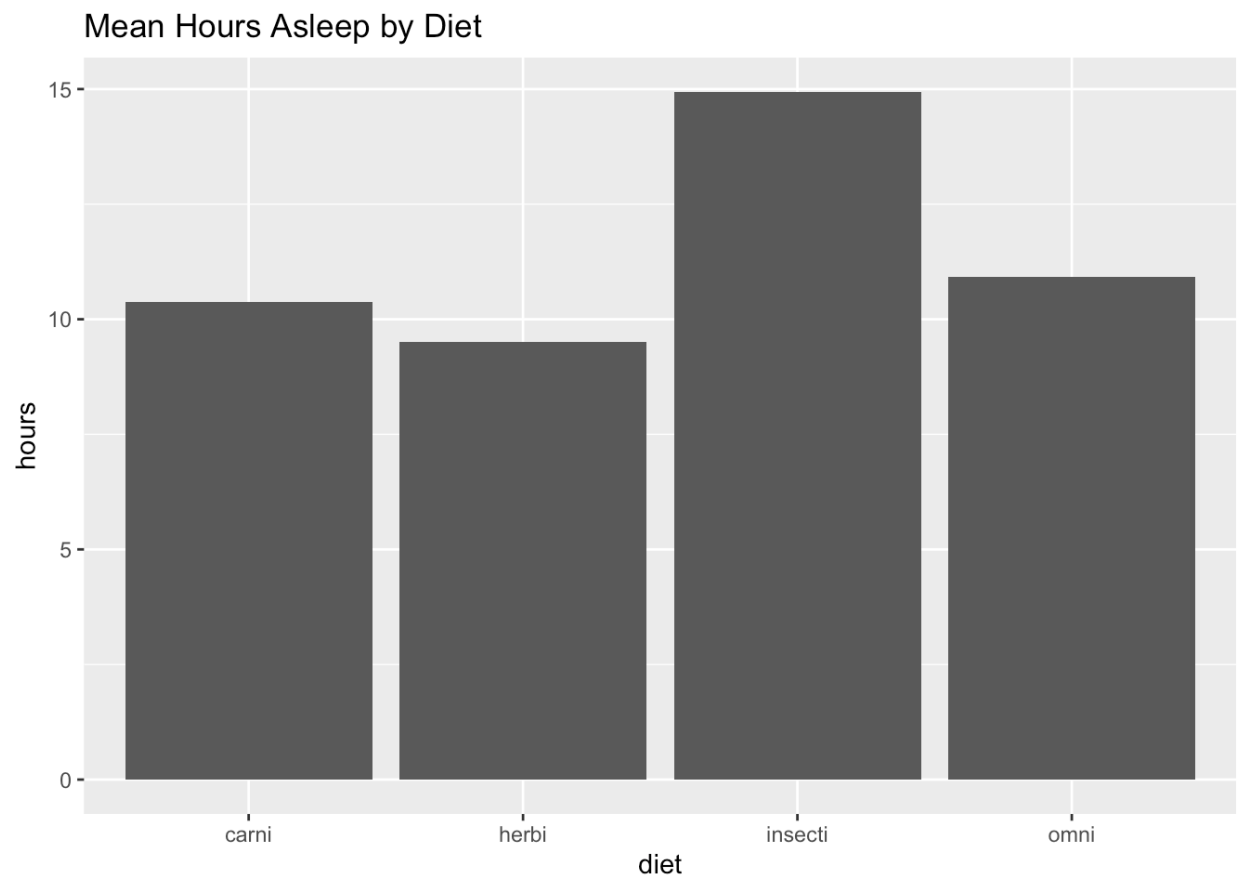
Let's say we want to see how much an animal sleeps on average by the kind of food it eats, based on the `msleep` dataset. The code below does just that! Recall that passing `stat = "identity"` to a `geom_bar()` layer tells `ggplot2` to display values as is, rather than count the number of occurrences. We can similarly use `stat = "summary"`, which tells `ggplot2` to summarize values according to a provided function. We can specify `fun = "mean"` to summarize our `y` axis variable by calculating mean values for each value in our `x` axis variable.

```
Filter our data to include only hours spent asleep, omitting NA values
msleep_means_df <- msleep %>%
 filter(status == "asleep") %>%
 na.omit()

Construct a bar plot calculating and displaying means
msleep_meanbar <-
 ggplot(msleep_means_df,
 aes(x = diet,
 y = hours)) +
 labs(title="Mean Hours Asleep by Diet") +
 geom_bar(stat = "summary",
 fun = "mean")
```

Copy to Clipboard

Here's how this plot looks. In the `msleep` dataset, insectivores (animals that eat insects) sleep for fifteen hours a day on average, which is far more than animals with other diets!



## Instructions

Checkpoint 1 Enabled

1.

We now want to calculate mean graduation rates for all students across all schools by year in our `graduation_df` dataset. We've filtered `graduation_df` to only retain rows where the `Demographic` column equals `Total Cohort` and the `Status` column equals `Graduated`.

Run the `head()` function on our new `graduation_means_df` data frame to examine it.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a bar plot using `graduation_means_df` named `graduation_meanbar`. Map `Year` to the `x` axis and `Pct` to the `y` axis. Use `stat = "summary"` and `fun = "mean"` to calculate and display mean values on the `y` axis.

Print the plot to see the change in mean graduation rates over time!

```

title: "Statistical Summaries"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)
library(dplyr)
```

```

# Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")
```

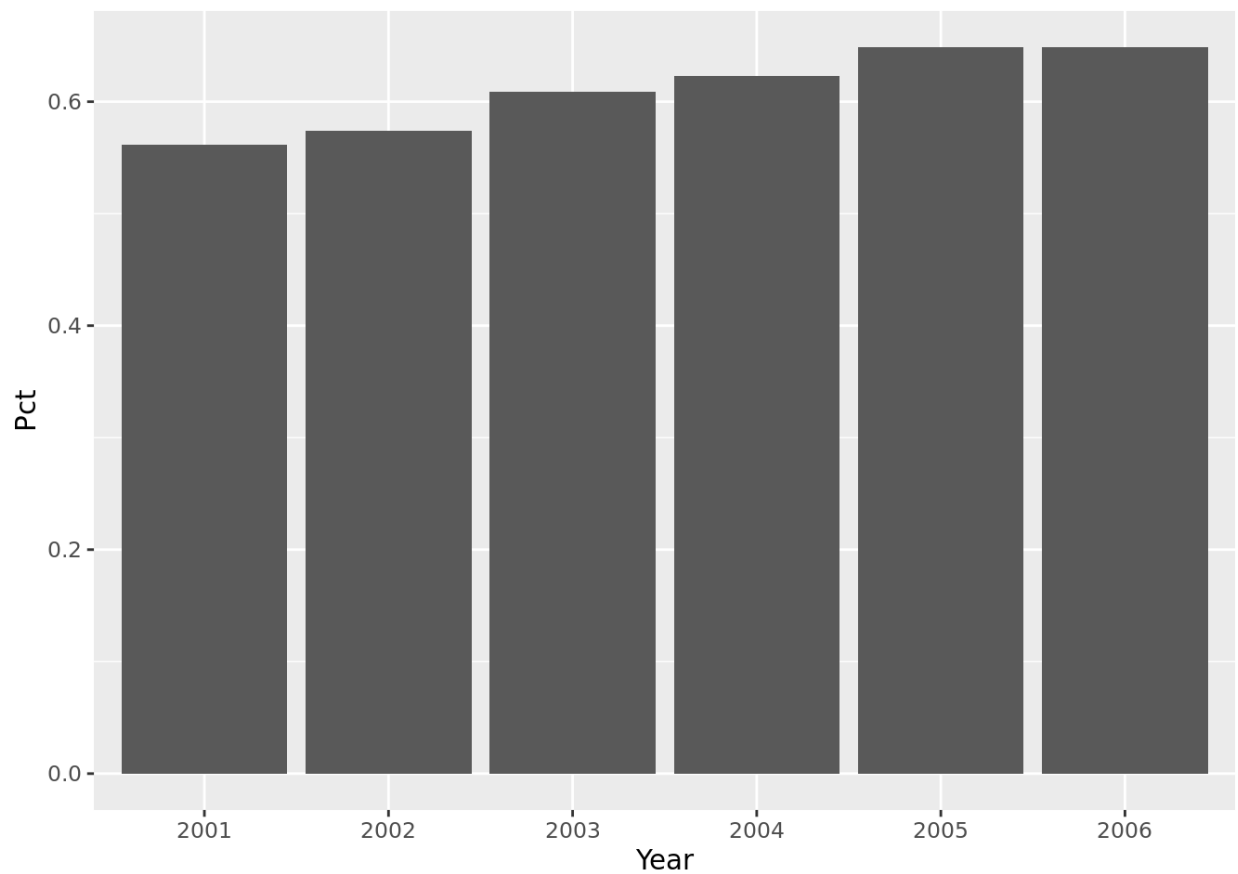
```{r message=FALSE, warning=FALSE}
# Process graduation_df for our bar plot showing means
graduation_means_df <- graduation_df %>%
  filter(Demographic == "Total Cohort") %>%
  filter(Status == "Graduated") %>%
  mutate(Year = factor(Year))

# Examine graduation_means_df using the head() function
head(graduation_means_df)

# Create a bar plot named graduation_meanbar showing means
graduation_meanbar <-
  ggplot(graduation_means_df,
    aes(x = Year, y = Pct)) +
  geom_bar(stat = "summary", fun = "mean")

# Print the bar plot to see what it looks like
graduation_meanbar
```

```



Intermediate Data Visualization in R



# Error Bars

Often, we'll want to show not only the mean of a value but also its standard error. This tells us how much variation there is around the mean – are most values close to the averages shown, or is there a wide range of values above and below the average?

To add error bars, we need to first calculate our standard errors. Continuing with our `msleep` dataset, the code below calculates means and standard errors for hours asleep by diet. We compute new

[variables](#)

Preview: Docs Loading link description

`mean.hours` representing the mean of `hours`, `mean.se` representing the standard error of our means, and `se.min` and `se.max` representing the lower and upper bounds of our error range.

```
msleep_error_df <- msleep %>%
 subset(status = "asleep") %>%
 na.omit() %>%
 group_by(diet) %>%
 summarize(mean.hours = mean(hours),
 mean.se = std.error(hours)) %>%
 mutate(se.min = mean.hours - mean.se,
 se.max = mean.hours + mean.se)
```

Copy to Clipboard

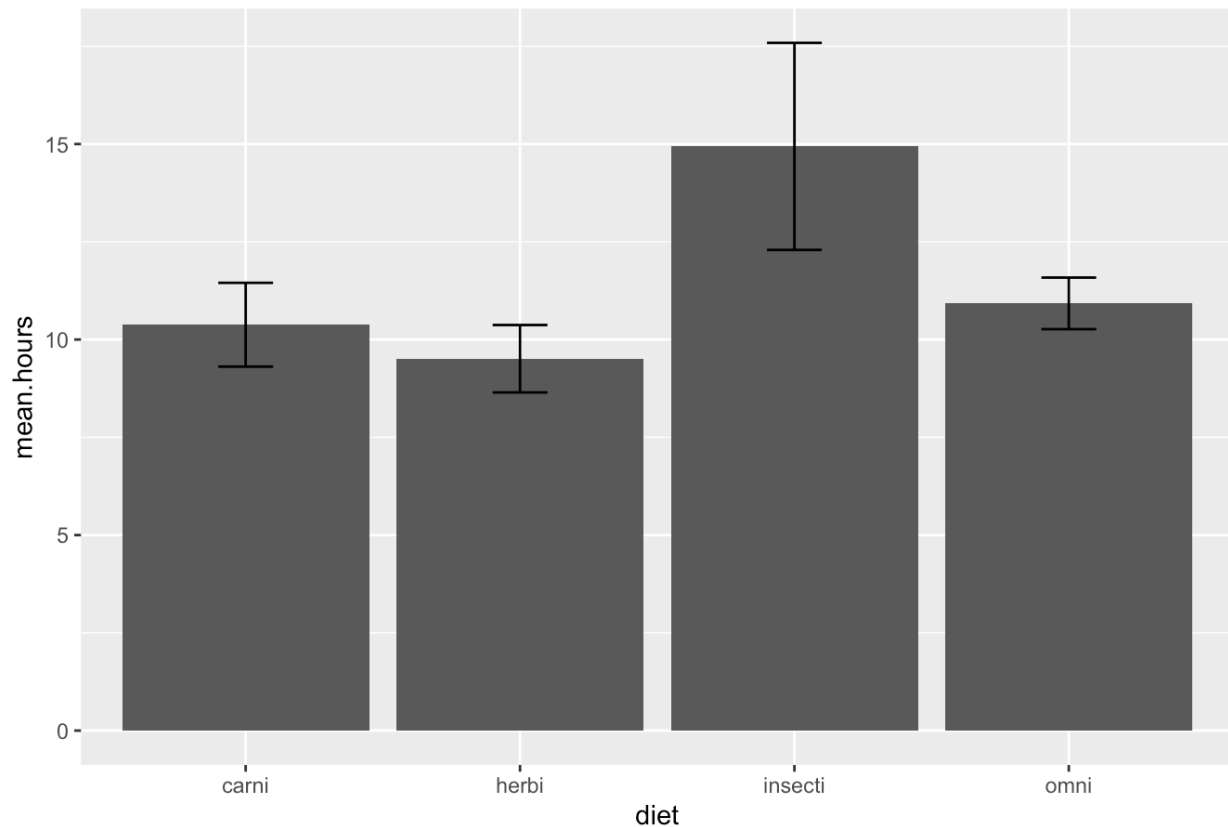
We can then create our bar plot showing means and standard errors as follows. We add a `geom_errorbar()` layer specifying `ymin` and `ymax` variables for our error bar and set its width to `0.2` of the bar's width. Because we've already calculated mean values in our summary data frame, we specify `stat = "identity"` to display the `mean.hours` values as is.

```
msleep_sebar <-
 ggplot(msleep_error_df,
 aes(x = diet,
 y = mean.hours)) +
 geom_bar(stat = "identity") +
 geom_errorbar(aes(ymin = se.min,
 ymax = se.max),
 width = 0.2) +
 labs(title = "Mean Hours Asleep by Diet")
```

Copy to Clipboard

This produces the following plot, which should look familiar from the last exercise! Now that we have error bars, we can see how much variation there is around the mean hours asleep for each diet group.

Mean Hours Asleep by Diet



### Instructions

Checkpoint 1 Enabled

1.

We now want to add error bars to our plot of mean graduation rates by year. We've created summary data frame called `graduation_error_df` containing means and standard errors for graduation rates by year. Examine `graduation_error_df` using the `head()` function.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Create a bar plot named `graduation_sebar` mapping `Year` to the `x` axis and `Mean.Pct` to the `y` axis, using our newly created `graduation_error_df` data frame. First, add a `geom_bar()` layer with the appropriate `stat`. Second, add a `geom_errorbar()` layer mapping `SE.Min` to `ymin` and `SE.Max` to `ymax` with a width of `0.2`.

Print `graduation_sebar` to see what it looks like.

```

title: "Error Bars"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)
library(dplyr)
library(plotrix)

# Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")
```
```

```

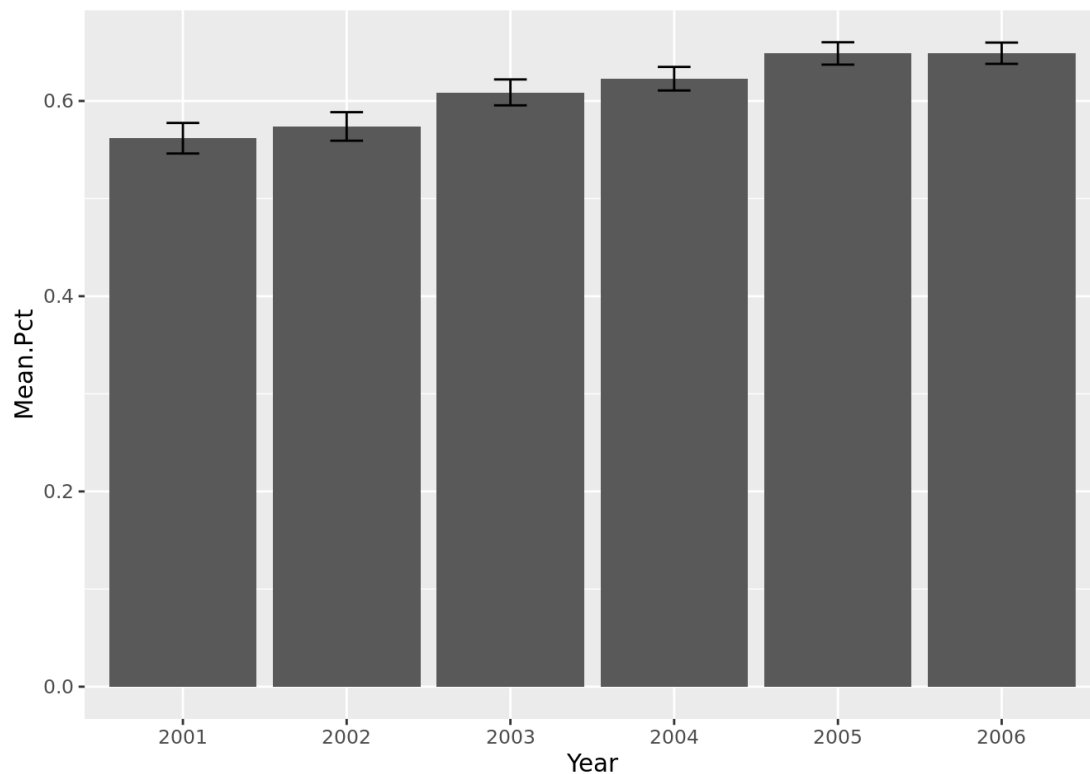
```{r message=FALSE, warning=FALSE}
# Process graduation_df to calculate means and standard errors
graduation_error_df <- graduation_df %>%
  filter(Demographic == "Total Cohort") %>%
  filter(Status == "Graduated") %>%
  group_by(Demographic, Year) %>%
  summarize(Mean.Pct = mean(Pct),
            Mean.SE = std.error(Pct)) %>%
  mutate(SE.Min = Mean.Pct - Mean.SE,
         SE.Max = Mean.Pct + Mean.SE,
         Year = factor(Year))

# Examine graduation_error_df using the head() function
head(graduation_error_df)

# Create a bar plot named graduation_sebar showing means and standard errors
graduation_sebar <-
  ggplot(graduation_error_df,
        aes(x = Year, y = Mean.Pct)) +
  geom_bar(stat = "identity") +
  geom_errorbar(aes(ymin = SE.Min,
                  ymax = SE.Max),
              width = 0.2)

# Print graduation_sebar to see what it looks like
graduation_sebar
```

```



## Customizing Discrete Axes

Frequently, we'll want to customize our axes to represent our data more clearly. For discrete

### [variables](#)

Preview: Docs A variable is used to store data that can be accessed later by subsequent code.

, such as categories on the `x` axis of a bar plot, we may want to specify a particular order these values appear in. Or, we might want to rename the axes labels so they better describe each value.

We can customize discrete variables on the `x` axis with the `scale_x_discrete()` layer. There is also a `scale_y_discrete()` layer that works the same way for variables on the `y` axis.

Let's take a look at the hours of sleep by diet plot we created in the last exercise. We can pass a vector to the `limits` argument in `scale_x_discrete()` to specify that we only want to show bars for `c("omni", "carni", "herbi")`, in that order rather than the default alphabetical ordering.

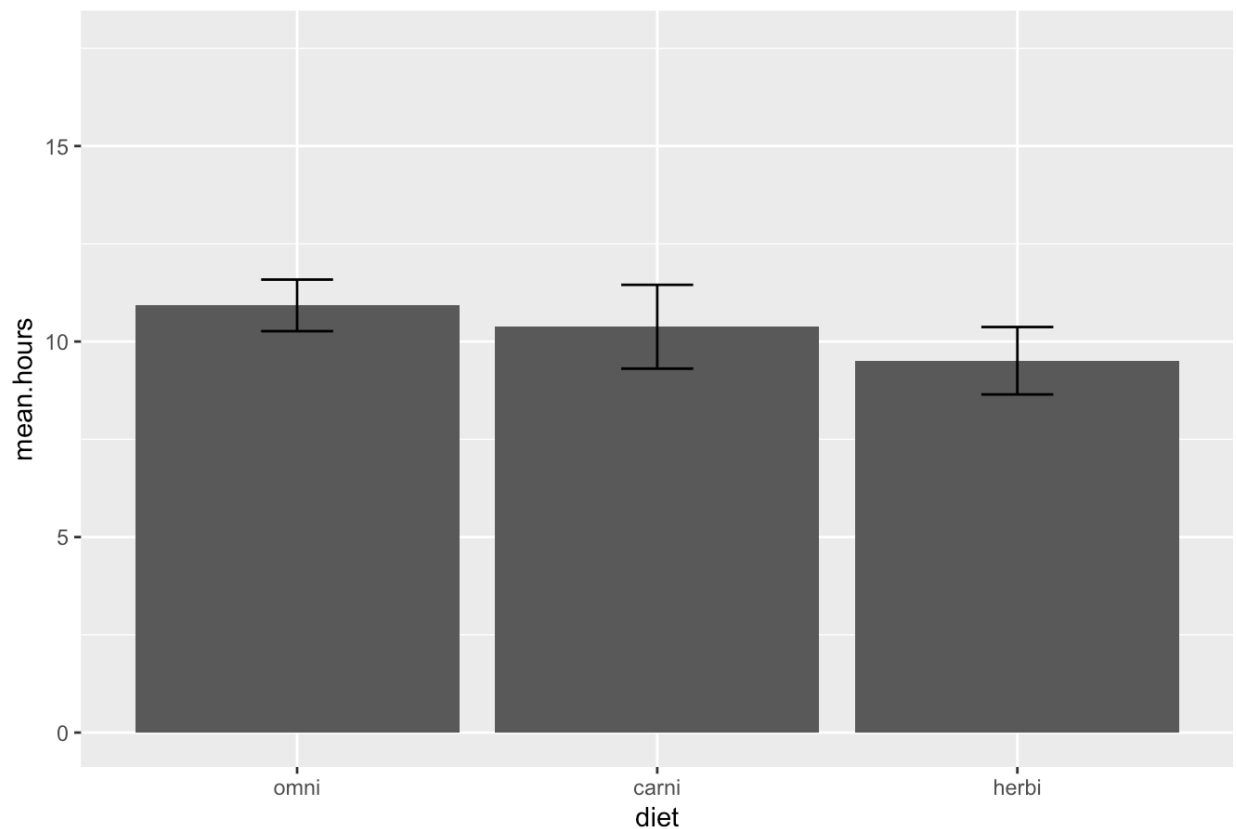
```
msleep_start <-
 ggplot(msleep_error_df,
 aes(x = diet,
 y = mean.hours)) +
 geom_bar(stat = "identity") +
 geom_errorbar(aes(ymin = se.min,
 ymax = se.max),
 width = 0.2) +
 labs(title = "Mean Hours Asleep by Diet")

msleep_discrete <-
 msleep_start +
 scale_x_discrete(
 limits = c("omni", "carni", "herbi"))
```

Copy to Clipboard

This produces the following plot. `"insecti"` is now omitted on the `x` axis. `"omni"` appears on the left, whereas it would appear on the right in the default alphabetical ordering.

Mean Hours Asleep by Diet



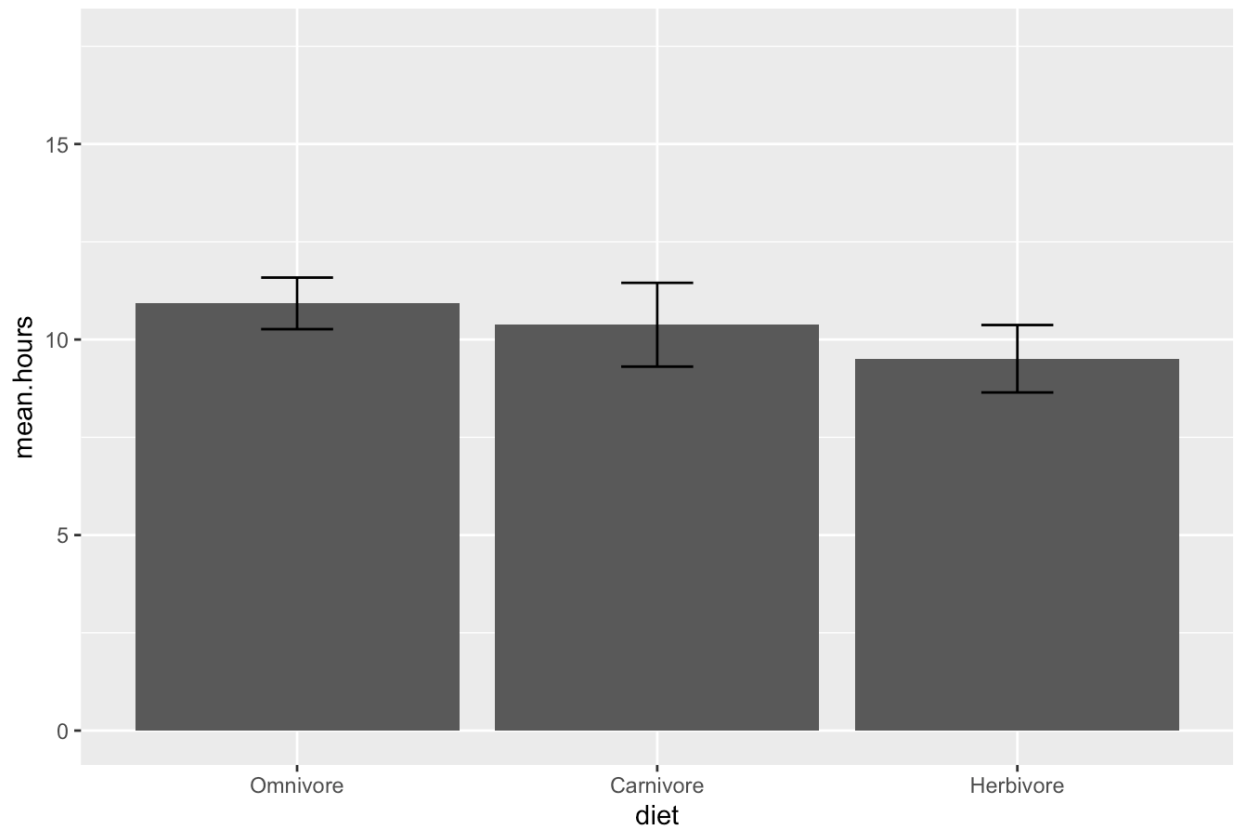
We might also want the labels to be more descriptive – for example, "carnivore" is a more familiar word compared to "carni". We can pass a vector of value-to-label mappings to the `labels` argument in `scale_x_discrete()` to specify what we want to show for each existing value.

```
msleep_discrete <-
 msleep_start +
 scale_x_discrete(
 limits = c("omni", "carni", "herbi"),
 labels = c("carni" = "Carnivore",
 "herbi" = "Herbivore",
 "omni" = "Omnivore"))
```

Copy to Clipboard

This produces the following plot. Now the labels are much more clear! The ordering we specify in `limits` is still preserved.

Mean Hours Asleep by Diet



### Instructions

Checkpoint 1 Enabled

1.

Let's return to our plot of mean graduation rates by year. We've loaded our preprocessed data frame and included starting code to create this plot again. Add a `scale_x_discrete()` geom and use its `limits` argument to only show graduation rates for years "2003", "2004", and "2005".

Then, print `graduation_discrete` to see what it looks like.

```

title: "Customizing Discrete Axes"
output: html_document

```{r message=FALSE, warning=FALSE}
# Load packages
library(ggplot2)
library(dplyr)
library(plotrix)

# Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
# Process graduation_df to calculate means and standard errors
graduation_error_df <- graduation_df %>%
  filter(Demographic == "Total Cohort") %>%
```

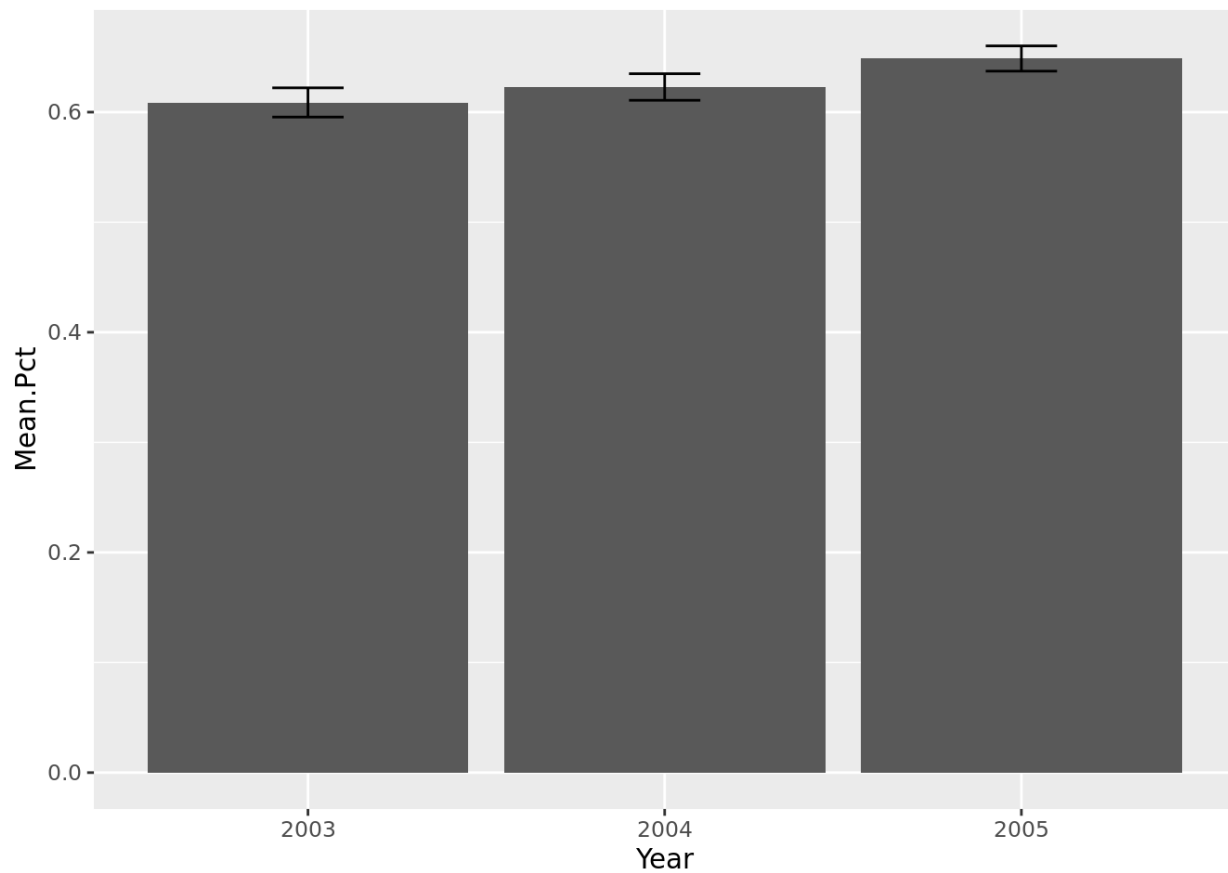
```

filter(Status == "Graduated") %>%
group_by(Demographic, Year) %>%
summarize(Mean.Pct = mean(Pct),
           Mean.SE = std.error(Pct)) %>%
mutate(SE.Min = Mean.Pct - Mean.SE,
       SE.Max = Mean.Pct + Mean.SE,
       Year = factor(Year))

# Complete the following bar plot by adding a scale_x_discrete() layer customizing x axis
labels
graduation_discrete <-
  ggplot(graduation_error_df,
         aes(x = Year, y = Mean.Pct)) +
  geom_bar(stat = "identity") +
  geom_errorbar(
    aes(ymin = SE.Min, ymax = SE.Max),
    width = 0.2) +
  scale_x_discrete(
    limits = c("2003", "2004", "2005"))

# Print graduation_discrete to see what it looks like
graduation_discrete
...

```



Facets

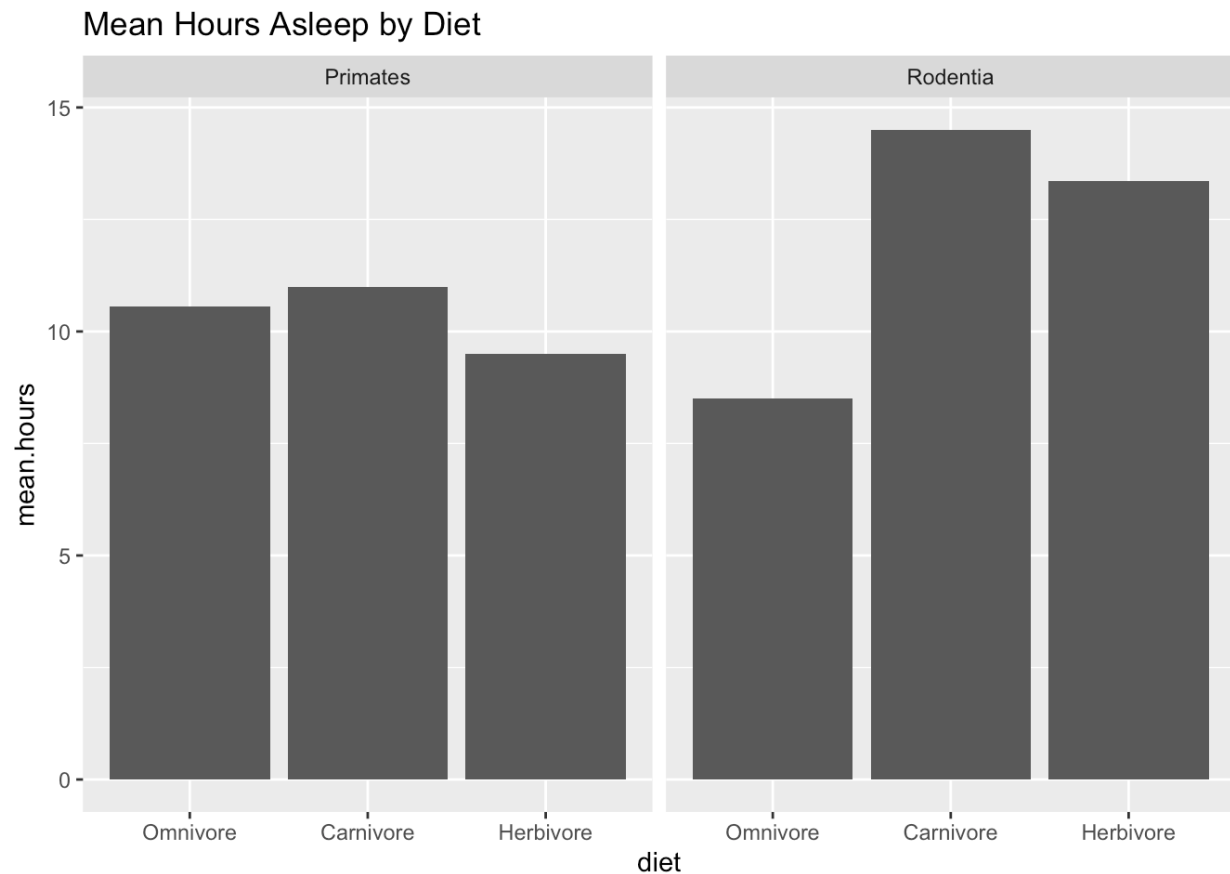
Facets allow us to visualize multiple discrete

[variables](#)

Preview: Docs Loading link description

in one plot, showing each value of the facet variable in a different section.

The plot below shows our familiar hours slept by diet plot, this time with the addition of a third variable describing the taxonomic order of animals in our data. We see that rodents of all three diet types sleep much more than primates!



We can add facets to our plot by adding a `facet_grid()` layer and specifying the variables it maps to. To show values of a facet variable as rows, we specify `facet_grid(rows = vars(row.var))`. To show values of a facet variable as columns, we specify `facet_grid(columns = vars(col.var))`. We can also show two facet variables in a grid, e.g. `facet_grid(rows = vars(row.var), columns = vars(col.var))`.

The code below produces the plot shown at the beginning of this exercise. First, we process our data frame to include the records we want and calculate mean hours asleep. Next, we create our sleep by diet plot, this time mapping the `order` variable to a `facet_grid()` layer split into columns.

```
msleep_facets_df <- msleep %>%  
  filter(status == "asleep") %>%  
  na.omit() %>%  
  group_by(diet, order) %>%  
  summarize(mean.hours = mean(hours)) %>%  
  filter(order %in% c("Primates", "Rodentia"))
```



```
msleep_facet <-
  ggplot(msleep_facets_df,
    aes(x = diet, y = mean.hours)) +
  geom_bar(stat = "identity") +
  labs(title = "Mean Hours Asleep by Diet") +
  scale_x_discrete(
    limits = c("omni", "carni", "herbi"),
    labels = c("carni" = "Carnivore",
               "herbi" = "Herbivore",
               "omni" = "Omnivore")) +
  facet_grid(cols = vars(order))
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Recall the clustered bar plot we created in Exercise 6, showing the percentage of students graduating by English proficiency, by year. For this exercise, we'll compare the graduation rates across two high schools. Complete the code provided, adding a `facet_grid()` layer to display outcomes by `School.ID`. Represent levels of this variable as rows. Print the plot to see what it looks like.

```
---
title: "Facets"
output: html_document
---

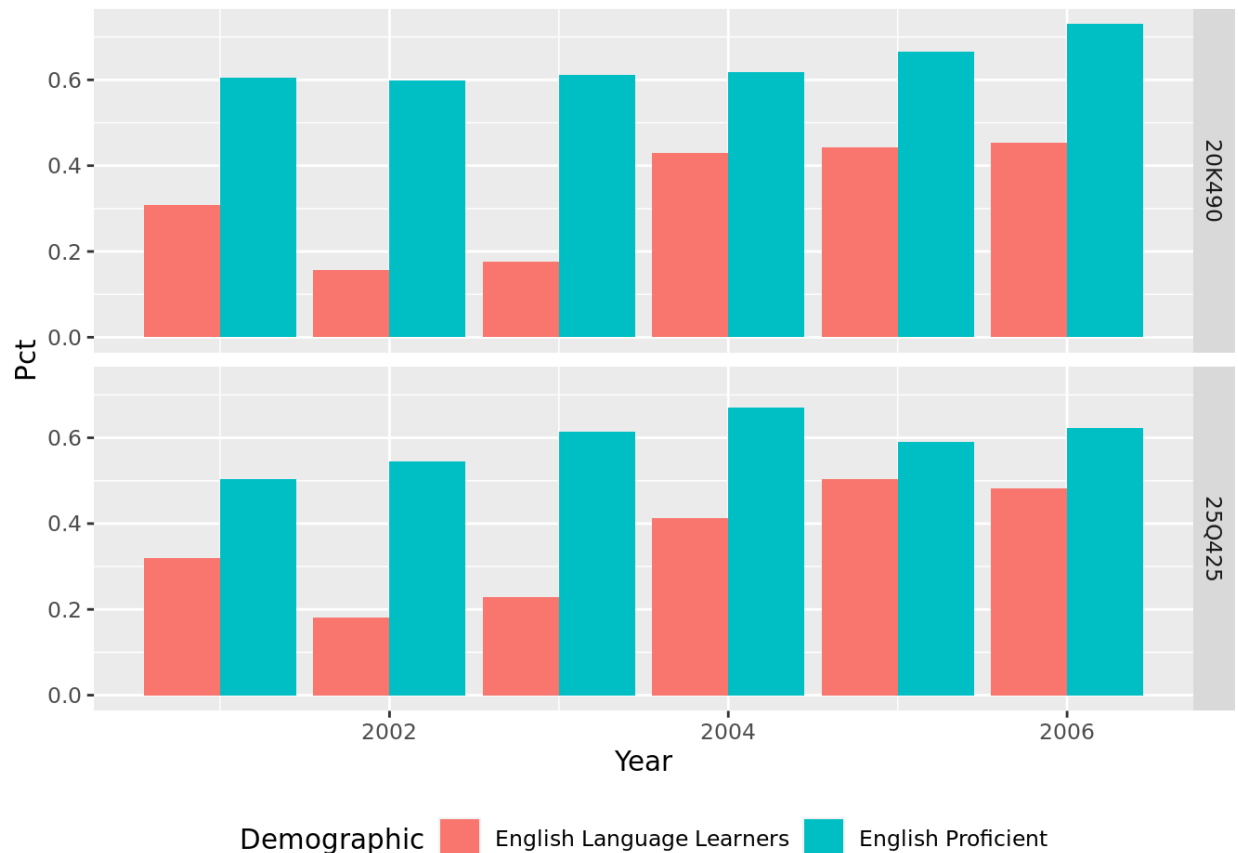
```{r message=FALSE, warning=FALSE}
Load packages
library(ggplot2)
library(dplyr)

Read in our graduation dataset
graduation_df <- read.csv("graduation_cleaned.csv")
```

```{r message=FALSE, warning=FALSE}
Process graduation_df to include the demographics we want to show
graduation_facets_df <- graduation_df %>%
 filter(Demographic %in% c("English Language Learners", "English Proficient")) %>%
 filter(School.ID %in% c("25Q425", "20K490")) %>%
 filter(Status == "Graduated")

Complete the bar plot below by adding a facet layer
graduation_facet <-
 ggplot(graduation_facets_df,
 aes(x = Year,
 y = Pct,
 fill = Demographic)) +
 geom_col(position = "dodge") +
 theme(legend.position = "bottom") +
 facet_grid(rows = vars(School.ID))

Print graduation_facet to see what it looks like
graduation_facet
```
```



Intermediate Data Visualization in R

Review

You've completed the Data Visualization in R lesson! You now know how to choose and implement different kinds of geoms in `ggplot2`, how to customize your plot axes, and how to visualize additional

[variables](#)

Preview: Docs Loading link description through facets.

Below is a summary of the key concepts you learned – great job!

How to create different geoms and when to use which type:

Histograms can be created using `geom_histogram()` to show the distribution of a continuous variable.

Heatmaps can be created using `geom_bin2d()` to show the distribution of the intersections of two continuous variables.

Box-and-whisker plots can be created using `geom_boxplot()` to show the distribution of a continuous variable by quantiles, e.g. 25th, 50th, and 75th percentiles.

Bar plots can be created using `geom_bar()`, which shows the count of observations for different values of a discrete variable by default. `geom_col()` will create bar plots showing the value of the variable on the y axis rather than counts.

Using the `position` argument and a `fill` aesthetic mapping, we can create stacked bar plots (`position = "stack"`), stacked bar plots showing ratios (`position = "fill"`), and clustered bar plots (`position = "dodge"`).

How to show different statistics in our data:

The `stat` argument allows us to display different kinds of values.
`stat = "identity"` will show the `y` axis variable values on a bar plot as is, rather than displaying the `x` axis value counts.
`stat = "summary"` combined with a function supplied in `fun` will display bar heights based on the summary function. For example, `stat = "summary", fun = "mean"` will calculate and display means.

How to add error bars to bar plots to show variance around a mean:

`geom_error()` creates error bars on bar plots when provided `ymin` and `ymax` variables representing the upper and lower bounds of error ranges.

How to customize discrete and continuous axes:

We can customize discrete axes using `scale_x_discrete()` and `scale_y_discrete()`.
We can customize continuous axes using `scale_x_continuous()` and `scale_y_continuous()`.
We can zoom in on a region of our data using `coord_cartesian()`.

How to show additional variables in panels of a grid using facets:

By adding `facet_grid()`, we can map up to two additional variables along facet columns and rows.

Instructions

The code included to the right creates the plot shown in the very first exercise, depicting how different animals spend the hours of their day. Feel free to experiment with this plot and modify it further!

Data Visualization in R Cumulative Project

https://www.youtube.com/watch?v=T44GWKIFFRI&ab_channel=Codecademy

In this project, we will be visualizing data associated with the New York City bike share program, Citi Bike!
Welcome to the cumulative project for visualization in R! Follow these instructions to download the project and complete it using RStudio.

Getting Started with RStudio

If you've never used RStudio before, we recommend watching this video to familiarize yourself with the tool: If you have not already done so, [download R](#), and [download RStudio](#).
Download the [Data Visualization project](#) zip file.
Unzip the file by double-clicking on it in your Downloads folder.
Navigate to the newly unzipped folder and open `Citibike_Project_Starting.Rmd` in RStudio to see the project instructions.
Follow the instructions by writing code in the code blocks found in `Citibike_Project_Starting.Rmd`.

If you are stuck on the project, we have included our solution in the file `Citibike_Project_Solution.Rmd`.

Welcome To Machine Learning In R

A preview of the machine learning section of this course.

Welcome to the section of the course that focuses on machine learning. Machine learning is a massive subject, and we're just scratching the tip of the iceberg in this section. In this section, you will learn how to make linear regression models using R. In addition to learning how to make the model, you will also learn how to interpret it. This is almost more critical than making the model itself — being able to communicate the findings that you get from your model is an essential skill of a data scientist.

Let's get started!

Linear Regression in R

Introduction to Linear Regression in R

Linear Regression is the workhorse of applied Data Science; it has long been the most commonly used method by scientists and can be applied to a wide variety of datasets and questions. Unlike more recently developed methods in Machine Learning, Linear Regression models can be used to predict new data points and help us understand the relative impact one variable has on another. For example, one well-designed regression model can answer:

How, and to what extent, does advertising in print media affect the total sales of a product?
What are the predicted total sales for a product, given the amount spent on print advertisement this month?

In this lesson, you will learn how to harness the malleability and explanatory power of Linear Regression models by following the four primary steps of statistical model building: **confirming data assumptions**, **building a model on training data**, **assessing model fit**, and **analyzing model results**. Using two real-life datasets, `conversion` and `advertising`, this lesson will also focus on the application of regression modeling for use in marketing data science. Let's get started!

Instructions

1. Two datasets, `conversion` and `advertising`, are included in workplace for you to explore. `conversion` is an example of data that a scientist would receive from a large social media platform, like Facebook or Twitter, and can be used to answer questions about the performance of newsfeed advertisements. `advertising` is an example of data that would be recorded by the marketing department of a company; a marketing data scientist would often be given this data and expected to answer questions around of the return on investment for various media channels, like podcast or tv advertisements. Familiarize yourself with both before moving forwards! Some suggestions:

Use `str(dataset_name)` to print out a list of variables and their associated datatypes

Use a combination of `mean()` and R's variable selection syntax to calculate the average value of a data variable. For instance, we could calculate the mean of `sales` in the `advertising` dataset using `mean(advertising$sales)`

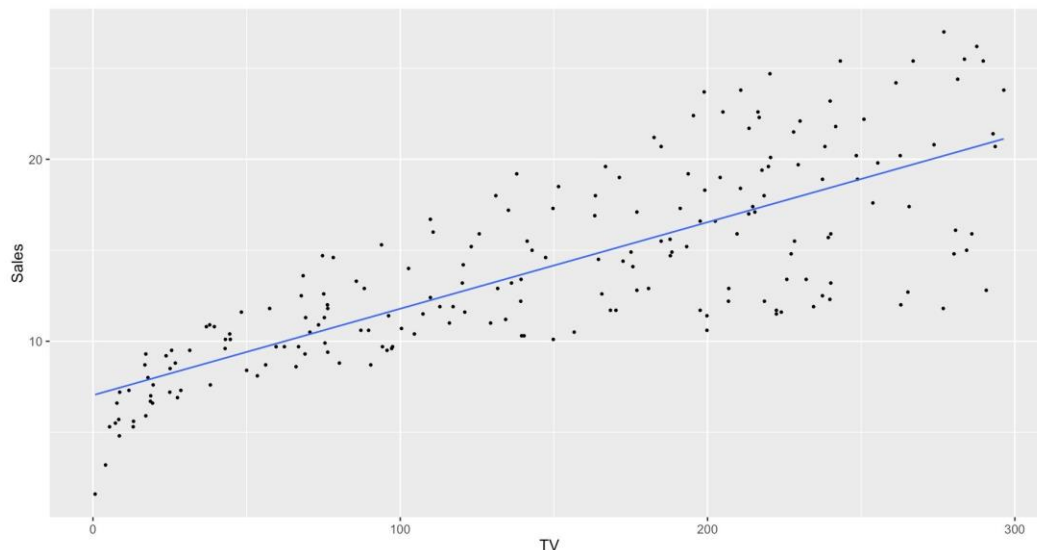
Use ggplot's `geom_bar` and `geom_point` functions to visually analyze the distribution of data values.

Linear Regression in R

Assumptions of Simple Linear Regression

While the linear regression is perhaps the most widely applied method in Data Science, it relies on a strict set of assumptions about the relationship between predictor and outcome. The most obvious (but crucial!) assumption is a **linear relationship** between the predictor and outcome. Following from this assumption is one key observation about any variables we want to include in our model which must be tested before building a model:

The expected value of the outcome variable is a straight-line function of exclusively the predictor variable. The best test for this relationship is quite straightforward-- we can just visualize the relationship between the predictor and outcome variables as a scatterplot. A linear relationship will resemble a straight line with a slope not equal to zero, like the relationship between spending on TV ads and the overall sales volume of the related product found in our `advertising` dataset.



We can also quantitatively test for a linear relationship by computing the **correlation coefficient**. The correlation coefficient is always between positive one and negative one. A coefficient close to 0 (roughly between -0.20 and 0.20) suggests a weak linear relationship between two variables. A coefficient closer to positive or negative one suggests a stronger linear relationship. In R, we can compute the correlation coefficient using the `cor.test()` method as follows:

```
coefficient <- cor.test(advertising$TV, advertising$Sales)
coefficient$estimate
# Output:
0.837
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Load the `conversion.csv` into the working environment using `read.csv()`. Save the result to a variable called `conversion`, and don't forget to set the `header` parameter to `TRUE`!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

A good statistical workflow *always* involves a thorough understanding of the data available to model and a qualitative analysis of relevant variables. Use `str()` to write out the structure of the dataset and list of variables types. Which variables seem like possible predictors of purchase, or `total_convert`?

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use a combination of the base `ggplot()` function and `geom_bar()` to plot the distribution of the `clicks` variable, a measure of how many times a user clicked on an advertisement. Save the result to a variable called `clicks_dist`. Call `clicks_dist`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Take a closer look at the `clicks_dist` visualization. What is the approximate range of the `clicks` variable? What seems like the most common value (otherwise called the *mode*) of `clicks`? Set `clicks_mode` equal to approximate value of the `clicks` mode.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Assign the result of calling `cor.test()`, with `conversion$total_convert` and `conversion$clicks` as input parameters, to a variable called `correlation`. Print out `correlation$estimate`. Does the coefficient value suggest that the variables have a linear relationship?

```
---
title: "Assumptions of Simple Linear Regression"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries and data
library(readr)
library(dplyr)
library(ggplot2)
```

```{r read_data}
conversion <- read.csv('conversion.csv', header= T)
str(conversion)
```

```{r create_viz}
save viz to object
```

```

clicks_dist <- conversion %>%
 ggplot(aes(clicks)) +
 geom_bar()

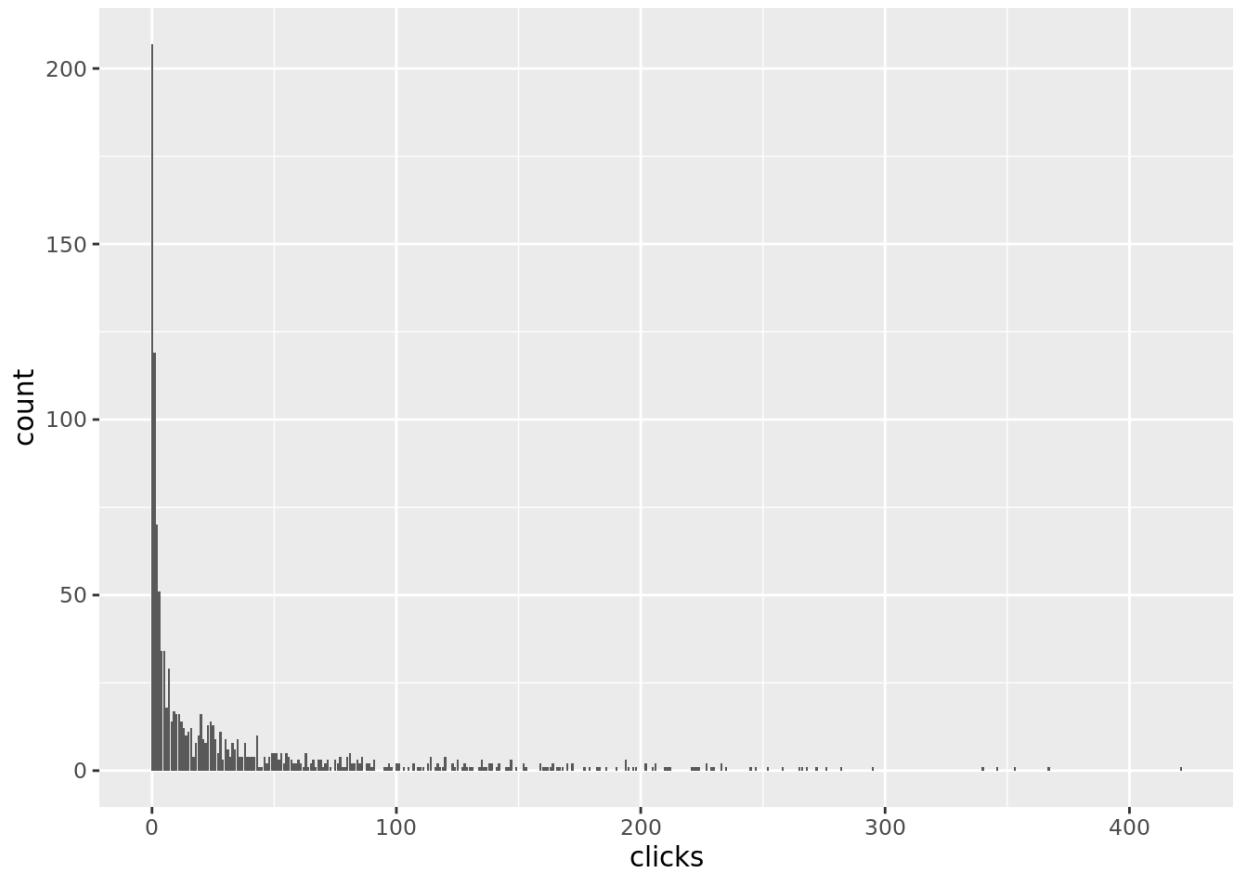
print out viz object
clicks_dist

declare mode of clicks
clicks_mode = 1
```

```{r compute_cor}
compute correlation
correlation <- cor.test(conversion$total_convert, conversion$clicks)

print out estimate value
correlation$estimate
```

```



Linear Regression in R

Assumptions of Linear Regression (Outliers)

Our next step is to check for outlier data points. Linear regression models also assume that there are no extreme values in the data set that are not representative of the actual relationship between predictor and outcome

[variables](#)

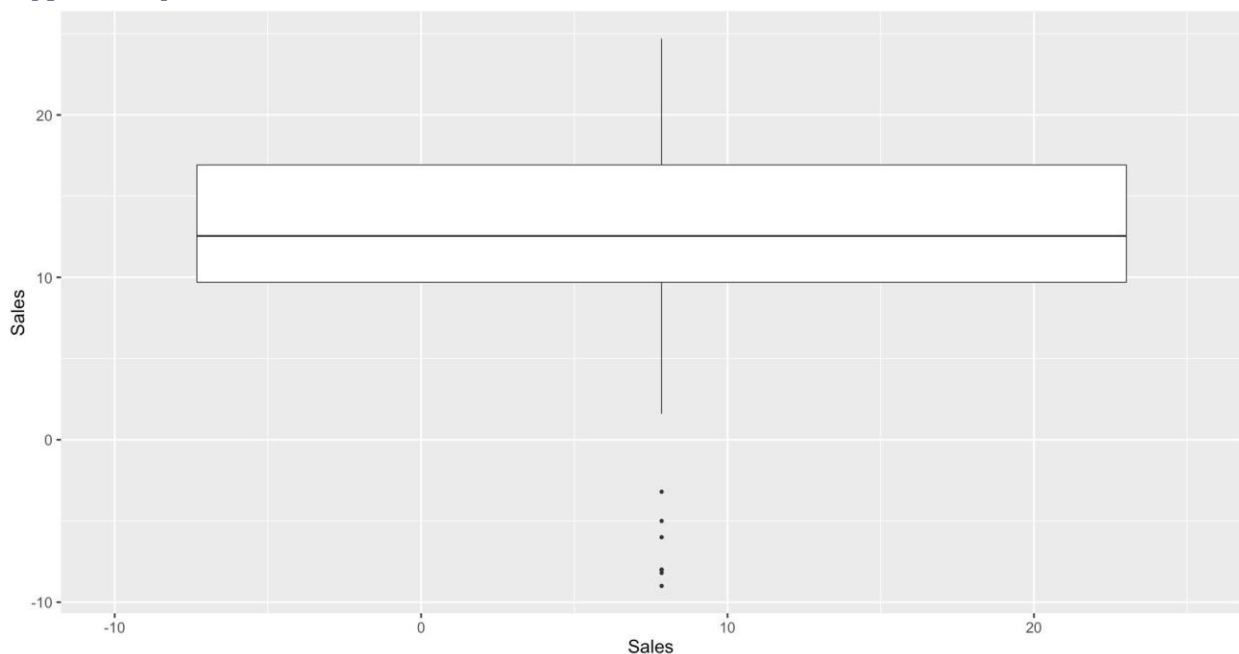
Preview: Docs Loading link description

A box-and-whisker plot is a common method used to quickly determine whether a data set contains any outliers, or data points that differ significantly from other observations in a dataset. An outlier may be caused by variability in measurement, or it might be a sign of an error in the collection of data.

Regardless, `ggplot`'s `geom_boxplot()` method allows for the easy creation of box-and-whisker plots. To plot the distribution of a single variable, like `advertising$sales`—the total number of sales for a product in a month—we pass in the same variable as both `x` and `y` in our call to `geom`:

```
plot <- advertising %>%  
  ggplot(aes(sales, sales)) +  
  geom_boxplot()
```

Copy to Clipboard



In this case, it looks like there are a handful of negative `sales` values in the dataset. This is not what we would expect given our understanding of the data; how could an entire market have negative average sales over an entire year? This seems like an error stemming from the collection of this data into a spreadsheet format. In this case, we will filter out these negative datapoints from our dataset using the `filter()` method. We can pass a boolean argument into `filter()` to exclude values that resolve to `false`.

```
advertising <- advertising %>% filter(Sales > 0)
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

Let's check our variables for any outliers. Use a combination of the base `ggplot()` function and `geom_boxplot()` to plot a boxplot of the `clicks` variable; assign the result to a variable called `clicks_bx_plot`. Don't forget to pass `clicks` in as the `y` variable, and afterwards call `clicks_bx_plot` in order to view the plot!

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Do any data points look like outliers? Set `threshold` equal to the value of `clicks` above which all data points fall outside of the whiskers of our box plot.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Use the `filter()` method to remove all outlying values from `clicks`. Save the resulting data frame to a variable called `convert_clean`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Let's check our work! Create another box plot of `clicks`, but this time, use the `convert_clean` dataset and save the plot to `clean_bx_plot`. Call `clean_bx_plot` and note any differences from `clicks_bx_plot`. Are the outliers gone?

```
---
title: "Assumptions of Linear Regression (Outliers)"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
load libraries and data
library(dplyr)
library(ggplot2)
conversion <- read.csv('conversion.csv', header= T)
```

```{r create_viz}
create viz object
clicks_bx_plot <- conversion %>%
 ggplot(aes(clicks, clicks)) +
 geom_boxplot()

print out object
clicks_bx_plot
```

```{r filter_data}
set threshold value
threshold <- 100

remove outliers
convert_clean <- conversion %>% filter(clicks < threshold)

creat second box plot
clean_bx_plot <- convert_clean %>%
 ggplot(aes(clicks, clicks)) +
 geom_boxplot()

clean_bx_plot
```
```

Building a Simple Linear Model

Simple linear regression is not a misnomer—it is an uncomplicated technique for predicting a continuous outcome variable, Y , on the basis of just one predictor variable, X . As detailed in previous exercises, a number of assumptions are made so that we can model the relationship between X and Y as a linear function. Using our `advertising` dataset, we could model the relationship between the amount spent on podcast advertising in a month and the number of respective products eventually sold as follows:

$$Y = \text{beta}_0 + \text{beta}_1 * X + \text{error}$$

Where...

Y: represents the dollar value of products sold

X: represents the amount spent on respective product podcast ads

Beta_0: is the intercept, or the number of products sold when no money has been spent on podcasts

Beta_1: is the coefficient, or the slope, of the line representing the relationship

Error: represents the random variation in the relationship between the two variables

To build this model in R, using the standard `lm()` package, we use the formula notation of $Y \sim X$:

```
model <- lm(sales ~ podcast, data = train)
```

But wait! Before building this model, we need to [split our data into test and training sets](#). For the development of this simple model, we'll use a standard 60/40 split of our data; where 60% is used to train the model, and 40% is used to test the model's accuracy and generalizability. We can randomly assign data points to test or training using base R's `sample()` method and list indexing functionality

```
# specify 60/40 split
sample <- sample(c(TRUE, FALSE), nrow(advertising), replace = T, prob = c(0.6, 0.4))
# subset data points into train and test sets
train <- advertising[sample, ]
test <- advertising[!sample, ]
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

First, let's split our `conversion_clean` dataset into 60/40 train/test subsets

Create a variable named `data_sample` by assigning the result of calling `sample()`, with `c(TRUE, FALSE)`, `nrow(conversion_clean)`, and `prob = c(0.6, 0.4)` as parameters.

Using list indexing, assign all data points in `sample` to a variable called `train`

Using list indexing, assign all data points in *not* in `sample` to a variable called `test`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Let's fit a linear model of the relationship between the number of products sold and the number of clicks on the respective product advertisement; this means that `conversion's total_convert` value, the total number of product purchases by a single user, will be our Y variable, or outcome variable. `clicks` the total number of times a user clicks on a version of an ad, will be our X variable, or predictor variable.

Assign the result of calling `lm()`, using `~` formula notation to set a linear relationship between `total_convert` and `clicks`, to a variable called `model`. Don't forget to set the `data` parameter equal to `train`!

```

title: "Building a Simple Linear Model"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
```

```{r split_train_test}
specify 60/40 split
data_sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob =
c(0.6,0.4))
subset data points into train and test sets
train <- conversion_clean[data_sample,]
test <- conversion_clean[!data_sample,]
```

```{r build_model}
build model
model <- lm(total_convert ~ clicks, data = train)
```

```

Linear Regression in R

Quantifying Model Fit

Once we have an understanding of the kind of relationship our model describes, we want to understand the extent to which this modeled relationship actually fits the data. This is typically referred to as the goodness-of-fit. In simple linear models, we can measure this quantitatively by assessing two things:

Residual standard error (RSE)
R squared (R^2)

The RSE is an **estimate of the standard deviation of the error** of the model (*error* in our mathematical definition of linear regression). Roughly speaking, it is the average amount that the response will deviate from the true regression line. We get the RSE at the bottom of `summary(model)`, we can also get it directly with

```
sigma(model)
```

```

#output
3.2

```

Copy to Clipboard

An RSE value of 3.2 means the actual sales in each market will deviate from the true regression line by approximately 3,200 units, on average. Is this too large of a deviation? Well, that's subjective, but when compared to the average value of sales over all markets the percentage error is 22%:

```
sigma(model)/mean(train$sales)
```

```

# output
[1] 0.2207373

```

Copy to Clipboard

The RSE provides an absolute measure of lack of fit of our model to the data. But since it is measured in the units of Y, it is not always clear what constitutes a good RSE.

The R^2 statistic provides an alternative measure of fit. It represents the **proportion of variance explained**, so it always takes on a value between 0 and 1, and is independent of the scale of Y, our outcome variable. Similar to RSE, the R^2 can be found at the bottom of `summary(model)` but we can also extract it directly by calling `summary(model)$r.squared`. The result below suggests that podcast advertising budget can explain 64% of the variability in the total `sales` value.

```
summary(model)$r.squared  
  
# output  
[1] 0.6372581
```

Copy to Clipboard

Instructions

Checkpoint 1 Enabled

1.

The code used to produce `model`, a simple linear model summarizing the relationship between `clicks` and `total_convert` is already included in your notebook.

Assign the result of `sigma(model) / mean(train$total_convert)` to `avg_rse`

Uncomment the following f-string, then run the file to see how we would contextualize the average RSE of our model

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Model fit is often quantified in comparison to other models, then used to determine which variation of a modeled relationship best fits the data. Let's build a second model so that we can contextualize our fit metrics.

Assign the result of building a simple linear model of `impressions`, the total number of times a user views a version of an advertisement, on `total_convert` to the variable `model_2`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's use a combination of R's variable selection syntax, the `$` character, and `summary()` to investigate the percent of variability explained by both `model` and `model2`.

Call extract the r-squared measure from `model`, and save the result to a variable called `r_sq`.

Call extract the r-squared measure from `model_2`, and save the result to a variable called `r_sq_2`.

Print out both r-square variables. Which model better explains a user's likelihood of purchasing a product they have been shown an advertisement for?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Uncomment the final f-string, then run the file to see how we would provide a narrative around the R^2 statistic and determine which model better explains user purchase behavior.

```
---  
title: "Quantifying Model Fit"  
output: html_notebook  
---  
  
```{r message=FALSE, warning=FALSE}  
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
```

```

sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
```

```{r compute_rse}
model <- lm(total_convert ~ clicks, data = train)
compute avg_rse
avg_rse <- sigma(model)/mean(train$total_convert)

#uncomment f-string below
sprintf("The percentage error of the model is %s. Any prediction drawn from this model
could be %s percent off from the actual observed value.", avg_rse, avg_rse)
```

```{r build_model2}
model_2 <- lm(total_convert ~ impressions, data = train)
```

```{r compute_rsquared}
compute r-squared
r_sq <- summary(model)$r.squared
r_sq_2 <- summary(model_2)$r.squared

print out r-squared values
r_sq
r_sq_2

uncomment f-string below
sprintf("Based on a pair of simple linear regression models, we have determined that %s
percent of the variation in user purchase behavior can be explained by the number of times
a user viewed on a relevant ad campaign; whereas only %s percent of this variation can be
explained by the number of times a user clicked on a relevant ad.", r_sq_2*100, r_sq*100)
```

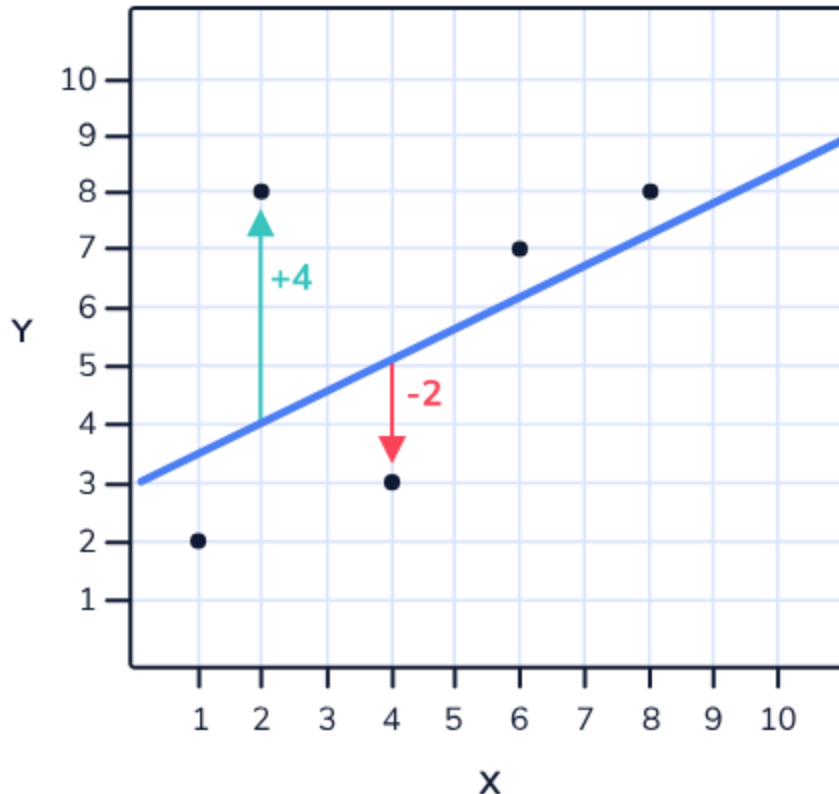
```

Linear Regression in R

Checking Model Residuals

Great! We can build a model! But... how do we know if it's any good? Also, if another data scientist builds a different model using a different independent variable, how can we tell which model is "best"? Even within Statistics, "best" can be a subjective qualifier. However, scientists who use regression models generally agree that the best model is the one that minimizes the distance between a data point and the estimation line drawn by a model. **The vertical distance between a datapoint and the line estimated by a regression model is called a residual**; residuals and their aggregations are the fundamental units of measures of regression model fit and accuracy.

Because residuals are based on cartesian distances, it often helps to visualize their values. For instance, consider the plot of a simple linear regression alongside its training data below. Note one point is 4 units above the regression estimate line; in this example, the residual for that point is 4. Meanwhile, another point is 2 units below the regression estimate line; the residual for that point is -2. A data point is best fit by the model which results in the smallest residual for that point.



When scientists make quantitative arguments for a best fit model, they rely on an aggregation, often the sum or average, of residual values across an entire dataset. While it is easy to be overwhelmed by the variety of measures used to argue that one model is better than the other, it is crucial to realize that all measures are grounded in the simple difference between regression estimate and observed data point. Let's produce a visualization of our own model of `clicks` on `total_convert` to better understand our model residuals.

Instructions

Checkpoint 1 Enabled

1.

First, let's pull out the points that make up the estimate line, and respective residual values, from our model. We can save them to columns back in our `train` dataset called `estimate` and `residuals` in our main `train` training dataset.

Call `predict()` on `model` and save the result to `train$estimate`.

Call `residuals()` on `model` and save the result to `train$residuals`

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Plot the values of `clicks` and `total_convert` using a combination of `ggplot()` and `geom_point()`. Save the result to a variable called `plot`. Don't forget to pass in `clicks` as `X` variable, and `total_convert` as a `Y` variable. Make sure you're using `train` as the dataset.

Call `plot` to view your visualization.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Plot the observed data points in our `train` dataset by adding another call to `geom_point()`. Make sure to explicitly pass in `estimate` as a `y` value, and set the `color` parameter equal to `"blue"`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Let's explicitly plot the vertical distance between `estimate` values and their respective observed data point. Add a call to `geom_segment()`, passing in `xend = clicks` and `yend = estimate` as arguments to `aes()`. Don't forget to set `color = "gray"`!

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Finally, we should provide another way to observe the size of residuals. Update our first call to `geom_point()` by passing in `size = abs(residuals)` as an argument to `aes()`. As `total_convert` increases, how do the value of model residuals change?

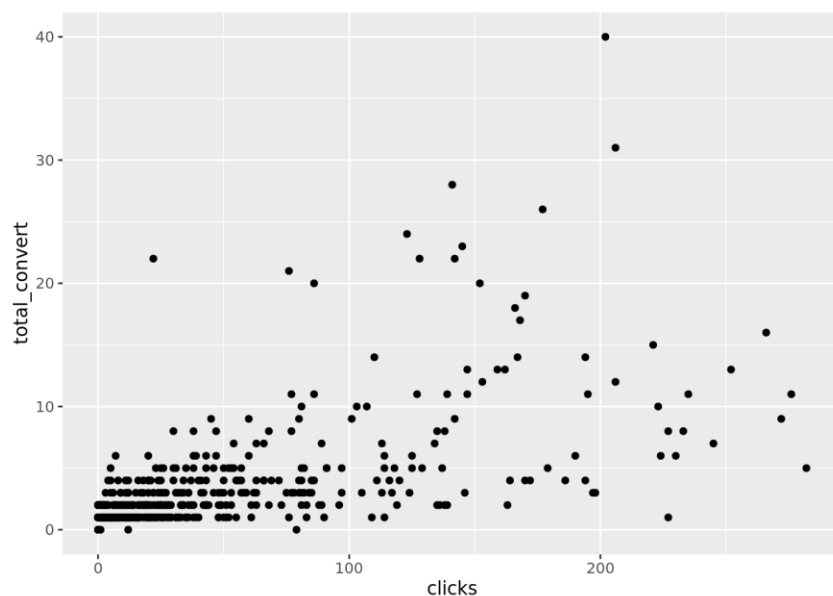
```
---
title: "Checking Model Residuals"
output: html_notebook
---

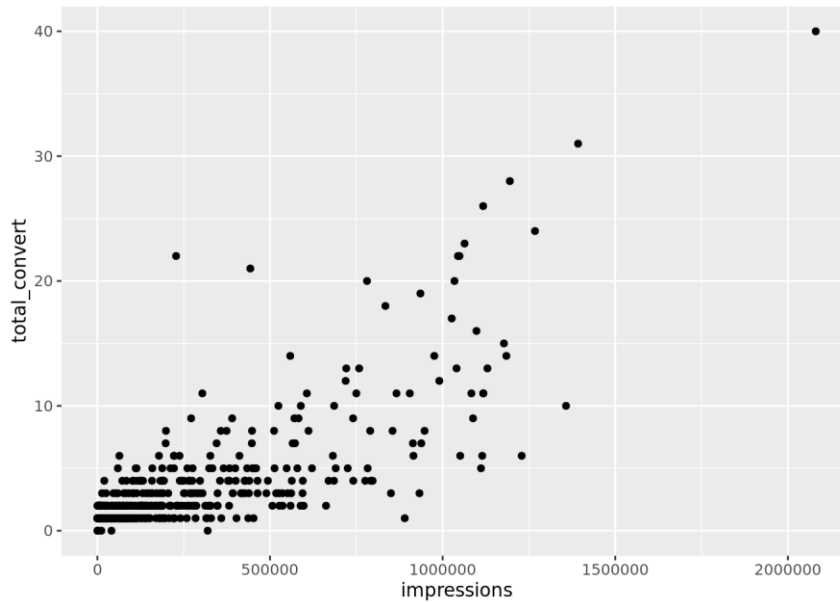
```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
```

```{r message=FALSE}
model <- lm(total_convert ~ clicks, data = train)

#save predicted and residual values to df
train$estimate <- predict(model)
train$residuals <- residuals(model)

#create visualization
plot <- train %>%
 ggplot(aes(clicks, total_convert)) +
 geom_point(aes(size = abs(residuals))) + #plot actual values
 geom_point(aes(y = estimate), color = "blue") +
 geom_segment(aes(xend = clicks, yend = estimate), color = "gray")
plot
```
```





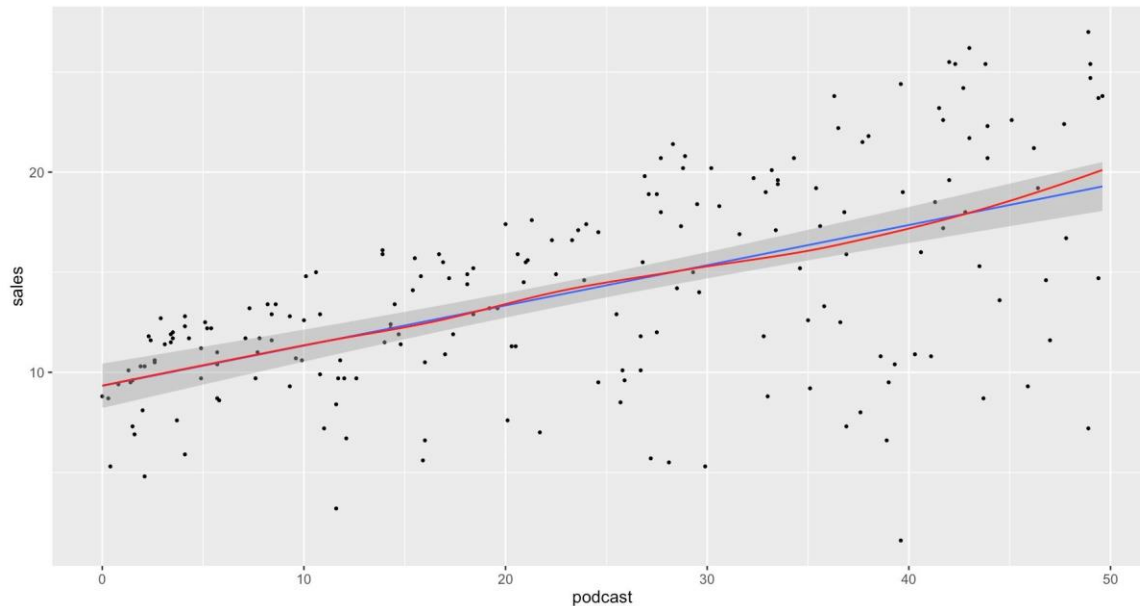
Linear Regression in R

Visualizing Model Fit

In addition to the quantitative measures that characterize our model accuracy, it is always a best practice to produce visual summaries to assess our model. First, we should always visualize our model within our data. For simple linear regression this is quite simple; we can use `geom_point()` to plot our observed values, and `geom_smooth(method = "lm")` to plot our model. In addition, we can include a second call to `geom_smooth()`, with parameters `(se = FALSE, color = "red")`. This combination of function calls allows us to compare the linearity of our model, visualized below as the blue line with the 95% confidence interval covering the shaded region, in comparison to a non-linear LOESS smoother visualized in red.

```
ggplot(train, aes(podcast, sales)) +  
  geom_point() +  
  geom_smooth(method = "lm") +  
  geom_smooth(se = FALSE, color = "red")
```

Copy to Clipboard



LOESS smoothers plot a line based on the weighted value of data points; the line produced by a LOESS smoother is similar to taking a moving average of data points as our x-axis variable increases. The smoother should not be used to predict new values, as it [relies heavily on our training data](#), but it is a helpful tool for visualizing where our linear model diverges from our training data.

Considering the LOESS smoother remains within the confidence interval of our model, we can assume the linear trend fits the essence of this relationship. However, we should note that as the podcast advertising budget gets closer to 0 there is a stronger reduction in sales beyond what the linear trend follows; this means that our model might be less accurate in instances where the podcast budget is very low.

Instructions

Checkpoint 1 Failed, try again

1.

We've plotted clicks against total converts. Let's add a LOESS smoother. Add two calls of `geom_smooth()` to `plot`. The first should use the parameter `method = "lm"`. The second should use the parameters `se = FALSE` and `color = "red"`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

How closely does the relationship between clicks and conversion follow a linear trend? Set the variable `linear_relationship` equal to either "a", "b", "c", or "d" depending on the statement that best characterizes the relations:

- A. The relationship is less linear when `clicks` approaches large values.
- B. There is a clear divergence from a linear relationship when `clicks` approaches zero or when `clicks` approaches infinity.
- C. The relationship between `clicks` and `total_conversion` is perfectly linear.
- D. There is no linear relationship between `clicks` and `total_conversion`

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's extend our linearity analysis to our `model2`, which describes the relationship between `impressions` and `total_conversion`. Add the two calls to `geom_smooth()` to `plot_2` to make a comparison to a LOESS model.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

How closely does the relationship between impressions and conversion follow a linear trend? Set the variable `linear_relationship_2` equal to "a", "b", "c", or "d" depending on the statement that best characterizes the relations:

- A. The relationship between `impressions` and `total_conversion` is perfectly linear.

- B. There is a clear divergence from a linear relationship when `impressions` approaches zero and when `impressions` is around 500,000.
- C. The relationship is less linear when `impressions` approaches very large values and when `impressions` is around 500,000.
- D. There is no linear relationship between `impressions` and `total_conversion`

```
---
title: "Visualizing Model Fit"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
```

```{r build_viz}
build plot of clicks on total_convert below
plot <- ggplot(train, aes(clicks, total_convert)) +
 geom_point() + geom_smooth(method = "lm") + geom_smooth(se = FALSE, color = "red")

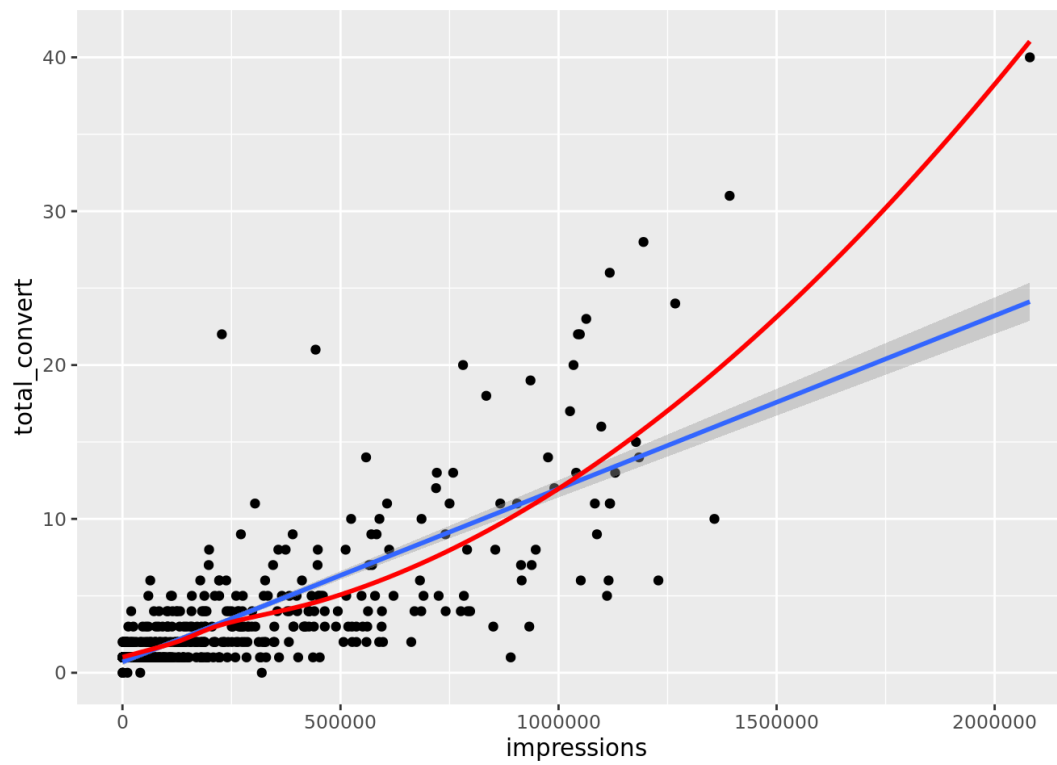
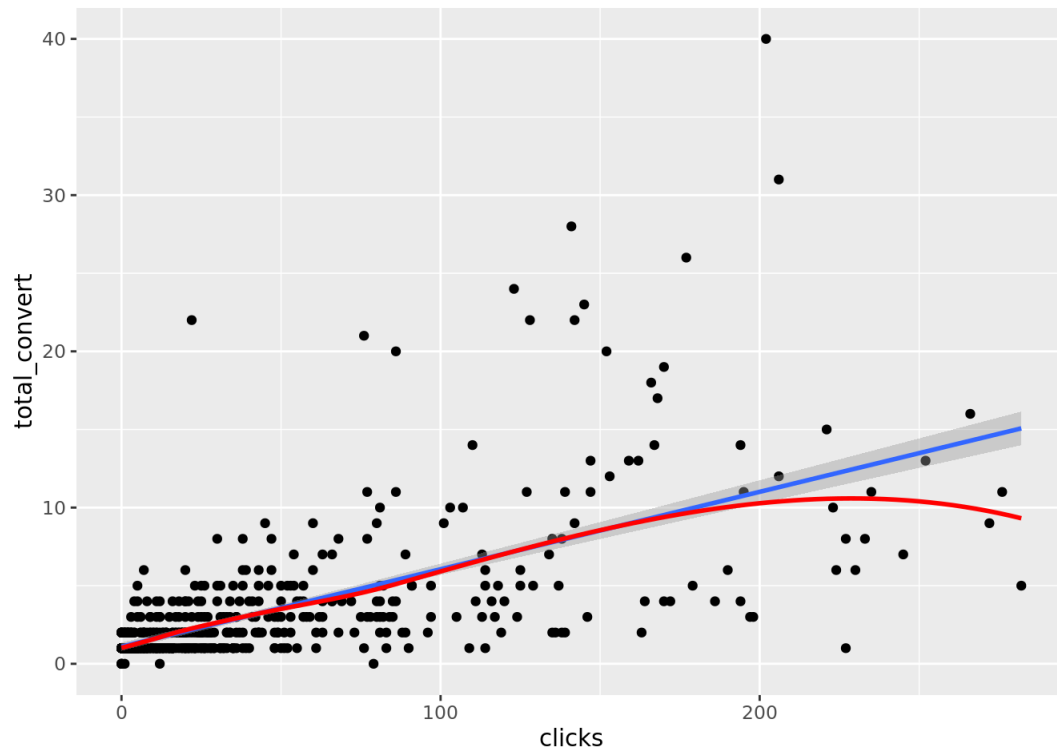
plot

linear_relationship <- "a"
```

```{r build_model2}
build plot of impressions on total_convert below
plot_2 <- ggplot(train, aes(impressions, total_convert)) +
 geom_point() + geom_smooth(method = "lm") + geom_smooth(se = FALSE, color = "red")

plot_2

linear_relationship_2 <- "c"
```
```



Linear Regression in R

Reading Model Results

Ready for the real fun? We've done our due diligence and confirmed that our data fulfills the assumptions of simple linear regression models; we've split our data into test and training subsets, and properly built a model using `y ~ x + b` notation; we've even taken the time to assess the fit of our model using both quantitative and qualitative approaches. Now we can finally analyze the results of our model and discover the relationship between user advertisement clicks and the purchase rate of related products!

We can view the results of a linear regression model in R by calling `summary()` on the `model` variable to which we saved the results of our call to `lm()`. The `summary()` function will print out a *lot* of information about our model—but don't be overwhelmed! There are four primary sections of quantitative results that are crucial to interpreting regression models:

Call

This section simply displays the call to `lm()` which created these model results. It's a helpful reminder of which version of an outcome-predictor pair is currently under analysis.

Residuals

As covered in our earlier exercises, a **residual** is the difference between the value of an outcome variable predicted by the model and the actual observed value of the variable. The `summary()` output displays a set of numbers that summarize the distribution of residuals in our model, including minimum/maximum residual values, the values first/third quantiles, and the median residual value for the model. We've already analyzed our residual values by creating a plot in an earlier exercise, but these summary values are a helpful reminder of the overall spread of our model errors.

Coefficients

Estimate

Coefficients are most important results in the interpretation of regression models. The number you see in the `Estimate` column, (a value of 0.048939 for `clicks`) is called a **regression coefficient**. Looking back to formal definition of a linear regression model:

$$Y = \beta_0 + \beta_1 * X + \text{error}$$

The regression coefficient is represented by the `beta_1` variable. This linear regression equation tells us that the regression coefficient represents the expected change in the dependent variable (in our case `total_convert`) for a one-unit increase in the independent variable (`clicks`). In other words, for every additional click on an advertisement, the expected sales of a related product are estimated to increase by 0.049 dollars. In addition to the size of the coefficient, it is also important to note the sign of the coefficient. If our `clicks` coefficient was negative, our model would be estimating that the sales of a product actually *decreases* every time an advertisement is clicked.

Std. Error

The column adjacent to `Estimate` is called `Std. Error`; the **standard error** of each coefficient is the estimate of the standard deviation of the coefficient. It is crucial to note that the standard error is not a quantity of interest by itself, but depends on the value of our regression coefficient

T-value and $Pr(>|t|)$

The `t value` and `Pr(>|t|)` inherently answer the same question—given the value of our variable's regression coefficient and its' standard error, does the variable explain a significant part of the change in our outcome variable? However, the `Pr(>|t|)` column purposely provides a more concise response to this question, using the asterisk notation that corresponds with the `Signif. codes` legend at the bottom of the Coefficients results section. In R

model output, one asterisk means “ $p < .05$ ”. Two asterisks mean “ $p < .01$ ”; and three asterisks mean “ $p < .001$ ”. These values are referred to as **p-values** in scientific literature. How can we use p-values to answer our question around model significance?

Asterisks in a regression table indicate the level of the **statistical significance** of a regression coefficient. Our understanding of statistical significance is based off of the idea of a random sample. When interpreting these asterisk values, we ask ourselves: if there truly is no relationship between clicks on an advertisement and product sales, then what are the chances that, across many user clicks on an ad, we see behavior that suggests that there is no relationship?

For our `clicks` variable, with *** in the `Pr(>|t|)` column, the answer is very unlikely. The value of ***, or $p < .001$, means that random sample resulting in the regression coefficient and standard error that we observed for `clicks`—given that there was truly no difference relationship between `clicks` and product purchase—would occur in less than one time in a random draw of 100, on average. Given that we would so rarely observe the situation that suggests that there is no relationship between our `click` and `total_convert`, we can say that there is a statistically significant relationship between the two variables. Generally speaking, **scientists accept that a variable coefficient with p-value less than 0.05 is statistically significant**.

Measures of Model Fit

At the bottom of the output of `summary()` are a series of labeled metrics, like Residual Standard Error (RSE) and Multiple R-squared, which quantify the fit of our model. Our previous exercises have covered how to interpret and plot many of these measures, but it’s a helpful reminder for them to be summarized along with other model output.

Wow! There is so much information conveyed within a simple call to our model results. Continue on to the second half of this lesson to practice the interpretation of model results, and learn more about how to select a best fit model.

Summary of a Model

```
Call:
lm(formula = total_convert ~ clicks, data = train)
```

→ Model definition

```
Residuals:

    Min       1Q   Median       3Q      Max
-10.0916  -0.6953  -0.2549  -0.0991   20.1449
```

→ Residual summary statistics

```
Coefficients:
      Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.205939   0.127139    9.485  <2e-16 ****
clicks       0.048939   0.002081   23.517  <2e-16 ****
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

→ Model coefficient table and p-value key

```
Residual standard error: 2.796 on 660 degrees of freedom
Multiple R-squared:  0.4559, Adjusted R-squared:  0.4551
F-statistic: 553 on 1 and 660 DF,  p-value: < 2.2e-16
```

→ Model accuracy metrics

Linear Regression in R

Assessing Simple Linear Regression

Let's practice our model interpretation skills! We know that for continuous independent

[variables](#)

Preview: Docs Loading link description

, like `podcasts`, the regression coefficient represents the difference in the predicted value of `sales` for each one-dollar increase in `podcasts`. Given the output of calling `summary(model)` below, we can correctly say that for every one dollar increase in podcast advertisement spending, total sales of the related project increases by 1.742 dollars.

```
summary(model)

#output
Call:
lm(formula = sales ~ radio, data = train)

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   9.57927    0.91176  10.506  < 2e-16 ***
podcast       1.74240    0.03255   5.353 3.71e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Copy to Clipboard

We could also extract the value of the `podcast` coefficient, the second coefficient returned by our model, using list indexing as follows:

```
podcast_coefficient <- model$coefficients[2]
```

Copy to Clipboard

It is important to note that the interpretation of the intercept coefficient is slightly different from that of variable coefficients. The intercept coefficient represents the value we would predict for our outcome variable, `sales`, given that podcast spending is equal to zero. **It is crucial to remember that the intercept coefficient is only interpretable if we can reasonably expect a zero value for all independent variables in a model.** Assuming, just as our simple linear model does, that spending on podcasts is the only variable that explains changes in `sales`, it does not make sense for any sales to occur without podcast spending. Therefore, for this model, our intercept coefficient is not interpretable.

However, in many cases, intercept coefficients are interpretable! As you've seen throughout this lesson, the analysis of any model results requires a thorough understanding of our data, the system that produces this data, and a critical approach to interpretation of coefficient values.

Instructions

Checkpoint 1 Enabled

1.

The code used to build two simple linear models, `model`, a regression of `clicks` on `total_convert`, and `model2`, a regression of `impressions` on `total_convert`, is already included in your notebook. Use `summary()` to print out the results of `model` and `model2`.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Save the results of calling `model$coefficients[2]` to a variable called `clicks_coefficient`; uncomment the following f-string to print out an example of how this estimate would be described in a written analysis.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

How might we interpret the coefficient estimate for our model intercept? Set the variable `intercept_coefficient` equal to the lowercase letter (ex. "a") associated with the statement that correctly interprets the estimate value:

A. The intercept coefficient is not significantly different from zero, therefore the model suggests that when `clicks` equal zero, the number of `total_converts` is likely to be around zero. This is reasonable because

if a user has not clicked on an ad, we do not expect them to purchase the related product.

B. The intercept coefficient is negative and significant; this means that when `clicks` equal zero, we expect the number of `total_converts` to be less than zero. This makes sense because the user did not click on an ad, so we expect zero or less products to be purchased.

C. The intercept coefficient is greater than zero, so when `clicks` equals zero we expect `total_converts` to be somewhere between 0.90 and 1.25. Even though a user does not click on an ad, they have been exposed to the product brand and might purchase the product by later searching for it through a search engine.

D. The intercept coefficient is close to zero; this means that when `clicks` equal zero, the number of `total_converts` is likely to be around zero. This is reasonable because if a user has not clicked on an ad, we do not expect them to purchase the related product.

```
---
title: "Assessing Simple Linear Regression"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
```

```{r message=FALSE}
model <- lm(total_convert ~ clicks, data = train)
model2 <- lm(total_convert ~ impressions, data = train)
summary(model)
summary(model2)

clicks_coefficient <- model$coefficients[2]
uncomment f-string below
sprintf("Based on a simple regression of `total_convert` by `clicks`, we estimate that for every additional click, the number of product purchases increases by %s.", clicks_coefficient)

intercept_coefficient <- "c"
```
```

Linear Regression in R

Making Predictions

Data Scientists are often interested in building models to make predictions on new data. While the `add_predictions()` function from the `modelr` package makes it easy to predict new values from a technical standpoint, it is far more difficult to develop and assess accurate predicted values.

The most common metric used to compute the accuracy of predicted values is mean squared error on test data. Similar to residual squared error (RSE) and R-squared, MSE measures the average squared difference between predicted and observed values. When we are working with just one model, it is helpful to compare the difference between MSE on our training dataset, and MSE on test data. We can calculate training MSE for model using a combination of `add_predictions()` and `summarise()`. `add_predictions()` creates and adds predicted values from a model to a column called `pred`. `summarise()` then allows us to calculate the mean of the squared difference between our observed values (`sales`) and predicted values (`pred`).

```
train %>%
  add_predictions(model) %>%
  summarise(MSE = mean((sales - pred)^2))

#output

      MSE
31.60713
```

Copy to Clipboard
We can use the same combination of

[functions](#)

Preview: Docs Loading link description

to calculate MSE for our test dataset, which results in a MSE of around 32.5. Testing MSE will almost always be higher than training MSE, as the model has been built off of training data; however, it is important to confirm that there is not a substantial difference between model training and test MSE. The value of using MSE to quantify prediction accuracy is more clear when comparing multiple models, as it allows us to determine which versions of a model best predicts an outcome variable. For instance, we could compute the MSE for a model of tv spending on sales.

```
model2 <- lm(sales ~ tv, data = train)

train %>%
  add_predictions(model2) %>%
  summarise(MSE = mean((sales - pred)^2))

#output

      MSE
27.28415
```

Copy to Clipboard

Comparing the train MSE for our tv-based model, at 27.28, to our train MSE for a podcast-based model, at 31.60, it is clear that the predictions from the tv-based model are more accurate, as the model's MSE is lower. If a data scientist

was trying to predict the expected volume of sales for a future business quarter, it would be a better idea for them to base their estimations off of a tv-based model.

Instructions

Checkpoint 1 Enabled

1.

The code used to build two simple linear models, `model`, a regression of clicks on `total_convert`, and `model2`, a regression of impressions on `total_convert`, is already included in your notebook.

Use `add_predictions()` and `summarize()` to calculate the test mean squared error of `model` (MSE), and save the result to a variable called `mse_clicks`.

Make sure to use the `test` dataset.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Use `add_predictions()` and `summarize()` to calculate the MSE of `model2`, and save the result to a variable called `mse_impressions`.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Print out both `mse_clicks` and `mse_impressions`. Which one is smaller? What does this tell us about the accuracy of our models on the test data?

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Let's plot the predicted test values of the model with the smallest MSE against the observed test data.

First, use a combination of `add_predictions()`, `ggplot()` and `geom_point()` to plot the observed values from the test data.

When creating the `ggplot()` add an `aes()` where the `x` and `y` values correspond to the variables used when making the correct model.

Save the visualization to a variable called `plot`, then print out the variable.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Now we can add in our predicted values! Add another call to `geom_point()`, explicitly passing in `pred` as the `y` value in `aes()`, and set the `color` parameter equal to blue.

```
---
title: "Making Predictions"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
library(modelr)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
test <- conversion_clean[!sample,]
```

```{r message=FALSE}
model <- lm(total_convert ~ clicks, data = train)
model2 <- lm(total_convert ~ impressions, data = train)
```
```

```

```{r message = FALSE}
mse_clicks <- test %>%
 add_predictions(model) %>%
 summarise(MSE = mean((total_convert - pred)^2))

mse_impressions <- test %>%
 add_predictions(model2) %>%
 summarise(MSE = mean((total_convert - pred)^2))

mse_clicks
mse_impressions

plot <- test %>%
 add_predictions(model2) %>%
 ggplot(aes(impressions, total_convert)) +
 geom_point() +
 geom_point(aes(y = pred), color = "blue")

plot
```

```

Linear Regression in R

Multiple Linear Regression

We've been able to really dig into the results of simple linear regression models and show how the results convey a substantial amount of information about the relationship between two. However, by this point you might be wondering—what if I think variables *other* than `podcast` have contribute to the total sales of a product? You might remember that the primary assumption behind simple linear models is that the expected value of the outcome variable is a straight-line function of *exclusively* the predictor variable. This means that our simple linear models assume that *all* variation in the outcome variable is explained by the predictor variable. In the case of our `sales` dataset, we know this is almost certainly not true; oftentimes more money is spent on TV or newspaper ads than on podcasts, so this spending might have an even larger effect than podcast spend.

Thankfully, there are methods to include the effects of `TV` and `newspaper` in linear regression models. We can expand our model definition from a *simple* model of one predictor variable to a *multiple* model of, you guessed it, *multiple* predictor variables. The formal definition of multiple linear regression models is a direct extension of the formula for simple linear regression:

$$Y = \beta_0 + \beta_1 X + \beta_2 X + \text{error}$$

As in a simple linear model, **Y** represents the dollar value of products sold, **X** represents the amount spent on respective product podcast ads, and **Beta_0** is the model intercept. Now, **Beta_1**, **Beta_2**, and **Beta_3** represent each the coefficients of predictor variables. To build a similar model in R, using the standard `lm()` package, we still use the formula notation of **Y ~ X**:

```
model <- lm(sales ~ podcast + tv, data = train)
```

Copy to Clipboard

While building a multiple regression model is a straightforward extension of the code used to a build a simple model and the output of the model results below looks quite similar, a bit more effort goes into the interpretation of the results of this model. Remember that in a simple linear regression model, the regression coefficient represents the expected change in the dependent variable for a one-unit increase in the independent variable. In other words, the coefficient for `podcast` represents the expected increase in `sales` given a one dollar increase podcast advertisement spend. Because multiple linear regression includes more than one predictor variable, the coefficient estimates must be interpreted differently. In multiple linear regression, the regression coefficient represents the expected change in the dependent variable for a one-unit increase in the independent variable, *holding all other*

variables in the model constant. Expand the width of your narrative panel to view the output of the multiple linear regression model below:

```
summary(model)

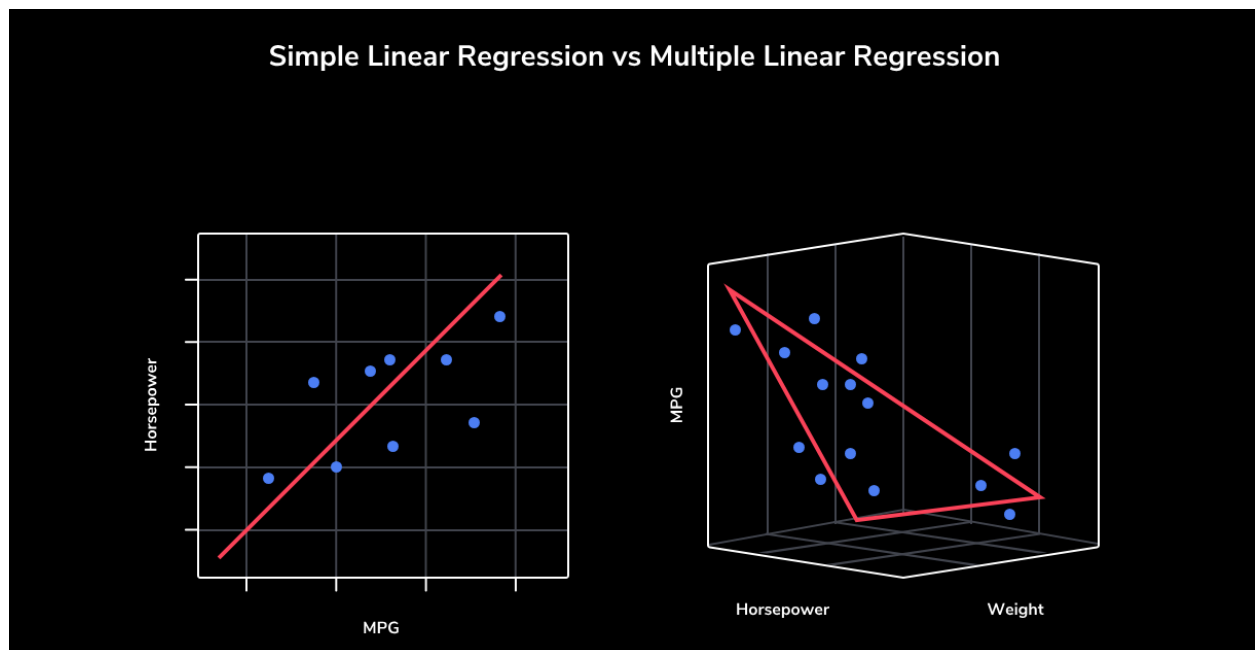
#output
Call:
lm(formula = sales ~ TV + podcast, data = train)

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.583386   1.024616   4.473 1.65e-05 ***
TV            3.006340   1.004924   7.380 1.62e-11 ***
podcast       1.049249   1.027665   5.395 3.10e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Copy to Clipboard

For example, a call to `summary(model)` shows that the coefficient for `podcasts` is equal to `1.04944`. This means that, when one more dollar is spent on podcast advertising, about `1.049` more dollars of the related product is sold, *given that there is no increase in the amount of money spent on tv advertisements*. In this way, multiple linear regression models allow us to isolate the unique effect of one predictor variable on the outcome variable.

As this example shows, the selection of variables in a regression model can have wide-ranging impacts on the results and interpretation of our models! Let's dive into one more exercise to practice building and interpreting multiple linear regression models.



Linear Regression in R

Assessing Multiple Linear Regression

Time to pull it all together! The interpretation of coefficients in multiple linear regression is slightly different than that of coefficients in simple linear regression. Coefficient of independent continuous

[variables](#)

Preview: Docs Loading link description

, like `podcasts`, represents the difference in the predicted value of sales for each one-dollar increase in podcasts, given that *all* other variables in the model, including `tv`, are held constant. Given the output of calling `summary(model)` below, we can correctly say that for every one dollar increase in podcast advertisement spending, while holding the amount spent on `tv` and `newspaper` constant, the total sales of the related product increases by 1.049 dollars.

```
summary(model)

#output
Call:
lm(formula = sales ~ TV + podcast + newspaper, data = train)

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  4.583386    1.024616   4.473 1.65e-05 ***
TV            3.006340    1.004924   7.380 1.62e-11 ***
podcast       1.049249    1.027665   5.395 3.10e-07 ***
newspaper     1.006340    1.002924   6.380 1.12e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Copy to Clipboard

In addition, the interpretation of boolean categorical variables differs slightly from that of continuous variables. The coefficient value associated with a boolean categorical variable represents the effect of changing from one category to another. For instance, the coefficient value of 1.006 for `newspaper` tells us that running print advertisements results in a 1.006 dollar increase in `sales`, holding the values of `TV` and `podcast` constant.

As we've suggested throughout this lesson, data scientists often build many variations of a model with different combinations of independent variables before ultimately committing to the model that best fits test data. Let's practice building, interpreting, and selecting the best fit multi-linear model for our `convert_clean` dataset!

Instructions

Checkpoint 1 Enabled

1.

Build a multiple linear regression model which regresses `impressions`, `clicks`, and `gender` on `total_convert`, using our `train` dataset. Save the result to a variable called `model`; then call `summary(model)` to view the model results.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

How might we interpret the coefficient estimate for `gender`? Set the variable `gender_coefficient` equal to the statement that most correctly interprets the estimate value — either "a", "b", or "c":

A. The coefficient of the `gender` variable is not statistically significant, so we cannot come to any substantive conclusion from its value.

B. The coefficient of the `gender` variable is negative, which means that as `total_convert`, `clicks`, and `impressions` increases, men are less likely to purchase an advertised product.

C. The coefficient of the `gender` variable is negative. This means that a man is less likely than a woman — with the same value of `clicks` and `impressions` — to purchase an advertised product.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Let's build a second, simpler model so that we can confirm adding `gender` to our model increases its accuracy. Build a multiple linear regression model which regresses `impressions`, and `clicks` on `total_convert`. Save the result to a variable called `model2`.

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Compute the R-squared value for `model` and `model2`, and save the results to `rsq_model` and `rsq_model2` respectively. Call both variables to view their values.

Checkpoint 5 Step instruction is unavailable until previous steps are completed

5.

Which model best fits our data? Set the variable `best_fit` equal to the larger r-squared value.

Checkpoint 6 Step instruction is unavailable until previous steps are completed

6.

Set the variable `gender_diff` equal to the difference between `rsq_model` and `rsq_model2`. Uncomment the f-string at the bottom of the file to see how we would provide a narrative around the effect of gender on interaction with online advertisements.

```
---
title: "Assessing Multiple Linear Regression"
output: html_notebook
---

```{r message=FALSE, warning=FALSE}
set sampling seed
set.seed(123)
load libraries and data
library(dplyr)
library(ggplot2)
conversion_clean <- read.csv('conversion_clean.csv', header= T)
sample training data
sample <- sample(c(TRUE, FALSE), nrow(conversion_clean), replace = T, prob = c(0.6,0.4))
train <- conversion_clean[sample,]
```

```{r message=FALSE}
build model below
model <- lm(total_convert ~ impressions + clicks + gender, data = train)
summary(model)
define gender_coefficient below
gender_coefficient <- "c"
```

```{r message=FALSE}
build second model below
model2 <- lm(total_convert ~ impressions + clicks, data = train)
compute r-squared below
rsq_model <- summary(model)$r.squared
rsq_model2 <- summary(model2)$r.squared

rsq_model
rsq_model2

define best_fit below
best_fit <- rsq_model

define gender_diff below
gender_diff <- rsq_model - rsq_model2

#sprintf("Based on the results of a series of multiple linear regressions on total_convert,
we estimate that user gender accounts for approximately %s percent of the variation in
product purchase rate.", gender_diff*100)
```
```

Linear Regression in R

Review

Whew, that's a wrap! You've covered a *lot* of material related to linear regression and its implementation in R. Here are the main concepts we've covered:

Statistical model building entails four main steps: 1) **confirming data assumptions**, 2) **building a model on training data**, 3) **assessing model fit**, and 4) **analyzing model results**.

We can use a combination of qualitative methods, such as box-plots, and quantitative methods, like the **correlation coefficient**, to assess that data meets our assumptions

We use the `lm()` method and $Y \sim X$ notation to build a linear regression model. The Y variable is referred to as the **outcome variable** of the model, and any X variable is referred to as the **predictor variable**.

We can use a similar set of qualitative and quantitative methods to evaluate the fit of our model, including a comparison of the plotted model to a LOESS smoother and statistics like **mean squared error (MSE)** and **R-squared**.

MSE and R-squared statistics are summaries of the overall value of the model residuals. A **residual** is the difference between the value of a data point predicted by a model and its actual observed value.

The results of a linear regression model include **regression coefficients**. These coefficients represent the effect their respective predictor variable has on the model's outcome variable.

In a simple linear regression, the regression coefficient represents the effect of a one-unit increase in the predictor variable.

The **intercept coefficient** represents the value of the outcome variable given that the predictor variable is equal to zero; this coefficient isn't always meaningful and depends on the situation being modeled.

The **p-value** associated with a regression coefficient helps us understand whether the effect of a variable is statistically significant.

Multiple linear regression is similar to simple regression, except that it includes multiple predictor [variables](#)

Preview: Docs Loading link description

In a multiple linear regression, the regression coefficient represents the effect of a one-unit increase in the respective predictor variable, given that all other predictor variables are held constant.

In both simple and multiple linear regression, boolean categorical variables represent the total effect of switching from one category to another.

Instructions

We've included the `advertising` and `conversion` datasets in the workspace, along with some sample code that builds a multiple linear regression model.

Take some time to extend the model by adding more predictor variables; or, come up with a robust assessment of its current fit to the data and see if adjustments to the model improve the fit. **How good of a fit can you achieve?**

Introduction to Regression Analysis

This article is a brief introduction to the formal theory (otherwise known as Math) behind regression analysis. What is regression analysis?

We often hear of new, complex “machine learning” methods that allow us to generate human language, very accurately predict changes in the stock market, or recognize that an image contains a person or specific object. While some of these amazing applications are results of genuinely new methodological developments, more frequently, these applications rely on the savvy use of a classic statistical technique— regression model analysis. The first regression model was specified by Adrien-Marie Legendre, a French mathematician, in 1805, and regression-based modeling has been the cornerstone of applied statistics ever since!

Regression analysis is a group of statistical methods that estimate the relationship between a **dependent variable** (otherwise known as the outcome variables) and one or more **independent variables** (often called predictor variables). The most frequently used regression analysis is **linear regression**, which involves requires a scientists to find a line of “**best fit**” which most closely traces the data according to a certain mathematical criterion.

Unlike many other models in Machine Learning, regression analyses can be used for two separate purposes. First, in the social sciences, it is common to use regression analyses to infer a causal relationship between a set of variables; second, in data science, regression models are frequently used to predict and forecast new values. Therefore, we can use regression analysis to answer a wide variety of questions, including the examples below.

- Is the relationship between two variables linear?
- Is there even a dependency between two variables?
- How strong is the relationship?
- Which variable contributes the most to the outcome measurement?
- How accurately can we predict future values?
- Is our outcome variable caused by another variable?

Estimating Coefficients

Regression analysis answers all these questions and more by estimating **regression coefficients** for every predictor variable used in a model. Let’s assume that we are analyzing a simple model that is made up of just one predictor, one outcome variable and one coefficient. This model is formally represented as follows:

$$Y=B_0+B_1 \cdot x+\text{error}$$

In this representation, the **beta values** (B_0 and B_1) represent the regression coefficients. In high school, we are often taught that a line can be formally represented as follows:

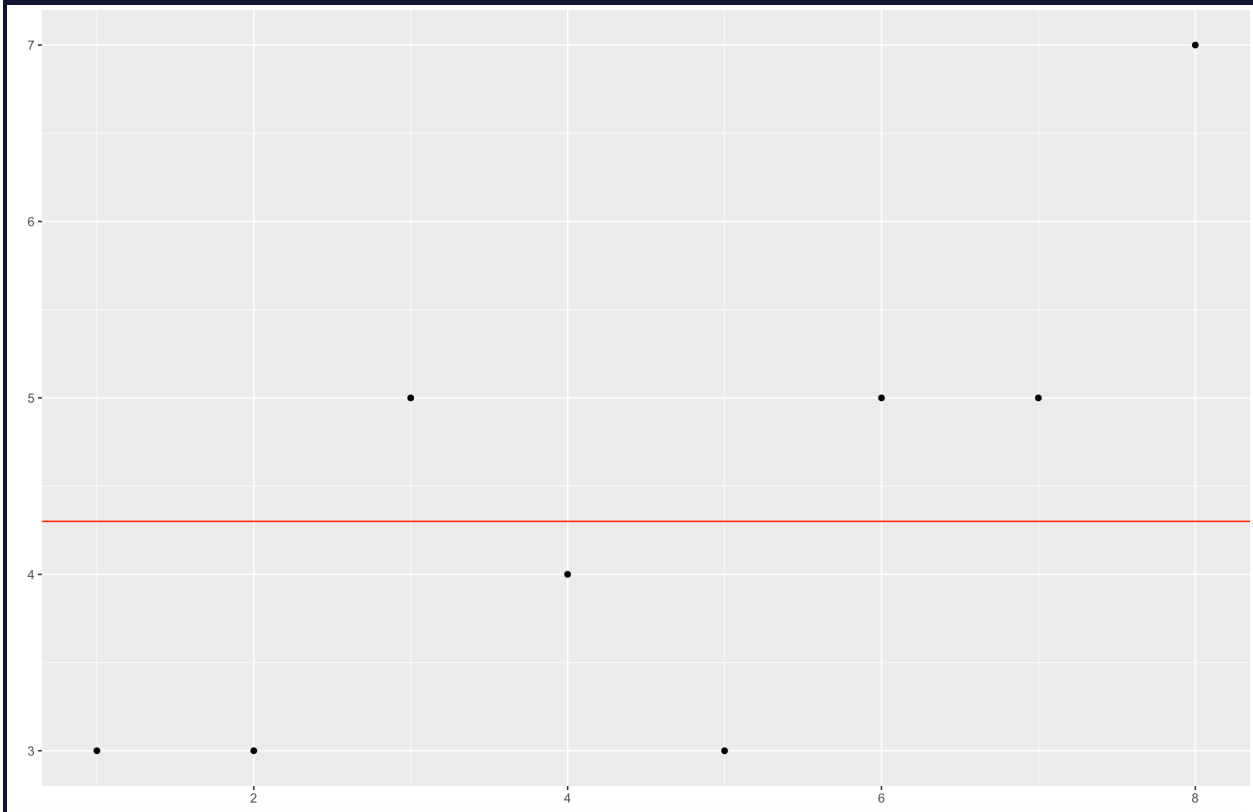
$$Y=m \cdot x+b$$

Our simple regression model follows the same formula. Except, in our case the **m**, or **slope** value, is represented by B_1 and the **b**, or intercept value, represented by B_0 . Regression analysis finds the best fit values for B_0 and B_1 and allows for us to describe the relationship between two variables using the resulting equation of the best fit line.

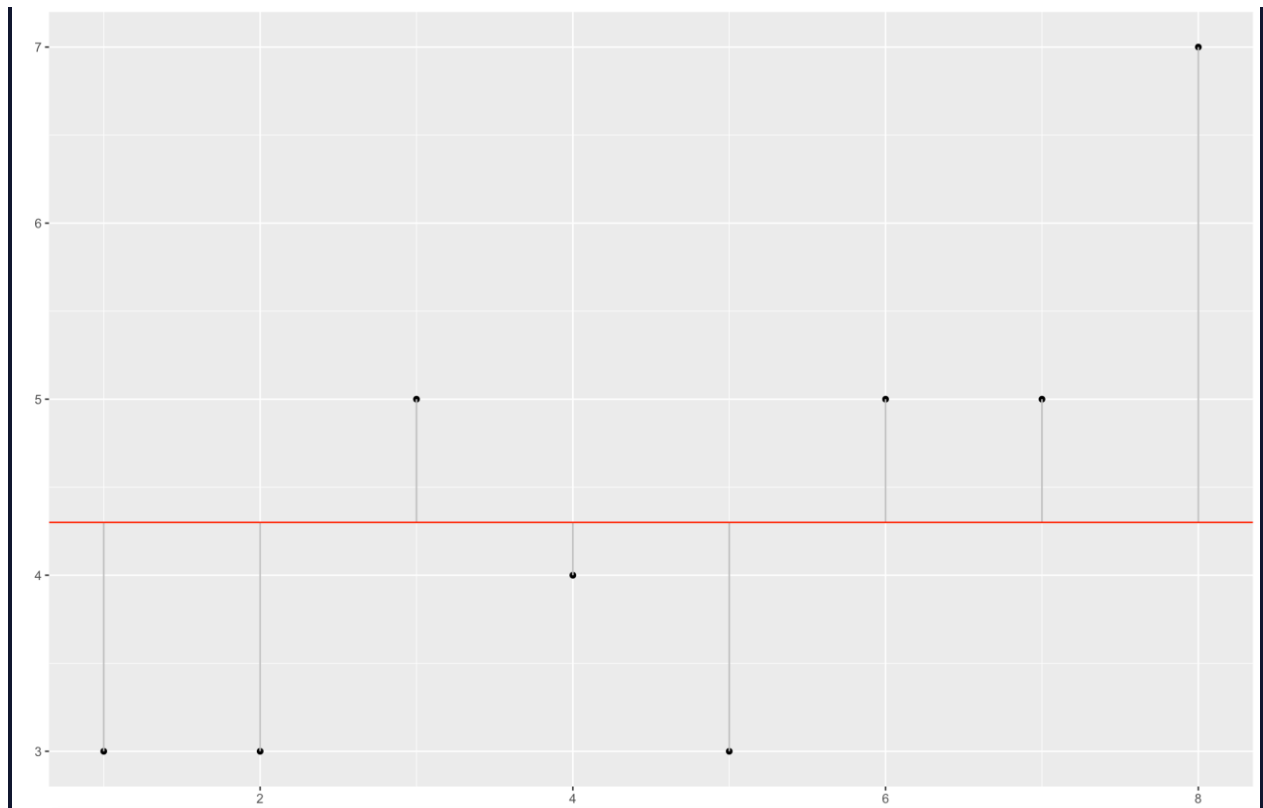
So how does a regression model find these beta values? In most cases, the best fit line is agreed to be the most that minimizes the **residual error**. A model will never perfectly fit the data, so there will always be some error, or difference between the actual observed data value and the value predicted by a model. This difference is commonly referred to as a **residual**, and is formally represented as follows:

$$\text{Error}_i=y_i-\hat{y}_i$$

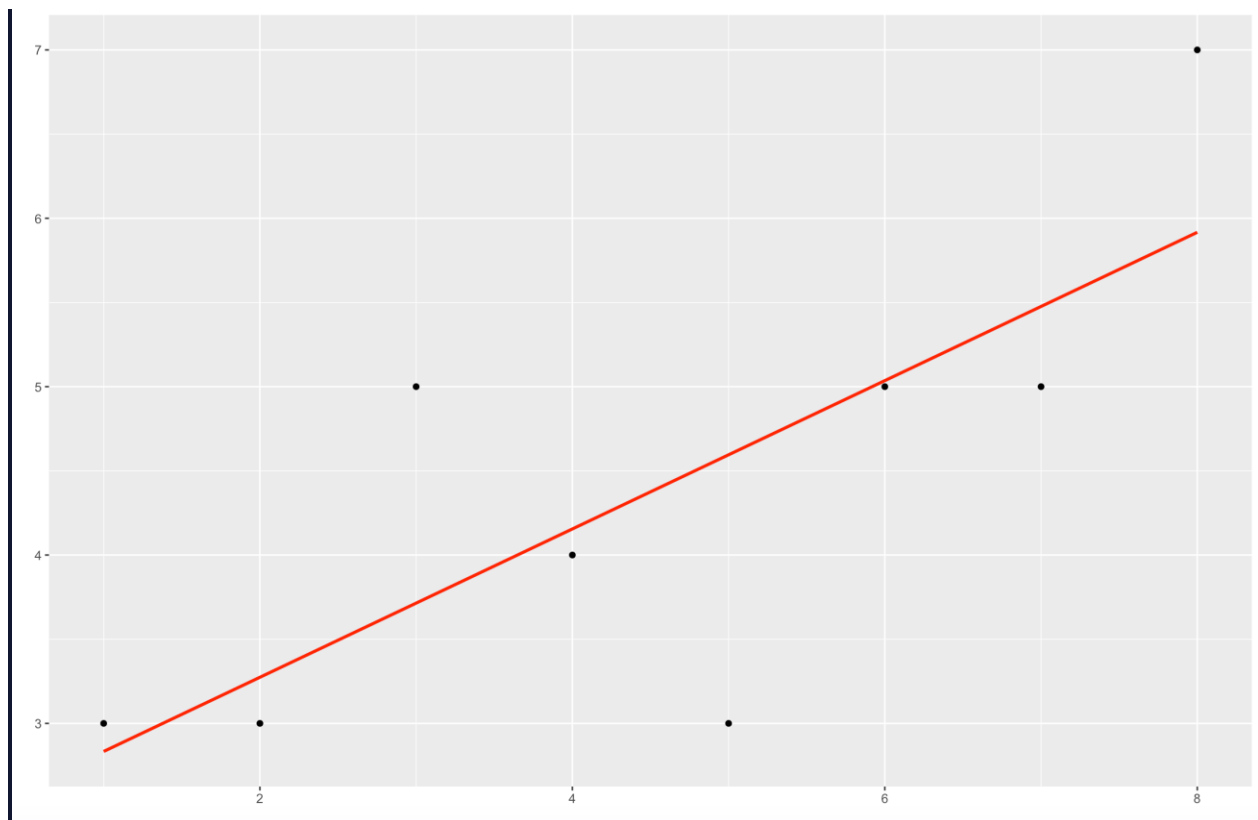
A very naive approach model for data prediction would just predict that the value of a new data point will equal the average of all observed data. This model would look like the example below, where the model's predictions are represented by a red line:



This model doesn't seem like it fits our data very well. If we drew a line from our observed data points to the prediction line, as shown below, then added up the absolute length of all these lines, our **sum of residual error** would be equal to 9. Any line that has a lower sum of residual error than our naive model would be considered a better fit line.



However, it is important to note that regression models use the **sum of squared error (SSE)**; in our example above, our naive model has a SSE of 13.92. We use squared error because a prediction can either be greater than or less than the actual value, producing a positive or negative error value. We avoided this issue in our calculation above by taking the absolute difference between observed and predicted values. But, If we did not square the error values or take their absolute difference, the sum of errors could decrease because of negative error values, not because a model fits the data best. While absolute difference solves this issue, it is far more common to fit a model based on squared errors. The square function adds a greater penalty to predicted values that are very incorrect, as squaring larger values results in a respectively larger result than squaring small values, this helps bias the model fitting calculation towards a line that produces accurate predictions for *all* values in a dataset, rather than a handful of most common observations. The example below shows a line fit by selecting the beta coefficients that have minimized SSE to 5.38. Doesn't that line look better?



Assess Model Fit

While theoretically the sum of square error gives us the best fit model given the data at hand, it doesn't guarantee that this best model is actually a "good", or relatively accurate, model. There are a number of statistics used to summarize model fit, but one of the most common measures is **R-squared**, sometimes called the coefficient of determination. R-squared quantifies the **proportion of the variability in the outcome variable that can be explained by a predictor variable**. More simply, it summarizes the difference between the sum of squared error for a line which is simply the average of all Y values, or outcome variables, and the sum of squared error for our proposed model. It can be formally represented as follows:

$$SS_{total} = \sum (y_i - \bar{y})^2$$

$$SS_{residual} = \sum (y_i - \hat{y}_i)^2$$

$$R^2 = 1 - SS_{residual} / SS_{total}$$

In this case, SS_{total} represents the squared difference between the actual data and the average value of all Y values, and $SS_{residual}$ represents the squared difference between the actual data and the values predicted by our model. We saw in our example model above that the SS_{total} for our model was 13.92, and the $SS_{residual}$ was 5.38. The R-squared calculation is simply one minus the proportion of our total and residual sum of squares; plugging in the values from our example, we get an R-squared value of 0.61:

$$R^2 = 1 - 5.38 / 13.92$$

This means that if predictor variable X can predict the outcome variable with consistent accuracy, then the proportion is high and the R-squared value will be close to 1. If the opposite is true, the R-squared value is closer to 0. We can provide a narrative around the R-squared value by saying that: "[R-squared value] percent of the variation in [outcome variable] is explained (or predicted) by [predictor variable]". In our example, 61% of the variation our Y value is explained by our X value. That's pretty good!

This is just the beginning of regression analysis! For the past two centuries researchers have been developing innovative approaches to regression-based analysis. By building up your theoretical and technical understanding of regression models, you too can join the ranks of statisticians utilizing the most widely and consistently applied methods to better understand our world.

Off Platform Project: Linear Regression in RStudio

Test your linear regression skills with this project in RStudio.

Welcome to an off-platform project focused on linear regression in RStudio! Follow these instructions to download the project and complete it using RStudio.

Getting Started with RStudio

If you've never used RStudio before, we recommend watching this video to familiarize yourself with the tool:

If you have not already done so, [download R](#), and [download RStudio](#).

Download the [Linear Regression project](#) zip file.

Unzip the file by double-clicking on it in your Downloads folder.

Navigate to the newly unzipped folder and open `Linear_Regression_Starting.Rmd` in RStudio to see the project instructions.

Follow the instructions by writing code in the code blocks found in `Linear Regression Starting.Rmd`.

If you are stuck on the project, we have included our solution in the file `Linear Regression Solution.Rmd`.

https://www.youtube.com/watch?v=T44GWKIFFRI&ab_channel=Codecademy

Capstone Project: Exploring Google Trends Data

Put all of your R knowledge to the test and create an R research project.

It's time to put your data analysis skills to the test. For this project, you will be asked to create a report using RStudio describing a research project that you conduct. On this page, we'll describe what data sources your project should use and the format of the report that you should produce. We'll also provide you with an example report that we created.

This project is meant to be fairly independent. No two projects will be identical – we want you to research something that you're personally interested in. No matter what topic you choose, this project should test your skills in the following areas:

- Finding and exploring new datasets

- Manipulating and cleaning those datasets to help answer your essential questions

- Creating visualizations from those datasets

- Building statistical models

While making your project, we expect that you will run into tasks that you've never done before in a Codecademy lesson. That's a good thing! Take this as an opportunity to apply all of your understanding of R and learn something new.

What Data To Use

For this project, we want you to investigate a topic of your choosing using Google Trends data. Using the R package `gtrendsR`, you can get data about any google search term. You can find information like the popularity of that search term over time, the location where that search term was commonly used, and other phrases related to that search term.

Make sure to download and install `gtrendsR` in RStudio. The code below shows an example of retrieving Google Trends data related to some popular television shows:

```
trends <- gtrends(keyword=c("mandalorian", "stranger things", "game of thrones", "the marvelous mrs maisel"))
```

`trends` is now a list of several data frames related to those search terms. For example, `trends$interest_over_time` is a data frame that contains information about the popularity of each search term over time. `trends$related_queries` contains information about similar search terms. You can learn more about `gtrendsR` by looking at the [documentation](#). Take a look at the `geo` argument – using that argument can add some interesting data to your data frames. You can also look at some of the code in our example report included below.

In addition to using Google Trends data, we want you to include data from one additional source that is related to your search terms. You'll see that in our example report, we found data related to the stock prices of Netflix, Amazon, Disney, and HBO. We were curious about how the shows in our search terms related to the stock prices of each company.

You don't have to use stock data. Try to find a data set that will help you answer the questions you have about your search terms.

The Format

When creating your report, make sure to include the following sections. Again, you can use our report as a template.

Introduction – Introduce the purpose of the project. What are the essential questions that you're trying to answer through statistical analysis? For our report, our essential question was "How do new releases affect the value of streaming services?"

Analysis – What did you do to try to answer your essential questions? What data sets did you use? How did you clean or manipulate those data sets? What assumptions did you make along the way? Show the visualizations that you made and describe what they show. Were you able to make any statistical models? This should be the bulk of your report – tell us what you did and whether or not you were able to answer your essential questions.

Conclusion and Further Work – Briefly summarize your findings and describe what you would do if you were to continue your project. Would you look for more datasets? Would you dig deeper into part of the data you didn't have a chance to work with?

Share Your Report

Because this project is so open-ended, we would love it if you shared your final result with fellow learners on [our forum](#). Post your report and take a look at what other projects other people have worked on – it might give you ideas for your next R project.

To post your report, write your report including the code blocks in an R Notebook in RStudio. Click on "Preview" to generate an HTML file. Share that file with us!

Our Example Report

Open [our report](#) to see an example of a completed project! The report should open in a new window in your browser. Good luck and have fun!